

Антипаттерны PostgreSQL и как их избежать

Джимми Ангелакас

Предисловие Джанни Чолли



MANNING



PostgreSQL Mistakes and How to Avoid Them

JIMMY ANGELAKOS
FOREWORD BY GIANNI CIOLLI



MANNING
SHELTER ISLAND

Антипаттерны PostgreSQL и как их избежать

ДЖИММИ АНГЕЛАКОС
Предисловие Джанни Чолли



Санкт-Петербург • Москва • Минск

2026

Джимми Ангелакос

Антипаттерны PostgreSQL и как их избежать

Серия «Библиотека программиста»

Перевел с английского Е. Бартов

Научный редактор С. Гонцов

ББК 32.973.233-018.2

УДК 004.65

Ангелакос Джимми

A64 Антипаттерны PostgreSQL и как их избежать. — СПб.: Питер, 2026. — 272 с.: ил. — (Серия «Библиотека программиста»).

ISBN 978-5-4461-4493-8

Исправление ошибок в базах данных PostgreSQL может быть трудоемким и рискованным процессом — особенно при внесении изменений в работающую систему в реальном времени. К счастью, можно учиться на ошибках, которые уже совершили другие! Автор подробно описывает, как выявлять и предотвращать самые распространенные, опасные и скрытые антипаттерны проектирования при работе с PostgreSQL. Узнайте, откуда берутся ошибки в таких ключевых областях, как типы данных, функции, безопасность и обеспечение высокой доступности СУБД. Для каждой ошибки вы найдете реальный кейс, иллюстрирующий истоки проблемы, и получите конкретные рекомендации по ее устранению. Особую ценность представляют наглядные примеры кода, схемы и сравнительные таблицы преимуществ и недостатков различных подходов.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ISBN 978-1633436879 англ.

Authorized translation of the English edition © 2024 Manning Publications.
This translation is published and sold by permission of Manning Publications,
the owner of all rights to publish and sell the same.

ISBN 978-5-4461-4493-8

© Перевод на русский язык ООО «Прогресс книга», 2026
© Издание на русском языке, оформление ООО «Прогресс книга», 2026
© Серия «Библиотека программиста», 2026

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес:
194044, Россия, г. Санкт-Петербург, Б. Сапсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 05.2026. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 15.04.26. Формат 70×100/16. Бумага офсетная. Усл. п. л. 21,930. Тираж 700. Заказ 0000.

Краткое содержание

Глава 1. Почему важен PostgreSQL — и почему нужно говорить об ошибках.....	29
Глава 2. Некорректное использование SQL	43
Глава 3. Некорректное использование типов данных.....	77
Глава 4. Ошибки при работе с таблицами и индексами	95
Глава 5. Неправильное использование возможностей	114
Глава 6. Нерекомендуемые практики, снижающие производительность ...	143
Глава 7. Нерекомендуемые практики администрирования.....	176
Глава 8. Нерекомендуемые практики безопасности.....	198
Глава 9. Нерекомендуемые практики, снижающие уровень доступности...	212
Глава 10. Нерекомендуемые практики обновления и миграции.....	231
Глава 11. О PostgreSQL, лучших практиках и о вас: заключительные выводы.....	246
Приложение А. База данных Frogge Emporium	258
Приложение В. Памятка.....	263

Оглавление

Предисловие к русскому изданию	12
О научном редакторе русского издания	15
От издательства	15
Предисловие	16
Вступление	18
Благодарности	20
О книге	22
Для кого эта книга	23
Структура книги	23
О коде в книге	25
Форум обсуждений liveBook	25
Автор онлайн	26
Об авторе	27
Об иллюстрации на обложке	28
Глава 1. Почему важен PostgreSQL — и почему нужно говорить об ошибках	29
1.1. Почему важно изучать PostgreSQL	29
1.2. Почему важно говорить об ошибках в PostgreSQL	31
1.3. Что вы узнаете	32
1.4. Типичные виды ошибок PostgreSQL	32
1.4.1. Ожидания от других баз данных	32

1.4.2. Непонимание особенностей PostgreSQL.....	33
1.4.3. Непонимание документации.....	34
1.4.4. Использование реликтов из стандарта SQL.....	34
1.4.5. Несоблюдение лучших практик.....	35
1.5. Как устроена эта книга.....	35
1.5.1. Ментальные модели.....	36
1.5.2. Пример ошибки.....	37
1.6. Пример базы данных: Frogge Emporium.....	41
Итоги.....	42
Глава 2. Некорректное использование SQL.....	43
2.1. Использование NOT IN для исключения.....	43
2.1.1. Влияние на производительность.....	47
2.1.2. Альтернатива.....	47
2.2. Выбор диапазонов с помощью BETWEEN.....	48
2.3. Отказ от использования CTE.....	51
2.4. Использование идентификаторов в верхнем регистре.....	54
2.5. Деление целых чисел.....	57
2.6. Подсчет значений NULL.....	59
2.7. Запросы к индексированным столбцам через выражения.....	61
2.8. UPSERT с NULL в составном уникальном ключе.....	64
2.9. Выборка и получение всех данных.....	67
2.10. Игнорирование преимуществ чекеров / линтеров или больших языковых моделей.....	69
2.10.1. Чекеры / линтеры кода.....	70
2.10.2. Большие языковые модели.....	73
Итоги.....	76
Глава 3. Некорректное использование типов данных.....	77
3.1. TIMESTAMP (WITHOUT TIME ZONE).....	77
3.2. TIME WITH TIME ZONE.....	81
3.3. CURRENT_TIME.....	82
3.4. CHAR(n).....	83
3.5. VARCHAR(n).....	85
3.6. Тип MONEY.....	87

3.7. Тип данных SERIAL.....	89
3.8. XML	91
Итоги.....	94
Глава 4. Ошибки при работе с таблицами и индексами.....	95
4.1. Наследование таблиц.....	95
4.2. Пренебрежение партиционированием таблиц	99
4.3. Партиционирование по нескольким ключам	104
4.4. Использование неправильного типа индекса	107
Итоги.....	113
Глава 5. Неправильное использование возможностей.....	114
5.1. Выбор кодировки SQL_ASCII	115
5.2. CREATE RULE.....	121
5.3. Реляционный JSON.....	125
5.4. Повсеместное использование UUID.....	128
5.5. Самописная multi-master репликация	131
5.6. Самописные распределенные системы	138
Итоги.....	142
Глава 6. Нерекомендуемые практики, снижающие производительность	143
6.1. Конфигурация по умолчанию в продакшене	144
6.2. Некорректное распределение памяти.....	147
6.3. Завышенное число разрешенных соединений	151
6.4. Простаивающие соединения	155
6.4.1. Для начала: что такое MVCC?	155
6.4.2. Проблема с простаивающими соединениями	156
6.5. Управление длительными транзакциями.....	158
6.5.1. Простой в транзакции	158
6.5.2. Длительные запросы.....	161
6.6. Высокая частота транзакций.....	162
6.6.1. Переполнение идентификаторов транзакций (XID)	163
6.6.2. Быстрое расходование XID	164
6.7. Отключение autovacuum/autoanalyze.....	165
6.8. Игнорирование возможностей EXPLAIN (ANALYZE)	168
6.9. Явная блокировка	169

6.10. Отсутствие индексов	170
6.11. Лишние индексы	172
6.12. Удаление индексов, которые используются в других местах	173
Итоги	174
Глава 7. Нерекомендуемые практики администрирования	176
7.1. Отсутствие мониторинга дискового пространства	176
7.1.1. Удаление журнала предзаписи	178
7.1.2. Потребители дискового пространства	179
7.1.3. Возможные действия	179
7.2. Запись логов в PGDATA	180
7.3. Игнорирование журналов	183
7.3.1. Некорректная конфигурация	183
7.3.2. Проблемы производительности	184
7.3.3. Блокировки	185
7.3.4. Повреждение данных	186
7.3.5. Безопасность	186
7.4. Отказ от мониторинга базы данных	187
7.5. Отказ от отслеживания статистики	189
7.6. Отказ от обновлений Postgres	192
7.7. Отказ от обновления системы	194
Итоги	196
Глава 8. Нерекомендуемые практики безопасности	198
8.1. Указание <code>psql -W</code> или <code>--password</code>	199
8.2. Установка <code>listen_addresses = '*'</code>	200
8.3. Использование <code>trust</code> в <code>pg_hba.conf</code>	202
8.4. База данных, принадлежащая суперпользователю	204
8.5. Неосторожное использование <code>SECURITY DEFINER</code>	207
8.6. Выбор небезопасного пути поиска	209
Итоги	210
Глава 9. Нерекомендуемые практики, снижающие уровень доступности	212
9.1. Отсутствие резервного копирования	213
9.2. Отсутствие точки восстановления на момент времени	216
9.3. Резервное копирование вручную	219

10 Оглавление

9.4. Отсутствие тестирования резервных копий.....	221
9.5. Отсутствие избыточности.....	224
9.6. Игнорирование специальных инструментов высокой доступности	227
Итоги.....	229
Глава 10. Нерекомендуемые практики обновления и миграции	231
10.1. Пропуск примечаний к выпуску	232
10.2. Недостаточное тестирование.....	236
10.3. Погружение в хаос кодировок.....	239
10.4. Разный BOOLEAN.....	241
10.5. Некорректная обработка различий в типах данных.....	244
Итоги.....	245
Глава 11. О PostgreSQL, лучших практиках и о вас:	
заключительные выводы	246
11.1. К какому типу пользователей вы относитесь	246
11.1.1. Любитель	247
11.1.2. Осторожный хранитель.....	248
11.1.3. Беспечный разработчик.....	249
11.1.4. Выживальщик	249
11.2. Будьте проактивны: действуйте на опережение.....	250
11.3. Итак, вам досталась проблемная база данных.....	252
11.3.1. «Так исторически сложилось»	252
11.3.2. Что дальше?	252
11.3.3. Начните с главного.....	253
11.4. Относитесь к Postgres с уважением, и она ответит вам тем же	255
Итоги.....	256
Приложение А. База данных Frogge Emporium	258
А.1. Схема базы данных Frogge Emporium.....	258
А.2. Данные базы Frogge Emporium.....	261
Приложение В. Памятка.....	263

Элпиде Анастасиу, моей любимой спутнице, которая приносила радость в мою жизнь последние десять лет. Без ее постоянной поддержки эта книга не увидела бы свет.

Посвящается памяти моего отца Янниса Ангелакоса (Giannis Angelakos), который пробудил во мне страсть к вычислительной технике в 1984 году, когда принес домой Sinclair ZX Spectrum. Он всегда поддерживал мои начинания и верил в меня.

Саймону Риггсу (Simon Riggs), моему наставнику, коллеге и другу, который дал мне старт в профессиональном мире PostgreSQL. Его доверие и наставничество стали бесценным вкладом в мой рост.

Предисловие к русскому изданию

В 2025–2026 годах PostgreSQL признан самой любимой базой данных разработчиков (по опросу на Stack Overflow Developer Survey 2025) и стабильно занимает 4-е место в мире в общем рейтинге СУБД по популярности (DB-Engines Ranking), причем с самой высокой скоростью роста популярности и применимости. Сейчас даже все крупнейшие коммерческие вендоры СУБД имеют в своих портфелях или во внутренних решениях продукты на базе PostgreSQL в дополнение к собственным коммерческим СУБД, что казалось невозможным еще лет 5–7 назад. Но откуда такая любовь, и главное, чем она подпитывается, что обеспечивает такие высокие показатели роста?

Тут нет особых тайн: большинство ответов достаточно очевидны.

1. Экономика и возможности легитимного использования: PostgreSQL бесплатен, и если ранее на какую-то коммерческую СУБД еще можно было «разориться», то при нынешних объемах данных стоимость лицензий на коммерческую СУБД часто может быть просто бессмысленно высокой. Но не только «бесплатность» важна как таковая, главное то, что PostgreSQL распространяется по одной из самых либеральных лицензий — PostgreSQL License, которая допускает свободное использование, модификацию и встраивание в коммерческие продукты без жестких лицензионных ограничений.
2. Инженерная универсальность: PostgreSQL является сильнейшим представителем классической реляционной СУБД (транзакционность, строгая целостность данных, развитая поддержка SQL, широкий набор механизмов контроля конкурентного доступа). Кроме того, он спроектирован как расширяемая платформа: можно добавлять типы данных, индексы, функции, процедуры и расширения. Благодаря этому PostgreSQL применяется в широком диапазоне сценариев: от небольших приложений или очень узкоспециализированных решений, например, с использованием расширения PostGIS для работы с геопространственными данными (с возможностью выполнения

над ними всех необходимых операций), до крупномасштабных систем, где важны надежность, предсказуемость поведения и качество диагностики.

3. Качество и зрелость СУБД для разработчиков и команд эксплуатации: зрелая и дисциплинированно развиваемая кодовая база с многолетней эксплуатацией. Изменения проходят публичное обсуждение и многоэтапное ревью, регрессии отслеживаются автоматическими тестами, в разработке СУБД делается сильнейший акцент на обеспечении обратной совместимости и предсказуемом поведении между версиями. В результате PostgreSQL ценят за стабильность под нагрузкой, корректность транзакционной модели и целостность данных, устойчивость к пограничным случаям, возможности качественной диагностики (планы, статистики), за понятный жизненный цикл и длительную поддержку выпущенных версий (что для open-source продуктов скорее исключение, чем правило). Кроме того, зрелость PostgreSQL характеризуется тем, что вокруг него сформировалось большое сообщество и рынок инструментов: библиотеки доступа к PostgreSQL, фреймворки для работы с данными через модели, системы мониторинга, средства резервного копирования, решения для высокой доступности и решения для миграций на PostgreSQL.

Существующая популярность PostgreSQL — это не какой-то временный тренд в ИТ, основанный на очередной модной технологии. Популярность поддерживает:

- понятная модель управления и релизного планирования: PostgreSQL имеет формализованную модель управления и публичные процессы разработки, предсказуемые релизы и долгий горизонт поддержки — новые мажорные релизы выходят регулярно, а поддержка длится годами (сейчас это 5 лет на мажорный релиз), что позволяет планировать обновления и эксплуатационные мероприятия на годы вперед;
- значительный накопленный объем разных DevOps практик, инструкций, руководств, касающихся того, как быстро развертывать и применять PostgreSQL, в том числе под разные узкие задачи и сценарии;
- огромные инвестиции крупных вендоров и в Postgres-экосистему (облака, сервисы), которые в том числе возможны именно из-за либеральной лицензии PostgreSQL, позволяющей компаниям без особых рисков строить свои сервисы и услуги, продукты, дистрибутивы, расширения вокруг PostgreSQL, тем самым усиливая его ценность и еще более разгоняя популярность.

При этом можно с уверенностью сказать, что в России и собственный интерес к PostgreSQL, поддерживаемый импортозамещением, и своя история развития: российские компании и отдельные российские разработчики составляют

заметную долю в сообществе ключевых разработчиков PostgreSQL. Сейчас в реестре отечественного ПО уже не менее 8 независимых поставщиков СУБД на базе PostgreSQL с общим количеством продуктов более 20. Если еще 5 лет назад люди улыбались или крутили пальцем у виска, когда речь заходила об использовании PostgreSQL для 1С, то сейчас PostgreSQL фактически стал новым стандартом для 1С и в некоторых случаях работает с 1С даже значительно быстрее, чем MS SQL.

А есть ли какие-либо минусы или обратная сторона медали у такого столь популярного open-source продукта, ведь не бывает же, что всегда все так идеально? Минусы, или, скажем так, сложившиеся архитектурные особенности, с которыми нужно «мириться», конечно же, есть: это и модель MVCC со своими ограничениями и спецификой (необходимостью использования VACUUM), и особый академический подход к СУБД, где много внимания уделяется чистоте и строгости соответствия SQL стандартам и типам данных, но при этом пользователю отдаются на полный откуп все вопросы, связанные с мониторингом (как оперативным, так и историческим), с резервным копированием, с обеспечением высокой доступности и с масштабируемостью, с выбором лучших практик для решения той или иной задачи. Особенность официальной документации PostgreSQL состоит в том, что она не говорит как, она говорит, какие функции есть и как они могут работать, что отличает ее от документации коммерческих вендоров, где часто есть информация о том, что нужно сделать для реализации того или иного сценария использования. Такой подход к документированию продукта и его очень высокая популярность в итоге привели к тому, что в интернете размещены тонны разных рекомендаций, инструкций, лучших практик, tutorиалов, с которыми еще нужно разобраться, оценить, понять, для какого сценария они годятся, и тут объемы доступной информации играют против ее полезности: многие, боясь этих объемов разношерстной информации, начинают к ней обращаться только тогда, когда что-то плохое уже случилось. И в данном случае книга «Антипаттерны PostgreSQL и как их избежать» — это тот доступный, крайне ценный и обобщенный опыт, представляющий собой альтернативу одновременно всем лучшим практикам сразу, который рассказывает, как точно не нужно делать, чтобы избежать потери производительности, повреждения или потери данных или значительных затрат на восстановление работоспособности вашего сервиса или продукта. Тут не описаны подробно лучшие практики, ценность которых становится ключевой для продукта с подобным типом развития и документирования, но вам точно будет легче их найти после прочтения этой книги. Но главное, книга призвана избавить вас от массы серьезных ошибок при проектировании и создании систем на базе PostgreSQL, которые будет крайне сложно и дорого исправлять.

*Сергей Юнцов, научный редактор
русского издания книги «Антипаттерны
PostgreSQL и как их избежать»*

О научном редакторе русского издания

Сергей Гонцов более 25 лет в ИТ — 25 лет интереснейшей работы в интеграторах, где каждый ИТ-проект — это новое испытание и новые смежные области знаний. Прошел путь от инженера поддержки одной из сложных государственных автоматизированных систем (ГАС) до технического владельца продуктов OrionSoft Proxima DB и Arenadata Prosperity (российских СУБД на базе PostgreSQL), участвовал во многих сложных и распределенных ИТ-проектах, постоянно сталкиваясь с разными вызовами использования реляционных СУБД, принимал участие в проектировании и создании инфраструктуры ЕМИАС, инженер по духу и увлеченный исследователь.

От издательства

Мы выражаем огромную благодарность клубу рецензентов ИТ-литературы ReadIT Club за помощь в работе над русскоязычным изданием книги и вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

Предисловие

Эта книга полностью оправдывает свое название: в ней вы увидите много примеров ошибок и предложений, как этих ошибок избежать. Однако заинтересованный читатель найдет здесь гораздо больше.

Примеры описаны четко и конкретно, они представляют собой случаи, которые могут произойти на практике. Это неудивительно: я знаю Джимми много лет, и мы вместе работали над множеством реальных проектов, помогая клиентам решать проблемы, с которыми они сталкивались. В какой-то момент мы оба были членами команды архитектуры решений, которая фокусируется на объединении технологий в согласованные системы. Такого рода работа расширяет кругозор, потому что дает заметные результаты и ощущение того, как эти результаты влияют на бизнес клиента. Кроме того, она требует внимания к деталям и весьма серьезных практических навыков. Работая в этой роли, вы постепенно укрепляете технические навыки и получаете ценный опыт работы с разнообразными продакшен-средами, к которым предъявляются высокие требования.

В книге *«Антипаттерны PostgreSQL и как их избежать»* вы поймете, как смотрит на проблему архитектор решений, — как он с помощью теории упорядочивает факты и сохраняет готовность в любой момент погрузиться в технические детали и решение практических вопросов, если того требует проблема.

Эту книгу рекомендуется прочитать всем, кому необходимо работать с PostgreSQL в продакшен-среде. Системный администратор, администратор СУБД, разработчик приложений, архитектор приложений и т. д. — все эти роли явно отличаются друг от друга с точки зрения целей их работы и требуемых навыков, но всех их объединяет общий путь, на котором детальное знание PostgreSQL действительно помогает, — многие мои клиенты убеждались в этом.

Названия глав в первую очередь относятся к конкретным практическим темам, таким как SQL, типы данных, таблицы и индексы; однако есть и более общие, например высокая доступность, производительность, администрирование,

безопасность, обновления и миграции. Начать чтение можно с более знакомой темы, а затем перейти к другим главам, не обязательно в том порядке, который представлен здесь. Более общие выводы приведены в последней главе, и чтобы подчеркнуть прикладной характер книги, в конце дано краткое резюме всего материала (своего рода шпаргалка).

Я уверен, что вы найдете эту книгу очень интересной и полезной, как и я, когда читал ее. Я благодарен Джимми Ангелаосу за то, что он не жалел усилий при ее написании, что, признаться, в его случае было немного проще, поскольку он мог использовать свой опыт популярного спикера на конференциях по PostgreSQL, чтобы понятно объяснять сложные вещи и выбирать правильные примеры.

*Д-р Джанни Чолли (DR. Gianni Ciolli), вице-президент, руководитель
направления по решениям высокой доступности в EDB*

Вступление

PostgreSQL (или Postgres для друзей) — это система управления базами данных (СУБД) общего назначения, за которой стоит более 25 лет разработки и процветающее активное сообщество. Мы называем ее системой общего назначения, потому что она не специализирована под конкретный сценарий применения. Скорее наоборот: множество проектных решений, охватывающих долгую историю ее разработки, склонялись в пользу сбалансированного поведения, чтобы Postgres мог показывать одинаково хорошую производительность во многих сценариях. Эта стратегия со временем оправдала себя, поэтому сформировалась чрезвычайно разнообразная пользовательская база, охватывающая большое число областей применения, часто они дополняются специализированными расширениями.

Согласно опросу разработчиков Stack Overflow 2024 года (<https://survey.stackoverflow.co/2024>), PostgreSQL уже второй год подряд признается самой предпочитаемой базой данных среди разработчиков в целом и профессиональных разработчиков в частности. В позапрошлом году она отобрала первое место по популярности у MySQL, и пока ничто не предвещает снижения темпов роста ее значимости. Это говорит о ее все большем влиянии и о том, что Postgres представляет собой инновационную силу, которая продолжает развиваться, чтобы соответствовать требованиям современного рынка баз данных и индустрии разработки ПО.

Я знал о PostgreSQL как о полнофункциональной СУБД с хорошей репутацией еще с конца 1990-х годов и начал экспериментировать с ней в начале 2000-х. Примерно с 2008 года по настоянию друга и коллеги я использую ее профессионально. С тех пор я ни разу не пожалел об этом и не переходил на другую систему баз данных.

Самая большая ошибка PostgreSQL

Давайте поговорим о слоне в комнате — о названии. После Ingres, предыдущей версии базы данных, созданной Майклом Стоунбрейкером (Michael Stonebraker), появился Postgres (Post-Ingres). Добавление возможностей SQL к Postgres привело к изменению названия на PostgreSQL, которое произносится как «пост-гресс-кью-эл». Многие участники и члены сообщества признали, что это название неудобно произносить, но пришли к выводу, что уже слишком поздно менять имя проекта. Тем не менее все согласились, что имя «Postgres» вполне приемлемо и эквивалентно полному варианту, поэтому в этой книге мы будем использовать оба названия взаимозаменяемо.

И да, никогда не говорите «Postgre».

Благодарности

Я хотел бы поблагодарить Джонатана Генника (Jonathan Gennick), редактора по привлечению авторов, который предложил превратить мой опыт в эту книгу. Его поддержка и проницательность заложили основу для этого проекта, а его помощь на протяжении всего процесса была бесценна.

Кэти Спосато (Katie Sposato), редактор по развитию, заслуживает моей искренней благодарности за ее профессиональные инструкции и вдумчивые советы. Она направляла меня в правильное русло с помощью практических советов и конструктивной обратной связи, и это помогло мне значительно улучшить ясность и структуру этой книги.

Не менее я благодарен Саймону Пейну (Simon Pane), техническому редактору, чей острый глаз и экспертность в технических вопросах гарантировали точность и достоверность содержания. Его скрупулезное внимание к деталям и глубокое понимание PostgreSQL сыграли ключевую роль в этой работе. Также выражаю благодарность Фрэнку Муру (Frank Moore), техническому корректору, за его тщательную вычитку и внимание к деталям, что дополнительно повысило качество книги.

Благодарю всех рецензентов: Адама Вана (Adam Wan), Алекса Элистратова (Alex Elistratov), Эй-Джей Бхандаля (AJ Bhandal), Амита Шарму (Amit Sharma), Андреаса Сакко (Andres Sacco), Эндрю Эленески (Andrew Eleneski), Ануджа Тьяги (Anuj Tyagi), Бассам Исмаила (Bassam Ismail), Билла Митчелла (Bill Mitchell), Корнела Гибана (Cornel Ghiban), Даниэля Васкеса (Daniel Vásquez), Дирка Гомеса (Dirk Gomez), Фернандо Бугни (Fernando Bugni), Фрэнка Мура (Frank Moore), Грегорио Пикколи (Gregorio Piccoli), Харша Ранджана (Harsh Ranjan), Генриетту Домбровскую (Henrietta Dombrovskaya), Ихсана Акина (Ihsan Akin), Иябо Синдику (Iyabo Sindiku), Джереми Чена (Jeremy Chen), Хосе Альберто Рейеса Кеведо (Jose Alberto Reyes Quevedo), Жозефа Пашода (Joseph Pachod), Кирана Кришнамурти (Kiran Krishnamurthy), Луку Феррари (Luca Ferrari), Маркуса Гезелле (Marcus Geselle), Маногара Саи Джасты (Manohar Sai Jasti),

Микаэля Дотре (Mikael Dautrey), Милорада Имбру (Milorad Imbra), Надира Доктора (Nadir Doctor), Нагу Ришьяндар Пангулури (Naga Rishyendar Panguluri), Пола Сноу (Paul Snow), Потито Колуччелли (Potito Coluccelli), Прабху Пателя (Prabhu Patel), Рани Шарим (Rani Sharim), Регину Обе (Regina Obe), Рубена Вандегинсте (Ruben Vandeginste), Сергея Смертина (Serge Smertin), Серхио Бритоса Аревало (Sergio Britos Arevalo), Симоне Сгуазца (Simone Sguazza), Викторию Долженко (Viktoria Dolzhenko), Уэса Шаддикса (Wes Shaddix) и Уильяма Джамира Силву (William Jamir Silva). Ваши отзывы помогли улучшить эту книгу.

Особая благодарность SomaFM (<https://soma.fm/>) — в частности, их каналу Synphaera — за идеальный саундтрек, необходимый для концентрации и глубокой фокусировки.

Наконец, я хотел бы вновь выразить свою признательность широкому и щедрому сообществу PostgreSQL и миру открытого исходного кода в целом. Ваш дух сотрудничества, готовность делиться знаниями и неизменная поддержка были источником вдохновения на протяжении всего моего пути.

Спасибо всем, кто помог сделать эту книгу реальностью.

О книге

В феврале 2023 года я выступил с докладом под названием «Не делайте этого» на ежегодной конференции FOSDEM (Free and Open Source Software Developers' European Meeting, <https://fosdem.org>) в бельгийском Брюсселе. В докладе рассматривались некоторые распространенные ошибки, подводные камни и заблуждения, с которыми могут столкнуться пользователи PostgreSQL, а также обсуждались возможные способы исправить их или обойти. Доклад был хорошо принят, и я представил его еще на двух конференциях. Но у меня осталось впечатление, что по этой теме можно рассказать гораздо больше. Однако объем материала стал таким, что по ощущениям он перестал подходить для выступлений на конференциях. Поиск более подходящего формата для этого контента и послужил толчком к обсуждениям, которые в конечном счете привели к написанию моей первой книги — *«Антипаттерны PostgreSQL и как их избежать»*.

Спойлер: в реальности ошибок и антипаттернов больше, чем перечислено в этой книге! Мне нравится думать, что буквально нет ничего, что нельзя было бы сделать неправильно. Но здесь упомянуты самые распространенные или самые опасные ошибки, с которыми вы, вероятно, столкнетесь.

Эта книга позволит вам изучить PostgreSQL, но ее формат, как мне кажется, более удобен, чем специализированный веб-сайт, разрозненные записи в блоге, ответы на вопросы на форумах или серия выступлений на конференциях. Книга дает возможность собрать мои мысли о правильной работе с PostgreSQL в одном месте и одновременно погрузиться глубже в эту тему и продвинуться дальше, чем любой из упомянутых источников. Кроме того, вам не придется тратить время на поиск нужного в интернете или смотреть многочасовые живые или плохо записанные выступления.

Описанные в этой книге ошибки — результат работы с PostgreSQL и ее программной экосистемой. Мне посчастливилось ежедневно взаимодействовать с Postgres в различных ролях во время работы в двух ведущих компаниях, специализирующихся на этой СУБД, что дало мне понимание того, как функционируют,

настраиваются и обслуживаются системы конечных пользователей, включая как хорошие практики, которым стоит следовать, так и плохие, которых следует избегать. Благодаря многолетнему опыту развертывания, наблюдения и исправления систем Postgres на практике я сформировал необходимое видение, чтобы написать эту книгу.

В любом случае, как говорится, «ни один человек не есть остров, но каждый — часть материка, часть целого¹», и книга была бы невозможна без наставничества, обмена знаниями и руководства, которые сообщество PostgreSQL так щедро предоставляло мне на протяжении многих лет.

Для кого эта книга

Чтобы извлечь максимальную пользу из этой книги, вы должны обладать общими знаниями о работе реляционных систем баз данных и практическим пониманием языка SQL. Это подразумевает способность проектировать простейшую реляционную базу данных с использованием SQL DDL (Data Definition Language) при базовом понимании моделирования данных. Соответственно, вы должны уметь писать SQL-запросы с предложениями `WHERE` и понимать концепции, лежащие в основе индексирования.

Для полноценной работы с книгой лучше иметь базовые знания о PostgreSQL. Вы должны уметь устанавливать и настраивать окружение PostgreSQL (только основы; не беспокойтесь о таких вещах, как тонкая настройка производительности). Также полезно знакомство с командной строкой UNIX (в частности, Linux). Вам пригодится умение работать с утилитами командной строки, понимание конвейеров (pipng) и перенаправления вывода.

Структура книги

Книга разделена на 11 глав, каждая из них посвящена критически важному аспекту использования PostgreSQL, подводным камням и лучшим практикам. Главы охватывают широкий спектр тем — от фундаментальных концепций до продвинутых вопросов, а два приложения предоставляют практические ресурсы для поддержки вашего обучения.

Вот краткий обзор каждой главы и приложения:

- *Глава 1. Почему важен PostgreSQL — и почему нужно говорить об ошибках* — даем обзор PostgreSQL и подчеркиваем цели этой книги, включая то, как

¹ Донн Джон. Избранное. В переводе Г. М. Кружкова. — *Примеч. ред.*

выявлять распространенные ошибки PostgreSQL, классифицировать их и извлекать из них уроки.

- *Глава 2. Некорректное использование SQL* — обсуждаем распространенные подводные камни SQL, способы написать корректный запрос и методы повышения производительности сложных запросов.
- *Глава 3. Некорректное использование типов данных* — исследуем последствия выбора неподходящего типа данных, проблемы с часовыми поясами и переходом на летнее время, а также типы данных, которые лучше избегать.
- *Глава 4. Ошибки при работе с таблицами и индексами* — освещаем расширенные возможности таблиц, а также важность партиционирования и выбор наиболее эффективных ключей и индексов для таблиц.
- *Глава 5. Неправильное использование возможностей* — анализируем сложности и риски, связанные с некорректным использованием возможностей PostgreSQL, таких как выбор кодировки, создание правил и нецелевое применение NoSQL-функций.
- *Глава 6. Нерекомендуемые практики, снижающие производительность* — освещаем ошибки: использование конфигураций по умолчанию в продакшен-среде, некорректное управление лимитами подключений, а также ошибочное управление транзакциями, индексами и настройками автоочистки.
- *Глава 7. Нерекомендуемые практики администрирования* — сосредоточиваемся на административных упущениях, включая неконтролируемое использование дискового пространства, игнорирование журналов, недостаточный статистический анализ и некорректное выполнение обновлений PostgreSQL.
- *Глава 8. Нерекомендуемые практики безопасности* — определяем риски безопасности и то, почему нельзя небрежно обращаться с паролями, предоставлять избыточные права доступа, подвергать базу данных рискам и небезопасно использовать функции.
- *Глава 9. Нерекомендуемые практики, снижающие уровень доступности* — обсуждаем распространенные проблемы доступности базы данных, включая пренебрежение резервным копированием, некорректное использование функций восстановления и отсутствие подготовки к сбоям базы данных.
- *Глава 10. Нерекомендуемые практики обновления и миграции* — разбираем потенциальные ловушки во время обновлений или миграций, такие как пропуск версий, недостаточное тестирование и недооценка различий между типами баз данных.
- *Глава 11. О PostgreSQL, лучших практиках и о вас: заключительные выводы* — обобщаем извлеченные уроки, предлагаем руководство по проактивному избеганию технического долга и подходы к улучшению унаследованных баз данных PostgreSQL.

- *Приложение А. База данных Frogge Emporium* — пример схемы и набора данных для практического применения концепций, обсуждаемых в книге.
- *Приложение В. Памятка* — быстрый справочник с советами и лучшими практиками PostgreSQL, который поможет избежать ошибок и работать эффективно.

О коде в книге

В этой книге содержится множество примеров исходного кода как в нумерованных кейсах — листингах, так и внутри обычного текста. В обоих случаях исходный код форматируется моноширинным шрифтом, чтобы отделить его от обычного текста. Иногда код также выделяется полужирным шрифтом, чтобы подчеркнуть изменения по сравнению с предыдущими шагами в главе, например, когда новая функциональность добавляется к существующей строке кода.

Во многих случаях исходный код был переформатирован: мы добавили переносы строк и изменили отступы, чтобы уместить его в доступное пространство страницы книги. В редких случаях даже этого было недостаточно, и листинги включают маркеры продолжения строк (↵). Кроме того, комментарии в исходном коде часто удаляются из листингов, когда код описывается в тексте. Многие листинги сопровождаются аннотациями к коду, которые выделяют важные концепции.

Вы можете получить исполняемые фрагменты кода из онлайн-версии этой книги на платформе liveBook по адресу <https://livebook.manning.com/book/postgresql-mistakes-and-how-to-avoid-them>. Полный код примеров из книги доступен для загрузки с веб-сайта Manning по адресу www.manning.com, а также с GitHub по адресу <https://github.com/vyruss/postgresql-mistakes>.

Форум обсуждений liveBook

Покупая книгу «*Антипаттерны PostgreSQL и как их избежать*», вы также получаете бесплатный доступ к платформе для онлайн-чтения liveBook издательства Manning (на английском языке). Эксклюзивные возможности liveBook позволяют оставлять комментарии как к книге в целом, так и к отдельным ее разделам или абзацам. Можно легко делать заметки для себя, задавать технические вопросы и отвечать на них, а также получать помощь от авторов и других пользователей. Чтобы получить доступ к форуму, посетите страницу <https://livebook.manning.com/book/postgresql-mistakes-and-how-to-avoid-them>. Информацию о форумах Manning и правилах поведения на них см. на <https://livebook.manning.com/discussion>.

В рамках своих обязательств перед читателями издательство Manning предоставляет ресурс для содержательного общения читателей и авторов. Эти обязательства не подразумевают конкретную степень участия автора, которое остается добровольным (и неоплачиваемым). Задавайте авторам хорошие вопросы, чтобы им было интересно участвовать в диалоге! Форум и архивы обсуждений доступны на сайте Manning, пока книга продолжает издаваться.

Автор онлайн

Джимми Ангелакоса (Jimmy Angelakos) можно найти в интернете на следующих платформах:

- *YouTube* — <https://youtube.com/JimmyAngelakos>
- *Mastodon* — <https://fosstodon.org/@vyruss>
- *Bluesky* — <https://bsky.app/profile/vyruss.org>
- *LinkedIn* — <https://linkedin.com/in/vyruss>
- *GitHub* — <https://github.com/vyruss>
- *Веб-сайт* — <https://vyruss.org/computing>
- *Блог* — <https://vyruss.org/blog>

Об авторе



Джимми Ангелакос — архитектор систем и баз данных, признанный эксперт по PostgreSQL. Более 25 лет работает с open-source инструментами и развивает их. Занимал ключевые роли в компаниях 2ndQuadrant и EDB, где получил богатый опыт в области архитектуры программного обеспечения и глубокие экспертные знания в PostgreSQL. Как активный член сообщества Джимми — контрибьютор проекта PostgreSQL и активный участник PostgreSQL Europe и PostgreSQL US. Регулярно делится своими идеями в качестве спикера на конференциях и мероприятиях, посвященных базам данных и open-source приложениям. Изучал информатику в Университете Абердина.

Об иллюстрации на обложке

Иллюстрация под названием Le Corse («Житель Корсики»), помещенная на обложку, взята из книги Луи Кюрме (Louis Curmer), опубликованной в 1841 году. Каждая иллюстрация в ней тщательно прорисована и раскрашена вручную.

В те времена было легко определить просто по одежде людей, где они живут и каков их род занятий или положение в обществе. Издательство Manning поощряет изобретательность и инициативность участников бизнеса в сфере ИТ и при создании обложек берет за основу богатый и разнообразный многовековой опыт региональной культуры, возвращенный к жизни с помощью изображений из коллекций, подобных этой.

Почему важен PostgreSQL — и почему нужно говорить об ошибках

В этой главе

- ✓ Каковы основы PostgreSQL
- ✓ Как выявлять ошибки PostgreSQL и почему важно их обсуждать
- ✓ Какие бывают ошибки PostgreSQL
- ✓ Как использовать эту книгу для обучения

Добро пожаловать на страницы книги *«Антипаттерны PostgreSQL и как их избежать»*! Наверняка вы приобрели эту книгу, чтобы узнать больше о PostgreSQL, и речь пойдет действительно о нем. Однако это не учебник в традиционном смысле и не руководство по администрированию — уже есть хорошие книги с таким ракурсом. Эта книга подчеркивает, что в реальности ошибки случаются, но на них хорошо учиться. В этой главе мы увидим, почему СУБД PostgreSQL актуальна именно сейчас, как эта книга рассматривает ошибки, связанные с PostgreSQL, и почему стоит обращать на них внимание.

1.1. Почему важно изучать PostgreSQL

Как мы уже упоминали ранее, PostgreSQL (также известный как Postgres) — это инновационная технология, которая начала смещать крупных поставщиков баз

данных с их позиций. Это надежная и стабильная база данных с богатым функционалом, способным значительно расширяться, и с каждым новым выпуском она приобретает все больше возможностей продукта уровня Enterprise. Эта технология получила широкое признание не только в сообществе специалистов по СУБД, но и в коммерческих кругах: сотни вендоров теперь предлагают помощь в разработке СУБД, услуги техподдержки, версии с расширенными функциями и даже хостинг СУБД, включая предоставление облачных или DBaaS-решений (database-as-a-service) на основе Postgres.

Важно отметить, что проект PostgreSQL — яркий пример свободного ПО, который управляется и развивается сообществом, что выражено в лицензии PostgreSQL. Эта лицензия особенно значима на фоне того, что большинство других поставщиков СУБД придерживаются стратегии проприетарности ПО. Кроме того, стремясь увеличить собственную прибыль и ограничить заработок конкурентов, многие новые и давно работающие проекты в области СУБД переходят на лицензии, несовместимые с принципами ПО на базе открытого исходного кода. Вы же можете свободно извлекать прибыль из PostgreSQL, продавая поддержку, хостинг, обучение или услуги; даже создать на его основе собственный продукт с открытым или закрытым исходным кодом без риска неприятных юридических сюрпризов. Отсутствие привязки к конкретному поставщику в сочетании с возможностью полностью построить свой бизнес на Postgres без лицензионных ограничений — это мощный фактор, ярко выделяющий PostgreSQL на фоне других конкурирующих СУБД.

Такой подход к лицензированию меняться не будет. Джонатан Кац (Jonathan Katz) из основной команды разработчиков отметил в марте 2024 года: «Проект PostgreSQL начинался как совместная инициатива с открытым исходным кодом и создан так, чтобы никто не смог его полностью контролировать. Этот принцип сохраняется все 30 лет и даже закреплён во всех политиках проекта» (<http://jkatz05.com/post/postgres/postgres-license-2024/>).

Мощность и расширяемость PostgreSQL позволили ему найти применение в самом широком спектре отраслей и приложений. В частности, крупные банковские, кредитные и розничные системы применяют ее в OLTP-сценариях (Online Transaction Processing, оперативная обработка транзакций) и пользуются его расширенными возможностями составлять запросы и формировать отчеты для масштабной бизнес-аналитики, включая OLAP-процессы (Online Analytical Processing, процессы оперативной аналитики). Такие расширения, как PostGIS для географических информационных систем (ГИС, GIS), колоночные СУБД Citus, TimescaleDB для работы с временными рядами и pgvector для векторного поиска, добавили множество специализированных функций, что еще больше расширило пользовательскую базу. Сетевые отели, киностудии, транспортные компании, страховые фирмы, государственные организации, медицинские учреждения и космические агентства используют ту или иную форму Postgres,

и этот список можно продолжать. Для этих задач чрезвычайно важны надежность, масштабируемость и отказоустойчивость. Разумеется, экосистему дополняют множество крупных и мелких вендоров, консультантов и подрядчиков, которые обеспечивают работу всех этих приложений.

1.2. Почему важно говорить об ошибках в PostgreSQL

Популярность PostgreSQL растет, ежедневно увеличивается число вариантов его применения, и мы наблюдаем множество пользователей, которые пытаются применять практики, усвоенные при работе с другими СУБД, или неправильно понимают особенности Postgres. Эти практики могут выражаться в том, что люди повторяют что-то, что работало в другой СУБД, или используют функциональность, не понимая полностью ее документации. Например, PostgreSQL поддерживает такие сложные типы данных, как массивы, тогда как в других СУБД пришлось бы вставлять несколько строк с повторяющимися данными. Иногда это просто случай использования неподходящего инструмента для задачи — скажем, неправильной функции или типа данных. Также может быть, что выбранное решение работает сейчас, но откажется работать в будущем по различным причинам, таким как чрезмерное увеличение размера таблицы.

Эти и другие факторы часто приводят к дорогостоящим ошибкам, если они происходят в производственной системе. Поэтому лучше знать об этих ошибках заранее и вовремя их выявлять, поскольку исправление также может отнимать много времени и быть связано с высокой степенью сложности или риска, например, если изменения вносятся в работающую систему.

Важно распространять информацию о потенциальных ошибках и подводных камнях, чтобы:

- сэкономить человеко-часы, которые будут затрачены на устранение последствий;
- безопасно хранить данные;
- гарантировать долгосрочную жизнеспособность проектов баз данных;
- устранить антипаттерны, вредные для проектирования и производительности;
- применять стандартные решения, а не изобретать велосипед;
- повысить осведомленность о лучших практиках работы с базами данных в вашей команде или организации;
- максимально эффективно использовать возможности PostgreSQL.

Также важно защищать репутацию вашей любимой СУБД! Слишком часто мы видим сообщения в блогах или статьи, критикующие PostgreSQL, авторы которых не поняли технологию. Пользователи пытаются сделать что-то нестандартным способом, терпят неудачу и приходят к ошибочному выводу, что PostgreSQL плох.

1.3. Что вы узнаете

Книга *«Антипаттерны PostgreSQL и как их избежать»*, надеюсь, поможет вам:

- узнать о потенциальных подводных камнях PostgreSQL, относящихся к различным аспектам конфигурации, администрирования и эксплуатации СУБД;
- научиться избегать упомянутых проблем до того, как проявятся последствия, или применять обходные решения, чтобы их исправить;
- приобрести более широкое понимание фундаментальных принципов работы баз данных, чтобы применять лучшие практики;
- глубже понять особенности и нюансы PostgreSQL, которые отличают ее от других баз данных;
- проследить, как лучшие практики влияют на использование PostgreSQL и его производительность.

Эти знания позволят вам в повседневной работе:

- экономить время и усилия, применяя лучшие практики и связанные с ними шаблоны использования;
- действовать на опережение и решать потенциальные проблемы до того, как они станут разрушительными;
- принимать профилактические меры и защищать себя от неправильного использования базы данных другими (случайно или умышленно);
- обучать коллег, делаясь этими знаниями, и сделать опыт работы с PostgreSQL более продуктивным и приятным для всех.

1.4. Типичные виды ошибок PostgreSQL

Давайте посмотрим, как можно случайно сделать ошибку в PostgreSQL.

1.4.1. Ожидания от других баз данных

Многие новички в Postgres чувствуют себя сбитыми с толку и разочарованными, делая первые шаги в работе с этой базой данных. Одна из возможных причин заключается в том, что другие базы данных часто требуют длительного

и сложного мастера установки, где вам приходится принимать важные (и часто необратимые) решения, прежде чем вы сможете приступить к использованию СУБД. В случае с PostgreSQL многие пользователи поражаются тому, что в большинстве случаев, как только вы ее установили, вы можете сразу же начать работать с сервером базы данных. Все просто: подключите клиент базы данных к серверу, и можно начать исследовать возможности. Однако это сопряжено с риском: PostgreSQL устанавливается с некоторыми разумными значениями по умолчанию (*sensible defaults*). Насколько они приемлемы, зависит от того, для чего вы планируете применять базу данных. Конфигурация по умолчанию почти наверняка не подходит для большинства сценариев использования в условиях продакшен-среды.

Еще один возможный источник путаницы — то, что большинство СУБД имеют собственные подходы к выполнению операций, часто отклоняющиеся от стандарта SQL (международного стандарта языка структурированных запросов; в настоящее время ISO/IEC 9075:2023) по историческим, коммерческим или техническим причинам реализации. У PostgreSQL также есть особенности работы, но при этом он общепризнан как наиболее соответствующая стандарту SQL база данных из доступных сейчас. Как следствие, люди иногда ожидают увидеть то, к чему привыкли при работе с другими СУБД. Например, пользователи могут ожидать, что создание пользователя базы данных автоматически создаст схему с тем же именем¹ и неявно назначит этому пользователю право владения всеми объектами в ней. Кроме того, хотя некоторые базы данных автоматически и без предупреждений преобразуют типы данных (например, позволяя вставлять значения 1 и 0 как целые числа в столбец типа `BOOLEAN`), PostgreSQL сообщит об ошибке в такой ситуации. Вполне закономерно, что пользователи удивляются, обнаружив, что ни одна из этих ожидаемых возможностей в PostgreSQL не работает.

1.4.2. Непонимание особенностей PostgreSQL

PostgreSQL — это СУБД общего назначения. Тем не менее это не означает, что она подходит для абсолютно любого мыслимого применения. Автору доводилось наблюдать, как люди пытались использовать ее:

- в качестве встраиваемой (*embedded*) базы данных;
- в качестве распределенной базы данных с собственной репликацией;
- в качестве сервера журналирования (лог-сервера);
- для хранения объемных видеофайлов (например, фильмов);

¹ Стандартное поведение Oracle. — *Примеч. науч. ред.*

- для сценариев работы в памяти (in-memory DB), для которых лучше подходит Redis;
- в качестве графовой базы данных (каюсь, грешен).

Да, в определенной степени она способна выполнять все эти задачи. Однако всегда следует осознавать, какие решения доступны для конкретной ниши, которую вы пытаетесь занять, и взвешивать все «за» и «против», когда вы выбираете между универсальным или узкоспециализированным решением для баз данных.

1.4.3. Непонимание документации

PostgreSQL обладает очень подробной документацией, что закреплено в управлении проектом. Есть четко установленное правило, что ни один патч не принимается без сопроводительной документации: если вы что-то создаете или изменяете, вы должны это задокументировать. Весь документационный материал доступен по адресу <https://postgresql.org/docs/>.

Зачастую эта официальная документация представлена в очень техническом или академическом стиле (что соответствует происхождению проекта PostgreSQL). Для неподготовленных пользователей, привыкших к обучающим руководствам или заметкам с более практическими примерами, документация может быть трудной для восприятия. Как следствие, возможно неверно понять суть той или иной функции или способ ее применения.

Также можно упустить важные примечания или не иметь того контекста, который более опытные пользователи приобрели на практике. Иногда части документации кажутся неполными, потому что необходимые для понимания сведения, термины или детали приходится искать в других разделах документации. Наконец, следует убедиться, что вы читаете документацию, специфичную для вашей версии PostgreSQL, чтобы избежать путаницы и недопонимания.

1.4.4. Использование реликтов из стандарта SQL

Тот факт, что что-то присутствует в официальном стандарте SQL, не означает, что вы обязаны это использовать. Многие пережитки сохранились в стандарте потому, что либо относятся к раннему периоду существования стандарта, либо когда-то казались целесообразными, либо были всеми забыты. Часть из них плохо определены с точки зрения реализации и работают по-разному в каждой СУБД. Некоторые функции SQL были сохранены для обратной совместимости, и причины, по которым их не следует предпочитать или использовать вообще, широко понятны. Хорошим примером этого служит тип `TIME WITH TIME ZONE`, действие которого объясняется в главе 3.

1.4.5. Несоблюдение лучших практик

За десятилетия наблюдений и на основе накопленного отраслью практического опыта сформировались лучшие практики проектирования, разработки, администрирования и сопровождения ИТ-систем. Во многих системах есть общие черты, которые предполагают схожие лучшие практики, которые, в свою очередь, имеет смысл применять к базам данных. Базы данных — это сложные системы со множеством аспектов, связанных с точностью, производительностью и безопасностью, и PostgreSQL не исключение из этого правила.

Несоблюдение лучших практик при проектировании схемы базы данных приложения, планировании сценариев использования или уровня параллелизма, внедрении решений по высокой доступности и аварийному восстановлению, а также при формировании политик безопасности может привести к серьезным проблемам. Это верный способ создать себе трудности в будущем.

1.5. Как устроена эта книга

Как я уже упоминал, большинство ошибок и идей в этой книге (если не все) были собраны в ходе наблюдений за тем, как в реальных условиях работают внедренные решения на базе PostgreSQL и как ведут себя их пользователи. Второй их источник — личный опыт автора и истории, которыми поделились другие специалисты отрасли и участники сообщества.

История, или нарратив, — это ключ для понимания обстановки и контекста теоретической или практической проблемы. Она задает основу для образа мышления того, кто пытается ее решить, и проливает свет на его процесс принятия решений. Отсюда мы можем перейти к общему варианту использования и к конкретной проблеме. Мы увидим то, как готовилась попытка решения и какая цепочка событий привела к ошибке. Затем мы глубже погрузимся в то, почему это ошибка, каковы ее причины и потенциальные последствия. Наконец, мы обсудим правильное решение и то, как его реализовать. Все это обсуждение при необходимости дополняется реальными образцами схемы базы данных и самих данных, а также кодом на SQL или PL/pgSQL, как для ошибочных решений, так и для правильного способа действий. Для каждого случая также показан ожидаемый результат.

СОВЕТ PL/pgSQL — это простой процедурный язык для PostgreSQL, который можно использовать для создания функций, процедур и триггеров. Он добавляет управляющие структуры к языку SQL; может применять все пользовательские типы, функции и операторы; способен выполнять сложные вычисления.

На момент написания книги актуальным релизом является PostgreSQL 17, поэтому именно эту версию мы будем использовать в книге. Разумеется, большая часть книги будет применима и к более старым, но все еще актуальным версиям PostgreSQL. В случаях, где есть различия, они будут указаны.

1.5.1. Ментальные модели

Полезно всегда помнить, что PostgreSQL — это клиент-серверная СУБД с многопроцессной архитектурой. Соответственно, любое клиентское приложение подключается к серверу базы данных (серверу PostgreSQL), работающему на одном узле, для выполнения запросов и получения данных. Это подключение обслуживается одним или несколькими фоновыми процессами PostgreSQL (backend-процессами `postgres`), работающими на стороне сервера. PostgreSQL включает и другие внутренние процессы, которые могут не взаимодействовать напрямую с клиентом. Многопроцессный дизайн позволяет выполнять задачи параллельно и задействовать все доступные CPU, но в некоторых аспектах ведет себя иначе, чем многопоточные системы баз данных. На рис. 1.1 показано, как выглядит эта модель.

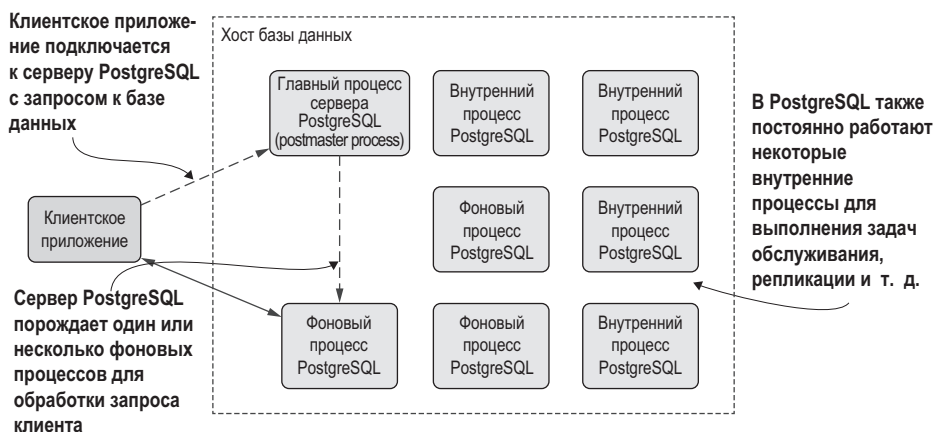


Рис. 1.1. Как работает клиент-серверная архитектура PostgreSQL

Эта книга построена следующим образом. Мы по этапам (по шагам) начинаем с описаний, которые задают контекст для нашего примера использования. Мы изучаем проблему и попытку ее решения, которая приводит к ошибке. Затем мы определяем, почему этот подход неверен, каковы его последствия и возможные обходные пути. Наконец, мы приходим к правильному решению и рассматриваем,

как его реализовать. На рис. 1.2 показан этот процесс, состоящий из этапов разбора проблемы и поиска решений, в тексте они обозначены как «Кейсы».

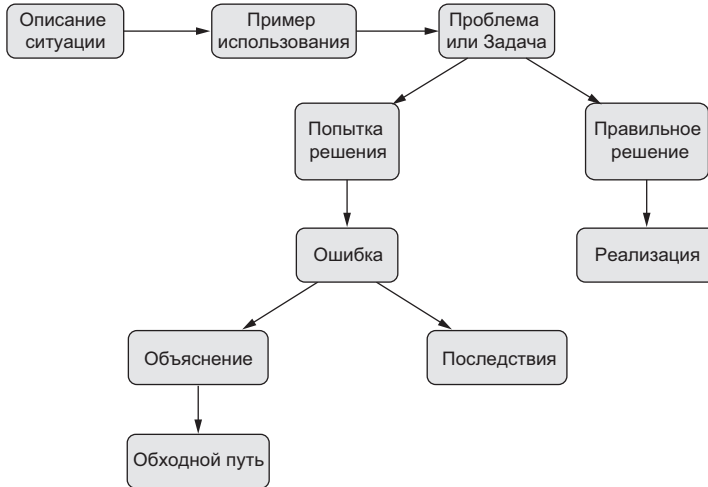


Рис. 1.2. Как устроена эта книга

1.5.2. Пример ошибки

Давайте теперь рассмотрим возможную ошибку, чтобы кратко проиллюстрировать, как эта книга проводит читателя через описание ситуации, проблему и шаги решения. Предположим, у вас есть таблица с сотнями тысяч записей тикетов службы поддержки, например такая (чрезмерно упрощенная) модель:

```
CREATE SCHEMA support
CREATE TABLE tickets (id int, content text, status smallint);
```

ПРИМЕЧАНИЕ Этот пример не особенно впечатляющий, и он применим не только к PostgreSQL, но из него видно, как эта книга подходит к решению проблем в соответствии с ментальной моделью, представленной на рис. 1.2.

Для нашего примера мы предположим, что `status = 10` означает «открыт» (open), а `status = 20` означает «закрыт» (closed). Давайте вставим несколько сотен тысяч строк с закрытыми тикетами:

```
INSERT INTO support.tickets
SELECT id, 'case description text', 20
FROM generate_series(1, 499750) AS id;
```

А теперь вставим несколько сотен строк недавних, все еще открытых тикетов:

```
INSERT INTO support.tickets
SELECT id, 'case description text', 10
FROM generate_series(499751, 500000) AS id;
```

Ради простоты мы предположим, что параллельное выполнение запросов невозможно, поэтому отключим его с помощью:

```
SET max_parallel_workers_per_gather = 0;
```

Мы будем отслеживать продолжительность выполнения запроса, включив тайминг в `psql`:

```
\timing
Timing is on.
```

Теперь предположим, что для приложения поддержки клиентов актуальны только открытые тикеты. Соответственно, вас интересуют только они, и вы хотите их подсчитать. Попробуем следующий подход:

Кейс 1.1. Попытка решить задачу

```
SELECT count(*)
FROM support.tickets
WHERE status = 10;
```

Действительно, такой подсчет открытых тикетов возвращает правильный результат. Но он выполняется медленно:

```
count
-----
      250
(1 row)

Time: 110.036 ms
```

Посмотрим, почему он медленный. Для этого выполним `EXPLAIN`, который покажет нам, как PostgreSQL будет выполнять запрос (*план запроса*):

```
EXPLAIN SELECT count(*)
FROM support.tickets
WHERE status = 10;

              QUERY PLAN
-----
Aggregate(cost=9927.28..9927.29 rows=1 width=8)
-> Seq Scan on tickets (cost=0.00..9927.28 rows=1 width=0)
    Filter: (status = 10)
(3 rows)
```

Здесь говорится о том, что для выполнения агрегации `count()` PostgreSQL планирует использовать последовательное сканирование (Seq Scan) таблицы `tickets` с последующей фильтрацией результатов по условию `status = 10`. Последовательные сканирования (также известные как полные сканирования таблиц) работают медленно. Поэтому вы решаете создать индекс. Ведь индексы ускоряют все, верно?

```
CREATE INDEX ON support.tickets(status);
CREATE INDEX
Time: 732.403 ms
```

Теперь, когда индекс создан, пробуем снова:

```
SELECT count(*)
FROM support.tickets
WHERE status = 10;
count
-----
      250
(1 row)
```

```
Time: 3.715 ms
```

Этот результат значительно лучше. `EXPLAIN` поясняет, почему теперь все работает гораздо быстрее: используется сканирование только по индексу (Index Only Scan):

```
-----
QUERY PLAN
-----
Aggregate (cost=4.44..4.45 rows=1 width=8)
-> Index Only Scan using tickets_status_idx on tickets
    └─(cost=0.42..4.44 rows=1 width=0)
        Index Cond: (status = 10)
(3 rows)
```

Однако этот индекс довольно большой относительно полезных данных.

Кейс 1.2. Почему решение не оптимально?

```
\x
\di+ support.tickets*
List of relations
-[ RECORD 1 ]-----
Schema      | support
Name        | tickets_status_idx
Type        | index
Owner       | frogge
Table       | tickets
Persistence | permanent
Access method | btree
Size        | 3408 kB
Description |
```

Давайте представим, что у нас есть сотни миллионов заявок в истории поддержки клиентов, при этом одновременно открытыми обычно бывают только около 250 последних. Крупные индексы, конечно же, занимают больше места на диске. Но они также работают медленнее, поскольку приходится обрабатывать больше данных, и они замедляют операции записи, потому что их нужно обновлять при каждом INSERT или UPDATE.

В нашем случае нас интересуют сравнительно немногие открытые заявки. Поэтому мы можем сэкономить на размере индекса, используя так называемый *частичный индекс* (*partial index*), и добавить только интересующие нас строки: WHERE status = 10.

Теперь мы удаляем предыдущий индекс и создаем новый частичный индекс.

Кейс 1.3. Правильное решение

```
DROP INDEX support.tickets_status_idx;
CREATE INDEX ON support.tickets(status)
WHERE status = 10;
```

Посмотрите, насколько этот индекс стал меньше! Это логично, поскольку мы индексируем только около 0,05 % от общего числа строк, верно?

```
\di+ support.tickets*
List of relations
-[ RECORD 1 ]-----
Schema      | support
Name        | tickets_status_idx
Type        | index
Owner       | frogge
Table       | tickets
Persistence | permanent
Access method | btree
Size        | 16 kB
Description |
```

Сводка показывает, как теперь выглядит время выполнения после уменьшения размера индекса более чем в 200 раз:

```
SELECT count(*)
FROM support.tickets
WHERE status = 10;
count
-----
    250
(1 row)

Time: 0.762 ms
```


И EXPLAIN теперь показывает следующее:

```

                                QUERY PLAN
-----
Aggregate (cost=4.16..4.17 rows=1 width=8)
  -> Index Only Scan using tickets_status_idx on tickets
      ↳(cost=0.14..4.16 rows=1 width=0)
(2 rows)

```

Операция по-прежнему представляет собой *Index Only Scan*, но с гораздо меньшим размером индекса, поэтому выполнение занимает значительно меньше времени.

Таким образом, мы увидели, что простое создание индекса на столбце технически работает, но это не всегда оптимальное решение, если учитывать такие факторы, как большие объемы данных и требования к производительности.

1.6. Пример базы данных: Frogge Emporium

Каждой книге о базах данных нужен рабочий демопример базы данных (на котором можно тренироваться), так зачем использовать то, что десятилетиями кочует по интернету? Возьмем Frogge Emporium — уважающая себя розничная сеть, которая, конечно же, в своем бизнесе имеет процессы, файлы, записи, архивы и наборы данных. Все это можно представить в виде набора таблиц реляционной БД. Среди таблиц в ее базе данных будут:

- филиалы (*branches*), в том числе:
 - запасы (*stock*);
 - заказчики (*customers*);
 - поставщики (*suppliers*);
- товары (*products*) со многими и многими:
 - заказами (*orders*);
- платежи (*payments*), где будут:
 - типы платежей (*payment_types*).

ПРИМЕЧАНИЕ Скрипт генерации тестовых данных в приложении А создает данные, используя текущую дату и двигаясь на один год назад. В результате сгенерированные вами тестовые данные будут отражать дату запуска скрипта, а значит, они не будут в точности совпадать с выводом, показанным в примерах этой книги. Эта особенность предусмотрена специально, чтобы скрипт оставался динамичным и полезным для тестирования в любое время.

Полную схему базы данных и скрипты для ее наполнения см. в приложении А. Далее все примеры поиска ошибок, проблем и решений приведены для демобазы данных Frogge Emporium.

Итоги

- PostgreSQL — это мощная, бесплатная и соответствующая стандартам база данных, которая с каждым днем меняет индустрию и набирает популярность.
- Важно изучать ошибки Postgres и учиться на них, чтобы экономить время и силы, а также защищать свои данные.
- Повышение осведомленности о потенциальных проблемах приносит пользу всем.
- Понимание того, как работает PostgreSQL и чем она отличается от других СУБД, позволяет избегать ошибок, особенно если вы переходите с другой системы, — не все СУБД устроены одинаково.
- Понятие многопроцессной клиент-серверной архитектуры PostgreSQL важно для дальнейшего изложения в этой книге.
- Следование лучшим практикам и тщательное изучение документации приведет вас к правильному техническому решению вашей проблемы.
- В книге используется повествование на основе практических сценариев, дополненных кодом, чтобы показать, как можно предотвратить ошибку или восстановиться после уже совершенной.

Некорректное использование SQL

В этой главе

- ✓ Какие есть подводные камни SQL
- ✓ Почему важно быть осторожным с результатами запросов
- ✓ Как повысить производительность на сложных запросах
- ✓ Как проверять запросы на корректность

Давайте начнем наше путешествие в мир ошибок PostgreSQL с того, что наиболее хорошо знакомо пользователям реляционных баз данных, — с языка структурированных запросов (SQL, structured query language). В этой главе мы рассмотрим, как некоторые вредные привычки работы с SQL могут привести к низкой скорости выполнения запросов или, что еще хуже, к неверным результатам. PostgreSQL требователен к тому, как написаны команды, и в большинстве случаев придерживается строгой интерпретации стандарта SQL. Поэтому вам нужно внимательно конструировать запросы и всегда проверять, действительно ли вы получаете желаемые результаты.

2.1. Использование *NOT IN* для исключения

Нам часто приходится выражать состояние через отрицательное выражение. Не лица, конечно, если вдруг вы об этом подумали, — речь идет о запросе. То есть

мы указываем в нем не то, что хотим видеть в результатах, а то, чего не хотим, — иными словами, инвертируем желаемое условие.

Терминология: предикат

Предикат (predicate) — это просто условное выражение, которое возвращает булево значение: либо TRUE, либо FALSE. Хорошим примером предиката является содержимое предложения WHERE.

Отрицательные предикаты выражаются с помощью ключевого слова NOT и могут использоваться для инверсии любого SQL-выражения, включая выражения на основе IN.

Кейс 2.1. Примеры запросов с IN/NOT IN и NULL

```
SELECT * FROM (VALUES ('Harper', 'salesperson'),
                     ('Parker', 'driver'),
                     ('Riley', 'electrician'),
                     ('Skyler', 'manager'))
              AS t(first_name, job_title)
WHERE job_title IN ('salesperson', 'driver', 'electrician');
```

first_name	job_title
Harper	salesperson
Parker	driver
Riley	electrician

(3 rows)

```
SELECT * FROM (VALUES ('Harper', 'salesperson'),
                     ('Parker', 'driver'),
                     ('Riley', 'electrician'),
                     ('Skyler', 'manager'))
              AS t(first_name, job_title)
WHERE job_title NOT IN ('manager');
```

first_name	job_title
Harper	salesperson
Parker	driver
Riley	electrician

(3 rows)

Мы видим, что NOT IN может немного сократить время на ввод текста запроса (в части формирования условия WHERE) и сделать запрос проще для чтения, особенно если вам нужно исключить лишь небольшое количество значений.

Теперь предположим, что у сотрудников Frogge Emporium именно такая задача. Они собираются провести акцию, и им нужно найти все адреса электронной почты заказчиков из штатов, где у них нет поставщиков, чтобы отправить им

промотовары с центрального склада. Таблица заказчиков `customer_contact_details` выглядит как табл. 2.1.

Таблица 2.1. Пример строки из `customer_contact_details`

id	email	street_address	city	state	country	phone_no
100	jordan.barber@example.com		Albany	NY	USA	

Таблица поставщиков `suppliers` выглядит как табл. 2.2.

Таблица 2.2. Пример строки из `suppliers`

id	company_name	state	country	phone_no	email
1	Omni Consumer Products	MI	USA		ocp@example.com

Для этого в Frogge Emporium составляют такой запрос:

```
SELECT email
FROM erp.customer_contact_details
WHERE state NOT IN (SELECT state
                    FROM erp.suppliers);
```

Здесь выбираются все адреса электронной почты из `customer_contact_details`, при этом штат (`state`) заказчика не содержится в списке штатов всех поставщиков. Однако при выполнении происходит следующее:

```
email
-----
(0 rows)
```

Почему возвращается ноль результатов, хотя заказчики в штатах, где нет поставщиков, точно есть?

Допустим, сотрудники Frogge Emporium знают, что у них нет поставщиков в Канзасе. Вот SQL-запрос, который проверяет, есть ли в нем хотя бы один заказчик:

```
SELECT email
FROM erp.customer_contact_details
WHERE state = 'KS'
LIMIT 1;
      email
-----
river.smith@example.com
(1 row)
```

Итак, заказчики в Канзасе есть, как минимум один, но он не выводится в запросе списка email-адресов заказчиков, в штатах которых нет поставщиков. Этот результат является следствием того, как PostgreSQL обрабатывает NULL в предикатах. В нашем примере не все поставщики находятся в США, поэтому у нас есть некоторые поставщики со значением NULL в поле `state`. Давайте проверим наличие хотя бы одного поставщика, который не находится в США:

```
SELECT *
FROM erp.suppliers
WHERE state IS NULL;
 id | company_name | state | country | phone_no | email
-----+-----+-----+-----+-----+-----
  2 | Yoyodyne     |      | Japan   |          | yoyodyne@example.com
(1 row)
```

Столбец `state` для этого поставщика имеет значение NULL. Это приводит к тому, что подзапрос `SELECT state FROM erp.suppliers` возвращает по крайней мере одно значение NULL.

Вероятно, самое неожиданное — то, что предикат `state NOT IN (SELECT state FROM erp.suppliers)` никогда не может вернуть TRUE, если есть хотя бы одно значение NULL. Выражение оценивается как *unknown* (неизвестно), если подзапрос возвращает какие-либо значения NULL, что фактически отрицает предикат и приводит к пустому набору результатов.

Почему так происходит? Давайте рассмотрим логику запросов с использованием `NOT IN`, учитывая оба случая — со значениями NULL и без них.

Без значений NULL:

```
SELECT email, state
FROM erp.customer_contact_details
WHERE state NOT IN ('Fake state')
LIMIT 1;
 email | state
-----+-----
jordan.barber@example.com | NY
(1 row)
```

Со значениями NULL:

```
SELECT email, state
FROM erp.customer_contact_details
WHERE state NOT IN ('Fake state', null)
LIMIT 1;
 email | state
-----+-----
(0 rows)
```

В первом запросе условие `WHERE` эквивалентно записи `NOT (state IN ('Fake state'))`, то есть фактически `NOT (FALSE)`, а значит, `TRUE`. Добавление `NULL` (во втором запросе) делает предикат неизвестным, потому что `NOT (state IN ('Fake state', null))` вычисляется как `NOT (NULL)`, что эквивалентно `NULL`, и поэтому не может быть `TRUE`. Мы должны помнить, что `SQL` — это язык запросов, а не язык программирования высокого уровня, такой как `Python`, где эквивалентное выражение `1 not in [2, None]` равно `True`.

2.1.1. Влияние на производительность

Запросы с использованием `NOT IN (SELECT ...)` плохо оптимизируются PostgreSQL. В частности, система не может автоматически преобразовать такие выражения в **anti-join** в плане запроса и выбирает хешированный или обычный подплан (`subplan`). Это приводит к неадекватному потреблению ресурсов, поскольку хешированный подплан работает быстро, но только для небольших выборок, а обычный подплан — очень медленно. Следовательно, такой план выполнения запроса может показать приемлемую производительность в небольших масштабах, но значительно замедлится, если вы превысите пороговое значение размера данных. В документации PostgreSQL по использованию `EXPLAIN` есть дополнительная информация о планах запросов (<https://www.postgresql.org/docs/current/using-explain.html>).

Терминология: anti-join

Допустим, у нас есть таблицы `a` и `b`. Anti-join — это запрос только для тех строк из таблицы `a`, которые не возвращают ни одной строки из коррелированного подзапроса к таблице `b`. Эта операция происходит из реляционной алгебры и обычно выражается в PostgreSQL с помощью предиката `NOT EXISTS` (<https://www.postgresql.org/docs/current/functions-subquery.html#FUNCTIONS-SUBQUERY-EXISTS>).

2.1.2. Альтернатива

Очевидно, что в большинстве случаев `NOT IN (SELECT ...)`, который может неочевидным образом возвращать `NULL`, — это не то, что вам нужно. Вариант запроса с более предсказуемыми результатами — с использованием `NOT EXISTS`:

```
SELECT ccd.email
FROM erp.customer_contact_details ccd
WHERE NOT EXISTS (SELECT FROM erp.suppliers s
                  WHERE ccd.state = s.state)
AND ccd.state IS NOT NULL;
```

```

            email
-----
river.smith@example.com
drew.anderson@example.com
(2 rows)

```

Этот запрос наконец дает правильные результаты. Он также должен выполняться быстрее, поскольку планировщик запросов на этот раз выбирает anti-join. План запроса NOT EXISTS выглядит следующим образом:

```

EXPLAIN SELECT ccd.email
FROM erp.customer_contact_details ccd
WHERE NOT EXISTS (SELECT FROM erp.suppliers s
                  WHERE ccd.state = s.state)
AND ccd.state IS NOT NULL;
                                QUERY PLAN
-----
Hash Anti Join      (cost=19.45..319.48 rows=2 width=25)
  Hash Cond: (ccd.state = s.state)
    -> Seq Scan on customer_contact_details ccd (cost=0.00..300.00 rows=2
        width=28)
        Filter: (state IS NOT NULL)
    -> Hash      (cost=14.20..14.20 rows=420 width=32)
        -> Seq Scan on suppliers s (cost=0.00..14.20 rows=420 width=32)
(6 rows)

```

Следующий альтернативный запрос явно демонстрирует механизм anti-join и позволяет лучше визуализировать, как строки исключаются из-за отсутствия совпадений:

```

SELECT ccd.email
FROM erp.customer_contact_details ccd
LEFT JOIN erp.suppliers s USING (state)
WHERE s.state IS NULL;

```

Мы видим, что есть веские причины не использовать синтаксис NOT IN. Однако если вы уверены, что ваш подзапрос не может возвращать значения NULL, или если вы предоставляете список постоянных значений (как в кейсе 2.1), его использование должно быть безопасным. Отрицательные предикаты могут быть эффективным способом исключения данных, но при этом важно учитывать значения NULL.

2.2. Выбор диапазонов с помощью BETWEEN

BETWEEN — это удобная функция SQL, которая позволяет задавать диапазон значений в относительно краткой форме, как в следующем примере, в котором выбираются заказчики с id от 1 до 100.

Кейс 2.2. Пример запроса с BETWEEN

```
SELECT *
FROM erp.customers
WHERE id BETWEEN 1 AND 100;
```

id	first_name	middle_name	last_name	marketing_consent
1	River	J	Smith	t
2	Drew		Anderson	t
...				
100	Jordan	A	Barber	t

(100 rows)

Сотрудники компании Frogge Emporium решили, что им нужен запрос, который запускается каждое утро для расчета общей суммы платежей, полученных компанией за предыдущий день. Они написали его следующим образом:

```
WITH t(today) AS (SELECT CURRENT_DATE::timestampz)
SELECT sum(amount)
FROM erp.payments, t
WHERE tstamp BETWEEN t.today - INTERVAL '1 day' AND t.today;
```

Этот запрос извлекает текущую дату и преобразует ее в метку времени (timestamp): так фиксируется начало текущего дня в полночь. Затем мы суммируем все суммы платежей, которые были получены между полночью вчера и полночью сегодня. Выполнение запроса дает следующее:

```
sum
-----
5179739.95
(1 row)
```

Все ли тут в порядке? Не совсем. Мы забыли, что оператор **BETWEEN** включает в себя оба конца диапазона — как верхнюю, так и нижнюю границы. Если вы помните возвращаемые результаты из кейса 2.2, мы получили, как легко заметить, и 1, и 100; это также называется *закрытым интервалом (closed interval)*. Чем это чревато? Давайте выберем последний платеж за вчерашний день:

```
WITH t(today) AS (SELECT CURRENT_DATE::timestampz)
SELECT max(tstamp)
FROM erp.payments, t
WHERE tstamp BETWEEN t.today - INTERVAL '1 day' AND t.today;
```

max
2024-05-27 00:00:00+01

(1 row)

А теперь давайте выберем первый платеж за сегодняшний день:

```
WITH t(today) AS (SELECT CURRENT_DATE::timestampz)
SELECT min(tstamp)
FROM erp.payments, t
WHERE tstamp BETWEEN t.today AND t.today + interval '1 day';
      min
-----
2024-05-27 00:00:00+01
(1 row)
```

Заметили, да? Это одна и та же метка времени. Таким образом, если платежи происходят ровно в полночь, вы посчитаете их дважды — и это очень плохо, если речь идет о деньгах. По сути, этот запрос эквивалентен следующему:

```
WITH t(today) AS (SELECT CURRENT_DATE::timestampz)
SELECT sum(amount)
FROM erp.payments, t
WHERE tstamp >= t.today - INTERVAL '1 day'
AND tstamp <= t.today;
```

Всякий раз, когда мы хотим выбрать строки для конкретных последовательно расположенных диапазонов, нам нужно убедиться, что мы не включаем пересекающиеся данные на их концах. Более безопасный способ написать этот запрос — исключить верхнюю границу с помощью явного определения диапазона, как в следующем примере (обратите внимание на отсутствие знака равенства в выражении `AND tstamp < t.today`):

```
WITH t(today) AS (SELECT CURRENT_DATE::timestampz)
SELECT sum(amount), count(amount)
FROM erp.payments, t
WHERE tstamp >= t.today - INTERVAL '1 day'
AND tstamp < t.today;
      sum      | count
-----+-----
5179680.00 | 86400
(1 row)
```

Давайте проверим разницу, изучив общие суммы платежей и число строк:

```
WITH t(today) AS (SELECT CURRENT_DATE::timestampz)
SELECT sum(amount), count(amount)
FROM erp.payments, t
WHERE tstamp BETWEEN t.today - INTERVAL '1 day' AND t.today;
      sum      | count
-----+-----
5179739.95 | 86401
(1 row)
```

Возможно, при таком подходе вам потребуется набирать чуть больше текста, но если вы явно определите диапазоны, то запрос будет выглядеть чище и легче и воспринимать его с первого взгляда станет проще. Чем сложнее становятся

ваши запросы, тем больше вы будете ценить ясность и понятность. Поэтому помните, что BETWEEN — это сокращение для «больше или равно И меньше или равно» (рис. 2.1).

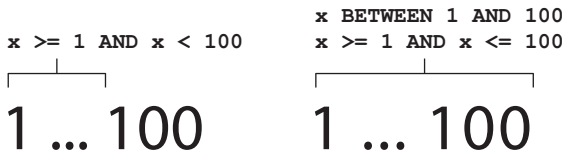


Рис. 2.1. BETWEEN означает «включительно»

2.3. Отказ от использования CTE

Обобщенные табличные выражения (CTE, Common Table Expressions), или синтаксис WITH, — это полезный «синтаксический сахар», который помогает привести запрос в более аккуратный вид. Если у вас есть сложные запросы, CTE могут сделать их более читабельными, разбивая SQL на меньшие, удобные части. CTE, в отличие от подзапросов, могут избавить вас от некоторого повторения, поскольку вы можете ссылаться на них несколько раз в пределах одного и того же запроса.

Давайте рассмотрим пример сложного и не очень читабельного запроса. Frogge Emporium необходимо составить список с адресами электронной почты заказчиков, которых уже уведомляли о неоплаченном счете за приобретенные услуги (то есть не товары). Изначально запрос выглядит следующим образом.

Кейс 2.3. Пример запроса без CTE

```
SELECT DISTINCT email
FROM erp.customer_contact_details ccd
JOIN erp.invoices i ON i.customer = ccd.id
JOIN erp.order_groups og ON i.order_group = og.id
JOIN erp.sent_emails se ON se.invoice = i.id
JOIN erp.orders o ON o.order_group = og.id
WHERE ccd.id IN (
    SELECT customer
    FROM erp.invoices
    WHERE paid = false
)
AND i.order_group IN (
    SELECT order_group
    FROM erp.invoices
    WHERE paid = false
)
AND se.email_type = 'Invoice reminder'
AND o.item IS NULL;
```

52 Глава 2. Некорректное использование SQL

Это вполне рабочий запрос, но он не очень удобен для чтения из-за своей запутанной структуры с множественными соединениями (JOIN), фильтрами и подзапросами. Запустив его с EXPLAIN ANALYZE, чтобы посмотреть, насколько он эффективен, мы получаем длинный вывод (см. рис. 2.2).

```
QUERY PLAN
-----
Unique (cost=17472.51..17472.66 rows=1 width=25) (actual time=42.954..49.193 rows=8 loops=1)
-> Nested Loop (cost=17472.51..17472.66 rows=1 width=25) (actual time=42.953..49.190 rows=8 loops=1)
-> Gather Merge (cost=17472.09..17472.21 rows=1 width=49) (actual time=42.925..49.168 rows=8 loops=1)
  Workers Planned: 1
  Workers Launched: 1
  -> Sort (cost=16472.00..16472.09 rows=1 width=49) (actual time=40.868..40.872 rows=4 loops=2)
    Sort Key: ccd.email
    Sort Method: quicksort. Memory: 25kB
    Worker 0: Sort Method: quicksort. Memory: 25kB
    -> Parallel Hash Semi Join (cost=12648.82..16472.07 rows=1 width=49) (actual time=40.756..40.864 rows=4 loops=2)
      Hash Cond: (i.customer = invoices.customer)
      -> Nested Loop (cost=8576.55..12599.79 rows=1 width=65) (actual time=32.546..32.653 rows=4 loops=2)
        -> Parallel Hash Join (cost=8576.26..12599.49 rows=1 width=32) (actual time=32.527..32.625 rows=4 loops=2)
          Hash Cond: (o.order_group = i.order_group)
          -> Parallel Seq Scan on orders o (cost=0.00..3820.59 rows=701 width=8) (actual time=6.956..7.012 rows=675 loops=2)
            Filter: (item IS NULL)
            Rows Removed by Filter: 124325
          -> Parallel Hash (cost=8576.21..8576.21 rows=4 width=24) (actual time=25.525..25.527 rows=675 loops=2)
            Buckets: 2048 (originally 1024) Batches: 1 (originally 1) Memory Usage: 152kB
            -> Hash Join (cost=4116.83..8576.21 rows=4 width=24) (actual time=25.018..25.299 rows=675 loops=2)
              Hash Cond: (i.id = oi.invoice)
              -> Parallel Hash Semi Join (cost=4072.28..8529.52 rows=775 width=32) (actual time=24.725..24.872 rows=675 loops=2)
                Hash Cond: (i.order_group = invoices.i_order_group)
                -> Parallel Seq Scan on invoices i (cost=0.00..4062.59 rows=14709 width=24) (actual time=6.985..6.724 rows=12980 loops=2)
                  Filter: (type = 'Invoice'::text)
                  Buckets: 2048 Batches: 1 Memory Usage: 80kB
                  -> Parallel Hash (cost=4062.59..4062.59 rows=775 width=8) (actual time=7.190..7.190 rows=675 loops=2)
                    Buckets: 2048 Batches: 1 Memory Usage: 80kB
                    -> Parallel Seq Scan on invoices invoices_1 (cost=0.00..4062.59 rows=775 width=8) (actual time=6.985..7.055 rows=675 loops=2)
                      Filter: (NOT paid)
                      Rows Removed by Filter: 124325
                -> Hash (cost=25.88..26.88 rows=1550 width=8) (actual time=0.201..0.201 rows=1350 loops=2)
                  Buckets: 2048 Batches: 1 Memory Usage: 69kB
                  -> Seq Scan on sent_emails se (cost=0.00..26.88 rows=1550 width=8) (actual time=0.008..0.147 rows=1550 loops=2)
                    Filter: (type = 'Invoice'::text)
                    Buckets: 2048 Batches: 1 Memory Usage: 80kB
                    -> Parallel Seq Scan on invoices (cost=0.00..4062.59 rows=775 width=8) (actual time=7.729..7.795 rows=675 loops=2)
                      Filter: (NOT paid)
                      Rows Removed by Filter: 124325
              -> Index Only Scan using order_groups_key on order_groups og (cost=0.42..0.45 rows=1 width=8) (actual time=0.002..0.002 rows=1 loops=8)
                Index Cond: (id = i.order_group)
                Heap Fetches: 8
      -> Parallel Hash (cost=4062.59..4062.59 rows=775 width=8) (actual time=7.940..7.941 rows=675 loops=2)
        Buckets: 2048 Batches: 1 Memory Usage: 80kB
        -> Parallel Seq Scan on invoices (cost=0.00..4062.59 rows=775 width=8) (actual time=7.729..7.795 rows=675 loops=2)
          Filter: (NOT paid)
          Rows Removed by Filter: 124325
        -> Index Only Scan using order_groups_key on order_groups og (cost=0.42..0.45 rows=1 width=8) (actual time=0.002..0.002 rows=1 loops=8)
          Index Cond: (id = i.order_group)
          Heap Fetches: 8
Planning Time: 2.229 ms
Execution Time: 49.244 ms
(45 rows)
```

Рис. 2.2. План запроса без CTE

Затраты времени на планирование и выполнение не очень хорошие: такова природа сложных запросов. Они сложнее не только для нашего понимания, но и для корректной работы оптимизатора запросов PostgreSQL.

Давайте перепишем запрос, чтобы он возвращал тот же результат, но на этот раз мы используем CTE и увидим, чем они хороши. Здесь нам нужен CTE `unp` для получения неоплаченных счетов и CTE `ni` для получения заказов на услуги («без товаров»).

Кейс 2.4. Пример запроса с CTE

```
EXPLAIN ANALYZE WITH
unp AS (
  SELECT id, customer c, order_group AS og
  FROM erp.invoices
  WHERE paid = false
),
ni AS (
  SELECT og.id
  FROM erp.order_groups og
```

```

JOIN erp.orders o ON o.order_group = og.id
WHERE o.item IS NULL
)
SELECT DISTINCT email
FROM erp.customer_contact_details ccd
JOIN unp ON unp.c = ccd.id
JOIN ni ON ni.id = unp.og
JOIN erp.sent_emails se ON se.invoice = unp.id
AND se.email_type = 'Invoice reminder';

```

Надеюсь, из нового запроса стало понятнее, что таблица `invoices` предоставляет данные о заказчиках и группах заказов только для неоплаченных счетов, в то время как таблица `order_groups` изолирует заказы на услуги, выбирая строки с `NULL`-значениями в столбце `item`. Затем мы соединяем наши CTE с таблицей `sent_emails` и фильтруем по письмам-напоминаниям о счетах. `EXPLAIN ANALYZE` нового запроса дает план, показанный на рис. 2.3. Он упростился, а планирование и выполнение запроса ускорились.

```

----- QUERY PLAN -----
Unique  (cost=8933.91..8936.10 rows=1 width=25) (actual time=18.857..25.156 rows=8 loops=1)
->  Nested Loop  (cost=8933.91..8936.10 rows=1 width=25) (actual time=18.836..25.133 rows=8 loops=1)
    Join Filter: (og.id = o.order_group)
    ->  Gather Merge  (cost=8933.48..8933.60 rows=1 width=41) (actual time=18.829..25.111 rows=8 loops=1)
        Workers Launched: 1
        ->  Sort  (cost=7933.48..7933.48 rows=1 width=41) (actual time=16.180..16.502 rows=4 loops=2)
            Sort Key: ccd.email
            Sort Method: quicksort  Memory: 25kB
            ->  Nested Loop  (cost=4199.62..7933.47 rows=1 width=41) (actual time=15.967..16.878 rows=4 loops=2)
                ->  Parallel Hash Join  (cost=4199.53..7932.56 rows=1 width=24) (actual time=15.945..16.851 rows=4 loops=2)
                    Hash Cond: (o.order_group = invoices.order_group)
                    ->  Parallel Seq Scan on orders o  (cost=0.00..5820.59 rows=781 width=8) (actual time=7.729..7.787 rows=675 loops=2)
                        Filter: (item IS NULL)
                        Rows Removed by Filter: 124525
                    ->  Parallel Hash  (cost=4109.28..4109.28 rows=4 width=16) (actual time=7.960..7.961 rows=675 loops=2)
                        Buckets: 1024  Batches: 1  Memory Usage: 184kB
                        ->  Hash Join  (cost=45.75..4109.28 rows=4 width=16) (actual time=7.655..7.813 rows=675 loops=2)
                            Hash Cond: (invoices.id = se.invoice)
                            ->  Parallel Seq Scan on invoices  (cost=0.00..4862.59 rows=775 width=24) (actual time=7.566..7.428 rows=675 loops=2)
                                Filter: (NOT paid)
                                Rows Removed by Filter: 124525
                            ->  Hash  (cost=26.88..26.88 rows=1550 width=8) (actual time=0.281..0.281 rows=1350 loops=2)
                                Buckets: 2048  Batches: 1  Memory Usage: 69kB
                                ->  Seq Scan on sent_emails se  (cost=0.00..26.88 rows=1550 width=8) (actual time=0.007..0.146 rows=1350 loops=2)
                                    Filter: (type = 'Invoice reminder'::text)
                                    Heap Fetches: 0
                            ->  Index Scan using customer_contact_details_pkey on customer_contact_details ccd  (cost=0.29..0.90 rows=1 width=51) (actual time=0.006..0.006 rows=1 loops=8)
                                Index Cond: (id = invoices.customer)
                ->  Index Only Scan using order_groups_pkey on order_groups og  (cost=0.42..2.48 rows=1 width=8) (actual time=0.682..0.682 rows=1 loops=8)
                    Index Cond: (id = invoices.order_group)
                    Heap Fetches: 0
Planning Time: 0.475 ms
Execution Time: 25.168 ms
134 rows

```

Рис. 2.3. План запроса с CTE

Терминология: селективность

Селективность — это мера того, сколько строк мы ожидаем получить по нашему запросу для конкретного предиката. Таким образом, низкая селективность означала бы относительно небольшое число строк, которые нужно просканировать и отфильтровать. В этом контексте высокая селективность, скорее всего, имеет смысл, противоположный бытовому значению выражения «высокая избирательность».

Здесь произошло следующее: оптимизатор определил, что, переупорядочив операции, можно быстрее получить эти данные. Применяя фильтры с низкой

селективностью на раннем этапе, мы отсекаем большую часть ненужных для запроса данных.

Тот факт, что мы теперь разместили CTE первыми, не означает, что они будут выполнены первыми. PostgreSQL эффективно *встроил* CTE или объединил их в общий план, чтобы найти оптимальный порядок операций на основе собранной статистики содержимого таблиц. Кроме того, мы заменяем операции IN более оптимизированными JOIN.

ПРИМЕЧАНИЕ Еще одно преимущество CTE заключается в том, что они вычисляются только один раз, независимо от того, как часто на них ссылаются в основном запросе или других CTE.

Иногда, когда мы хорошо знаем, что содержат наши таблицы и какова селективность каждого (под)запроса, мы можем создавать целевые CTE. Например, если мы знаем, что действительно ресурсоемкий запрос или функция возвращают несколько элементов, которые многократно переиспользуются и важны для определения селективности остальной части запроса, мы можем указать, что CTE должен быть материализован (**MATERIALIZED**), чтобы принудительно выполнить его вычисление на раннем этапе. Наконец, CTE обычно проще запускать автономно, чем коррелированный подзапрос, — так удобнее оптимизировать запросы (в том числе рекурсивные SQL-запросы) и устранять неполадки.

ПРИМЕЧАНИЕ Не выбирайте для своих CTE имена, которые конфликтуют с именами существующих объектов, таких как таблицы или представления, потому что внутри запроса имя CTE переопределяет имя объекта и будет использоваться вместо него. Это может привести к неверным результатам и путанице. Вам также следует избегать совпадений с именами SQL-функций и т. п.

2.4. Использование идентификаторов в верхнем регистре

По умолчанию PostgreSQL игнорирует заглавные буквы и всегда преобразует каждый введенный вами идентификатор в нижний регистр. Поэтому если вы пришли из среды определенной СУБД, где принято писать **IN_UPPERCASE**, вам может показаться логичным перенести эту привычку в PostgreSQL.

Однако верхний регистр может в конечном счете вызвать проблемы и сломать код. PostgreSQL сохраняет заглавные буквы только тогда, когда вы заключаете идентификатор в двойные кавычки. Такие идентификаторы называются *отделенными (delimited identifiers)*, и они чувствительны к регистру. А непоследовательность в использовании кавычек может привести к ошибкам. Так может легко получиться, если вы создадите запросы вручную и забудете заключить

идентификаторы в двойные кавычки, а ORM¹, который вы используете, сделает это сам.

Давайте рассмотрим пример. Предположим, администратор СУБД создает некоторые таблицы:

```
SET search_path = erp, "$user", public;
CREATE TABLE Customers (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    first_name text,
    middle_name text,
    last_name text,
    marketing_consent boolean DEFAULT false
);

CREATE TABLE "Invoices" (
    "Invoice ID" bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    "Amount" numeric NOT NULL,
    "Customer" bigint NOT NULL REFERENCES Customers(id),
    "Paid" boolean NOT NULL DEFAULT false
);
```

В результате создаются две таблицы: одна называется `customers` (без учета регистра), а другая — `"Invoices"` (с учетом регистра). Если мы попытаемся использовать таблицу `customers`, произойдет следующее:

```
TABLE Customers;
id | first_name | middle_name | last_name | marketing_consent
-----+-----+-----+-----+-----
(0 rows)
```

Здесь все хорошо. В действительности имя таблицы, как и любой другой идентификатор в PostgreSQL, было преобразовано в нижний регистр:

```
\d customers
```

Table "erp.customers"				
Column	Type	Collation	Nullable	Default
id	bigint		not null	generated always as →identity
first_name	text			
middle_name	text			
last_name	text			
marketing_consent	boolean			false

```
Indexes:
    "customers_pkey" PRIMARY KEY, btree (id)
```

¹ ORM (Object-Relational Mapping) в контексте PostgreSQL — это обычно библиотека или фреймворк, которые позволяют работать с таблицами и строками PostgreSQL как с объектами вашего языка программирования. — *Примеч. науч. ред.*

Теперь посмотрим на другую таблицу:

```
TABLE Invoices;
ERROR:  relation "invoices" does not exist
LINE 1: TABLE Invoices;
          ^
```

Здесь уже есть проблема.

ПРИМЕЧАНИЕ SQL-команду `TABLE table_name` можно использовать как сокращение для `SELECT * FROM table_name`.

То же самое относится и к именам столбцов:

```
CREATE TABLE "Invoices" (
    "Invoice ID" bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    "Amount" numeric NOT NULL,
    "Customer" bigint NOT NULL REFERENCES Customers(id),
    "Paid" boolean NOT NULL DEFAULT false
);
```

Здесь мы создали все имена столбцов в смешанном регистре, и поскольку мы заключили их в кавычки, они стали чувствительными к регистру:

```
SELECT count(Customer)
FROM "Invoices";
ERROR:  column "customer" does not exist
LINE 1: SELECT count(Customer)
                  ^
HINT:  Perhaps you meant to reference the column "Invoices.Customer".
```

Таким образом, забыв использовать идентификатор столбца в двойных кавычках, мы получаем ошибку. Если вы создали идентификатор с заглавными буквами в кавычках, вам придется всегда писать его точно так же, включая регистр, иначе запрос не сработает.

```
SELECT count("Customer")
FROM "Invoices";
count
-----
      1
(1 row)
```

Лучше всего не заключать идентификаторы в кавычки. Однако если вам нужны идентификаторы в кавычках, чтобы соответствовать ожиданиям коллег, — например, если они хотят видеть в отчетах столбцы `"Entry Date"` или `"Reconciliation Amount"`, — это легко сделать через псевдонимы столбцов в ваших отчетных запросах. Вам не потребуется помещать в базу данных замысловатые имена из их отчетов, и они ничего не заметят:


```
SELECT reconc_amt AS "Reconciliation Amount", entry_date AS "Entry Date"
FROM financials
WHERE entry_date = '2023-02-02';
```

Вот что получается:

```
Reconciliation Amount | Entry Date
-----+-----
                235.11 | 2023-02-02
(1 row)
```

2.5. Деление целых чисел

Допустим, мы хотим выяснить, сколько заказов мы получили на физические товарные позиции по сравнению с заказами на услуги. Поэтому мы пишем следующий запрос к таблице `orders`:

```
WITH
i AS (
    SELECT count(*) c
    FROM erp.orders
    WHERE item IS NOT NULL
),
s AS (
    SELECT count(*) c
    FROM erp.orders
    WHERE service IS NOT NULL
)
SELECT
    i.c AS "Item orders",
    s.c AS "Service orders",
    i.c + s.c AS "Total",
    i.c / (i.c + s.c) * 100 AS "Item order %",
    s.c / (i.c + s.c) * 100 AS "Service order %"
FROM i, s;
```

Результат:

```
Item orders | Service orders | Total | Item order % | Service order %
-----+-----+-----+-----+-----
        248650 |          1350 | 250000 |             0 |             0
(1 row)
```

Погодите, тут явно ошибка. Посмотрите на последние два столбца: вычисления не сходятся. У нас точно есть заказы, и в последних двух столбцах должны быть %. По факту при вычислении % мы разделили два целых числа — такая операция при вычислении вручную дала бы дробь, но в SQL это работает иначе.

Документация PostgreSQL сообщает нам следующее об операторе деления `/`: «Для целочисленных типов (`int` и `bigint`) деление усекает результат в сторону

нуля» (<https://www.postgresql.org/docs/current/functions-math.html#FUNCTIONS-MATH-OP-TABLE>). При таком подходе вы будете получать весьма неточные числовые результаты там, где меньше всего ожидаете.

В этом случае необходимо привести числа к типу, к которому можно применять операцию деления без таких последствий, например к `float` или к числу с двойной точностью (`double precision`):

```
SELECT
    10 / 4 AS "int division",
    pg_typeof(10 / 4) AS "-> output type",
    10::double precision / 4 AS "double division",
    pg_typeof(10::double precision / 4) AS "-> output type";
```

int division	-> output type	double division	-> output type
2	integer	2.5	double precision

(1 row)

Вы можете возразить: мы же преобразовали только одно из целых чисел в `double precision`, и это сработало. По умолчанию при таком сценарии Postgres автоматически приводит другое целое число к совместимому типу, чтобы разделить его на число с двойной точностью.

ПРИМЕЧАНИЕ В PostgreSQL можно использовать сокращенный синтаксис приведения типов: вместо стандартного `CAST(expr AS type)` можно писать `expr::type`. Функция `pg_typeof()` показывает, какой тип данных имеет переданное ей значение.

Вооружившись этими знаниями, мы теперь пишем наш запрос так:

```
WITH
i AS (
    SELECT count(*)::float c
    FROM erp.orders
    WHERE item IS NOT NULL
),
s AS (
    SELECT count(*)::float c
    FROM erp.orders
    WHERE service IS NOT NULL
)
SELECT
    i.c AS "Item orders",
    s.c AS "Service orders",
    i.c + s.c AS "Total",
    round((i.c / (i.c + s.c) * 100)::numeric, 1) AS "Item order %",
    round((s.c / (i.c + s.c) * 100)::numeric, 1) AS "Service order %"
FROM i, s;
```

Мы получаем правильный результат с округлением и приведением типа:

Item orders	Service orders	Total	Item order %	Service order %
248650	1350	250000	99.5	0.5

(1 row)

С этим частично связана и другая потенциально болезненная ошибка. Она возникает, когда вы допускаете деление на ноль, — операция, которую математика не одобряет и которую точно не любят компьютеры. В коде приложения обычно нужно гарантировать, чтобы на вход в базу данных не приходило такое значение, но если это все же случится, вот что будет:

```
WITH zero AS (SELECT count(*) FROM erp.customers WHERE 1=2)
SELECT 10 / zero.count
FROM zero;
ERROR:
division by zero
```

ПРИМЕЧАНИЕ Здесь WHERE 1=2 — это невозможное условие, которое не может быть выполнено, поэтому подсчет count() подзапроса нулевых строк возвращает 0.

Каково решение? Можно сделать результат неопределенным (другими словами, NULL), если вы столкнетесь с нулем, как в нашем первом столбце здесь, или заменить 0 на другое значение, заведомо безопасное, например 1, как во втором столбце:

```
WITH zero AS (SELECT count(*) FROM erp.customers WHERE 1=2)
SELECT
    10 / NULLIF(zero.count, 0) AS nullified,
    10 / COALESCE(NULLIF(zero.count, 0), 1) AS replaced
FROM zero;
nullified | replaced
-----+-----
          |      10
(1 row)
```

ПРИМЕЧАНИЕ NULLIF возвращает NULL, если первый аргумент равен второму, а COALESCE возвращает первый не NULL аргумент.

2.6. Подсчет значений NULL

При попытке подсчитать значения или строки вам нужно точно знать, допускает ли значение NULL столбец, который вы используете для агрегатной функции count(). Компании Frogge Emporium нужно выяснить число заказов, размещенных с начала года. Ее запрос выглядит так:

```
SELECT count(item)
FROM erp.orders
WHERE placed_at > date_trunc('year', CURRENT_DATE);
```

Когда сотрудники Frogge Emporium запускают его, они получают следующий результат:

```
count
-----
198908
(1 row)
```

Однако некоторые заказы были сделаны не на товары (`item`), а на услуги (`service`). К этим заказам не прикреплен товар, поэтому их столбец `item` содержит `NULL`:

```
SELECT id, status, placed_at, item, service
FROM erp.orders
WHERE service IS NOT NULL
LIMIT 1;
 id | status | placed_at | item | service
-----+-----+-----+-----+-----
168521 | Placed | 2024-05-26 08:55:21+01 |      | 21
(1 row)
```

Проверим, сколько всего заказов действительно есть, подсчитав строки вместо товаров:

```
SELECT count(*)
FROM erp.orders
WHERE placed_at > date_trunc('year', CURRENT_DATE);
count
-----
200000
(1 row)
```

Такой результат получается потому, что функция `count()` игнорировала ранее значения `NULL` (невозможно посчитать то, чего нет) в расчетах по столбцу `item` (допускающему `NULL`). В этом случае (когда в некоторых столбцах могут быть значения `NULL`) нужно считать общее число строк, чтобы получить правильный результат.

У такого способа также есть интересный побочный эффект: можно при необходимости подсчитать, какой процент ваших заказов приходится на услуги (зная, что заказы на товары будут иметь `NULL` в столбце `service`), например, так:

```
SELECT round(count(service)::numeric
/ count(*)::numeric * 100, 1) AS "Service orders %"
FROM erp.orders
WHERE placed_at > date_trunc('year', CURRENT_DATE);
Service orders %
-----
0.5
(1 row)
```

2.7. Запросы к индексированным столбцам через выражения

Когда на столбце есть индекс, определенные типы запросов выполняются чрезвычайно быстро. Однако если обращаться к тому же индексированному столбцу через выражения, например, пропуская его через функцию или приводя его к другому типу данных (не соответствующему типу в индексе), то PostgreSQL может вообще не задействовать индекс.

Это сильно снижает производительность. По сути, вы платите за наличие индекса (например, нужно больше дискового пространства, записи вносятся в таблицу медленнее и т. д.), не получая ничего взамен.

Рассмотрим пример с таблицей `payments`, где содержатся данные о платежах Frogge Emporium с временными метками. Если мы выберем платежи по определенной временной метке, то получим практически мгновенный ответ с минимальными затратами ресурсов:

```
EXPLAIN (ANALYZE, FORMAT YAML)
SELECT * FROM erp.payments
WHERE tstamp = '2023-10-18 03:40:34.000';
                                QUERY PLAN
```

```
-----+
- Plan:                                     +
  Node Type: "Index Scan"                  +
  Parallel Aware: false                    +
  Async Capable: false                     +
  Scan Direction: "Forward"                +
  Index Name: "payments_tstamp_idx"        +
  Relation Name: "payments"                +
  Alias: "payments"                        +
  Startup Cost: 0.42                       +
  Total Cost: 8.46                         +
  Plan Rows: 2                             +
  Plan Width: 31                           +
  Actual Startup Time: 0.010                +
  Actual Total Time: 0.011                 +
  Actual Rows: 0                           +
  Actual Loops: 1                          +
  Index Cond: "(tstamp = '2023-10-18 03:40:34+01':::timestamp with time zone)" +
  Rows Removed by Index Recheck: 0         +
  Planning Time: 0.051                     +
  Triggers:                                +
  Execution Time: 0.022                     +
(1 row)
```

Этот план задействовал в качестве метода доступа сканирование по индексу, которое выполняется мгновенно.

62 Глава 2. Некорректное использование SQL

Теперь предположим, что сотрудники Frogge хотят изменить запрос так, чтобы им не приходилось указывать временную метку с точностью до миллисекунд. Они решают использовать функцию `date_trunc()` и снизить точность до уровня секунд. Такой запрос вернет данные о платежах за конкретную секунду:

```
EXPLAIN (ANALYZE, FORMAT YAML)
SELECT * FROM erp.payments
WHERE date_trunc('s', tstamp) = '2023-10-18 03:40:34';
QUERY PLAN
```

```
- Plan:                                                                 +
  Node Type: "Seq Scan"                                             +
  Parallel Aware: false                                           +
  Async Capable: false                                           +
  Relation Name: "payments"                                       +
  Alias: "payments"                                              +
  Startup Cost: 0.00                                              +
  Total Cost: 5589.00                                             +
  Plan Rows: 1250                                                 +
  Plan Width: 31                                                  +
  Actual Startup Time: 1.900                                       +
  Actual Total Time: 35.880                                       +
  Actual Rows: 1                                                  +
  Actual Loops: 1                                                 +
  Filter: "(date_trunc('s'::text, tstamp) = '2023-10-18 03:40:34+01' +
➡::timestamp with time zone)"                                     +
  Rows Removed by Filter: 249999                                  +
  Planning Time: 0.038                                           +
  Triggers:                                                         +
  Execution Time: 035.892                                         +
(1 row)
```

Что здесь произошло? Мы видим, что этот запрос не использовал индекс и вместо этого выполнил полное сканирование таблицы, — и это значительно медленнее.

Так вышло потому, что индекс создан для прямого поиска по сохраненным значениям временных меток. Если изменить предикат, то тип значения больше не будет соответствовать типу в индексе, и поэтому индекс не может использоваться.

Один из возможных обходных путей — приводить тип не столбца, а значения, с которым его сравнивают, например, так:

```
EXPLAIN (ANALYZE, FORMAT YAML)
SELECT * FROM erp.payments
WHERE tstamp >= '2023-10-18 03:40:34'::timestampz
AND tstamp < '2023-10-18 03:40:34'::timestampz + INTERVAL '1 s';
QUERY PLAN
```

```

- Plan:
  Node Type: "Index Scan"
  Parallel Aware: false
  Async Capable: false
  Scan Direction: "Forward"
  Index Name: "payments_tstamp_idx"
  Relation Name: "payments"
  Alias: "payments"
  Startup Cost: 0.42
  Total Cost: 8.44
  Plan Rows: 1
  Plan Width: 31
  Actual Startup Time: 0.014
  Actual Total Time: 0.015
  Actual Rows: 1
  Actual Loops: 1
  Index Cond: "((tstamp >= '2023-10-18 03:40:34+01'::timestamp with time
  ➡zone) AND (tstamp < ('2023-10-18 03:40:34+01'::timestamp with time zone
  ➡+ '00:00:01'::interval)))"
  Rows Removed by Index Recheck: 0
  Planning Time: 0.058
  Triggers:
  Execution Time: 0.023
(1 row)

```

Здесь мы видим, что запрос использует индекс и снова обеспечивает максимальную производительность, поскольку функция теперь сравнивает не столбец, а значение.

Кроме того, стоит помнить еще об одной замечательной функции PostgreSQL: индексе по выражению. Если мы знаем, что всегда будем запрашивать этот столбец через `date_trunc`, например, чтобы найти все платежи за определенную минуту, мы можем использовать индекс по выражению, чтобы включить функцию и ускорить эти запросы:

```
CREATE INDEX ON erp.payments(date_trunc('m', tstamp AT TIME ZONE 'UTC+1'));
```

Такой индекс затем можно запрашивать следующим образом:

```
SELECT * FROM erp.payments
WHERE date_trunc('m', tstamp AT TIME ZONE 'UTC+1')
      = '2023-10-18 03:40' AT TIME ZONE 'UTC+1';
```

Однако будьте осторожны: такой индекс может быть неэффективным и неиспользуемым — он не будет применяться к запросам, которые не используют те же функции и приведения типов, с которыми индекс был создан!

Когда в запросе к индексированному столбцу вызывается функция или меняется его тип, базе данных приходится вычислять функцию для каждой строки отдельно. При таком подходе индекс не используется, поскольку преобразованные

данные не соответствуют проиндексированным данным. Следовательно, базе данных приходится выполнять дополнительную работу для обработки большего числа строк.

2.8. UPSERT с NULL в составном уникальном ключе

UPSERT (сокращение от UPDATE + INSERT) — это общепринятое название операции записи данных в таблицу, когда нужно гарантировать: если строка уже существует, она должна быть обновлена, а если не существует — должна быть вставлена. Чтобы применить такую логику, требуется уникальное ограничение (UNIQUE) или уникальный индекс. В PostgreSQL UPSERT технически выполняется через INSERT ... ON CONFLICT ..., а также с помощью более новой команды MERGE.

Во время обновления интерфейса, с помощью которого сотрудники работают с системой управления запасами, компания Frogge Emporium поручила разработчику создать UPSERT-запрос для следующей таблицы inventory, чтобы поддерживать данные о точном уровне запасов на складах компании:

```
CREATE TABLE erp.inventory (
    product_id int NOT NULL,
    warehouse_id int,
    area text,
    quantity int NOT NULL,
    updated_at timestamptz NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX ON erp.inventory (product_id, warehouse_id, area);
```

Уникальный индекс, созданный для отслеживания запасов, представляет собой комбинацию product (товар), warehouse (склад) и area (зона). Идея заключается в том, что каждый товар может находиться на нескольких складах и в разных зонах в пределах одного склада. Поскольку некоторые товары могут просто складироваться в общей зоне хранения, а не в конкретном месте, например в морозильной камере, столбец area может иметь значение NULL (при создании таблицы для столбца area нет ограничения NOT NULL).

В UPSERT-запросе, реализованном разработчиком, использовался оператор INSERT ON CONFLICT DO следующим образом:

```
INSERT INTO erp.inventory (product_id, warehouse_id, area, quantity)
VALUES (99999, 1, 'freezer', 10)
ON CONFLICT (product_id, warehouse_id, area) DO UPDATE
    SET quantity = EXCLUDED.quantity;
```

Запрос вернул:

```
INSERT 0 1
```


Пока все хорошо. Теперь попробуем обновить количество товара с `product_id` 99999 на складе `warehouse_id` 1 до 20 с помощью этого UPSERT-запроса:

```
INSERT INTO erp.inventory (product_id, warehouse_id, area, quantity,
    updated_at)
VALUES (99999, 1, 'freezer', 20, now())
ON CONFLICT (product_id, warehouse_id, area) DO UPDATE
    SET quantity = EXCLUDED.quantity;
```

Этот запрос вернул:

```
INSERT 0 1.
```

Проверим таблицу `inventory`:

```
TABLE erp.inventory;
product_id | warehouse_id | area    | quantity | updated_at
-----+-----+-----+-----+-----
    99999 | 1             | freezer |      20 | 2024-10-28 21:04:26.04033.
          |               |        |        | .8+00
```

Хорошо, похоже, значение обновилось, а значит, UPSERT выполнен успешно.

Разместим количество этого товара в общей зоне хранения того же склада с помощью:

```
INSERT INTO erp.inventory (product_id, warehouse_id, area, quantity)
VALUES (99999, 1, null, 5)
ON CONFLICT (product_id, warehouse_id, area) DO UPDATE
    SET quantity = EXCLUDED.quantity;
```

Получаем:

```
TABLE erp.inventory;
product_id | warehouse_id | area    | quantity | updated_at
-----+-----+-----+-----+-----
    99999 | 1             | freezer |      20 | 2024-10-28 21:04:26.040338+00
    99999 | 1             |        |       5 | 2024-10-28 21:05:14.020143+00
```

Теперь обновим это количество до 7:

```
INSERT INTO erp.inventory (product_id, warehouse_id, area, quantity,
    updated_at)
VALUES (99999, 1, null, 7, now())
ON CONFLICT (product_id, warehouse_id, area) DO UPDATE
    SET quantity = EXCLUDED.quantity;
```

Проверим:

```
TABLE erp.inventory;
product_id | warehouse_id | area | quantity | updated_at
-----+-----+-----+-----+-----
99999 | 1 | freezer | 20 | 2024-10-28 21:04:26.040338+00
99999 | 1 | | 5 | 2024-10-28 21:05:14.020143+00
99999 | 1 | | 7 | 2024-10-28 21:07:57.067498+00
```

Хм, что не так? Если товар с таким же `product_id`, `warehouse_id` и `area` уже существует, его количество обновляется, в противном случае — вставляется новая строка (как и задумано при `UPSERT`), однако у нас появилась новая строка. Мы столкнулись с критической проблемой, которая возникает при работе со значениями `NULL`.

Проблема связана с тем, как PostgreSQL трактует `NULL` в составном уникальном индексе. В SQL значение `NULL` означает отсутствие значения или «неизвестно». Следовательно, PostgreSQL (или любая другая база данных) не может сравнивать `NULL` на равенство. Поэтому две строки, где в колонке `area` стоит `NULL`, считаются разными, хотя по смыслу выглядят так, будто должны представлять одну и ту же запись в базе.

В результате вместо обновления существующей строки неожиданно создаются несколько строк с `NULL` в поле `area`.

До PostgreSQL 14 это была настоящая головоломка, поскольку поведение `ON CONFLICT` при значениях `NULL` было непредсказуемым. В одних случаях он вставлял новые строки, а в других — нет. Это могло приводить к ошибкам в приложениях и проблемам с целостностью данных.

Начиная с версии 15, PostgreSQL позволяет явно определять то, как следует разрешать конфликты для `ON CONFLICT` с уникальными ограничениями, содержащими поля `NULL`. Все, что нужно сделать, — это добавить предложение `NULLS NOT DISTINCT` при создании индекса. Этот оператор изменяет унаследованное поведение по умолчанию, которое рассматривало значения `NULL` как различные, и таким образом `UPSERT` работает корректно.

Это можно проиллюстрировать, если удалить проблемную строку и заново создать индекс с новой опцией:

```
DELETE FROM erp.inventory WHERE quantity = 7;
DROP INDEX erp.inventory_product_id_warehouse_id_area_idx;
CREATE UNIQUE INDEX ON erp.inventory (product_id, warehouse_id, area)
NULLS NOT DISTINCT;
```

Теперь `UPSERT` работает корректно:

```
INSERT INTO erp.inventory (product_id, warehouse_id, area, quantity,
    updated_at)
VALUES (99999, 1, null, 7, now())
```

```
ON CONFLICT (product_id, warehouse_id, area) DO UPDATE
    SET quantity = EXCLUDED.quantity;
```

```
TABLE inventory;
```

product_id	warehouse_id	area	quantity	updated_at
99999	1	freezer	20	2024-10-28 21:04:26.040338+00
99999	1		7	2024-10-28 21:22:07.914464+00

С таким индексом поведение UPSERT теперь гарантирует, что строки с NULL в столбце `area` рассматриваются как дубликаты, если по остальным столбцам они совпадают (проверка ограничения `UNIQUE`). Соответственно, база данных будет корректно обновлять существующие строки вместо вставки новых.

Хотя в более поздних версиях PostgreSQL и есть такой обходный путь, полагаться на NULL в составных уникальных ограничениях, скорее всего, плохая практика. Обычно лучше заменить NULL на осмысленное значение (например, `'common_area'`) или перестроить схему таблицы.

2.9. Выборка и получение всех данных

Как правило, не следует делать `SELECT` большего числа столбцов, чем необходимо для выполнения запроса. Более того, не стоит вытаскивать и затем передавать большие объемы данных из базы на клиентскую сторону, а затем выполнять операции вроде сортировки и фильтрации вне базы данных. Старайтесь минимизировать объем данных, которые выбираются и получаются на каждом уровне, это хорошо скажется на скорости выполнения запросов.

Допустим, разработчик хочет получить первичный ключ всех тикетов со `status = 10`, но при этом использует `SELECT *` вместо `SELECT id`, поскольку знает, что может просто отбросить ненужные столбцы в коде приложения. Напишем пример кода, чтобы посмотреть, действительно ли получение всех результатов может повлиять на производительность.

Подготовим чистое окружение Python¹:

```
virtualenv ve
. ve/bin/activate
pip install psycopg
```

Теперь напишем код для обоих сценариев: выборка всех столбцов из таблицы против выборки только нужного нам столбца (подключение будет всегда выполняться к локальному PostgreSQL (по умолчанию через Unix-socket или localhost, в зависимости от текущих настроек)).

¹ Команды подготовки Python окружения в вашем Linux дистрибутиве могут отличаться. — *Примеч. науч. ред.*

Кейс 2.5. fetch.py: получение всех столбцов

```
import psycopg, datetime
with psycopg.connect("dbname=frogge user=frogge") as conn:
    with conn.cursor(row_factory=psycopg.rows.dict_row) as cur:
        t1 = datetime.datetime.now()
        cur.execute('''SELECT *
                        FROM support.tickets
                        WHERE status = 10''')
        res = cur.fetchall()
        tkts = []
        for row in res:
            tkts += row['id'],
        t2 = datetime.datetime.now()
        print(f"SELECT *" took {t2-t1} seconds.')

with conn.cursor(row_factory=psycopg.rows.dict_row) as cur:
    t3 = datetime.datetime.now()
    cur.execute('''SELECT id
                    FROM support.tickets
                    WHERE status = 10''')
    res = cur.fetchall()
    tkts = []
    for row in res:
        tkts += row['id'],
    t4 = datetime.datetime.now()
    print(f"SELECT id" took {t4-t3} seconds.')
```

Запуск кода дает следующий результат¹:

```
$ python3 fetch.py
"SELECT *" took 0:00:00.244571 seconds.
"SELECT id" took 0:00:00.093342 seconds.
```

Из этого можно сделать вывод: выборка и передача данных, которые мы потом все равно выкидываем, дает штраф по производительности из-за дополнительных затрат ресурсов и на стороне базы, и на стороне приложения просто потому, что приходится передавать больше данных, сериализовать/десериализовать их, размещать в памяти и т. п. Кроме того, помните, что это результат на локальной машине; если речь идет о сетевых соединениях, которые работают еще медленнее, разница в производительности может оказаться еще больше.

Более грубая ошибка возникает, когда разработчик выгружает в приложение всю таблицу целиком, чтобы «избежать хлопот с написанием SQL-запросов», потому что он хороший разработчик и знает, как эффективно фильтровать данные на стороне приложения. Поверьте, такие ситуации реальны.

¹ Не забудьте про пароль для подключения к PostgreSQL из python скрипта, самый быстрый вариант — выполнить предварительно команду: `export PGPASSWORD='пароль пользователя frogge'`. — *Примеч. науч. ред.*

Кейс 2.6. fetch2.py: получение всех строк

```
import psycopg, datetime
with psycopg.connect("dbname=frogge user=frogge") as conn:
    with conn.cursor(row_factory=psycopg.rows.dict_row) as cur:
        t1 = datetime.datetime.now()
        cur.execute('''SELECT *
                        FROM support.tickets''')
        res = cur.fetchmany(10000)
        while (res):
            tkts = []
            for row in res:
                if row['status'] == 10:
                    tkts += row['id'],
            res = cur.fetchmany(10000)
        t2 = datetime.datetime.now()
        print(f'"SELECT *" with no predicate took {t2-t1} seconds.')
```

Запустим этот код:

```
$ python3 fetch2.py
"SELECT *" with no predicate took 0:06:41.919264 seconds.
```

Такое время выполнения — катастрофа. Да, это намеренно вопиющий пример: очень большая таблица, и разработчик фактически отказался от использования индексов. В реальной жизни это обычно заканчивается тем, что разработчик или конечный пользователь жалуется ИТ-директору: «PostgreSQL тормозит», — после чего начинаются закупки ненужного топового железа, чтобы «ускорить» плохой запрос.

Используйте СУБД для того, в чем она хороша: для извлечения данных! PostgreSQL почти 30 лет развивает оптимизацию запросов, и этим нужно пользоваться.

Почему PostgreSQL выбирает Index Scan вместо Index Only Scan

Если вы выбираете только проиндексированный столбец, операция выполнится как быстрый **Index-Only scan** (данные столбца получаются только из индекса, в таблицу заходить вообще не нужно). Если вы добавите столбцы в выборку, СУБД будет вынуждена выполнить **Index Scan**, то есть читать уже не только индекс, но и саму таблицу, — и это может быть заметно медленнее. Об этом полезно помнить при написании подзапросов или использовании CTE, которые должны брать только минимально необходимый объем данных и которые необходимо оптимизировать по максимуму.

2.10. Игнорирование преимуществ чекеров / линтеров или больших языковых моделей

Современные инструменты, такие как чекеры, линтеры и ИИ-ассистенты, могут найти множество распространенных ошибок до того, как проблемы появятся.

Инструменты могут значительно улучшить качество кода, поддерживать лучшие практики и предотвращать ошибки. Однако они не заменят человека, который понимает нюансы PostgreSQL и принимает обоснованные решения.

2.10.1. Чекеры / линтеры кода

Каждый уважающий себя разработчик знает, как ценна вторая (или более) пара глаз, которые смотрят на код, поскольку так можно найти дополнительные идеи или выявить ошибки, ускользнувшие от внимания. К сожалению, не всегда легко или вообще возможно найти одного или нескольких людей и показать им свой SQL-запрос для получения обратной связи.

Однако вы можете получить второе мнение бесплатно: существуют приложения, которые просмотрят и прокомментируют ваш код. Даже если вы считаете, что механические глаза не так хороши, как человеческие, по крайней мере, вы ничего не теряете, пропуская свой код через чекер или линтер.

SQLFLUFF

SQLFluff — это инструмент для линтинга и форматирования SQL-кода с поддержкой Postgres. Ниже приведен пример из репозитория SQLFluff, в котором с его помощью выявляются синтаксические ошибки и ошибки форматирования:

```
$ pip install sqlfluff
```

Этой командой устанавливается инструмент. Далее создадим запрос и сохраним его в файл:

```
$ echo " SELECT a + b FROM tbl; " > test.sql
```

Теперь проверим этот запрос линтером:

```
$ sqlfluff lint test.sql --dialect ansi
==[test.sql] FAIL
L: 1 | P: 1 | LT01 | Expected only single space before 'SELECT' keyword.
    | Found ' '. [layout.spacing]
L: 1 | P: 1 | LT02 | First line should not be indented.
    | [layout.indent]
L: 1 | P: 1 | LT13 | Files must not begin with newlines or whitespace.
    | [layout.start_of_file]
L: 1 | P: 11 | LT01 | Expected only single space before binary operator
    | Found '+'. [layout.spacing]
L: 1 | P: 14 | LT01 | Expected only single space before naked identifier.
    | Found ' '. [layout.spacing]
L: 1 | P: 27 | LT01 | Unnecessary trailing whitespace at end of file.
    | [layout.spacing]
L: 1 | P: 27 | LT12 | Files must end with a single trailing newline.
    | [layout.end_of_file]
All Finished ? ?!
```

Линтер обнаружил проблемы с форматированием запроса и сообщил, где именно в файле проблема.

SQLFluff можно найти по адресу <https://github.com/sqlfluff/sqlfluff>.

PLPGSQL_CHECK

plpgsql_check — это линтер для кода PL/pgSQL, реализованный как расширение PostgreSQL. Поскольку он использует собственный парсер и механизм оценки выражений Postgres, он может выявлять ошибки, которые возникли бы при фактическом запуске кода на сервере. Среди его прочих функций есть активный режим проверки, вызываемый функцией `plpgsql_check_function_tb()`, — она напрямую проверяет функцию и возвращает результат в табличной форме. Пример ниже демонстрирует, как `plpgsql_check` может отлавливать ошибки:

```
postgres=# CREATE EXTENSION plpgsql_check;
LOAD
```

Теперь расширение включено; создадим таблицу и функцию:

```
postgres=# CREATE TABLE t1(a int, b int);
CREATE TABLE
postgres=#
CREATE OR REPLACE FUNCTION public.f1()
RETURNS void
LANGUAGE plpgsql
AS $function$
DECLARE r record;
BEGIN
    FOR r IN SELECT * FROM t1
    LOOP
        RAISE NOTICE '%', r.c; -- there is bug - table t1 missing "c" column
    END LOOP;
END;
$function$;
```

```
CREATE FUNCTION
postgres=# \sf+ f1
CREATE OR REPLACE FUNCTION public.f1()
RETURNS void
LANGUAGE plpgsql
1 AS $function$
2 DECLARE r record;
3 BEGIN
4     FOR r IN SELECT * FROM t1
5     LOOP
6         RAISE NOTICE '%', r.c; -- there is bug - table t1 missing "c" column
7     END LOOP;
8 END;
9 $function$
```

Как мы видим, функция содержит ошибку. Запустим ее:

```
postgres=# select f1(); -- execution doesn't find a bug due to empty table t1
f1
-----
(1 row)
```

Линтер может выполнить проверку функции:

```
postgres=# \x
Expanded display is on.
postgres=# select * from plpgsql_check_function_tb('f1()');
-[ RECORD 1 ]-----
functionid | f1
lineno     | 6
statement  | RAISE
sqlstate   | 42703
message    | record "r" has no field "c"
detail     | [null]
hint       | [null]
level      | error
position   | 0
query      | [null]
```

Как мы видим, он обнаружил проблемное условие в строке 6 листинга функции. Он также предлагает пассивный режим, в котором модуль, загруженный в Postgres, сканирует весь код перед его исполнением в run time. Очевидно, что такой вариант пригоден только для отладки при тестировании, и его лучше не использовать в продакшене.

Линтер `plpgsql_check` для PL/pgSQL можно найти по адресу: https://github.com/okbob/plpgsql_check.

SQUAWK

`squawk` — это инструмент, который сканирует код с планируемым изменением схемы и предупреждает о потенциальных проблемах, которые может вызвать серия команд DDL на текущей базе данных.

Рассмотрим несколько примеров того, как он может отлавливать потенциальные проблемы до их возникновения, из его собственной документации:

```
> squawk example.sql
example.sql:2:1: warning: prefer-text-field
  2 | --
  3 | -- Create model Bar
  4 | --
  5 | CREATE TABLE "core_bar" (
  6 |   "id" serial NOT NULL PRIMARY KEY,
  7 |   "alpha" varchar(100) NOT NULL
  8 | );
```


note: Changing the size of a varchar field requires an ACCESS EXCLUSIVE lock.
 help: Use a text field with a check constraint.

example.sql:9:2: warning: require-concurrent-index-creation

```
9 |  
10 | CREATE INDEX "field_name_idx" ON "table_name" ("field_name");
```

note: Creating an index blocks writes.
 note: Create the index CONCURRENTLY.

example.sql:11:2: warning: disallowed-unique-constraint

```
11 |  
12 | ALTER TABLE table_name ADD CONSTRAINT field_name_constraint  
    ↳ UNIQUE (field_name);
```

note: Adding a UNIQUE constraint requires an ACCESS EXCLUSIVE lock
 ↳ which blocks reads.
 help: Create an index CONCURRENTLY and create the constraint using
 ↳ the index.

Инструмент проверки миграций Squawk можно найти по адресу: <https://github.com/sbdchd/squawk>.

2.10.2. Большие языковые модели

Меня впечатляет, как генеративный ИИ, в частности большие языковые модели (large language models, LLM), обученные на массивах текстов и данных — всего того, что их создатели нашли в интернете, могут обрабатывать запросы на естественном языке и выдавать связный результат. SQL-код PostgreSQL не исключение, и уже сейчас мы видим, как можно предоставить LLM некоторую информацию, а затем попросить ее написать SQL-запросы для вашей задачи.

LLM, безусловно, могут быть полезным инструментом для обсуждения идей и проверки хода мыслей или, если вы застряли, для того чтобы получить некоторое вдохновение в SQL. Для интереса предоставим ChatGPT несколько фрагментов схемы, расскажем, чего мы хотим достичь с помощью нашего запроса, и попросим его сгенерировать SQL для этого.

JA

У меня есть следующая схема PostgreSQL:

```
CREATE TABLE Customers (  
  id bigint PRIMARY KEY  
    GENERATED ALWAYS AS IDENTITY,  
  first_name text,  
  middle_name text,  
  last_name text,  
  marketing_consent boolean DEFAULT false  
);  
CREATE TABLE "Invoices" (  
  "Invoice ID" bigint PRIMARY KEY
```

```

GENERATED ALWAYS AS IDENTITY,
"Amount" numeric NOT NULL,
"Customer" bigint NOT NULL,
"Paid" boolean NOT NULL DEFAULT false
);

```

Я хочу найти, у скольких клиентов есть неоплаченные счета.

После нескольких итераций ChatGPT останавливается на таком выводе:



Чтобы найти количество клиентов с неоплаченными счетами, вы можете использовать SQL-запрос, который соединяет таблицы Customers и Invoices и находит количество уникальных клиентов с неоплаченными счетами. Вот запрос для достижения этой цели:

```

SELECT COUNT(DISTINCT c.id) AS number_of_customers_with_unpaid_invoices
FROM Customers c
JOIN "Invoices" i ON c.id = i."Customer"
WHERE i."Paid" = false;

```

Этот запрос выполняет следующие шаги:

1. Он выбирает все столбцы из таблицы Customers и присваивает ей псевдоним c.
2. Он соединяет таблицу Customers с таблицей Invoices по столбцу Customer, который связывает клиентов с их счетами.
3. Он фильтрует результаты, чтобы включить только строки, где столбец Paid в таблице Invoices имеет значение false, что указывает на неоплаченные счета.
4. Наконец, он суммирует уникальные идентификаторы клиентов в отфильтрованных результатах и дает вам количество клиентов с неоплаченными счетами.

Результатом этого запроса будет количество уникальных клиентов, у которых есть неоплаченные счета.

На первый взгляд это выглядит отлично: просто и гладко. Однако если посмотреть более внимательно, видны некоторые недостатки:

1. Наш запрос определенно не выбирает все столбцы из таблицы Customers.
2. Наблюдается непоследовательность в использовании кавычек для имен таблиц в текстовом объяснении запроса — для PostgreSQL это проблема.
3. Предполагается существование связи по внешнему ключу между таблицами Customers и "Invoices", хотя она неочевидна.
4. Делается предположение о порядке выполнения в Postgres, а как мы знаем, этого нельзя утверждать, не посмотрев план через EXPLAIN.

Здесь, безусловно, есть потенциал, и нет сомнений, что эти модели будут усовершенствованы. Однако в конечном счете необходимо помнить, что это генератор текста, который продолжает добавлять к своему выводу то, что, согласно модели, *с наибольшей вероятностью* должно следовать дальше. Он не имеет понятия о правильности и будет писать все, что, по его мнению, звучит достаточно правдоподобно, без попытки проверить свой вывод на корректность. Иногда он выдумывает вещи, которые явно ошибочны в SQL или PostgreSQL и даже не выполняются.

Учитывайте последствия

Когда вы вставляете свою схему или код в онлайн-инструмент анализа или чат-бот/LLM, вы фактически загружаете их на чужой сервер — со всеми последствиями, которые могут быть для любых применимых правил безопасности, — и в условиях работы LLM, вероятно, упоминается, что теперь им разрешено использовать ваши входные данные для дальнейшего обучения своего продукта и т. д.!

Помните о разнице между запуском локального ПО и передачей данных онлайн-сервису, чтобы он что-то сделал для вас (рис. 2.4).

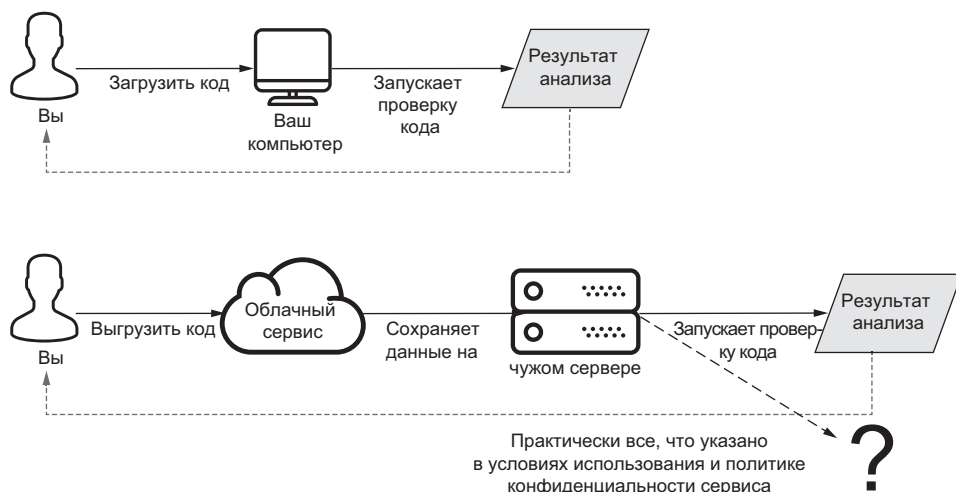


Рис. 2.4. Ваш компьютер против онлайн-сервиса

Наконец, также учитывайте, что все это вполне может в какой-то момент перестать работать. Если все начнут использовать генеративный ИИ, то в интернете будет только сгенерированный текст, и это «отравит» массив обучающих данных

LLM (poisoning the well). Помните, что в большинстве случаев, если вы подаете выходные данные ИИ на вход обучения ИИ, вы в результате получаете мусор.

Итоги

- Не используйте **NOT IN** для исключения списка значений, где может быть хотя бы одно значение **NULL**, поскольку это приведет к возврату пустого набора результатов; лучше — **NOT EXISTS**.
- Фильтрация с помощью **BETWEEN** — например, между двумя временными метками — может возвращать перекрывающиеся результаты в последующих запросах, поскольку его диапазоны включительные.
- **СТЕ** могут не только улучшать читабельность запросов, но и повышать скорость их выполнения, позволяя оптимизатору решать, какие части запроса объединить и переупорядочить.
- Заключение идентификаторов в кавычки делает их чувствительными к регистру, в то время как соглашение в **Postgres** предполагает, что все идентификаторы нечувствительны к регистру, и это может снижать удобство пользования и приводить к ошибкам.
- Когда выполняется деление целых чисел, это дает часто неожиданный, усеченный целочисленный результат.
- Функция **count()** игнорирует значения **NULL**, поэтому если вы подсчитываете значения в столбце, допускающем **NULL**, вы получите не количество подходящих для запроса строк, а количество строк, в которых в этом поле не было **NULL**.
- Когда вы запрашиваете индексируемые данные, к которым было применено приведение к типу, **Postgres** может вообще не задействовать индекс.
- Избегайте использования столбцов, допускающих **NULL**, в составных уникальных ключах. Хотя **PostgreSQL 15** ввел инструкцию **NULLS NOT DISTINCT** для решения проблем с **UPSERT**, все же лучше применять более понятную схему базы данных.
- Не замедляйте свои запросы, выбирая больше необходимого минимума данных, и не передавайте в клиентское приложение больше этого минимума данных; выполняйте операции фильтрации и управления данными на стороне базы данных.
- Используйте инструменты для проверки и линтинга SQL-кода на корректность и потенциальное влияние на продакшен, а также не отказывайтесь от потенциала генеративного ИИ, чтобы облегчить свою работу. В то же время помните о его ограничениях — всегда перепроверяйте выдачу и оставляйте за собой последнее слово.

Некорректное использование типов данных

В этой главе

- ✓ Как исключить неправильные типы данных
- ✓ Как решить проблемы с часовыми поясами и переходом на летнее время
- ✓ Какие типы данных следует полностью избегать

В PostgreSQL очень много типов данных, и, вероятно, эта СУБД поддерживает больше типов, чем другие. Но она идет еще дальше и позволяет вам создавать свои типы данных с их собственными индексами, функциями и операциями! Теперь мы рассмотрим некоторые популярные типы данных и то, как их корректное или некорректное использование может привести к серьезным ошибкам.

3.1. ***TIMESTAMP (WITHOUT TIME ZONE)***

Начнем с типов данных для хранения даты и времени. Временные метки (timestamps) — это по-настоящему популярный тип данных, который используется для одновременного хранения и даты, и времени. Если вы введете `TIMESTAMP`, PostgreSQL по умолчанию предположит, что вы хотите использовать `TIMESTAMP WITHOUT TIME ZONE`, поскольку этого требует стандарт SQL.

Сотрудники Frogge Emporium решили использовать `TIMESTAMP` для того, чтобы хранить время, когда открывается и закрывается заявка в службе поддержки клиентов. Из-за особенностей системы обслуживания клиентов, которая существует в компании, она фиксирует время открытия заявки по местному времени клиента.

Возьмем, к примеру, эту заявку, открытую клиентом на Западном побережье США 28 октября 2023 года в 16:00 по тихоокеанскому летнему времени (что на 8 часов отстает от всемирного координированного времени, или UTC-8) и закрытую агентом службы поддержки (который находился в Великобритании) 29 октября 2023 года в 09:00 по среднему времени по Гринвичу (GMT, или UTC+0):

```
-[ RECORD 1 ]-----
id           | 132591
content      | Kindly close our account, as we don't need it anymore. Thank you
status       | 20
opened_at    | 2023-10-28 16:00:00
closed_at    | 2023-10-29 09:00:00
```

Если мы попытаемся рассчитать продолжительность времени, которое потребовалось для закрытия заявки (например, для контроля качества), мы сделаем запрос:

```
SELECT pg_typeof(closed_at - opened_at), closed_at - opened_at
FROM support.tickets
WHERE id = 132591;
```

Результат будет таким:

```
pg_typeof | ?column?
-----+-----
interval  | 17:00:00
(1 row)
```

Семнадцать часов? Тут явно ошибка. С точки зрения клиента, заявка была бы закрыта 29 октября 2023 года в 02:00 по тихоокеанскому летнему времени (UTC-8), что составляет всего 10 часов.

Почему мы получили такой результат? `TIMESTAMP WITHOUT TIME ZONE`, также известный как *наивная временная метка*, не хранит информацию о часовом поясе. В результате выполнять арифметические операции (как вычитание здесь) между временными метками, введенными в разных часовых поясах, бессмысленно, поскольку мы получим неверные результаты.

Теперь предположим, что это поведение было учтено и разработчики приложений договорились с администратором базы данных хранить время только в часовом поясе `Europe/London` независимо от местоположения пользователя

приложения. Так вычисления уже можно выполнять, поскольку все находится в одном часовом поясе. Наша запись станет такой:

```
-[ RECORD 1 ]-----
id          | 132591
content     | Kindly close our account, as we don't need it anymore. Thank you
status      | 20
opened_at   | 2023-10-29 00:00:00
closed_at   | 2023-10-29 09:00:00
```

Время, в течение которого заявка была открыта, составило:

```
pg_typeof | ?column?
-----+-----
interval  | 09:00:00
(1 row)
```

Ой, опять не то! На самом деле в Лондоне летнее время (DST) закончилось в 2:00 ночи 29 октября, и часы были переведены на один час назад. Таким образом, 29-го числа у нас фактически был 1:00 по британскому летнему времени (BST, UTC+1), а затем снова 1:00 по GMT (UTC+0).

Даже если мы обойдем проблему летнего времени, используя везде UTC, все равно неправильно применять `TIMESTAMP` для его хранения. База данных не знает, что хранит UTC, и не сможет преобразовывать время между часовыми поясами для правильных вычислений.

Самый простой способ решить эту проблему — использовать `TIMESTAMPTZ` или `TIMESTAMP WITH TIME ZONE` в качестве типа данных. Таким образом мы можем вводить временные метки в любом часовом поясе, а база данных учтет все преобразования, когда потребуются сделать расчеты времени. Теперь наша таблица и данные будут выглядеть следующим образом при правильной вставке данных:

```
INSERT INTO support.tickets (content, status, opened_at, closed_at) VALUES
('Kindly close our account, as we don''t need it anymore. Thank you',
'10',
'2023-10-28 16:00 PDT',
'2023-10-29 09:00 GMT');
```

ПРИМЕЧАНИЕ Обратите внимание, что в предыдущем запросе мы смогли вставить апостроф без конфликта с одинарными кавычками, ограничивающими строку, просто удвоив его: `''`.

При просмотре базы данных из клиентского приложения в часовом поясе `Europe/London` запись выглядит так:

```
-[ RECORD 1 ]-----
id          | 1
```

```

content | Kindly close our account, as we don't need it anymore. Thank you
status  | 10
opened_at | 2023-10-29 00:00:00+01
closed_at | 2023-10-29 09:00:00+00

```

Расчет продолжительности дает правильный результат:

```

pg_typeof | ?column?
-----+-----
interval  | 10:00:00
(1 row)

```

`TIMESTAMP WITH TIME ZONE` хранит полный момент времени, поэтому база может корректно рассчитывать, сколько прошло между двумя такими моментами. В отличие от него, `TIMESTAMP WITHOUT TIME ZONE` больше похож на фотографию календаря (без года) и часов, которая фиксирует время и дату без дополнительного контекста.

`TIMESTAMP TZ` также удобен тем, что, хотя он показывает время в часовом поясе клиента, он может отобразить его для конкретного часового пояса по вашему желанию:

```

SELECT opened_at AT TIME ZONE 'PDT' AS "Ticket opened",
       closed_at AT TIME ZONE 'PDT' AS "Ticket closed"
FROM support.tickets;

```

Вывод будет следующим:

```

Ticket opened | Ticket closed
-----+-----
2023-10-28 16:00:00 | 2023-10-29 02:00:00

```

Кроме того, вам не нужно дополнительное пространство для хранения; оба типа `TIMESTAMP` имеют длину 8 байт. Благодаря этой компактности вы можете использовать `TIMESTAMP TZ` в качестве естественного первичного ключа для временных рядов. И нужен ли вам *суррогатный* (то есть искусственный) ключ, когда `TIMESTAMP TZ` может служить уникальным идентификатором тикетов? В качестве дополнительного бонуса он прекрасно партиционируется и индексируется, поэтому вы можете использовать его в качестве практического ключа партиционирования, а затем применять этот ключ для действительно эффективных запросов.

Подводя итог, скажем, что `TIMESTAMP TZ` — предпочтительный тип данных для записи конкретного момента времени. Наивный `TIMESTAMP` бесполезен для временных вычислений и не имеет преимуществ в производительности или хранении по сравнению с типом, учитывающим часовые пояса.

3.2. TIME WITH TIME ZONE

Для некоторых данных достаточно захватить только время без даты. Поэтому легко предположить, что `TIME WITH TIME ZONE` или `TIMETZ` — это хороший вариант для поля данных; в конце концов, мы видели ранее, что игнорирование часового пояса может вызывать проблемы.

Предположим, что у Frogge Emporium есть таблица, в которой хранится энергопотребление каждого филиала, измеренное «умным» счетчиком. Счетчик предоставляет время каждого замера, а поступающие от него данные сохраняются в колонку типа `TIMETZ`. Для филиала 41 у нас есть следующие последовательные показания:

branch_id	reading_time	reading	unit
41	01:17:27.612383+01	54921.8	kWh
41	01:17:21.356247+00	54988.0	kWh

Первое и второе записанное время находятся по разные границы перехода на летнее время. Мы знаем, что между отсечками примерно один час, но давайте посмотрим, что произойдет, когда мы попытаемся вычесть из одного времени другое, чтобы найти интервал между ними:

```
SELECT '01:17:27.612383+01'::timetz - '01:17:21.356247+00'::timetz;
```

Это приводит к ошибке:

```
ERROR:
operator does not exist: time with time zone - time with time zone
LINE 1: select '01:17:27.612383+01'::timetz - '01:17:21.356247+00':::...
                                     ^
```

HINT: No operator matches the given name and argument types. You might need to add explicit type casts.

Итак, мы видим, что:

- хранимое смещение времени может меняться при переходе на летнее время;
- Postgres не может выполнять временные вычисления с этим типом.

Из этого мы можем понять, что полезность `TIMETZ` сомнительна. Если мы вместо этого используем (наивный) тип `TIME`, мы столкнемся с теми же проблемами при вычислении через границы перехода на летнее время, как мы только что видели с наивным `TIMESTAMP`. Дело в том, что в реальном мире часовые пояса имеют мало смысла без дат, которые добавляют необходимый контекст.

Короче говоря, `TIMETZ` включен в PostgreSQL только для соответствия стандарту SQL. Поскольку он также занимает 8 байт пространства для хранения, он нецелесообразен, и вместо него я рекомендую использовать `TIMESTAMPTZ`.

3.3. CURRENT_TIME

Да, `CURRENT_TIME` — это не совсем тип данных, но он хорошо связывается с предыдущим разделом о `TIME WITH TIME ZONE`. `CURRENT_TIME` — это временная функция, которая возвращает текущее время суток в виде типа данных `TIME WITH TIME ZONE`:

```
SELECT CURRENT_TIME, pg_typeof(CURRENT_TIME);
```

Результат:

current_time	pg_typeof
-----+-----	
20:46:27.094953+00	time with time zone

Если вы решитесь использовать `CURRENT_TIME`, вы столкнетесь с теми же проблемами, что и при `TIMETZ`. Более правильное решение — использовать временную метку, которая представляет полный момент времени. Если вам больше не понадобится часть с датой, вы можете просто отбросить ее с помощью `EXTRACT()` или `date_part()` либо программно на стороне вашего приложения. В любом случае пространство для хранения поля будет одинаковым.

Чтобы использовать правильную конструкцию даты/времени, вам необходимо знать, что возвращает каждая функция (табл. 3.1).

Таблица 3.1. Функции времени PostgreSQL и возвращаемые типы

Функция	Возвращаемый тип	Пример вывода
<code>CURRENT_TIMESTAMP</code> или <code>now()</code>	timestamp with time zone	2023-11-20 21:03:34.349275+00
<code>CURRENT_DATE</code>	date	2023-11-20
<code>CURRENT_TIME</code>	time with time zone	21:03:34.349275+00
<code>LOCALTIMESTAMP</code>	timestamp without time zone	2023-11-20 21:03:34.349275
<code>LOCALTIME</code>	time without time zone	21:03:34.349275

Таким образом, функция `CURRENT_TIME` и возвращаемый ею тип данных (`TIMETZ`) не особо полезны, и вам, вероятно, лучше использовать что-то более подходящее из таблицы выше.

Вы можете найти больше информации о функциях даты/времени в документации PostgreSQL по адресу <https://www.postgresql.org/docs/current/functions-datetime.html>.

3.4. CHAR(n)

Теперь обсудим символьные типы PostgreSQL, или то, что вы используете для хранения текстовых строк внутри базы данных, начиная с CHAR(n) или CHARACTER(n). Это тип фиксированной длины с дополнением пробелами; он всегда имеет конкретную длину *n*, и если строка короче *n* символов, оставшая часть поля дополняется пробелами. Вообще я рекомендую избегать CHARACTER(n), и в следующих нескольких абзацах станет понятно почему.

ПРИМЕЧАНИЕ Поскольку это тип символов с дополнением пробелами длиной *n*, он также известен как тип BPCHAR(n) (*blank padded character type*). Ранее он использовался только как внутреннее обозначение типа, но теперь документирован, начиная с PostgreSQL 16.

Строка 'postgres' внутри CHAR длиной 10 выглядит следующим образом:

```
SELECT 'postgres'::CHAR(10);
      bpchar
-----
 postgres
(1 row)
```

Рисунок 3.1 делает мои объяснения немного понятнее (здесь пробелы представлены символом `␣`).

```
p o s t g r e s ␣ ␣
_ _ _ _ _ _ _ _ _ _
```

Рис. 3.1. Дополнение строки с помощью CHAR(10)

Поскольку у 'postgres' длина 8 символов, в конец добавляются 2 пробельных символа, чтобы дополнить поле до длины 10. Эти пробелы игнорируются или рассматриваются как *семантически незначимые* при сравнении строк следующим образом:

```
SELECT 'postgres'::CHAR(10) = 'postgres'::CHAR(20);
      ?column?
-----
      t
(1 row)
```

Но будьте внимательны: при сопоставлении по шаблону через LIKE и при проверке регулярными выражениями (regex) дополнение пробелами не игнорируется!

```
SELECT 'postgres'::CHAR(10) LIKE '%ostgres';
?column?
-----
f
(1 row)
```

LIKE не соответствует строке, заканчивающейся на `ostgres`, потому что в конце нашего значения `CHAR(10)` два пробела.

Аналогично поиск строки, заканчивающейся на `ostgres`, по регулярному выражению завершится неудачей:

```
SELECT 'postgres'::CHAR(10) ~ '.*ostgres';
?column?
-----
f
(1 row)
```

Мы видим, что то же самое справедливо и для POSIX-регулярных выражений с использованием оператора сопоставления `~`.

Еще одна неприятная особенность заключается в том, что если вы приводите строку длиннее n к типу `CHAR(n)`, она будет обрезана без каких-либо предупреждений или ошибок, так как этого требует стандарт SQL:

```
SELECT 'I heart PostgreSQL'::CHAR(10);
bpchar
-----
I heart Po
(1 row)
```

Даже если вам нужно сделать точную длину строки в столбце ровно в n символов, `CHAR(n)` в этом не поможет, поскольку он охотно принимает и более короткие строки.

Что еще хуже у этого типа, так это то, что внутри PostgreSQL он даже не хранится как поле фиксированной ширины. Поскольку символам может потребоваться более 1 байта (в зависимости от кодировки символов), строка в базе представляется как значение переменной длины. Таким образом, вы можете впустую тратить дисковое пространство, храня ненужные пробелы. От этого падает производительность, при использовании `CHAR(n)` ваш сервер тратит дополнительное время вычислений на удаление пробелов, чтобы выполнить строковые операции и сравнения.

Наконец, индексы, созданные для столбцов `CHAR(n)`, могут не работать для запросов с параметром `TEXT`, которые передаются в базу данных из коннектора или драйвера PostgreSQL (аналогично тому, что мы видели в разделе 2.7).

В итоге практически для каждого случая вам лучше просто использовать TEXT — текстовый тип переменной неограниченной длины (что немного сбивает с толку, потому что он эквивалентен VARCHAR без указания ограничения). TEXT дает вам больше гибкости (не ограничивает вас «на будущее») и может быть быстрее, чем при использовании CHAR фиксированной длины.

3.5. VARCHAR(n)

CHARACTER VARYING(n) или VARCHAR(n) — это поле переменной длины для текстовых данных с ограничением длины, поэтому вы можете хранить любую строку длиной до этого предела. Соответственно, вставка более длинных строк приведет к ошибке:

```
CREATE TEMP TABLE test1 (col VARCHAR(5));
CREATE TABLE

INSERT INTO test1 VALUES ('12345678');
ERROR:  value too long for type character varying(5)
```

При этом, как ни странно, если лишние символы сверх *n* — пробелы, строка будет просто обрезана до длины *n* без сообщения об ошибке, как предписано стандартом SQL:

```
INSERT INTO test1 VALUES ('1234 ');
INSERT 0 1
TABLE test1;
  col
-----
1234
```

Мы видим, что вставленная строка '1234 ' состоит из пяти символов, заканчивающихся пробелом. Такое поведение характерно и для CHAR(n). Если более длинное значение приводится к VARCHAR(n), оно тоже обрезается без предупреждения или ошибки:

```
SELECT 'Just use TEXT'::VARCHAR(10);
 varchar
-----
Just use T
(1 row)
```

В отличие от CHAR(n), VARCHAR(n) не хранит никаких завершающих пробелов, поэтому проблема пустой траты места для хранения и проблема сравнения строк, которые мы видели с CHAR(n), исключены.

Однако вы снова не получаете абсолютно никакой выгоды от установки ограничения длины, поскольку хранение в базе идентично TEXT. Что еще хуже,

как раз когда вы думаете, что все уладили с таблицей `suppliers` и ее столбцом `company_name VARCHAR(50)`, появляется поставщик, и он раздражен тем, что не видит полного названия своей компании при входе на ваш портал (название компании просто не помещается в `VARCHAR(50)`):

```
SELECT length('Peterson's and Sons and Friends Bits & Parts Limited');
length
-----
      52
(1 row)
```

Использование `VARCHAR(n)` создает лишние проблемы: изменение лимита требует `ALTER TABLE` и связанных с DDL блокировок, а уменьшить ограничение обычно вообще невозможно.

Если вы непременно хотите ограничить длину поля, допустим, чтобы соответствовать требованиям, просто установите ограничение `CHECK`, которое затем можно будет легко изменить. Ограничения целостности `CHECK` задают требования к значению, которое может храниться в столбце, например, так:

```
DROP TABLE test1;
DROP TABLE

CREATE TEMP TABLE test1 (col TEXT CHECK(length(col)<=5));
CREATE TABLE

INSERT INTO test1 VALUES ('12345678');
ERROR:  new row for relation "test1" violates check constraint
➡ "test1_col_check"
DETAIL:  Failing row contains (12345678).
```

В качестве альтернативы вы можете использовать `CREATE DOMAIN` поверх типа данных `TEXT` для применения ограничений. Домены (Domains) — это типы данных с уже заданными ограничениями, которые полезны для того, чтобы исключить повторения определений `CHECK`.

Еще раз обращаю внимание: суть здесь в том, чтобы избегать типа `VARCHAR(n)`, — он может в конечном счете загнать вас в такой тупик, выход из которого будет трудным и долгим, и у этого типа нет преимуществ в производительности по сравнению с типом без ограничений. Поэтому просто используйте `TEXT`.

ПРИМЕЧАНИЕ Для получения дополнительной информации о символьных типах в PostgreSQL обязательно ознакомьтесь с подсказкой по производительности в документации (<https://www.postgresql.org/docs/current/datatype-character.html#DATATYPE-CHARACTER>).

Тип BPCHAR

Начиная с PostgreSQL 16, документирован и тип BPCHAR, который хранит строки с обрезанными пробелами, а физическая длина такого значения может быть переменной. То есть подобно VARCHAR, BPCHAR также принимает строки любой длины, но завершающие пробелы семантически незначимы. Имейте в виду, что это означает существенно иное поведение по сравнению с VARCHAR. Проиллюстрируем разницу между BPCHAR и VARCHAR в том, как они обрабатывают завершающие пробелы:

```
CREATE TEMP TABLE trailing_ws (v VARCHAR, b BPCHAR);
CREATE TABLE

INSERT INTO trailing_ws VALUES ('vvv  ', 'bbb ');
INSERT 0 1

SELECT v, ('vvv' = v), b, ('bbb' = b) FROM trailing_ws;
   v   | ?column? |   b   | ?column?
-----+-----+-----+-----
vvv    | f        | bbb   | t
(1 row)
```

Мы видим, что BPCHAR игнорирует завершающие пробелы и рассматривает 'bbb ' (с тремя пробелами) так же, как и 'bbb', тогда как VARCHAR считает, что строки 'vvv ' и 'vvv' различаются.

3.6. Тип MONEY

Наши друзья из Frogge Emporium хотят хранить данные о полученных оплатах. Естественно, они думают о типе данных MONEY. Он может хранить денежную сумму с определенной дробной точностью, установленной базой данных. Frogge Emporium создает следующую таблицу:

```
CREATE TABLE erp.payments (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    tstamp timestamp with time zone NOT NULL,
    amount money NOT NULL,
    invoice bigint NOT NULL
);
```

Вставив тестовую оплату, мы получаем:

```
INSERT INTO erp.payments (tstamp, amount, invoice)
VALUES (now(), 99.99, 0);
INSERT 0 1
TABLE erp.payments;
 id |          tstamp          | amount | invoice
-----+-----+-----+-----
  1 | 2023-11-20 21:25:12.501561+00 | £99.99 |      0
(1 row)
```

Разберемся, что здесь произошло. В тестовой базе данных автора настройка локали PostgreSQL LC_MONETARY унаследовала системное значение по умолчанию en_GB, которое выбирает британский фунт (GBP). Таким образом, база данных предположила, что мы вставили £99.99 в этой валюте. Это ошибка некорректного ведущего символа: тип MONEY фактически не хранит тип валюты, а использует то, что настроено на сервере.

Пока забудем об этом и посмотрим, что произойдет, если мы захотим применить 25 % скидку на все старые платежи заказчика в качестве благодарности. Формула должна быть проста — мы знаем, что скидка рассчитывается так:

```
SELECT 99.99 * 0.25;
?column?
-----
    24.9975
(1 row)
```

Применим эту же функцию к значению, хранящемуся в нашей таблице:

```
SELECT amount * 0.25
FROM erp.payments
WHERE id = 1;
?column?
-----
    £25.00
(1 row)
```

Ой, ошибочка. Тип MONEY не может обрабатывать доли пенни, цента или любой другой денежной единицы, поэтому вы не сможете учесть все деньги, что неприемлемо для большинства целей. Из этого ясно, что MONEY также не сможет точно конвертировать валюты, когда округление недопустимо.

Оказывается, что тип MONEY, как ни парадоксально, — откровенно плохой тип для хранения данных о деньгах. Более того, тип данных MONEY охотно принимает недопустимые числа на вход:

```
SELECT ',123,456,,7,8.1,0,9':MONEY;
money
-----
£12,345,678.11
(1 row)
```

Такое поведение для денежного типа данных ужасно. Вам точно нужно это в базе данных? На самом деле именно из-за проблем, описанных выше, главные разработчики PostgreSQL несколько раз пытались объявить тип MONEY устаревшим. Но в ответ на это каждый раз поступали настойчивые просьбы вернуть этот тип, потому что у кого-то оставались базы данных с ним и люди не хотели менять его на более целесообразный.

ПРИМЕЧАНИЕ Для работы с деньгами я также не рекомендую использовать такие типы с плавающей точкой, как `real` или `double precision`, поскольку они подвержены ошибкам округления. По определению, эти числовые типы неточны и их приближения чисел не подходят для хранения точных величин, например валют.

Хорошее решение — использовать `NUMERIC` вместо `MONEY`. Другой неплохой вариант — хранить валюту в соседнем столбце, рядом с денежным значением.

СОВЕТ В PostgreSQL нет разницы между `NUMERIC` и `DECIMAL`.

3.7. Тип данных SERIAL

`SERIAL` — это расширение PostgreSQL, то есть нестандартный способ попросить базу данных создать поле с автоинкрементным целым числом. То же самое относится и к его старшему брату `BIGSERIAL`, который автоматически увеличивает значение `BIGINT`. Раньше это было удобным вариантом сокращенной записи, но сегодня такая запись приносит больше проблем, чем пользы. Чтобы лучше разобраться, давайте посмотрим, как это работает, создав таблицу с первичным ключом `SERIAL`:

```
CREATE TEMP TABLE transactions (
  id SERIAL PRIMARY KEY,
  amount numeric NOT NULL
);
CREATE TABLE
```

```
\d transactions
```

Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('transactions_id_seq'::regclass)
amount	numeric		not null	

```
Indexes:
    "transactions_pkey" PRIMARY KEY, btree (id)
```

Как и ожидалось, автоматически создана последовательность `transactions_id_seq` для генерации значений для столбца `id`.

Теперь предоставим другому пользователю разрешение на изменение этой таблицы:

```
CREATE USER jimmy;
CREATE ROLE

GRANT ALL ON TABLE transactions TO jimmy;
GRANT
```

Посмотрим, сможет ли jimmy вставить сюда данные после переключения ролей:

```
SET ROLE jimmy;
SET
```

```
INSERT INTO transactions (amount) VALUES (10.00);
ERROR: permission denied for sequence transactions_id_seq
```

Как видим, другие пользователи не могут вставлять данные в таблицу, даже если мы им это разрешим, потому что у них нет права обращаться к автоматически созданной последовательности. Это серьезный недостаток: правами доступа для автоинкрементных последовательностей, созданных через **SERIAL**, необходимо управлять отдельно от самой таблицы.

Более того, если вы используете **CREATE TABLE ... LIKE** для создания похожей таблицы, в новой таблице будет та же самая последовательность!

```
CREATE TEMP TABLE new_tx (LIKE transactions INCLUDING ALL);
CREATE TABLE
```

```
\d new_tx
```

```

                                Table "pg_temp_4.new_tx"
  Column | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
   id    | integer |           | not null | nextval('transactions_id_seq'::
                                regclass)
  amount | numeric |           | not null |
Indexes:
    "new_tx_pkey" PRIMARY KEY, btree (id)
```

Результат неожиданный и вряд ли желаемый: в обеих таблицах используется одна и та же последовательность **transactions_id_seq**, и вставки в одну таблицу будут автоматически увеличивать счетчик и для другой. Кроме того, у него есть еще один неприятный эффект — вы не можете удалить исходную таблицу, потому что от нее зависит последовательность в новой. Чтобы избежать этих проблем, можно использовать столбцы-идентификаторы (**identity columns**) вместо **serial**-типов, вот так:

```
DROP TABLE new_tx;
DROP TABLE
```

```
CREATE TEMP TABLE new_tx (
    id int GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    amount numeric not null
);
CREATE TABLE
```

```
\d new_tx
```

```

Table "pg_temp_4.new_tx"
Column | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
id      | integer   |           | not null | generated by default as identity
amount  | numeric   |           | not null |
Indexes:
    "new_tx_pkey" PRIMARY KEY, btree (id)

```

Со столбцом-идентификатором не требуется знать имя последовательности, чтобы ею управлять:

```

ALTER TABLE new_tx ALTER COLUMN id RESTART WITH 1000;
ALTER TABLE

```

```

INSERT INTO new_tx (amount) VALUES (10.00);
INSERT 0 1
TABLE new_tx;
  id | amount
-----+-----
1000 | 10.00
(1 row)

```

Если вы теперь заново попробуете конструкцию `CREATE TABLE ... LIKE` с таблицей, содержащей столбцы-идентификаторы (`identity columns`), новая таблица получит собственные новые последовательности, поэтому вы не столкнетесь с какими-либо проблемами.

И напоследок: если приложению нужно генерировать непрерывную последовательность идентификаторов без пропусков (например, номера чеков), то лучше формировать такую последовательность на стороне приложения, чтобы гарантировать корректность. Дело в том, что последовательности PostgreSQL будут выдавать новые значения даже для транзакций, которые потом «не закоммитились» и были отменены. В итоге вам придется искать фактический последний идентификатор в таблице и переустанавливать последовательность (`reset sequence`), чтобы выдать следующее число.

3.8. XML

Начнем с того, что в PostgreSQL действительно есть тип данных XML, как бы сомнительно это ни выглядело, и многие аргументы против использования XML вне базы данных применимы и здесь. Начнем с того, что речь идет о проблемах, которые вообще не зависят от типа СУБД, в частности, XML как формат документов вобрал в себя худшее из обоих миров: он в целом такой же неэффективный, как и обычный текстовый файл (если не хуже), и он может быть нечитабелен для человека почти настолько же, насколько и бинарный файл.

Есть причины избегать этого формата и в практической плоскости, в частности:

- он требует единственного корневого элемента, и это означает, что для таких операций, как конкатенация XML-содержимого, нужен специальный парсинг и сами они становятся значительно сложнее;
- он использует концепцию пространств имен и вводит дополнительную проблему обработки коллизий пространств имен;
- как текстовый формат он может допускать несоответствия в текстовом представлении числовых данных. Хороший пример — включение чисел, введенных в разных локалях: есть высокая вероятность того, что правила записи чисел будут разными;
- он одновременно поддерживает несколько различных способов экранирования символов, и правила экранирования различаются в зависимости от того, находитесь ли вы внутри текста, атрибута, комментария, CDATA и т. д.;
- часто XML формируется неправильно и его нужно парсить. XML чаще редактируют вручную, чем JSON, который обычно используется для сериализации/десериализации объектов приложения через функции библиотек преобразования.

К этому можно добавить, что XML — плохой выбор для приложения из-за кошмара с выделением памяти при парсинге: вам придется разбираться в плюсах и минусах DOM против SAX/стриминговых парсеров и т. п.

Есть и дополнительные риски уже на стороне СУБД, конкретно в PostgreSQL.

- Объявление типа документа DOCTYPE (DTD) проблематично, потому что требует внешнего или встроенного определения DTD (которое будет полностью нерелевантно для вашей базы данных). PostgreSQL не проверяет входные значения на соответствие ему и не поддерживает другие языки XML-схем, такие как XML Schema.
- Кодировка символов, указанная в XML-декларации, может отличаться от кодировки, в которой был сохранен XML-текст. Дополнительная сложность в том, что на стороне клиента и сервера, а также на стороне XML-данных могут использоваться разные кодировки символов. По умолчанию PostgreSQL преобразует все символьные данные между клиентом и сервером, включая строковые представления значений в вашем XML, и это может сделать объявление кодировки XML недействительным. Это возлагает на вас ответственность за соответствие всех трех кодировок и XML-декларации.
- То же самое относится к атрибуту идентификации языка `xml:lang`; он просто нерелевантен внутри базы данных PostgreSQL.

- Для типа XML нет применимых операторов сравнения, поскольку общеизвестного и надежного алгоритма сравнения XML-данных не существует даже за пределами Postgres. Как следствие, вы не можете извлекать строки по значению XML-столбца. Таким образом, основной доступный и далеко не идеальный вариант — это преобразовывать все в TEXT и сравнивать строки.
- XML можно запрашивать с помощью XPath 1.0, но смысла в этом нет, поскольку тип не индексируется. Сравните это с JSONB, который отлично индексируется с использованием типов индексов GIN и GiST.
- Он очень многословен, что приводит к большому количеству избыточных байтов и, как следствие, к бесполезной трате места для хранения.
- В PostgreSQL XML не предлагает никаких плюсов по сравнению с гораздо более компактным представлением тех же данных в формате JSON. Рассмотрим этот фрагмент, который даже не включает базовые накладные расходы и декларации, необходимые для того, чтобы считаться XML-документом:

```
SELECT
  '<property>
    <key>color</key>
    <value>00000</value>
  </property>'::XML,
  pg_column_size(
    '<property>
      <key>color</key>
      <value>00000</value>
    </property>'::XML
  );
```

xml		pg_column_size
<property>	+	72
<key>color</key>	+	
<value>00000</value>	+	
</property>		
(1 row)		

Сравните это с более лаконичным способом представления почти тех же данных:

```
SELECT
  '{"color":0}'::JSON,
  pg_column_size(
    '{"color":0}'::JSON
  );
```

json	pg_column_size
{"color":0}	15
(1 row)	

ПРИМЕЧАНИЕ `pg_column_size()` — это функция, с помощью которой мы измеряем пространство в байтах, необходимое для хранения любого отдельного значения данных.

В заключение хочу сказать, что XML подвержен ошибкам, неэффективен и медлителен. Не используйте XML для серьезных задач, особенно внутри базы данных Postgres. Вам будет гораздо удобнее структурировать данные через JSON или, что еще лучше, JSONB. Подробнее о JSONB можно узнать в документации PostgreSQL (<https://www.postgresql.org/docs/current/datatype-json.html>).

Итоги

- **TIMESTAMP (WITHOUT TIME ZONE)** не дает никаких преимуществ, так как может привести к ошибкам в расчетах времени из-за отсутствия контекста часового пояса и перехода на летнее время (DST). **TIMESTAMP WITH TIME ZONE** — это правильный тип данных для записи временных меток как конкретных моментов времени.
- У **TIMETZ (TIME WITH TIME ZONE)** и **CURRENT_TIME** сомнительная полезность, потому что часовые пояса не имеют смысла без контекста дат. Опять же предпочтительнее использовать **TIMESTAMPTZ**, даже если нам не нужно отображать часть даты временной метки.
- **MONEY** не хранит информацию о валюте, и у него ограниченная и недоработанная реализация. Вместо него лучше использовать **NUMERIC** или другие числовые форматы, которые лучше фиксируют точные значения, также есть смысл хранить значение валюты в отдельном столбце.
- Два **serial**-типа, **SERIAL** и **BIGSERIAL**, были фактически вытеснены **identity**-столбцами (столбцами-идентификаторами), которые ведут себя более предсказуемо в плане управления ролями и использования последовательностей, а также более прозрачно показывают, к какой таблице принадлежит последовательность.
- Вы не экономите место для хранения, используя ограниченные символьные типы **CHAR(n)** и **VARCHAR(n)**, а пробелы, хранимые с **CHAR(n)**, могут снизить производительность. Кроме того, из-за специфических особенностей SQL вы рискуете загнать себя в тупик ограничениями максимальной длины. Лучше выбрать **TEXT**.
- XML — очень плохой вариант для хранения документов, если только вы не просто копируете неизменяемые XML-данные внутри базы данных. Если вы намерены запрашивать данные или манипулировать ими, лучше использовать **JSON(B)**.

4

Ошибки при работе с таблицами и индексами

В этой главе

- ✓ Что такое наследование таблиц — необычная фишка
- ✓ Почему важно партиционирование и как правильно его реализовать
- ✓ Как использовать правильный тип ключа и индекса для таблиц

Некоторые особенности PostgreSQL дают ему дополнительную гибкость по сравнению с другими реляционными СУБД (RDBMS), позволяя строить мощные и многофункциональные схемы баз данных. Однако эти же самые фишки могут стать подводными камнями, если вы приходите в Postgres с багажом представлений из других систем или если неправильно поняли документацию.

4.1. Наследование таблиц

На момент написания книги PostgreSQL везде описывается как «мощная, открытая объектно-реляционная система управления базами данных». Эта формулировка может сбить кого-то с толку, так как кажется описанием из другой эпохи в области вычислений. Вполне возможно, это устаревшая отсылка к объектно-ориентированному программированию (ООП), которое было невероятно

популярно в прошлые годы, как некоторые из вас, наверное, помнят. Разработчики PostgreSQL старались поддерживать ООП, и одной из продвинутых фиш в духе ООП было наследование таблиц (table inheritance).

Проще говоря, этот механизм позволяет создавать таблицы, которые наследуют столбцы от других таблиц — точно так же, как объектные классы могут наследовать переменные и методы от своих родительских классов. Использовать его так же просто, как CREATE TABLE (...) INHERITS Например, если мы рассматриваем встречу как разновидность события, мы можем создать таблицу meetings, которая включает атрибуты таблицы events.

Кейс 4.1. Наследование таблиц

```
CREATE SCHEMA calendar;
SET search_path = calendar, "$user", public;
```

```
CREATE TABLE events (
    id int PRIMARY KEY,
    scheduled_time timestamptz,
    status smallint
);
CREATE TABLE
```

```
CREATE TABLE meetings (
    invited_emails text[],
    confirmed_emails text[],
    location text
) INHERITS (events);
CREATE TABLE
```

\d events

Table "calendar.events"				
Column	Type	Collation	Nullable	Default
id	integer			
scheduled_time	timestamp with time zone			
status	smallint			

Number of child tables: 1 (Use \d+ to list them.)

\d meetings

Table "calendar.meetings"				
Column	Type	Collation	Nullable	Default
id	integer			
scheduled_time	timestamp with time zone			
status	smallint			
invited_emails	text[]			
confirmed_emails	text[]			
location	text			

Inherits: events

Таким образом, мы видим, что связь между таблицами установлена и отображается в PostgreSQL. Посмотрим, что мы можем сделать теперь. Что произойдет, когда мы вставим встречу — meeting?

```
INSERT INTO meetings (id, scheduled_time, status, invited_emails, location)
VALUES (1981, '2024-02-01 09:00', 10,
        ARRAY['kerry.moss@example.com', 'morgan.avenal@example.com'],
        'https://webmeetings.example.com/pas/ok/1981');
INSERT 0 1
```

```
TABLE meetings;
-[ RECORD 1 ]-----+-----
id              | 1981
scheduled_time  | 2024-02-01 09:00:00+00
status          | 10
invited_emails  | {kerry.moss@example.com,morgan.avenal@example.com}
confirmed_emails | 
location        | https://webmeetings.example.com/pas/ok/1981
```

```
TABLE events;
-[ RECORD 1 ]--+-+-----
id              | 1981
scheduled_time  | 2024-02-01 09:00:00+00
status          | 10
```

Мы видим, что вставка встречи (meeting) также генерирует строку в таблице events с соответствующими унаследованными полями, заполненными значениями. Это довольно удобно и, должно быть, казалось хорошей идеей до появления инструментов ORM.

До PostgreSQL 10 наследование таблиц также использовали для реализации партиционирования — через определение связей между родительской таблицей и (дочерними) таблицами-партициями. Однако начиная с PostgreSQL 10, когда появилось декларативное партиционирование, практически не осталось причин использовать значительно более сложный путь через наследование для партиционирования таблиц.

Посмотрим, что произойдет, когда, по нашему мнению, эта таблица станет слишком большой и мы попытаемся создать партиционированную версию и присоединить текущую таблицу в качестве первой партиции:

```
CREATE TABLE events_partitioned (
    id int,
    scheduled_time timestamptz,
    status smallint
) PARTITION BY RANGE (scheduled_time);
CREATE TABLE
ALTER TABLE events_partitioned ATTACH PARTITION events
FOR VALUES FROM ('2000-01-01') TO ('2025-01-01');
ERROR:  cannot attach inheritance parent as partition
```

Ой! Оказывается, наследование принципиально несовместимо с декларативным партиционированием. Вы не можете ни наследовать от партиционированной таблицы, ни добавить к ней возможности наследования. Так что же делать теперь, когда вы застряли с наследованием таблиц? Вам потребуется найти какой-то способ отменить его.

К счастью, это не так уж сложно: вы можете просто заменить отношение, определенное наследованием таблиц, на связи по внешнему ключу. Не стоит переживать из-за этого, потому что, отказавшись от наследования таблиц, вы не лишитесь никакой важной функциональности. Все, что вам нужно сделать, это создать новую таблицу для хранения данных и добавить столбец внешнего ключа. Сделаем это для таблицы `meetings`:

```
CREATE TABLE new_meetings (LIKE meetings);  
CREATE TABLE
```

```
ALTER TABLE new_meetings ADD event_id int;  
ALTER TABLE
```

Затем скопируем данные из старой таблицы в новую (что может занять много времени):

```
INSERT INTO new_meetings  
SELECT *, id FROM meetings;  
INSERT 0 520628
```

После этого вы можете создать все необходимые ограничения, индексы, триггеры и т. д. для `new_meetings`.

Есть недокументированный прием, которым вы можете воспользоваться, если у вас таблица огромная и вы делаете это на живой системе. Суть приема в том, чтобы создать ограничения внешнего ключа без его немедленной валидации. Этот прием, конечно, предполагает, что вы доверяете данным, которые находятся внутри ваших таблиц, — в частности, уверены, что данные в столбце внешнего ключа валидны (это, вероятно, будет так, поскольку таблица — точная копия исходной). Однако такие действия обычно не рекомендуются, поскольку они связаны с вмешательством в системные каталоги PostgreSQL.

ПРИМЕЧАНИЕ Изменения системных каталогов PostgreSQL — это то, чего следует избегать, потому что если вы не совсем понимаете, что делаете, то рискуете внести изменение, которое приведет к повреждению данных или сделает базу данных непригодной для использования.

С учетом описанных рисков вот что можно сделать:

```
ALTER TABLE new_meetings ADD  
CONSTRAINT event_id_fk FOREIGN KEY (event_id)
```

```
REFERENCES events (id) NOT VALID;
UPDATE pg_constraint SET convalidated = true WHERE conname = 'event_id_fk';
```

Если это живая система, то, вероятно, стоит создать триггеры для репликации изменений, поступающих в `meetings` и в `new_meetings`. Кроме того, вам понадобятся триггеры для репликации изменений в `events`, поскольку теперь, когда есть внешний ключ, изменения в `meetings` выполняются просто.

Наконец, все операции DDL выполняются сразу, внутри одного блока кода:

```
DO $
BEGIN
    ALTER TABLE meetings RENAME TO old_meetings;
    ALTER TABLE new_meetings RENAME TO meetings;
    DROP TABLE old_meetings;
    COMMIT;
END $ LANGUAGE plpgsql;
```

Вот и все — теперь вы освободились от наследования таблиц и можете экспериментировать с партиционированием (об этом поговорим в следующем разделе).

4.2. Пренебрежение партиционированием таблиц

Мы уже обсуждали партиционирование таблиц, но в чем вообще его суть? В контексте реляционных баз данных это просто разделение одной таблицы на несколько отдельных независимых таблиц. Это также называют горизонтальным партиционированием: строки, которые различаются по какому-то признаку, оказываются в разных таблицах. Я десятилетиями слышал жалобы от пользователей баз данных других вендоров на то, что PostgreSQL не поддерживает партиционирование, и всегда поражался, насколько мало людей знает о том, что такая поддержка там все-таки есть.

В PostgreSQL 10 в 2017 году появилось декларативное партиционирование (возможность задавать его прямо через `CREATE TABLE`), а менее изящные способы реализовать партиционирование существовали как минимум со времен PostgreSQL 8.1, выпущенного еще в 2005 году! PostgreSQL продолжал улучшать фишки для партиционирования в последующих версиях, например добавил поддержку внешних ключей в версии 12 и повысил производительность для партиционированных таблиц.

Теперь к вопросу о том, когда отказ от партиционирования ваших таблиц будет ошибкой. У Frogge Emporium есть большая таблица `payments`. Предположим, что не имеет смысла индексировать ее, потому что она получает очень большой объем данных, и Frogge не хочет замедлять скорость выполнения `INSERT`. Поиск платежа выглядит следующим образом:

```

SELECT count(*) FROM erp.payments;
      count
-----
150001000
(1 row)

SET jit=off; SET max_parallel_workers_per_gather=0;
SET
SET

EXPLAIN (ANALYZE, BUFFERS)
  SELECT * FROM erp.payments WHERE tstamp='2022-03-09 22:58:20.431946+00';

                                QUERY PLAN
-----
Seq Scan on payments (cost=0.00..3026445.20 rows=1 width=31) (actual
➔time=38723.267..62110.334 rows=1 loops=1)
  Filter: (tstamp = '2022-03-09 22:58:20.431946+00'::timestamp with time
➔zone)
    Rows Removed by Filter: 150000999
    Buffers: shared hit=812 read=1151157
Planning Time: 0.115 ms
Execution Time: 62110.373 ms
(6 rows)

```

Ого! Поиск строки в этой таблице занимает целую минуту. Что еще хуже, мы, похоже, прочитали 1 151 157 буферов, или целых 9 ГБ данных с диска. Аналогично, если мы хотим удалить самые старые данные за месяц, которые здесь датируются сентябрем 2021 года, мы должны написать:

```

\timing
Timing is on.

DELETE FROM erp.payments WHERE tstamp < '2021-10-01';
DELETE 1209894
Time: 65914.030 ms (01:05.914)

```

Это ужасный результат для OLTP-базы данных с высокими требованиями к производительности. Партиционирование таблицы может помочь с этой рабочей нагрузкой, поскольку оно позволяет разбить ее на меньшие таблицы, — в нашем случае они лучше работают и их проще поддерживать.

Декларативное партиционирование в PostgreSQL заключается в том, что для таблицы указываются:

- метод партиционирования;
- ключ партиционирования — один или несколько столбцов или выражений;
- границы партиций.

И делается это просто с помощью DDL. Создадим пустую партиционированную версию таблицы `payments` следующим образом:

```
CREATE TABLE erp.payments_p (
    id bigint GENERATED ALWAYS AS IDENTITY,
    tstamp timestamp with time zone NOT NULL,
    amount numeric NOT NULL,
    invoice bigint NOT NULL
) PARTITION BY RANGE (tstamp);
CREATE TABLE
```

Еще одна причина для партиционирования — жесткие ограничения PostgreSQL по размеру таблиц. Отмечу, что достичь этих лимитов сложно, но все же возможно, учитывая постоянно растущие объемы обрабатываемых данных.

PostgreSQL поддерживает неограниченный общий объем данных — и это отлично. Он также поддерживает до 1,4 миллиарда таблиц на одну базу данных, независимо от того, насколько это вообще здравая идея. Однако максимальный размер отдельной таблицы — вполне реальное ограничение, и этот размер при стандартном размере блока в 8192 байта составляет 32 терабайта. Ограничение на максимальное число строк в таблице менее однозначно: это столько строк, сколько помещается в 4,2 миллиарда блоков.

Вот здесь и начинается планирование размеров (dimensioning). Важно делать это заранее, чтобы через несколько месяцев или лет не оказаться с неуправляемо большой таблицей. Нужно вооружиться калькулятором и прикинуть скорость поступления данных в систему с точки зрения как количества новых строк, так и объема данных в байтах. Еще один фактор, который нужно учесть, — это прогнозируемый рост, который может даже не зависеть от ИТ-системы. Например, сейчас у компании Frogge Emporium есть 25 розничных точек, а прогнозируется рост до 200 в течение следующих четырех лет. Наконец, необходимо учитывать требования к хранению данных: например, законодательство может обязывать хранить записи за 10 лет.

Ваш расчет планирования мощностей определит решение для метода и ключа партиционирования. Допустим, вы определили, что с каждого устройства из сети датчиков в 1000 единиц вы получаете 1440 показаний в день. Вы можете экстраполировать это число, чтобы увидеть, сколько показаний вы принимаете за год. Очевидно, вам необходимо следить за тем, чтобы оценка оставалась актуальной и, если что, быть готовым ее пересчитать.

Компания Frogge Emporium определила, что оптимальный размер партии — один месяц. Она может использовать поле `tstamp` в качестве ключа партиционирования и разделить таблицу методом RANGE-партиционирования, который хорошо подходит для диапазонов времени, идентификаторов и т. п. Соответственно,

команда Frogge Emporium пишет скрипт для новых ежемесячных партиций для только что созданной таблицы:

```
DO $$
BEGIN
  FOR i IN 0..28
  LOOP
    EXECUTE format('CREATE TABLE erp.%s PARTITION OF erp.payments_p
FOR VALUES FROM ( ''%s'' ) TO ( ''%s'' ),
'payments_p_' ||
extract('year' FROM date_trunc('month', now()) -
(i * INTERVAL '1 month')) || '_' ||
extract('month' FROM date_trunc('month', now()) -
(i * INTERVAL '1 month')),
date_trunc('month', now()) - (i * INTERVAL '1 month'),
date_trunc('month', now()) + ((1 - i) * INTERVAL '1 month'));
  END LOOP;
END;
$$;
DO
```

Цикл скрипта начинается за 28 месяцев до текущего и создает партицию для каждого месяца следующим образом:

```
CREATE TABLE erp.payments_p_<year>_<month>
PARTITION OF erp.payments_p
FOR VALUES FROM (<first day of month>) TO (<last day of month>);
```

Получившаяся структура партиций выглядит так:

```
payments_p
├─ payments_p_2021_1
├─ payments_p_2021_2
├─ payments_p_2021_3
├─ ...
├─ payments_p_2022_1
├─ payments_p_2022_2
└─ ...
```

Далее мы копируем данные из непартиционированной таблицы в наши новые таблицы (это занимает много времени):

```
INSERT INTO erp.payments_p (tstamp, amount, invoice)
  SELECT tstamp, amount, invoice FROM erp.payments;
INSERT 0 150001000
```

Теперь посмотрим, сколько времени займет поиск той же строки в партиционированной таблице:

```
EXPLAIN (ANALYZE, BUFFERS)
  SELECT * FROM erp.payments_p
```

```
WHERE tstamp='2022-03-09 22:58:20.431946+00';
```

QUERY PLAN

```
-----
Seq Scan on payments_p_2022_3 payments_p (cost=0.00..111450.00 rows=1
➔ width=32) (actual time=1780.649..2535.625 rows=1 loops=1)
  Filter: (tstamp = '2022-03-09 22:58:20.431946+00'::timestamp with time
    ➔ zone)
  Rows Removed by Filter: 5349599
  Buffers: shared read=44580
Planning:
  Buffers: shared hit=20 dirtied=2
Planning Time: 0.406 ms
Execution Time: 2535.658 ms
(8 rows)
```

Запрос к партиционированной таблице занял всего 2,5 секунды, что в 24,5 раза быстрее, чем обращение к непартиционированной таблице. Поскольку PostgreSQL выполняет так называемое *отсечение партиций* (*partition pruning*), то есть не ищет в партициях, которые заведомо не могут содержать искомую строку, ему потребовалось проверить только партицию `payments_p_2022_3` и прочитать всего 44 580 страниц данных, или 348 МБ с диска. Отсечение партиций стало возможным благодаря тому, что мы включили ключ партиционирования в качестве предиката в запрос, и это позволило внутреннему планировщику запросов понять, какие строки мы *не* ищем.

Посмотрим, насколько быстрее получится удалить месячную партицию, чем удалить данные за этот месяц:

```
\timing
Timing is on.

DROP TABLE erp.payments_p_2021_9;
DROP TABLE
Time: 43.113 ms
```

Скорость получилась потрясающей плюс значительно упростилось управление данными и их обслуживание.

ПРИМЕЧАНИЕ Подробнее о том, как выбрать метод и ключ партиционирования, можно прочитать в официальной документации PostgreSQL (<https://www.postgresql.org/docs/current/ddl-partitioning.html>).

В чем может помочь партиционирование?

- *Производительность* — благодаря отсечению партиций (*partition pruning*) последовательные сканы и сканы по индексу выполняются по существенно меньшим объемам данных.

- *Обслуживание* — вы можете использовать `DROP TABLE` для удаления старых данных, а самоочистка через `VACUUM` для нескольких небольших таблиц может выполняться параллельно и завершаться быстрее, чем одна очень долгая операция для одной огромной таблицы.
- *Ограничения размера диска* — вы можете размещать партиции в разных *табличных пространствах* (tablespaces), которые в PostgreSQL могут находиться на разных файловых системах или дисках. Следовательно, вы можете размещать разные партиции на более медленных и дешевых дисках и решать, на каких из них создавать индексы.
- *Обход проблемы чрезмерно больших таблиц* — таблицы разбиваются на файлы по 1 ГБ, и PostgreSQL циклично выполняет код, одинаковый для каждого сегмента размером 1 ГБ, поэтому для таблицы размером 32 ТБ этот код выполнился бы 32 000 раз.

Нужно понимать, что неудачный выбор метода партиционирования, ключа партиционирования или размера партиций может, наоборот, снизить производительность. Например, следует отдавать предпочтение ключам с достаточной кардинальностью для равномерного распределения данных по партициям. Кроме того, если ваши запросы не включают ключ партиционирования в выражение с `WHERE`, они не будут отсекаать партиции, и это ухудшит план выполнения.

Плохо выбранные стратегия и ключ партиционирования могут привести к тому, что будет слишком много партиций с малым количеством строк в каждой. Если ключ партиционирования не группирует данные эффективно, вы столкнетесь с повышенными накладными расходами из-за чрезмерного сканирования партиций, операции ввода-вывода увеличатся и, как следствие, запросы станут более медленными.

Таким образом, тщательно продуманное партиционирование может упростить управление таблицами, помочь обойти ограничения PostgreSQL при работе с большими данными и значительно повысить скорость выполнения запросов.

4.3. Партиционирование по нескольким ключам

В разделе документации PostgreSQL о создании таблиц с партиционированием легко найти синтаксис для партиционирования по нескольким ключам. Иногда имеет смысл партиционировать таблицу по нескольким столбцам, чтобы увеличить гранулярность или число партиций, настроенных под специфику данных.

Компания Frogge Emporium хочет партиционировать свою таблицу энергопотребления как по месяцам, так и по филиалам, чтобы данные в каждой таблице были конкретнее. Попробуем партиционировать по нескольким ключам:


```
CREATE TABLE erp.energy_usage (  
    branch_id integer NOT NULL,  
    reading_time timestamptz DEFAULT CURRENT_TIMESTAMP,  
    reading_numeric NOT NULL,  
    unit varchar DEFAULT 'kwh' NOT NULL  
)  
PARTITION BY RANGE (reading_time, branch_id);  
CREATE TABLE
```

Теперь, когда базовая таблица готова, создадим партицию для января 2024 года и филиалов с ID от 1 до 10:

```
CREATE TABLE erp.energy_usage_2024_01_01to10  
PARTITION OF erp.energy_usage  
FOR VALUES FROM ('2024-01-01', 1) TO ('2024-02-01', 10);  
CREATE TABLE
```

Сработало хорошо. Теперь создадим следующую партицию для того же месяца для данных из филиалов с ID от 11 до 20:

```
CREATE TABLE erp.energy_usage_2024_01_11to20  
PARTITION OF erp.energy_usage  
FOR VALUES FROM ('2024-01-01', 11) TO ('2024-02-01', 20);  
ERROR: partition "energy_usage_2024_01_11to20" would overlap partition  
➔"energy_usage_2024_01_01to10"  
LINE 3: FOR VALUES FROM ('2024-01-01', 11) TO ('2024-02-01', 20);
```

Мы получили ошибку! PostgreSQL фактически сообщает, что значение `reading_time` (2024-02-01 10:00:00) может попадать только в первую партицию, независимо от `branch_id`.

Почему так? Более внимательный анализ наших действий показывает, что мы определили границы партий по двум ключам. Такой вариант просто ограничивает значения для всех ключей, которые могут попасть в каждую партицию. Будьте осторожны! Это не многоуровневое партиционирование.

Для нашей ситуации (одна партиция в месяц для определенного набора филиалов) нужно *субпартиционирование* (sub-partitioning). Субпартиционирование — это, проще говоря, партиционирование партий. Поскольку каждая партиция представляет собой отдельную таблицу, она сама может быть партиционированной таблицей.

ВНИМАНИЕ Партиционирование по нескольким ключам не то же самое, что многоуровневое партиционирование.

Рассмотрим правильный способ сделать субпартиционирование. Мы создаем базовую таблицу с `reading_time` в качестве ключа партиционирования:

```
CREATE TABLE erp.energy_usage (  
    branch_id integer NOT NULL,  
    reading_time timestamptz DEFAULT CURRENT_TIMESTAMP,  
    reading_numeric NOT NULL,  
    unit varchar DEFAULT 'kwh' NOT NULL  
)  
PARTITION BY RANGE (reading_time);  
CREATE TABLE
```

Затем мы создаем партии для каждого месяца, а после этого дополнительно партиционируем эти ежемесячные партии по `branch_id`:

```
CREATE TABLE erp.energy_usage_2024_01  
PARTITION OF erp.energy_usage  
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01')  
PARTITION BY RANGE (branch_id);  
CREATE TABLE
```

```
CREATE TABLE erp.energy_usage_2024_01_01to10  
PARTITION OF erp.energy_usage_2024_01  
FOR VALUES FROM (1) TO (10);  
CREATE TABLE
```

```
CREATE TABLE erp.energy_usage_2024_01_11to20  
PARTITION OF erp.energy_usage_2024_01  
FOR VALUES FROM (11) TO (20);  
CREATE TABLE
```

Продолжим в том же духе — теперь наша схема выглядит отлично!

```
\dt erp.energy*  
  
List of relations  
Schema | Name | Type | Owner  
-----+-----+-----+-----  
erp | energy_usage | partitioned table | frogge  
erp | energy_usage_2024_01 | partitioned table | frogge  
erp | energy_usage_2024_01_01to10 | table | frogge  
erp | energy_usage_2024_01_11to20 | table | frogge  
(4 rows)
```

Команда `\dt+ erp.energy_usage_2024_01` для партиционированной таблицы января 2024 года возвращает следующее:

```
Partition of: erp.energy_usage FOR VALUES FROM ('2024-01-01 00:00:00+00') TO  
➡('2024-02-01 00:00:00+00')  
Partition constraint: ((reading_time IS NOT NULL) AND (reading_time >=  
➡'2024-01-01 00:00:00+00'::timestamp with time zone) AND (reading_time < '2024-  
02-01 00:00:00+00'::timestamp with time zone))  
Partition key: RANGE (branch_id)  
Partitions: erp.energy_usage_2024_01_01to10 FOR VALUES FROM (1) TO (10),  
             erp.energy_usage_2024_01_11to20 FOR VALUES FROM (11) TO (20)
```

Если мы продолжим в том же духе, партиционирование будет выглядеть так:

```
energy_usage
├── energy_usage_2024_01
│   ├── energy_usage_2024_01_01to10
│   └── energy_usage_2024_01_11to20
├── energy_usage_2024_02
│   ├── energy_usage_2024_02_01to10
│   └── energy_usage_2024_02_11to20
├── energy_usage_2024_03
│   └── ...
└── ...
```

Партиционирование по нескольким колонкам, безусловно, бывает полезно, например, чтобы держать научные данные отдельно и обеспечивать быстрый доступ за счет отсеечения партий (при условии, что в запросе в `WHERE` указаны предикаты по обеим колонкам). При этом в документации легко упустить такой важный момент: «Например, для `PARTITION BY RANGE (x,y)` граница партии `FROM (1, 2) TO (3, 4)` допускает $x=1$ с любым $y \geq 2$, $x=2$ с любым непустым y и $x=3$ с любым $y < 4$ ».

В большинстве случаев субпартиционирование будет для вас удобнее такого синтаксиса, поэтому полезно о нем знать.

4.4. Использование неправильного типа индекса

PostgreSQL предлагает широкое разнообразие индексов наряду с невероятно мощной возможностью писать собственные типы индексов. У каждого встроенного типа индекса есть свой алгоритм, который подходит для конкретных случаев, а именно для улучшения скорости определенных типов запросов на конкретных типах данных.

Тип индекса по умолчанию — B-Tree (B-Tree Index), и снова я с удивлением обнаружил, что многие считают его единственным в PostgreSQL. B-Tree индексы могут ускорять выполнение запросов с операциями равенства или с упорядочивающими сравнениями (для сортировки/диапазонов). У них также могут быть некоторые функциональные возможности для запросов с оператором `LIKE`, когда нужно сопоставить данные с образцом через оператор `~` для регулярных выражений.

Посмотрим, что мы можем сделать с типом индекса по умолчанию для набора данных, такого как метаданные исследований ArXiv Open Access, где содержатся записи для 2,4 миллиона статей, включая названия, авторов, категории, аннотации и т. д.

ПРИМЕЧАНИЕ Вы можете создать учетную запись Kaggle для загрузки набора данных по адресу: <https://www.kaggle.com/datasets/Cornell-University/arxiv/>

После распаковки набора данных JSON объемом 4,2 ГБ мы запускаем следующую команду для экранирования обратных слешей и повышения совместимости нашего JSON-файла с Postgres:

```
sed -i 's/\\/\\\\/g' 'arxiv-metadata-oai-snapshot.json'
```

Затем мы копируем его в нашу таблицу базы данных с именем `arxiv`, помещая все в столбец `jsonb` с именем `data`:

```
CREATE TABLE test.arxiv (data jsonb);
CREATE TABLE

\copy test.arxiv FROM 'arxiv-metadata-oai-snapshot.json'
COPY 2417693
```

Мы можем получить доступ к элементам каждой записи, например к названию, следующим образом:

```
SELECT data ->> 'title' FROM test.arxiv TABLESAMPLE BERNOULLI (0.1) LIMIT 1;
           ?column?
```

```
-----
Modeling of hydrogen and hydroxyl group migration on graphene
(1 row)
```

Теперь создадим индекс для ускорения поиска по этим названиям, индексируя только элемент `title` из всего JSON-документа. Лучше использовать регистронезависимый поиск, поэтому преобразуем все в нижний регистр и укажем, что нам нужен индекс для `text_pattern_ops`:

```
CREATE INDEX ON test.arxiv (lower(data->>'title') text_pattern_ops);
CREATE INDEX

ANALYZE test.arxiv;
ANALYZE
```

После этого мы можем искать статьи с названиями, начинающимися, например, с `Modeling of hydrogen`. Важно всегда вводить в условия `WHERE` то же выражение, которое использовалось при создании индекса:

```
EXPLAIN ANALYZE
  SELECT data->>'doi'
  FROM test.arxiv
  WHERE lower(data->>'title') LIKE 'modeling of hydrogen%';
          QUERY PLAN

-----
Index Scan using arxiv_lower_idx on arxiv  (cost=0.56..9.19 rows=242
➔width=32) (actual time=0.085..0.085 rows=1 loops=1)
  Index Cond: ((lower((data ->> 'title'::text)) ~>~
➔'modeling of hydrogen'::text) AND (lower((data ->> 'title'::text)) ~~~
➔'modeling of hydrogeo'::text))
```

```

Filter: (lower((data ->> 'title'::text)) ~ 'modeling of
        ↳hydrogen%'::text)
Planning Time: 0.755 ms
Execution Time: 0.134 ms
(5 rows)

```

Выглядит неплохо. Теперь попробуем фильтровать по названиям, *содержащим* modeling of hydrogen (обратите внимание на изменение шаблона запроса на '%modeling of hydrogen%'):

```

SET jit=off; SET max_parallel_workers_per_gather=0;
SET
SET

EXPLAIN ANALYZE
  SELECT data->'doi'
  FROM test.arxiv
  WHERE lower(data->>'title') LIKE '%modeling of hydrogen%';
          QUERY PLAN

```

```

-----
Seq Scan on arxiv (cost=0.00..488032.26 rows=242 width=32) (actual
↳time=539.217..8624.051 rows=5 loops=1)
  Filter: (lower((data ->> 'title'::text)) ~ '%modeling of
        ↳hydrogen%'::text)
  Rows Removed by Filter: 2417688
  Planning Time: 0.084 ms
  Execution Time: 8624.065 ms
(5 rows)

```

А вот тут получилось значительно хуже. Индекс не использовался вообще, потому что B-Tree индексы подходят только для поиска по равенству и сортировке, даже с указанием `text_pattern_ops`. (Вы заметили, что в плане предыдущего запроса PostgreSQL искал ключи индекса между 'modeling of hydrogen%' и 'modeling of hydrogeo%'?)

Таким образом, тип индекса по умолчанию не подходит для поиска подстрок, которые не находятся в начале (или конце) значения, что делает его практически непригодным для полнотекстового поиска. Но даже если бы он волшебным образом мог это делать, мы столкнулись бы с проблемой при попытке проиндексировать, например, аннотацию вместо названия:

```

CREATE INDEX ON test.arxiv (lower(data->>'abstract') text_pattern_ops);
ERROR:  index row size 2728 exceeds btree version 4 maximum 2704 for index
↳"arxiv_lower_idx1"
DETAIL:  Index row references tuple (213838,4) in relation "arxiv".
HINT:  Values larger than 1/3 of a buffer page cannot be indexed.
Consider a function index of an MD5 hash of the value, or use full text indexing.

```

Действительно, индексировать такие длинные значения невозможно, что делает B-Tree индекс в большинстве случаев неподходящим для индексации

документов. И довольно хорошо то, что PostgreSQL прямо подсказывает нам, когда мы, возможно, что-то делаем не так!

Обратим также внимание на размер B-Tree индекса:

```
\di+ test.arxiv*
List of relations
-[ RECORD 1 ]-----
Schema      | test
Name         | arxiv_lower_idx
Type         | index
Owner        | frogge
Table        | arxiv
Persistence  | permanent
Access method | btree
Size         | 239 MB
Description  |
```

Правильный индекс для этой задачи — обобщенный обратный индекс (*Generalized Inverted Index, GIN*) PostgreSQL, который отлично подходит для полнотекстового поиска с типом данных `tsvector`. Создание индекса по названиям выглядит следующим образом:

```
DROP INDEX test.arxiv_lower_idx;
DROP INDEX

CREATE INDEX ON test.arxiv
    USING gin (to_tsvector('english', data->>'title'));
CREATE INDEX
```

Выполнение запроса показывает, насколько лучше GIN-индекс справляется с подобными задачами:

```
EXPLAIN ANALYZE
    SELECT data->>'doi'
    FROM test.arxiv
    WHERE to_tsvector('english', data->>'title')
    @@ plainto_tsquery('english', 'modeling of hydrogen');
               QUERY PLAN
-----
Bitmap Heap Scan on arxiv (cost=38.89..292.85 rows=60 width=32) (actual
=>time=1.829..4.949 rows=236 loops=1)
    Recheck Cond: (to_tsvector('english'::regconfig, (data ->>
        =>'title'::text))@@ '''model' & 'hydrogen'::tsquery)
    Heap Blocks: exact=236
    -> Bitmap Index Scan on arxiv_to_tsvector_idx (cost=0.00..38.87
        => rows=60 width=0) (actual time=1.783..1.783 rows=236 loops=1)
        Index Cond: (to_tsvector('english'::regconfig, (data ->>
            =>'title'::text)) @@ '''model' & 'hydrogen'::tsquery)
Planning Time: 3.455 ms
Execution Time: 4.989 ms
(7 rows)
```

Впечатляющая производительность для миллионов документов. Полагаю, производительность для аннотаций будет сопоставимой. Как вариант, можно объединить оба элемента и проиндексировать их вместе.

И хорошая новость — размер индекса получился меньше, чем у B-Tree индекса:

```
\di+ test.arxiv*
List of relations
-[ RECORD 1 ]-----
Schema      | test
Name        | arxiv_to_tsvector_idx
Type        | index
Owner       | frogge
Table       | arxiv
Persistence | permanent
Access method | gin
Size        | 100 MB
Description |
```

К сожалению, GIN-индекс также может ошибочно или некорректно использоваться. Помимо полнотекстового поиска, GIN применяют и для индексации целых JSON-документов, чтобы ускорить доступ к содержащимся в них элементам. И очень легко ошибочно предположить, что один такой «большой» индекс сможет покрывать вообще все сценарии:

```
DROP INDEX test.arxiv_to_tsvector_idx;
DROP INDEX
```

```
CREATE INDEX ON test.arxiv USING gin (data);
CREATE INDEX
```

```
\di+ test.arxiv*
List of relations
-[ RECORD 1 ]-----
Schema      | test
Name        | arxiv_data_idx
Type        | index
Owner       | frogge
Table       | arxiv
Persistence | permanent
Access method | gin
Size        | 1680 MB
Description |
```

Теперь используем этот индекс для поиска конкретных идентификаторов цифровых объектов (DOI, Digital Object Identifier) статей:

```
EXPLAIN ANALYZE
SELECT *
FROM test.arxiv
WHERE data @> '{"doi": "10.1039/c0cp01009j"}';
QUERY PLAN
```

112 Глава 4. Ошибки при работе с таблицами и индексами

```
Bitmap Heap Scan on arxiv (cost=56.90..1011.01 rows=242 width=1345)
➔(actual time=0.127..0.127 rows=1 loops=1)
  Recheck Cond: (data @> '{"doi": "10.1039/c0cp01009j"}'::jsonb)
  Heap Blocks: exact=1
  -> Bitmap Index Scan on arxiv_data_idx (cost=0.00..56.84 rows=242
    ➔ width=0) (actual time=0.116..0.116 rows=1 loops=1)
      Index Cond: (data @> '{"doi": "10.1039/c0cp01009j"}'::jsonb)
Planning Time: 2.345 ms
Execution Time: 0.159 ms
(7 rows)
```

Отличная производительность. Но если мы ищем только DOI, действительно ли это правильный выбор? Давайте сравним с B-Tree индексом:

```
DROP INDEX test.arxiv_data_idx;
DROP INDEX

CREATE INDEX ON test.arxiv((data->>'doi'));
CREATE INDEX
```

```
\di+ test.arxiv*
List of relations
```

```
-[ RECORD 1 ]-+-----
Schema      | test
Name        | arxiv_expr_idx
Type        | index
Owner       | frogge
Table       | arxiv
Persistence | permanent
Access method | btree
Size        | 61 MB
Description |
```

Очень небольшой итоговый размер индекса (61МБ против 1680МБ)! Теперь найдем наш DOI:

```
EXPLAIN ANALYZE
SELECT * FROM test.arxiv
WHERE data->>'doi' = '10.1039/c0cp01009j';
               QUERY PLAN
-----
Bitmap Heap Scan on arxiv (cost=254.11..42289.79 rows=12088 width=1345)
➔(actual time=0.043..0.044 rows=1 loops=1)
  Recheck Cond: ((data ->> 'doi'::text) = '10.1039/c0cp01009j'::text)
  Heap Blocks: exact=1
  -> Bitmap Index Scan on arxiv_expr_idx (cost=0.00..251.09 rows=12088
    ➔ width=0) (actual time=0.034..0.034 rows=1 loops=1)
      Index Cond: ((data ->> 'doi'::text) = '10.1039/c0cp01009j'::text)
Planning Time: 0.554 ms
Execution Time: 0.076 ms
(7 rows)
```


Вот это скорость! Со сравнением идентификаторов прекрасно справляются В-деревья. Помните, что для каждого типа запроса лучше использовать свой тип индекса с учетом его сильных сторон.

Можно также обратиться к документации PostgreSQL, чтобы определить, когда GiST index (Generalized Search Tree Index, обобщенный/расширяемый индексный метод) будет выгоднее по сравнению с GIN, — например, при работе с огромными наборами данных, где время выполнения запросов не критично. Индексы GiST оказываются полезными для типов диапазонов в Postgres, в частности `daterange`. По схожей причине можно использовать lossy-индекс BRIN (Block Range Index), который значительно меньше B-Tree индекса, для поиска по диапазонам значений, таким как временные метки, если этого достаточно для запросов, которые планируются. Поскольку обновление индексов, особенно сложных его типов (типа GIN), выполняется медленно, важно помнить: индексировать следует только то, что необходимо, и с учетом той скорости выполнения, которую вы считаете приемлемой.

Дополнительную информацию об этих типах индексов и об их использовании можно найти по ссылкам:

- <https://www.postgresql.org/docs/current/textsearch-indexes.html>
- <https://www.postgresql.org/docs/current/brin.html>

Итоги

- Если вы обнаружили наследование таблиц и думаете, что вам это нужно, вы, скорее всего, ошибаетесь. Реализуйте отношения «родитель-потомок» с помощью внешних ключей и триггеров, если это необходимо.
- Партиционирование таблиц помогает упростить управление и сопровождение больших таблиц и значительно ускорить запросы, которые по ним выполняются.
- Будьте осторожны при создании субпартиций, поскольку партиционирование по нескольким ключам — это не то же самое. К сожалению, из документации не совсем ясно, что конкретно делает выражение для партиционирования по нескольким ключам, когда его лучше всего использовать и каковы его последствия.
- У каждого типа индекса в PostgreSQL есть свои сильные и слабые стороны. Подстраивая стратегию индексирования под ваши данные и под тип запросов, которые нужно выполнять, вы можете оптимизировать и производительность, и расход дискового пространства. Если сделать все правильно, запросы могут работать на порядки быстрее.

5

Неправильное использование возможностей

В этой главе

- ✓ К чему приводит выбор кодировки SQL_ASCII
- ✓ Как создаются правила и какие здесь есть подводные камни
- ✓ Где можно ошибиться, используя возможности NoSQL для SQL-запросов
- ✓ Как импровизация с распределенными/multi-master системами может привести к проблемам

Богатый набор возможностей PostgreSQL и делает его таким мощным инструментом для обработки данных. Это далеко не просто «классическая» реляционная СУБД (RDBMS): дополнительные фишки вроде NoSQL-возможностей, логической репликации, подключения во внешние СУБД и источники данных через различные FDW (foreign data wrappers) и правила (rules) дают вам гибкость, позволяющую проектировать самые разные системы, ориентированные на базы данных (и, разумеется, допускать ошибки в таких дизайнах!).

5.1. Выбор кодировки SQL_ASCII

С самого начала компьютерной эры в кодировке символов или в числовом представлении текстовых символов использовались различные схемы преобразования символов в числовые значения для хранения. По мере распространения компьютеров по всему миру естественным образом возникала необходимость создания большего количества (почти всегда несовместимых между собой) *кодových страниц* (code pages) или *наборов символов* (character sets) для письменных знаков разных языков.

Кодировка / кодовая страница / набор символов

Набор символов (character set) — это просто группа символов и знаков, необходимых для правильного представления комбинации языка и локали. В некоторых устаревших средах такие наборы также известны как *кодovые страницы (code page)*. Термин «кодировка» (encoding) описывает, как этот набор символов представлен внутри — одним или несколькими байтами на символ. Так, например, существует стандарт Unicode, задающий универсальный набор символов и их кодовые точки (номер/значение символа в стандарте Unicode); он согласован со стандартом ISO/IEC 10646 (UCS). Текст, состоящий из этих кодовых точек, может быть представлен в двоичном виде как последовательность байтов кодировки UTF-8.

PostgreSQL поддерживает множество наборов символов или текстовых кодировок — в основном из-за необходимости установить совместимость с унаследованными данными, которые уже используют эти кодировки. Их также называют серверными кодировками (server-side encodings). В PostgreSQL вы можете выбрать глобальную кодировку по умолчанию для всего сервера, которая задается во время инициализации кластера (initdb), в то же время отдельные базы данных могут иметь собственную кодировку. В зависимости от кодировки на один символ может приходиться один или несколько байтов.

Если локаль не задана, что иногда бывает у плохо настроенных инстансов в облаке, Postgres по умолчанию будет использовать SQL_ASCII. Ваши базы данных тогда будут выглядеть примерно так:

```
\l
List of databases
-[ RECORD 1 ]-----+
Name           | frogge
Owner          | frogge
Encoding       | SQL_ASCII
Locale Provider | libc
Collate        | C
ctype         | C
```

```

ICU Locale      |
ICU Rules      |
Access privileges |
-[ RECORD 2 ]-----+-----
Name           | postgres
Owner          | postgres
Encoding       | SQL_ASCII
Locale Provider | libc
Collate        | C
Ctype          | C
ICU Locale     |
ICU Rules     |
Access privileges |

```

Сотрудники Frogge Emporium хотят хранить текстовое содержимое заявок в таблице `tickets`, но их клиентская база работает на нескольких языках. Поэтому они решают остановиться на кодировке базы данных `SQL_ASCII`, которая, кажется, без возражений принимает от клиента символы из любого набора.

В документации PostgreSQL для 17-й версии (<https://www.postgresql.org/docs/17/multibyte.html>) мы можем увидеть в разделе 23.3.1 («Поддерживаемые кодировки») таблицу наборов символов PostgreSQL (см. таблицу 5.1).

Таблица 5.1. Наборы символов PostgreSQL

Имя	Описание	Язык	Поддержка на сервере?	Поддержка ICU?	Байт на символ	Псевдонимы
SQL_ASCII	Не указано (см. текст)	Любой	Да	Нет	1	

А в разделе 23.3.4 («Возможные перекодировки наборов символов») мы находим преобразования наборов символов (таблица 5.2).

Таблица 5.2. Доступные преобразования наборов символов

Серверная кодировка	Доступные клиентские кодировки
SQL_ASCII	Любые (преобразование выполняться не будет)

Посмотрим, что происходит, когда мы принимаем записи из любой клиентской кодировки в нашу базу данных `SQL_ASCII`. В следующем фрагменте мы сгенерируем текстовый ввод в трех разных кодировках на стороне приложения, а затем вставим его прямо в нашу базу данных.

Кейс 5.1. sql_ascii_in.py: вставка многоязычного текста

```
import pycopg
english_text = ("Good evening, I would like to return my last order "
               + "please.").encode('iso-8859-1')
greek_text = ("Καλησπέρα, θα ήθελα να επιστρέψω την τελευταία μου παραγγελία "
              + "παρακαλώ.").encode('windows-1253')
japanese_text = ("こんばんは、前回の注文を返品したいのですがお願いします。"
                 ).encode('shift_jis')

with pycopg.connect("dbname=frogge user=frogge") as conn:
    with conn.cursor() as cur:
        cur.execute('''INSERT INTO support.tickets (content, status)
                     VALUES (%s, 20), (%s, 20), (%s, 20)''',
                     (english_text, greek_text, japanese_text))
```

Хорошо, похоже, сработало. Посмотрим, что мы поместили в нашу базу данных:

```
SELECT id, content FROM support.tickets;
id | content
-----+-----
1 | \x476f6f64206576656e696e672c204920776f756c64206c696b6520746f207265747
  | .5726e206d79206c617374206f7264657220706c656173652e
2 | \xcae1ebe7f3f0ddf1e12c20e8e120dee8e5ebe120ede120e5f0e9f3f4f1ddf8f920f.
  | 4e7ed20f4e5ebe5f5f4e1dfe120ecff520f0e1f1e1e3e3e5ebdfe120f0e1f1e1eae1.ebfe2e
3 | \x82b182f182ce82f182cd8141914f89f182cc928d95b682f095d4956982b582bd82a
  | .282cc82c582b782aa82a88ae882a282b582dc82b78142
(3 rows)
```

Погодите, это немного сбивает с толку. Что это за шестнадцатеричные крокозябры? Ах! Я же говорил, что текстовая кодировка — числовое представление текстовых символов, так что все логично. Должно быть, это числовые представления строк в указанных нами кодировках.

Давайте теперь прочитаем их из приложения! Поскольку эти последовательности байтов представляют собой текст, извлечем их, приведем к типу `bytea`, а затем выведем как текст UTF-8.

Кейс 5.2. sql_ascii_out.py: чтение многоязычного текста

```
import pycopg

with pycopg.connect("dbname=frogge user=frogge") as conn:
    with conn.cursor(row_factory=pycopg.rows.dict_row) as cur:
        cur.execute('''SELECT id, content::bytea FROM support.tickets''')
        res = cur.fetchall()
        for row in res:
            print(row['id'], row['content'].decode('UTF-8'))
```


в UTF-8 или, по крайней мере, что в базу данных устаревших приложений данные отправляются в UTF-8.

К сожалению, если локаль в PostgreSQL не указана, то СУБД по умолчанию использует SQL_ASCII, и если ваш текст написан латинскими буквами, то он будет в основном работать нормально — до тех пор, пока вы не попытаетесь выполнить какие-либо преобразования. Однако если настройки локали системы правильные, PostgreSQL будет использовать для нее UTF8.

Итак, что же делать, если у вас есть устаревшая база данных и вы застряли с SQL_ASCII? Если вы знаете наиболее вероятную кодировку данных, вы, пожалуй, сможете перенести их в UTF8. Извлечь данные и преобразовать их в UTF-8 не так уж сложно:

```
SELECT id,
       convert_from(convert(content::bytea, 'ISO-8859-1', 'UTF8'), 'UTF8')
FROM support.tickets
WHERE id = 1;
 id |
-----+-----
  1 | Good evening, I would like to return my last order please.
```

Это хорошо — теперь вы можете извлекать строки в формате UTF-8 для экспорта текста. Кодировку базы данных изменить нельзя, поэтому изменение кодировки обычно выполняется путем миграции данных в «правильную» кодировку в новую базу. И даже в неправильно настроенной системе вы все равно можете создать базу данных с кодировкой UTF8 и с указанием имени локали en_US.UTF-8, которая, в свою очередь, задает правила сравнения/сортировки и классификации символов:

```
CREATE DATABASE frogge_new ENCODING UTF8 LC_COLLATE 'en_US.UTF-8'
LC_CTYPE 'en_US.UTF-8' TEMPLATE template0;
CREATE DATABASE
\x\l frogge*
Expanded display is on.
List of databases
-[ RECORD 1 ]-----+-----
Name          | frogge
Owner          | frogge
Encoding       | SQL_ASCII
Locale Provider | libc
Collate        | C
Ctype          | C
ICU Locale     |
ICU Rules      |
Access privileges |
-[ RECORD 2 ]-----+-----
Name          | frogge_new
Owner         | frogge
```

```

Encoding          | UTF8
Locale Provider   | libc
Collate           | en_US.UTF-8
Ctype             | en_US.UTF-8
ICU Locale        |
ICU Rules         |
Access privileges |

```

Когда мы создаем нашу схему в новой базе данных, строки в Unicode работают предсказуемо:

```

INSERT INTO support.tickets (content, status) VALUES
('Good evening, I would like to return my last order please.', 20),
('Καλησπέρα, θα ήθελα να επιστρέψω την τελευταία μου παραγγελία παρακαλώ.', 20),
('こんにちは、前回の注文を返品したいのですがお願いします。', 20);
INSERT 0 3
SELECT id, content FROM support.tickets;
 id | content
-----+-----
  1 | Good evening, I would like to return my last order please.
  2 | Καλησπέρα, θα ήθελα να επιστρέψω την τελευταία μου παραγγελία παρακαλώ.
  3 | こんにちは、前回の注文を返品したいのですがお願いします。
(3 rows)

```

Кодировка клиента

Я уже упоминал, что это кодировки для сервера. Однако PostgreSQL также позволяет выбрать кодировку для клиента (например, с помощью `SET client_encoding`), и он автоматически выполняет любое преобразование набора символов между сервером и клиентом за вас (если комбинация кодировок поддерживается; см. документацию по адресу <https://www.postgresql.org/docs/current/multibyte.html#MULTIBYTE-CONVERSIONS-SUPPORTED>). Подразумевается, что это позволяет вам использовать Postgres с устаревшими приложениями, которые работают не с Unicode, а с другими кодировками.

Однако вам нужно убедиться, что вы записываете данные на сервер, используя правильную кодировку. Например, наш скрипт в Кейсе 5.1 не будет работать с базой данных UTF-8, если предварительно не перекодировать текст в UTF-8.

Подводя итог, скажем, что `SQL_ASCII` — это не кодировка базы данных, это скорее особый режим, который означает «неизвестная/непроверяемая однобайтовая кодировка». При `SQL_ASCII` не выполняется преобразование или проверка кодировки, и в результате в одной таблице легко получить смесь разных фактических кодировок без точного знания, в какой кодировке были записаны конкретные значения, и надежно восстановить такой исходный текст будет невозможно. Если вы не взаимодействуете с устаревшей системой и не знаете наверняка, какой в ней набор символов, для вас самый безопасный вариант — всегда использовать

UTF8. Всегда проверяйте, в какой кодировке вы создаете базу данных. Если вы переносите базу данных, конвертируйте строковые данные в UTF8. Наконец, внимательно следите за правилами сортировки (collation) — они могут влиять на классификацию символов и порядок сортировки¹.

ПРИМЕЧАНИЕ Подробнее о поддержке правил сортировки в PostgreSQL см. здесь: <https://www.postgresql.org/docs/current/collation.html>

5.2. CREATE RULE

PostgreSQL предлагает мощную систему правил, которая позволяет переписывать и изменять выполнение запросов. Это расширение Postgres поверх стандарта SQL, которое позволяет выполнять действия, например, «когда мы выполняем UPDATE таблицы a, также надо выполнять INSERT в таблицу b» или «когда мы выполняем SELECT из таблицы x, вместо этого выполнить SELECT из таблицы y».

ПРИМЕЧАНИЕ Такая система правил эксклюзивна для PostgreSQL. Она не связана с CREATE RULE в других базах данных.

У Frogge Emporium возникла проблема с тем, что сотрудники вставляют данные в таблицу заказов вручную, а не через пользовательский интерфейс администрирования продаж, из-за этого не выполняется проверка остатков.

Чтобы найти «хулиганов», в компании установили правило, которое регистрирует ручные вставки и записывает, кто это сделал, когда и с какой строкой. Сначала подготовим текущую таблицу audit.audit_log для поддержки ручных вставок, добавив новый столбец manual_insert_id:

```
ALTER TABLE audit.audit_log
ADD COLUMN manual_insert_id bigint;
```

¹ В современных установках PostgreSQL все чаще отдают предпочтение правилам сортировки, предоставляемым провайдером ICU, потому что он дает более предсказуемое и Unicode-корректное поведение, что особенно важно для многоязычных данных и для сохранения одинакового поведения БД в операциях сравнения/сортировки, например при переносе базы на новый сервер под управлением другой версии ОС с иными системными локалями, тогда как libc опирается на локали ОС и может менять поведение при смене дистрибутива, версии glibc или набора локалей. При этом в обоих случаях обновления ICU/libc способны изменить версию колляции (collation version), что требует особого внимания: индексы, зависящие от колляции, могут потребовать перестройки (REINDEX). Практическое правило такое: для «человекочитаемых» текстов (имена, адреса, описания) используйте ICU-колляции (например, ru-RU/ru-RU-x-icu), а для технических идентификаторов, кодов, логинов и ключей — C.UTF-8 (или быстрые встроенные колляции), чтобы получить максимальную предсказуемость и производительность. — *Примеч. науч. ред.*

Теперь создадим само правило, которое будет записывать, кто вставил данные в таблицу заказов вручную:

```
CREATE OR REPLACE RULE log_order_insertions AS
  ON INSERT TO erp.order_groups
  DO ALSO
    INSERT INTO audit.audit_log (what, who, manual_insert_id, tstamp)
    VALUES ('Manual order insertion',
            CURRENT_ROLE::text, NEW.id, clock_timestamp());
```

Протестируем правило: вставим строку в `order_groups` и сразу ее выберем:

```
\x
Expanded display is on.
INSERT INTO erp.order_groups (status, placed_at, updated_at, customer)
  VALUES ('Placed', clock_timestamp(), clock_timestamp(), 135);
SELECT * FROM erp.order_groups ORDER BY placed_at DESC FETCH FIRST ROW ONLY;
INSERT 0 1
-[ RECORD 1 ]-----
id           | 500001
status       | Placed
placed_at    | 2024-06-04 21:21:57.542003+01
updated_at   | 2024-06-04 21:21:57.542003+01
customer     | 135
```

Мы видим, что вставили строку с `id 500001`. Посмотрим, что мы залогировали в таблице `audit_log`:

```
TABLE audit.audit_log;
-[ RECORD 1 ]-----
what      | Manual order insertion
who       | frogge
id        | 500002
tstamp    | 2024-06-04 21:21:57.52818+01
```

Как это произошло? Мы видим в логге, что была вставлена строка с `id 500002`, хотя точно знаем, что вставляли строку с `id 500001`.

Дело в том, что мы выполнили запрос `INSERT`, который автоматически выбрал `nextval()` из последовательности `order_groups_id_seq` и присвоил значение `id`, затем правило переписало запрос, и тот фактически стал двумя запросами:

```
INSERT INTO erp.order_groups (id, status, placed_at, updated_at, customer)
  VALUES (nextval('erp.order_groups_id_seq'), 'Placed', clock_timestamp(),
          clock_timestamp(), 135);
INSERT INTO audit.audit_log (what, who, manual_insert_id, tstamp)
  VALUES ('Manual order insertion',
          CURRENT_ROLE::text, nextval('erp.order_groups_id_seq'),
          clock_timestamp());
```

Таким образом, мы видим, что правило сработало не так, как мы ожидали, потому что оно изменило наш SQL с непреднамеренным побочным эффектом — инкрементированием последовательности `order_groups_id_seq`. На это также указывала подсказка — разный временной штамп, который мы увидели в `tstamp` таблицы `audit_log`. Мы можем извлечь из этого два урока: во-первых, подобные побочные эффекты могут быть опасны, особенно если, к примеру, правило выполняет `DELETE`, во-вторых, применение волатильных (`VOLATILE`) функций с правилами рискованно, поскольку они могут выполняться несколько раз. В таких ситуациях лучше использовать `TRIGGER`, потому что его поведение хорошо изучено и предсказуемо.

ПРИМЕЧАНИЕ Функции PostgreSQL помечаются одной из категорий изменчивости (volatility): `VOLATILE`¹, `STABLE` или `IMMUTABLE`. Функции `VOLATILE` могут делать что угодно, включая модификацию базы данных, тогда как функции `STABLE` и `IMMUTABLE` не могут модифицировать базу данных. В то же время функции `STABLE` должны возвращать одинаковые результаты для одинаковых аргументов для всех строк в пределах одного оператора. Функции `IMMUTABLE` должны всегда возвращать одинаковые результаты для одинаковых аргументов. Объявление правильной категории волатильности крайне важно для повышения скорости обработки запросов через оптимизатор Postgres.

Рассмотрим еще одну особенность использования `RULE`. Предположим, Frogge хочет предотвратить обновление заказов старше одного года. Наиболее частая ошибка и заблуждение в данной задаче — это то, что «старше одного года» не равно «прошлый год и старше».

Сначала обновим наши тестовые данные (так как они стареют и могут не удовлетворять условиям), обновим 30 % `updated_at`, так что в этих 30 % также был разброс даты обновления от 3 до 12 месяцев назад от текущей даты.

```
UPDATE erp.order_groups
SET updated_at = now() - (interval '3 months' + random() * interval '9 months')
WHERE updated_at < now() - interval '1 year'
AND random() < 0.3;
```

Теперь непосредственно создадим само правило, запрещающее обновлять заказы старше 1 года:

```
CREATE OR REPLACE RULE dont_update_old_orders AS
  ON UPDATE TO erp.order_groups
  WHERE old.updated_at < now() - INTERVAL '1 year'
  DO INSTEAD NOTHING;
```

¹ **VOLATILE** — это самая изменчивая категория: функция может вернуть разный результат при каждом вызове, даже в рамках одного SQL-запроса, и/или может иметь побочные эффекты. Поэтому PostgreSQL рассматривает ее как «нельзя кешировать/предвычислять, надо вызывать заново при каждом месте, где она встречается». — *Примеч. науч. ред.*

Найдем для подтверждения корректности текущих данных хотя бы 1 заказ старше года:

```
SELECT *
FROM erp.order_groups
TABLESAMPLE BERNOULLI (10)
WHERE updated_at < now() - INTERVAL '1 year' LIMIT 1;
-[ RECORD 1 ]-----
id          | 651
status      | Fulfilled
placed_at   | 2023-01-23 15:36:48.270475+00
updated_at  | 2023-01-23 15:36:48.270475+00
customer    | 88
```

Хорошо, теперь внимательно посмотрим, как созданное правило повлияет на наш запрос обновления данных старше 10 месяцев (специально выбран интервал, который в том числе захватывает данные, не подпадающие под условие запрета). В начале и в конце теста пересчитаем количество завершенных заказов старше 1 года.

```
SELECT count (*) FROM erp.order_groups
WHERE updated_at < now() - INTERVAL '1 year';
count
-----
24687
UPDATE erp.order_groups SET status = 'Cancelled'
WHERE updated_at < now() - interval '10 months';
UPDATE 2208
```

А запрос-то выполнялся...

```
SELECT count (*) FROM erp.order_groups
WHERE updated_at < now() - INTERVAL '1 year';
count
-----
24687
```

При этом количество заказов старше года осталось неизменным, то есть правило работает корректно. Очень важно понимать, что RULE не «запрещают» и не блокируют выполнение запроса. Мы увидели, что запрос действительно выполнялся и затронул все строки заказов старше 10 месяцев, но при этом не старше года. То есть запрос был выполнен, но с добавленным условием `updated_at < now() - INTERVAL '1 year'`, которое отражено в плане:

```
QUERY PLAN
-----
Update on order_groups (cost=0.00..9753.65 rows=0 width=0) (actual
time=61.876..61.876 rows=0 loops=1)
-> Seq Scan on order_groups (cost=0.00..9753.65 rows=45462 width=10)
(actual time=46.175..49.918 rows=3206 loops=1)
    Filter: ((updated_at < (now() - '10 mons'::interval)) AND ((updated_at
< (now() - '1 year'::interval)) IS NOT TRUE))
```

У системы правил (RULE) есть и другие подводные камни. Например, ничто не мешает вам создавать циклические правила:

- правило 1: при вставке в *x* вместо этого вставлять в *y*;
- правило 2: при вставке в *y* вместо этого вставлять в *x*.

Справедливости ради, правила действительно нужны в одном случае: чтобы разрешить запись в представления (VIEW). Вы можете создавать правила, определяющие действия ON INSERT, ON UPDATE или ON DELETE для представлений, которые фактически выполняются над базовыми таблицами, стоящими за этим представлением.

Безусловно, самая большая проблема системы правил — она сложна для понимания, а следовательно, это благодатная почва для ошибок. Правила не применяют условную логику, **а лишь переписывают SQL** для модификации запросов и добавления дополнительных действий.

Итог простой: не используйте правила! В большинстве случаев они не будут делать то, что вы от них ожидаете, поэтому вместо них вам пригодятся триггеры. Правила — это внутренний механизм, нужный Postgres для реализации представлений, и пользователям лучше его не трогать.

5.3. Реляционный JSON

Добавление типов json и jsonb для хранения документов в PostgreSQL открыло целый спектр возможностей NoSQL, в том числе для комбинирования реляционных и бессхемных баз данных.

Сейчас, когда JS-языки распространены повсеместно, вполне естественно, что разработчики тяготеют к JSON, поскольку это нативный и хорошо им знакомый формат данных. Однако мощь PostgreSQL, позволяющая комбинировать возможности SQL и NoSQL, может стать недостатком, если эти возможности комбинировать неправильно.

В мире реляционных СУБД есть известный антипаттерн Entity–Attribute–Value (EAV, «сущность–атрибут–значение»), также известный как «открытая схема» (*open schema*). Этот паттерн позволяет эффективно хранить наборы данных с разреженными атрибутами (то есть не каждая сущность или объект имеют одинаковые атрибуты, даже если они относятся к одному типу объекта) или разреженными значениями (много значений NULL). Параллели с NoSQL тут очевидны. Однако EAV считается антипаттерном в мире RDBMS, потому что он требует более сложных запросов для извлечения данных, и часто из-за этого запросы пишутся неправильно или работают плохо.

Польза JSON для решения похожих задач, например для приема больших объемов данных, где заранее неизвестны атрибуты или их заполнение под вопросом, не вызывает сомнений. Тем не менее легко увлечься и начать использовать JSON даже для данных, которые вполне подходят под реляционную модель. Хороший маркер того, что вы свернули не туда, — когда вы начинаете задумываться об использовании SQL JOIN для извлечения данных, хранящихся в JSON: ведь NoSQL или бессхемные форматы как раз и предназначены для того, чтобы избавиться от джойнов!

Я предпочитаю называть этот антипаттерн *реляционным JSON (relational JSON)*. Давайте рассмотрим примеры. Пользователь создал таблицы для хранения данных о клиентских счетах и продажах, где для максимальной гибкости каждая строка целиком находится внутри JSON-поля:

```
CREATE TABLE test.accounts (
    json_account jsonb NOT NULL
);

CREATE TABLE test.sales (
    json_sale jsonb NOT NULL
);
```

Данные о клиентах, которые попадают в `accounts.json_account`, выглядят так:

```
{
  "id": 52101,
  "name": "Freddie",
  "balance": 1500.37,
  "migrated_account": {
    "system": "legacy",
    "migrated_on": "2023-12-20"
  }
}

{
  "id": 8101,
  "name": "Emory Trenneman",
  "opened_on": "March 12, 2019",
  "balance": 2530.00
}
```

А записи о продажах в `sales.json_sale` выглядят примерно так:

```
{
  "id": 133045,
  "account_id": 565,
  "timestamp": "2024-01-30 12:54:10",
  "point_of_sale": 311,
  "amount": 35.99
}
```

```
{
  "id": 133046,
  "amount": 2138,
  "account_id": 8101,
  "timestamp": "2024-01-30 12:55:46+0 UTC",
  "point_of_sale": 2
}
```

Для опытного пользователя баз данных некоторые проблемы здесь сразу же бросаются в глаза. Например, база не сможет обеспечить ссылочную целостность и даже не сможет гарантировать, что значения имеют корректный тип или попадают в допустимые диапазоны. Теперь всю эту валидацию и контроль придется делать на стороне приложения.

Пользователь из нашего примера хочет выяснить имена клиентов из старой системы, которые оформили заказ на сумму более 10 000 и на балансе которых менее 20 000. Соответственно, формулируется следующий запрос:

```
SELECT a.json_account ->> 'name' AS "Name"
FROM test.accounts a
JOIN test.sales s
  ON (s.json_sale ->> 'account_id')::int = (a.json_account ->> 'id')::int
WHERE (a.json_account ->> 'balance')::numeric < 20000
AND (s.json_sale ->> 'amount')::numeric > 10000
AND (NOT a.json_account['migrated_account'] ->> 'system' = 'legacy'
  OR a.json_account -> 'migrated_account' IS NULL);
```

Это запутанный и трудночитаемый запрос. Работает ли он? Да, вполне. Однако следует помнить о верном типе индекса: в предыдущем разделе мы видели, как неправильный выбор может приводить к перерасходу дискового пространства или к снижению производительности. Для такого варианта нам придется проиндексировать все содержимое таблиц с помощью больших индексов GIN или GiST. Но даже в этом случае индекс не поможет запросу, потому что план превратится в Merge Join с Materialize и Sort операциями над Seq Scan.

Таким образом, даже если вы создадите GIN-индекс и будете использовать правильные операторы jsonb, сохраняя его преимущества, как показано ниже, вы сможете несколько ускорить запрос, но он все равно останется вложенным циклом вместо правильного реляционного соединения через индекс:

```
SELECT a.json_account ->> 'name' AS "Name"
FROM test.accounts a
JOIN test.sales s
  ON (s.json_sale -> 'account_id') @> (a.json_account -> 'id')
WHERE (a.json_account -> 'balance')::numeric < 20000
AND (s.json_sale ->> 'amount')::numeric > 10000
AND (NOT a.json_account ->
  'migrated_account' @> '{"system": "legacy"}'::jsonb OR NOT
  a.json_account ? 'migrated_account');
```

Вообще, как вариант, можно было бы создать индекс по выражениям на ключах JSON, причем на таких ключах, которые в каждой строке могут быть, а могут и отсутствовать. Если уж вы собираетесь проделать всю эту работу, то почему бы не хранить все в реляционной форме и избежать хлопот с написанием сложных, трудных для восприятия запросов? Используйте `json(b)` по назначению, например для замены EAV или в качестве второго формата хранения для извлечения целого объекта, и не втискивайте в него реляционные данные и запросы.

5.4. Повсеместное использование UUID

В PostgreSQL есть интересный и часто некорректно употребляемый тип данных (data type) UUID (также известный в других системах как GUID), который представляет собой последовательность из 32 шестнадцатеричных цифр для использования в качестве 128-битного идентификатора. Разновидность UUID, которая изначально поддерживалась в PostgreSQL, — это UUIDv4¹, и она выдает такие значения, как `9b287b2a-276d-4ba0-bcc7-b917246169a0`.

Если не вдаваться в детали, тип данных `uuid` кажется идеальным как надежный идентификатор: ничтожно мала вероятность дублирования и огромен диапазон значений. Также важно, что механизм его формирования непрозрачен, поскольку он генерируется непоследовательно, а значит, его сложнее угадать, что делает этот тип данных популярным для безопасности через неясность (*security-via-obscurity*).

Для справки: рассмотрим размеры некоторых типов данных PostgreSQL, как показано в табл. 5.3.

Таблица 5.3. Размеры типов данных в PostgreSQL

Тип данных	Размер в байтах
<code>boolean</code>	1
<code>int</code>	4
<code>bigint</code>	8
<code>timestampz</code>	8
<code>double precision</code>	8
<code>uuid</code>	16
<code>text</code>	1 + длина строки в байтах (+4, если > 127 байт)

¹ В PostgreSQL 18 дополнительно реализован UUIDv7. — *Примеч. науч. ред.*

Создадим таблицу с uuid-идентификаторами и затем проиндексируем их, используя в качестве первичного ключа:

```
\timing
Timing is on.

CREATE TABLE test.tab (id uuid, content text);
CREATE
Time: 11.737 ms
INSERT INTO test.tab
  SELECT gen_random_uuid(), 'test' FROM generate_series(1,100000000);
INSERT 0 100000000
Time: 387838.234 ms (06:27.838)

ALTER TABLE test.tab ADD PRIMARY KEY (id);
ALTER TABLE
Time: 75875.825 ms (01:15.876)
```

Вот столько времени это заняло. Теперь выясним размер нашего ключа и индекса:

```
\x
Expanded display is on.

SELECT pg_column_size(id) FROM test.tab LIMIT 1;
-[ RECORD 1 ]-----
pg_column_size | 16

Time: 2.492 ms
```

```
\di+ test.tab_pkey
List of relations
-[ RECORD 1 ]-----
Schema      | test
Name        | tab_pkey
Type        | index
Owner       | frogge
Table       | tab
Persistence | permanent
Access method | btree
Size        | 3008 MB
Description |
```

Хорошо, теперь, когда у нас получилась отправная точка для расчетов, удалим таблицу и попробуем снова с серийными идентификаторами типа `bigint`:

```
CREATE TABLE test.tab (id bigint, content text);
CREATE TABLE
Time: 5.902 ms

INSERT INTO test.tab SELECT generate_series(1,100000000), 'test';
INSERT 0 100000000
Time: 83137.075 ms (01:23.137)
```

```
ALTER TABLE test.tab ADD PRIMARY KEY (id);
ALTER TABLE
Time: 38123.742 ms (00:38.124)
```

Работает заметно быстрее. Проверим размеры ключа и индекса:

```
SELECT pg_column_size(id) FROM test.tab LIMIT 1;
-[ RECORD 1 ]---
pg_column_size | 8
Time: 1.195 ms
```

```
\di+ test.tab_pkey
List of relations
-[ RECORD 1 ]-+-----
Schema      | test
Name        | tab_pkey
Type        | index
Owner       | frogge
Table       | tab
Persistence | permanent
Access method | btree
Size        | 2142 MB
Description |
```

Представим это в табл. 5.4 для удобного сравнения.

Таблица 5.4. Различия между UUID и серийными bigint

	bigint	uuid	Разница
INSERT	01:23	06:28	на 367 % медленнее
CREATE INDEX	00:38	01:16	на 100 % медленнее
Размер индекса	2142 Мб	3008 Мб	на 40 % больше

Такие результаты, конечно, ожидаемы: UUIDv4 больше, чем **bigint**, и требует генерации следующего значения случайным образом (а с действительно хорошей случайностью компьютеры справляются не идеально, и это имеет стоимость). Большое количество UUID в таблице может также «выталкивать» остальные столбцы за пределы строки, из-за чего данные начнут чаще уходить в *TOAST*, и в целом все станет менее эффективно. Если система требует серьезной оптимизации производительности, эти моменты определенно стоит учитывать.

В большинстве случаев вам даже не нужен такой большой диапазон идентификаторов, потому что количество строк, которые вам нужно хранить, будет намного меньше доступного пространства UUID. Даже **bigint** в PostgreSQL с доступными 18 квинтиллионами (и даже чуть больше) значений может предоставить более чем достаточный диапазон для ваших задач.

Терминология: TOAST

TOAST (The Oversized-Attribute Storage Technique) — это механизм PostgreSQL для хранения больших значений внутри столбцов в условиях, когда размер страницы всего 8 КБ, а отдельные строки не могут «перетекать» на следующую страницу. TOAST поддерживает встроенное (in-line) сжатие значений, а также хранение вне строки (out-of-line) — в отдельной связанной таблице.

Более подробное описание TOAST можно прочитать в документации: <https://www.postgresql.org/docs/current/storage-toast.html>.

Не стоит сразу же прибегать к UUID как к хорошему варианту для идентификаторов. Это требует компромиссов, которые могут слишком сильно повлиять на приложение.

В каком же сценарии UUID можно считать полезным? Например, при использовании PostgreSQL 18 UUIDv7 может служить идентификатором для операций INSERT из нескольких узлов распределенной системы (поскольку не требуется координация между узлами относительно идентификаторов); такой идентификатор генерируется быстро и без коллизий.

СОВЕТ Snowflake ID и UUIDv7 кодируют в себе точные временные метки и могут быть отсортированы по времени. Кроме того, Snowflake кодирует специфичный для машины ID и локальный порядковый номер, что позволяет лучше обнаруживать коллизии и контролировать распределенные системы на более высоком уровне.

5.5. Самописная multi-master репликация

Люди в ИТ постоянно задают мне вопросы: «Могу ли я писать в одну и ту же файловую систему, но на нескольких дисках?», «Могу ли я писать в одну и ту же базу данных, но на нескольких серверах?» и т. д. По сути, эти вопросы означают, что люди хотят иметь общее хранилище, но так, чтобы у него была высокая доступность. Когда я объясняю, что без компромиссов совместить оба требования нельзя, они начинают жаловаться, что они теряют производительность из-за ожидания блокировок или обеспечения консистентности. Это фундаментальная проблема, и, к сожалению, многие не понимают: вы можете получить только два из трех свойств ниже:

- согласованность (consistency);
- доступность (availability);
- устойчивость к разделению (partition tolerance).

И вдобавок, приходится выбирать одно из двух: консистентность или низкую задержку (low latency).

Теоремы CAP/PACELC

В 1998 году Эрик Брюэр (Eric Brewer) предположил, что распределенные системы могут одновременно обеспечить только две из трех следующих гарантий: согласованность (**C**onsistency — чтение возвращает самые свежие данные или ошибку), доступность (**A**vailability — каждый запрос получает ответ, но без гарантии, что это самые свежие данные) и устойчивость к разделению (**P**artition tolerance — гарантия работы СУБД, несмотря на потерю или задержку сообщений по сети). Позже это было формально доказано, и теперь известно как теорема **CAP**.

В 2010 году Дэниел Абади (Daniel Abadi) пошел еще дальше и заявил, что при разделении базы (**P**artition) возможна или доступность (**A**), или согласованность (**C**); в противном случае (**E**lse) приходится выбирать: либо высокая задержка (**L**atency), либо потеря согласованности (**C**onsistency). Эта теорема значительно актуальнее для нашего обсуждения СУБД, и в частности для PostgreSQL, поскольку мы ожидаем, что он в основном будет работать онлайн. Теорема известна как **PACELC**, и она была официально доказана.

Главный вывод этих теорем в том, что при построении распределенных систем неизбежны компромиссы.

Репликация в PostgreSQL реализована по архитектуре shared-nothing (без общего хранилища), что позволяет обеспечить избыточность и высокую доступность. Следовательно, сейчас единственный разумный способ настроить multi-master кластер (то есть кластер баз данных с несколькими основными узлами) — установить соединения для репликации между несколькими независимыми инстансами СУБД, работающими на разных узлах.

PostgreSQL предлагает замечательную возможность — встроенную логическую репликацию. Для PostgreSQL также существует расширение pglogical, которое дает дополнительные функции по сравнению со встроенной логической репликацией. Поэтому наши друзья из Frogge Emporium подумали: почему бы не настроить однонаправленную логическую репликацию в обе стороны, чтобы сделать ее двунаправленной, то есть получить bi-directional или даже multi-master.

Настроить это в обе стороны действительно несложно, если следовать руководству; мы не будем углубляться в детали (тем более что я бы не хотел, чтобы вы этим реально пользовались), но давайте посмотрим, что происходит дальше.

Первая проблема связана с концепцией *replication origins*, то есть с тем, из какой базы данных или узла изначально пришла реплицируемая транзакция. Важно понимать, что логическая репликация PostgreSQL читает изменения из WAL на

стороне издателя, декодирует их и передает подписчику как логический поток изменений, а не как «сырой» WAL. Если при настройке публикаций и подписок в схеме с двусторонней или многосвязной репликацией не отсекают изменения, пришедшие с других узлов (не учитывать `origin`), то узел начнет повторно «реплицировать обратно» собственные изменения, которые ранее были получены от соседей и применены локально. В результате изменения будут пинг-понгом ходить по кластеру, образуя петлю обратной связи: узлы выполняют лишние операции записи, генерируют лишний локальный WAL, и со временем это может привести к разрастанию `pg_wal/`архивов и заполнению диска.

ПРИМЕЧАНИЕ Write-Ahead Log (WAL, журнал предзаписи) — это журнал транзакций PostgreSQL, который нужен для целостности данных. Подробнее о WAL можно узнать здесь: <https://www.postgresql.org/docs/current/wal-intro.html>.

После того как эта проблема изменений при двунаправленной логической репликации будет решена с помощью применения параметров `origin=NONE` для встроенной логической репликации или `forward_origins='{'}` для `pglogical`, попробуем настроить двунаправленную multi-master репликацию между хостами `alpha` и `beta`, которые уже могут соединяться друг с другом, и заполним таблицу тестовыми данными.

Чтобы настроить логическую репликацию, необходимо учитывать такие параметры, как `wal_level=logical`, `listen_addresses`, и аутентификацию на основе хостов (Host-Based Authentication, HBA) согласно документации (<https://www.postgresql.org/docs/current/logical-replication.html>).

На хосте `alpha` мы создаем нашу публикацию и таблицу:

```
CREATE PUBLICATION multi_alpha FOR ALL TABLES;
CREATE PUBLICATION

CREATE TABLE support.tickets (
    id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    content text,
    status smallint,
    opened_at timestampz DEFAULT CURRENT_TIMESTAMP NOT NULL,
    closed_at timestampz
);
CREATE TABLE

INSERT INTO support.tickets (content) VALUES ('first'), ('second');
INSERT 0 2
SELECT id, content FROM support.tickets;
 id | content
----+-----
  1 | first
  2 | second
(2 rows)
```

Отлично, это то, что нужно (см. рис. 5.1).

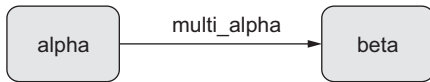


Рис. 5.1. Однонаправленная логическая репликация

Теперь перейдем на хост **beta**, создадим ту же схему, подпишемся на **alpha** и также создадим публикацию:

```

CREATE TABLE support.tickets (
  id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  content text,
  status smallint,
  opened_at timestampz DEFAULT CURRENT_TIMESTAMP NOT NULL,
  closed_at timestampz
);
CREATE TABLE

CREATE SUBSCRIPTION multi_alpha
  CONNECTION 'host=alpha dbname=frogge'
  PUBLICATION multi_alpha
  WITH (origin=NONE);
NOTICE: created replication slot "multi_alpha" on publisher
CREATE SUBSCRIPTION

SELECT id, content FROM support.tickets;
 id | content
----+-----
  1 | first
  2 | second
(2 rows)
CREATE PUBLICATION multi_beta FOR ALL TABLES;
CREATE PUBLICATION
  
```

На рис. 5.2 показана получившаяся после настройки схема.

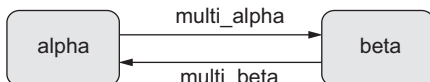


Рис. 5.2. Двухнаправленная логическая репликация

Теперь осталось подписать **alpha** на **beta**, и у нас будет двухнаправленная репликация. На **alpha**:

```
CREATE SUBSCRIPTION multi_beta
    CONNECTION 'host=beta dbname=frogge'
    PUBLICATION multi_beta
    WITH (origin=NONE);
WARNING:
subscription "multi_beta" requested copy_data with origin = NONE
➔but might copy data that had a different origin
DETAIL:
The subscription being created subscribes to a publication
➔("multi_beta") that contains tables that are written to by other
➔subscriptions.
HINT:
Verify that initial data copied from the publisher tables did not
➔come from other origins.
NOTICE:
created replication slot "multi_beta" on publisher
CREATE SUBSCRIPTION
```

Так... Точно все хорошо? Postgres предупреждает, что из-за игнорирования других источников (origins) он не может быть полностью уверен, что копируемые строки действительно были сформированы на (originated on) beta. Вставим строку на alpha:

```
INSERT INTO support.tickets (content) VALUES ('third');
INSERT 0 1
```

```
SELECT id, content FROM support.tickets;
 id | content
----+-----
  1 | first
  2 | second
  3 | third
(3 rows)
```

Убедимся, что она дошла до beta:

```
SELECT id, content FROM support.tickets;
 id | content
----+-----
  1 | first
  2 | second
  3 | third
(3 rows)
```

Теперь у нас установлена двунаправленная логическая репликация, и мы можем писать на оба узла, поэтому также вставим строку на beta:

```
INSERT INTO support.tickets (content) VALUES ('fourth');
ERROR:duplicate key value violates unique constraint "tickets_pkey"
DETAIL: Key (id)=(1) already exists.
```

Ой. Что случилось? Похоже, что **beta** попытался вставить строку, но генератор последовательности первичного ключа все еще находился на значении 1, а у нас уже был этот ключ в таблице, потому что он был вставлен узлом **alpha**. Для логической репликации обычно требуется первичный ключ или другое уникальное ограничение, и реплики, как правило, получают значения из генераторов последовательностей. Необходимо убедиться, что ни один узел не использует повторно номера последовательностей с другого узла, потому что, как мы только что видели, это создает конфликт данных.

Синхронизация последовательностей — это сложная задача; вам придется либо поддерживать синхронизацию по всему кластеру при каждой записи (для производительности это будет катастрофой), либо предварительно выделять диапазоны каждому узлу, что при ручном управлении создаст большую административную нагрузку. Важные вопросы: как разделить диапазоны, что происходит, когда эти диапазоны заканчиваются, и как работать с добавлением новых узлов?

Посмотрим, что произойдет, когда мы попробуем еще один способ, о котором вы наверняка знаете, — изменить схему базы данных. Меняем на **beta** имя столбца с `opened_at` на `logged_at`:

```
ALTER TABLE support.tickets RENAME opened_at TO logged_at;
ALTER TABLE
```

Тем временем какой-то ничего не подозревающий пользователь на **alpha** вставляет строку:

```
INSERT INTO support.tickets (content) VALUES ('fourth');
INSERT 0 1
```

```
SELECT id, content, opened_at FROM support.tickets WHERE content='fourth';
 id | content |          opened_at
-----+-----+-----
  4 | fourth | 2024-02-01 19:24:09.162218+00
(1 row)
```

Пока все хорошо. Проверим на **beta**:

```
SELECT id, content, logged_at FROM support.tickets WHERE content='fourth';
 id | content | logged_at
-----+-----+-----
(0 rows)
```

Строка не прошла. Проверка лога PostgreSQL на **beta** показывает:

```
Feb 1 19:24:10 beta postgres: ERROR: logical replication target
➔relation "support.tickets" is missing replicated column: "opened_at"
Feb 1 19:24:10 beta postgres: CONTEXT: processing remote data
➔for replication origin "pg_16596" during message type "INSERT" in
➔transaction 879, finished at 0/1DB9628
```



```
Feb 1 19:24:10 beta postgres: LOG: background worker "logical
➔replication worker" (PID 22139) exited with exit code 1
```

Таким образом, допустив даже на короткое время несовместимость схем, мы не только нарушили согласованность данных в нашем кластере, но и запустили цикл сбоев/перезапусков в рабочем процессе логической репликации PostgreSQL (crash/restart loop).

Без согласованной репликации DDL (Data Definition Language) не получится создать собственный распределенный multi-master кластер Postgres. Для изменения схемы необходимо координировать процессы. Это все усложняет, поскольку управление должно быть предельно выверенным, чтобы, с одной стороны, избежать конфликтов и сбоев репликации, а с другой — проверить соответствие требованиям к времени бесперебойной работы.

В истории PostgreSQL были разные попытки реализовать multi-master решения, но они работали на более высоких уровнях, чем сама база данных. Например, в 2007 году выпустили два инструмента репликации: Londiste на Python и Bucardo на (жуть какая-то!) Perl.

Если исходить из практичности, для enterprise-сценариев, где важна полная производительность PostgreSQL, обычно хочется использовать нативное multi-master решение, выполняющее репликацию на уровне базы. Самое продвинутое и проверенное решение — EnterpriseDB Postgres Distributed (PGD), изначально разработанное как BDR от 2ndQuadrant; на момент написания это проприетарное ПО. Новый претендент — Spock от pgEdge (source-available), также форкнутый из кода pglogical 2ndQuadrant.¹

В любом случае вам лучше сесть и подумать, действительно ли вам нужна multi-master репликация. Возможность приложения записывать «в любую ноду» звучит заманчиво, но в реальности это сопровождается массой неочевидных проблем, которые могут появиться как со стороны администрирования базы данных, так и со стороны приложения — оно должно видеть multi-master архитектуру, и далеко не все приложения поддерживают multi-master репликацию. Единственные реальные сценарии, которые пришли мне в голову, это:

- *предельно высокая доступность* — благодаря мгновенно доступной избыточности и возможности выполнять минорные и мажорные обновления версий без простоя;
- *геораспределение данных с минимальной задержкой* — потому что чтение и запись в базу данных происходят локально для каждого физического места.

¹ На российском рынке присутствует СУБД от компании «Постгрес Профессиональный» — СУБД POSTGRES PRO ENTERPRISE, которая также предоставляет возможность настройки multi-master кластера. — *Примеч. науч. ред.*

5.6. Самописные распределенные системы

Распределенная система не обязательно должна быть multi-master. Я считаю, что обычные возможности кластеризации PostgreSQL через потоковую и логическую репликацию уже делают систему распределенной. Следовательно, упомянутые ранее теоремы CAP и PACELC все еще применимы.

Могу вас заверить, что попытки написать собственный механизм репликации или, что еще хуже, использовать внешние системы Extract/Transform/Load (ETL и ELT) или инструменты Change Data Capture (CDC) не дадут ни более богатого функционала, ни лучшей производительности по сравнению с нативной репликацией PostgreSQL.

Но даже при использовании этих надежных и производительных методов все равно можно нарваться на неприятности. В этом примере у нас уже настроена потоковая репликация между основным хостом **alpha** и резервным хостом **beta**. На **alpha** мы создаем таблицу и вставляем данные:

```
CREATE TABLE test.accounts (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    balance numeric NOT NULL,
    updated_at timestamp with time zone DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE

INSERT INTO test.accounts (balance) VALUES (100);
TABLE test.accounts;
INSERT 0 1
 id | balance |          updated_at
-----+-----+-----
  1 |    100 | 2024-01-17 19:08:31.0284+00
(1 row)
```

Очень распространенный паттерн, который неоднократно встречается в продакшен-системах: приложение читает с одной из реплик для чтения (резервного сервера PostgreSQL) и пишет на основной сервер (primary). Мы специально для эксперимента имитируем минутную задержку репликации между двумя узлами, установив параметр конфигурации `recovery_min_apply_delay` в значение 60 000 (миллисекунд) на резервном сервере **beta**. Теперь приложение считывает баланс счета с резервного сервера, чтобы списать средства, и обновляет его на основном сервере (поскольку резервный сервер доступен только для чтения).

На резервном сервере **beta** мы читаем баланс:

```
SELECT balance FROM test.accounts WHERE id=1;
 balance
-----
    100
(1 row)
```

На основном узле **alpha** мы обновляем баланс:

```
UPDATE test.accounts SET balance=90, updated_at=now() WHERE id=1;
TABLE test.accounts;
UPDATE 1
 id | balance |          updated_at
-----+-----+-----
  1 |      90 | 2024-02-17 19:09:03.172174+00
(1 row)
```

Тем временем другая часть приложения пытается прочесть баланс (опять же с резервного сервера), чтобы начислить уже на основном сервере ежегодный бонус на счет. На резервном сервере **beta**:

```
SELECT balance FROM test.accounts WHERE id=1;
 balance
-----
      100
(1 row)
```

Приложение ошибочно полагает, что баланс счета равен 100, и начисляет бонус +50 к этой сумме на **alpha**:

```
UPDATE test.accounts SET balance=150, updated_at=now() WHERE id=1;
TABLE test.accounts;
UPDATE 1
 id | balance |          updated_at
-----+-----+-----
  1 |      150 | 2024-01-17 19:09:34.442339+00
(1 row)
```

В итоге бизнес потерял 10 у. е., а клиент получил неожиданный подарок из-за нарушения свойства согласованности (**C**onsistency) требований ACID (**A**tomicity, **C**onsistency, **I**solation, **D**urability) для баз данных.

Если PostgreSQL соответствует ACID, то что же пошло не так? Дело в том, что сервер PostgreSQL действительно гарантирует и обеспечивает безопасность ACID в рамках одного экземпляра/узла, но в этом примере приложение пыталось выполнить *атомарную (или неделимую) (atomic (or indivisible))* транзакцию между двумя разными экземплярами базы данных, то есть по факту выполнялась *распределенная транзакция (distributed transaction)*.

Распределенные транзакции устроены не так просто. Для такого примера нужна двухфазная транзакция по протоколу Two-Phase Commit (2PC) с использованием **PREPARE TRANSACTION** и связанных с ним команд. Однако такие транзакции сложны и трудно применимы в системах с задержками или лагом репликации, поскольку они должны быть кратковременными. Кроме того, их пришлось бы запускать исключительно на основном сервере, что сводит на нет плюсы от реплик для

чтения. Вдобавок они держат блокировки на объектах базы, что может привести к задержкам в приложении, а транзакции нередко могут оставаться «висящими», то есть незавершенными (например, когда компонент приложения аварийно завершает работу или перезапускается и теряет свое состояние).

Проведем еще один эксперимент, на основном сервере alpha почистим таблицу accounts и затем вставим в нее 100 000 000 строк:

```
TRUNCATE test.accounts;
TRUNCATE TABLE
```

```
INSERT INTO test.accounts (balance) SELECT generate_series(1,100000000);
INSERT 0 100000000
```

Теперь запустим длительный «отчетный» запрос (который делает две агрегации по одной и той же таблице) на резервном сервере beta:

```
SELECT sum(balance) FROM test.accounts
UNION
SELECT avg(balance) FROM test.accounts;
```

Тем временем пользователи продолжают работать с базой данных и на alpha вносятся некоторые изменения:

```
DELETE FROM test.accounts WHERE id BETWEEN 40000001 AND 40000020;
DELETE 20
```

Плюс сама система по расписанию почистила данные. Воспроизведем это вручную через VACUUM:

```
VACUUM test.accounts;
VACUUM
```

Эти команды удалили несколько счетов и очистили данные. В то же время на beta наш запрос завершился ошибкой:

```
SELECT sum(balance) FROM test.accounts
UNION SELECT avg(balance)
FROM test.accounts;
ERROR:
DETAIL: canceling statement due to conflict with recovery
User query might have needed to see row versions that must be removed.
```

Почему это произошло? Это явление известно как (*распределенная*) *аномалия сериализации (distributed serialization anomaly)*, или нарушение свойства *Isolation* из требований ACID. Конкретно — это конфликт *чтения-записи (read-write conflict)*, или *неповторяемое чтение (unrepeatable read)*, когда одна транзакция прочитала данные, а вторая транзакция перезаписала часть этих данных до того, как первая успела завершиться.

В большинстве случаев, вероятно, это можно предотвратить через синхронную репликацию, но это очень дорогой компромисс с точки зрения производительности, который можно рассматривать только тогда, когда завышенные требования к корректности и согласованности превосходят все остальное. Даже на самых быстрых сетях синхронная репликация данных между узлами PostgreSQL создает задержку, которая станет узким местом производительности базы данных гораздо раньше, чем сеть или репликация PostgreSQL достигнут своих пределов¹. Другая крайность — согласиться на *чтение с возможным отставанием от актуальных данных (stale read)* в обмен на низкую задержку.

Такие условия применимы и к multi-master системам. Проблема заключается в том, что логические «транзакции» на стороне приложения фактически размазаны между несколькими узлами базы данных. Если приложение выполняет атомарную операцию, используя один узел зараз, это нормально, поскольку возможные конфликты решаются обычными методами управления внутри одного узла СУБД. Однако когда приложение пытается выбрать несколько узлов в рамках одной транзакции, нам требуются строгие требования к согласованности и изоляции, что возвращает нас к RACELC.

Использовать реплики для чтения, чтобы распределить часть нагрузки, — это нормально и совершенно допустимо, но не внутри атомарной операции, вызываемой приложением, которая в данном случае становится распределенной транзакцией. Нет хорошего или простого способа избежать подобных проблем без надежного глобального менеджера транзакций. Глобальные менеджеры транзакций неизбежно становятся узкими местами для производительности и могут стать единой точкой отказа.

В заключение скажем, что создание распределенных баз данных — непростая задача из-за ограничений, которые накладывают законы физики. Индустрия долгое время боролась за создание подобных решений, которые находят компромиссы в конкурирующих приоритетах. Если у вас не слишком узкий сценарий использования (например, одна таблица, установленные правила защиты и правила управления конфликтами), вы вряд ли сможете с ходу сделать хорошую распределенную систему. Затраты на ее реализацию или устранение ошибок, скорее всего, многократно увеличат объем проекта по сравнению с прошлой системой.

¹ Причем чем больше будет мелких и частых транзакций, тем быстрее и значительнее проявится падение производительности. Показательное исследование на эту тему провела компания CYBERTEC в 2020 году, выступив с этой темой на pgconf.de в 2022-м. — *Примеч. науч. ред.*

Итоги

- `SQL_ASCII` — это не столько кодировка символов, сколько ее отсутствие. Если вы не хотите рисковать необратимым смещением кодировок и вам нравится легкость автоматического преобразования наборов символов, используйте для своей базы данных UTF-8.
- Поведение правил `RULE` в PostgreSQL никак не связано с поведением правил в других СУБД. В PostgreSQL они сложны для понимания и существуют в основном как внутренний механизм Postgres. В большинстве случаев они ведут себя не так, как ожидает пользователь, и вместо них лучше использовать триггеры (`TRIGGER`).
- `JSON(B)` с реляционными шаблонами доступа работают хуже — данные сложнее читать, а запросы выполняются медленнее. Лучше не смешивать парадигмы SQL и NoSQL и использовать каждую по назначению.
- `UUID (uuid)` занимают больше места для хранения, чем даже `bigint`, и их индексирование менее эффективно. Они могут обеспечить диапазон значений, который попросту избыточен для вашего сценария, при этом для гарантии уникальности может быть достаточно целочисленного индекса.
- Двунаправленная или `multi-master`, репликация сложнее, чем кажется. Для того чтобы ее практически применить в продакшене, есть очень длинный список предварительных условий, соображений и нюансов. Если все же ваш кейс действительно оправдывает `multi-master`, лучше не изобретать велосипед, а использовать готовое и стабильное `multi-master` решение, разработчики которого уже подумали над возможными проблемами.
- Все предыдущие пункты применимы и к самописным распределенным системам. Не стоит браться за их создание, если у нас не очень узконаправленный сценарий использования. Важно учитывать возможность аномалий при сериализации распределенных транзакций и необходимость значительных компромиссов согласно теоремам CAP и PACELC.

Нерекомендуемые практики, снижающие производительность

В этой главе

- ✓ Как использовать конфигурацию PostgreSQL по умолчанию в продакшене
- ✓ Как управлять лимитами подключений и их жизненным циклом
- ✓ Почему транзакции выполняются слишком долго
- ✓ Какие будут последствия, если отключить автоочистку `autovacuum` даже на короткое время
- ✓ Что такое явная блокировка и какие риски с ней связаны
- ✓ Как влияет на производительность слишком малое или слишком большое число индексов

Дочитав до главы 6, вы уже поняли, что PostgreSQL — мощнейшая СУБД: гибкая база данных, подходящая для множества сценариев и способная работать очень быстро, если «правильно с ней разговаривать». Под «правильно» я имею в виду понимать, как она устроена и как работает, чтобы избегать таких паттернов кода, конфигураций и эксплуатации, которые приводят к неоптимальным результатам. Теперь самое время разобрать несколько действительно распространенных

способов «убить» производительность в Postgres. Для кого-то они покажутся очевидными, для кого-то — неожиданными, так что разобрать их явно стоит.

6.1. Конфигурация по умолчанию в продакшене

Начнем с самого начала: вы только что установили PostgreSQL на ваш новенький облачный вычислительный инстанс — он оптимизирован для работы с памятью, в нем 16 ГБ ОЗУ, и он полностью готов к работе с СУБД. Вы решаете восстановить вашу базу данных в этом инстансе из последнего дампа.

Давайте сгенерируем достаточно крупную базу данных с помощью хорошо знакомой `pgbench` — встроенной утилиты для тестирования производительности. Сначала создадим базу из командной строки от имени `postgres`:

```
createdb pgbench
```

Теперь у нас есть пустая база данных для заполнения:

```
pgbench -i -s 1000 pgbench
dropping old tables...
NOTICE: table "pgbench_accounts" does not exist, skipping
NOTICE: table "pgbench_branches" does not exist, skipping
NOTICE: table "pgbench_history" does not exist, skipping
NOTICE: table "pgbench_tellers" does not exist, skipping
creating tables...
generating data (client-side)...
100000000 of 100000000 tuples (100%) done (elapsed 548.92 s, remaining
➔0.00 s)
vacuuming...
creating primary keys...
done in 761.00 s (drop tables 0.00 s, create tables 0.03 s, client-side
➔generate 549.59 s, vacuum 0.74 s, primary keys 210.64 s).
```

ПРИМЕЧАНИЕ Время выполнения здесь не так важно, поскольку мы просто запускаем код, чтобы получить некоторые данные для дампа и восстановления и посмотреть, сколько времени займет восстановление.

После заполнения мы делаем дампы:

```
pg_dump pgbench > /tmp/pgbench.dump
ls -lah /tmp/pgbench.dump
-rw-rw-r-- 1 postgres postgres 9.9G May 12 19:16 /tmp/pgbench.dump
```

И теперь у нас есть набор данных в `pgbench.dump`. Очистим и восстановим базу:

```
dropdb pgbench
createdb pgbench
```


На этот раз мы хотим засечь, сколько времени займет восстановление нашего дампа:

```
time psql pgbench < /tmp/pgbench.dump
SET
SET
SET
SET
SET
set_config
-----

(1 row)

SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
CREATE TABLE
ALTER TABLE
COPY 100000000
COPY 1000
COPY 0
COPY 10000
ALTER TABLE
ALTER TABLE
ALTER TABLE

real    44m15.279s
user    0m13.108s
sys     0m20.280s
```

Хорошо, мы получили некоторый временной показатель. Хорош он или нет?

Давайте изменим несколько параметров конфигурации и попробуем снова — посмотрим, сможем ли улучшить результат. В `postgresql.conf` поменяем значения некоторых параметров на:

```
shared_buffers = 3GB ← Default: 128MB
work_mem = 128MB ← Default: 4MB
maintenance_work_mem = 512MB ← Default: 64MB
```

Нам нужно перезапустить PostgreSQL, чтобы изменение `shared_buffers` вступило в силу. После этого посмотрим, сможем ли мы добиться улучшения:

```
dropdb pgbench
createdb pgbench
time psql pgbench < /tmp/pgbench.dump
```

...

```
real    26m17.669s
user    0m12.229s
sys     0m19.520s
```

Разница значительна: теперь процесс занимает менее 60 % времени, которое потребовалось со значениями по умолчанию в `postgresql.conf`.

Но разве конфигурация по умолчанию не состоит из наиболее часто используемых значений или тех, которые эмпирически дали наилучшие результаты?

Нет. В отличие от других «раздутых» систем баз данных, с которыми вы можете быть знакомы, основная идея, стоящая за «дефолтной» конфигурацией PostgreSQL, заключается в том, чтобы иметь базовый набор настроек, с которыми СУБД будет «просто работать» на любой машине (в разумных пределах — 486SX/25 1994 года с 24 МБ ОЗУ будет лишь чуть-чуть ниже минимальных требований). Как видно в документации, выделение памяти по умолчанию для таких конфигурационных параметров, как `shared_buffers`, составляет 128 МБ, а `work_mem` — всего 4 МБ. То есть конфигурация по умолчанию нацелена в первую очередь на то, чтобы Postgres вообще запускался и работал почти на любом, даже очень скромном железе.

То, что мы только что сделали, — это не столько «тюнинг базы», сколько базовая инженерная настройка при тестировании/подготовке PostgreSQL к продакшену. Новое значение `shared_buffers` увеличило объем памяти, доступной Postgres для буферов (блоков, читаемых с диска и (или) используемых в операциях), `work_mem` увеличил объем неразделяемой памяти, доступной для отдельных операций, а `maintenance_work_mem` увеличил память, доступную для операций типа `CREATE INDEX` или `VACUUM`.

Разумеется, это не означает, что вы должны выбрать какие-то значения параметров и придерживаться их раз и навсегда, но конфигурации по умолчанию точно будет катастрофически недостаточно для большинства видов рабочих нагрузок в продакшене. Кроме того, существует гораздо большее количество настраиваемых параметров, нежели просто настройки памяти.

При выполнении специальных операций, например массовых загрузок данных, иногда даже полезно на время отойти от «правильных» продакшен-принципов

и настроить систему на максимальную производительность ценой надежности или устойчивости к падениям (потому что в период загрузки данных, выполняемых обычно в окне технического обслуживания, избыточная надежность вам не так уж и важна).

Разумеется, кроме загрузки данных есть множество других, более разнообразных профилей нагрузки, в которых начинают играть роль другие факторы, например конкурентность. Но как мы показали на этом примере и как будет видно в следующих разделах этой главы, систему можно адаптировать под разные сценарии. Просто не рассчитывайте, что конфигурации по умолчанию достаточно для сколько-нибудь реалистичной нагрузки.

6.2. Некорректное распределение памяти

Итак, мы знаем, что не следует использовать PostgreSQL в продакшене с настройками по умолчанию. В частности, в предыдущем разделе мы выделили больше оперативной памяти для некоторых целей, и это помогло повысить производительность.

Запустим базовый стандартный тест `pgbench` на 5 минут (300 секунд) с только что восстановленной базой данных, чтобы посмотреть, какие показатели производительности мы получим:

```
pgbench -U postgres -h test01 -c 100 -j 24 -T 300 pgbench
pgbench (17.0 (Ubuntu 17.0-1.pgdg24.04+1))
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 1000
query mode: simple
number of clients: 100
number of threads: 24
maximum number of tries: 1
duration: 300 s
number of transactions actually processed: 323961
number of failed transactions: 0 (0.000%)
latency average = 92.347 ms
latency stddev = 60.183 ms
initial connection time = 824.391 ms
tps = 1082.342941 (without initial connection time)
```

СОВЕТ Убедитесь, что вы запускаете тесты с другого хоста, чтобы избежать перегрузки сервера базы данных излишней работой. В идеале хост, с которого вы запускаете тесты, должен быть достаточно мощным, чтобы иметь возможность перенасытить сервер базы данных запросами.

В этой системе с выбранными ранее настройками конфигурации мы получаем 1082 транзакции в секунду (TPS) при одновременном подключении 100 клиентов. Дадим нашим запросам максимум ресурсов для безупречного выполнения, поэтому увеличим `shared_buffers` и `work_mem` и посмотрим, сможет ли это еще улучшить производительность:

```
shared_buffers = 12GB
work_mem = 2GB
```

Перезапустим Postgres, чтобы изменения вступили в силу, и перезапустим `pgbench`:

```
pgbench -U postgres -h test01 -c 100 -j 24 -T 300 pgbench
pgbench (17.0 (Ubuntu 17.0-1.pgdg24.04+1))
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 1000
query mode: simple
number of clients: 100
number of threads: 24
maximum number of tries: 1
duration: 300 s
number of transactions actually processed: 319069
number of failed transactions: 0 (0.000%)
latency average = 93.930 ms
latency stddev = 65.995 ms
initial connection time = 297.914 ms
tps = 1062.819169 (without initial connection time)
```

Мы видим, что TPS остался примерно на том же уровне или стал даже чуть хуже. Выходит, мы изменили параметры не в ту сторону!

Теперь выясним, что произойдет, если 1000 клиентов (то есть в 10 раз больше) обратятся к базе данных с этими же настройками. Для этого установим `max_connections = 1000` и перезапустим кластер, чтобы изменения вступили в силу. Смоделируем тяжелую нагрузку:

```
pgbench -U postgres -h test01 -c 1000 -j 24 -T 300 -P 10 pgbench
pgbench (17.0 (Ubuntu 17.0-1.pgdg24.04+1))
starting vacuum...end.
progress: 10.0 s, 147.8 tps, lat 2532.664 ms stddev 1511.880, 0 failed
^C
```

После первых 10 секунд тест зависает, и мы видим, что что-то не так. Мы отменяем выполнение с помощью `Ctrl+C`, просматриваем системные логи на сервере СУБД и обнаруживаем следующее:

```
kernel: Out of memory: Killed process 15255 (postgres) total-vm:1674628kB,
➔anon-rss:3208kB, file-rss:5760kB, shmem-rss:0kB, UID:107 pgtables:152kB
➔oom_score_adj:0
kernel: systemd-journald[436]: /dev/kmsg buffer overrun, some messages lost.
kernel: systemd-journald[436]: Under memory pressure, flushing caches.
systemd[1]: postgresql@17-main.service: A process of this unit has been
➔killed by the OOM killer.
systemd[1]: postgresql@17-main.service: Failed with result 'oom-kill'.
systemd[1]: postgresql@17-main.service: Consumed 1min 18.570s CPU time.
```

ПРЕДУПРЕЖДЕНИЕ Запуск тяжелой нагрузки, как в предыдущем стресс-тесте, особенно при неправильно настроенных параметрах выделения памяти в PostgreSQL, может привести к падению системы непредсказуемым образом.

OOM killer (из сообщения об ошибке) — это механизм Linux Out-of-Memory (OOM) killer, который запускается ядром Linux, чтобы освободить память, когда в системе ее становится критически мало. Он выбирает процессы для завершения на основе данных собственного механизма ООМ-скоринга (можно посмотреть в `/proc/<PID>/oom_score`). И существует вполне реальная вероятность, что он может «убить» не текущий бэкэнд-процесс PostgreSQL, который занимает много оперативной памяти, а какой-то другой критичный процесс или приложение в системе, которое случайно использует много памяти.

Но почему у нас закончилась память? Помните, мы выделили 12 ГБ для `shared_buffers` и 2 ГБ для `work_mem`? Это означает, что мы потенциально выделили 2 ГБ оперативной памяти для *каждого узла* плана выполнения для *каждого выполняющегося запроса*. Учтите, что каждая из 1000 подключенных сессий может выполнять запрос, использующий множество узлов. Очевидно, что у нас недостаточно доступной оперативной памяти для этого, и тут нужно понимать, что ситуация может быстро выйти из-под контроля, если у нас есть сложные запросы со многими узлами плана и (или) с параллельным их выполнением.

ПРИМЕЧАНИЕ Важно не допускать срабатывания OOM Killer, потому что когда бэкэнд-процесс принудительно закрывается, PostgreSQL автоматически завершает все другие активные бэкэнд-процессы, чтобы избежать работы с разделяемой памятью, которая потенциально могла быть повреждена. Затем Postgres переходит в режим аварийного восстановления (будучи автоматически перезапущенным `systemd` или другим менеджером), что, в свою очередь, может спровоцировать переключение на реплику (если у вас кластер) или создать еще какие-нибудь неприятности.

Следует выделять ровно столько `work_mem`, сколько требуется для запросов, и не больше. Как и многое в этой книге, это полностью зависит от профиля рабочей нагрузки. Если вы выберете небольшое значение `work_mem`, то более сложные

запросы типа тех, что включают сортировку и агрегацию, могут исчерпать оперативную память для своих операций и перейти во временные файлы на диске, что значительно медленнее, чем выполнение операции в памяти. Однако мы воочию убедились, что выбор слишком большого размера `work_mem` ставит под угрозу запросы, потому что система может исчерпать память и «убить» их выполнение или, что еще хуже, сам PostgreSQL или какой-то другой критичный процесс. К счастью, `work_mem` можно устанавливать через SET на уровне сессии, поэтому мы можем настроить систему с некоторыми разумными значениями по умолчанию и увеличивать `work_mem` только для тех запросов, которым действительно нужна дополнительная память.

Аналогично существуют рекомендации вроде «идеальный `shared_buffers` = 25 % доступной оперативной памяти» и прочие формулы. Следовать им вслепую также не стоит, потому что правильное значение опять зависит от нагрузки: в нагрузках OLTP (Online Transaction Processing) можно выиграть благодаря большому объему разделяемой памяти, куда поместится используемая часть данных, тогда как в нагрузках OLAP (Online Analytical Processing) те же большие размеры `shared_buffers` могут быть потрачены впустую, поскольку такие операции поступают в каждый буфер только один раз. У вас могут быть даже смешанные нагрузки, и тогда потребуется тщательно искать баланс распределения памяти. В нашем случае выбранное значение для параметра `shared_buffers` явно избыточно, поскольку оно вообще не помогает с нашей рабочей нагрузкой.

ПРЕДУПРЕЖДЕНИЕ Будьте внимательны и всегда указывайте единицы измерения после изменяемых значений. Если вы этого не сделаете, то, например, единицей измерения по умолчанию для `shared_buffers` будет считаться сам буфер (8192 байта), и вы укажете, сколько буферов выделить; следовательно, существует очень большая разница между 1024MB и 1024 (буферами), что составляет всего 8 МБ.

Ситуация становится еще запутаннее, поскольку в других местах, таких как `work_mem` и `maintenance_work_mem`, Postgres использует в качестве единицы измерения KB; для `min_wal_size` и `max_wal_size` — MB; и т. д.

Итак, крайне важно корректно распределить память, в противном случае вы оказываетесь между молотом и наковальней, поскольку значения по умолчанию, скорее всего, будут недостаточными для вашей рабочей нагрузки, но значительно их увеличив, вы получите еще более серьезные проблемы с производительностью и стабильностью. Как правило, вам придется действовать методом проб и ошибок: только специфика именно ваших рабочих нагрузок и будет определять значения для ваших параметров. Также напомним: никогда не экспериментируйте в продакшене — делайте это только в тестовой или предпрод-среде с похожими данными и похожими рабочими нагрузками.

6.3. Завышенное число разрешенных соединений

Довольно часто можно встретить конфигурации PostgreSQL с большими лимитами разрешенных подключений¹ к базе данных, такими как `max_connections = 2000`. По моему опыту, многие разработчики, похоже, верят, что чем больше соединений разрешается обрабатывать базе данных, тем лучше для приложения.

Проведем эксперимент, чтобы посмотреть, как PostgreSQL справится с большим числом клиентских соединений. Как мы видели в предыдущем разделе, наш подход к распределению памяти оказался опасным, поэтому сейчас мы выберем более осмысленные значения для нашего сервера. Выделив небольшой объем для `shared_buffers` и особенно для `work_mem`, поскольку этот параметр связан с числом выполняемых запросов, мы снижаем вероятность исчерпания памяти. Мы также изменим лимит разрешенных соединений на 2000:

```
shared_buffers = 1GB
work_mem = 4MB
max_connections = 2000
```

После перезапуска Postgres, который необходим для вступления наших изменений в силу, мы снова запускаем `pgbench` и проводим серию тестов с разным числом клиентских подключений, начиная с 2000:

```
pgbench -U postgres -h test01 -c 2000 -j 24 -T 300 pgbench
pgbench (17.0 (Ubuntu 17.0-1.pgdg24.04+1))
starting vacuum...end.
transaction type: <builtin: TPC-B (sort of)>
scaling factor: 1000
query mode: simple
number of clients: 2000
number of threads: 24
maximum number of tries: 1
duration: 300 s
number of transactions actually processed: 84835
number of failed transactions: 0 (0.000%)
latency average = 5535.267 ms
latency stddev = 13696.921 ms
initial connection time = 69772.527 ms
tps = 352.083626 (without initial connection time)
```

¹ В контексте PostgreSQL термины «подключение» и «соединение» (например, клиентские подключения или клиентские соединения, максимальное количество разрешенных соединений или подключений) в большинстве случаев взаимозаменяемы. При этом в отдельных случаях, когда речь идет о процессе подключения, то есть когда соединение еще не установлено, это «подключение», в случаях, когда речь идет уже о состоявшемся подключении, это «соединение». — *Примеч. науч. ред.*

Далее пробуем 1500, 1000, 500 клиентских подключений и так далее. Результаты показаны в табл. 6.1.

Таблица 6.1. Результаты тестирования, показывающие TPS и задержку при различном числе подключений

Количество клиентских подключений	TPS	Средняя задержка, мс
2000	352,1	5535,267
1500	506,8	2869,986
1000	586,1	1694,616
500	628,4	794,688
250	635,7	392,554
125	556,9	224,411
63	613,0	102,758
32	572,3	55,907
16	301,8	53,009

Построим график на основе этой таблицы и изучим результаты. Мы видим, что TPS (transactions per second, число транзакций в секунду) примерно выходит на плато между 1500 и 32 подключениями, а на крайних значениях — 2000 и 16 клиентских подключений — он заметно ниже.

Однако задержка демонстрирует явную логарифмическую зависимость, снижаясь с очень плохого показателя в 5,5 секунды до стабильного около 53 миллисекунд при 32 подключениях и ниже, как показано на рис. 6.1.

О чем же это нам говорит? То, что мы вызвали чрезмерную конкурентность за ресурсы (масштабируя клиентские подключения до 2000 подключений), которая оказалась слишком высокой для нашего относительно скромного сервера. При обслуживании 2000 подключений сервер испытывал значительные трудности, что отражается в низком TPS и высокой задержке. Тем не менее не кажется ли, что он лучше справлялся с нагрузкой между 1500 и 32 подключениями?

Да, на первый взгляд кажется, что сервер справлялся лучше, однако именно значения задержки полностью показывают, насколько он был перегружен: восстановление наступает лишь при 32 (и менее) клиентских подключениях. Удерживать плато по TPS ему удалось, только пожертвовав задержкой, то есть поддерживать высокий TPS при большом числе одновременно активных соединений он смог лишь ценой резкого роста latency, что, кстати, также позволяет

частично скрыть влияние сетевой задержки, поскольку множественные соединения делают ее менее значимой для общей пропускной способности. Низкое значение TPS при 16 подключениях просто означает, что мы не давали серверу столько работы, сколько он мог бы выполнить.

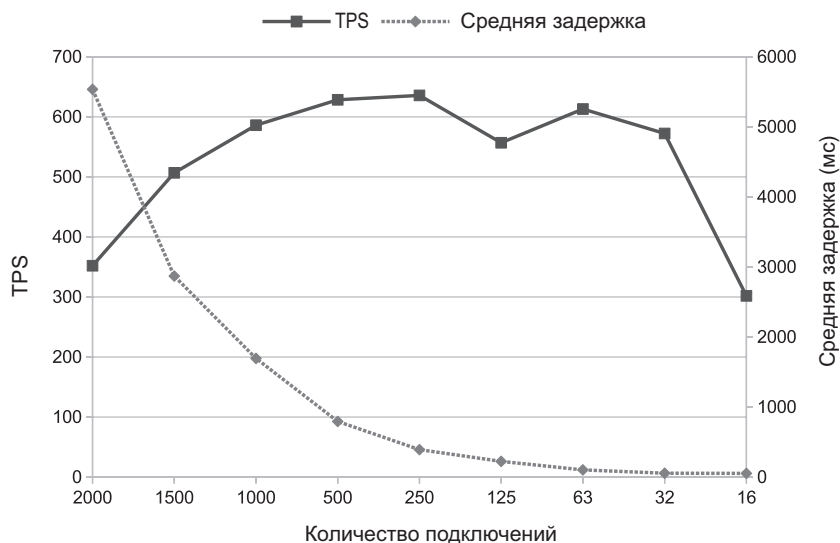


Рис. 6.1. График результатов тестирования TPS и задержки при различном числе подключений

Давайте отвлечемся от многозадачности: компьютер может делать только то, на что у него хватает ресурсов. Если вы требуете выполнить 2000 задач одновременно, но у вас нет 2000 ядер CPU, вы должны понимать, что требование нереалистично. Есть эмпирическое правило: число подключений в PostgreSQL не должно превышать более чем в четыре раза число ядер. Но опять же, не следуйте моим рекомендациям слепо, поскольку оптимальный коэффициент у вас обязательно будет зависеть от конкретной рабочей нагрузки.

Конкуренция за легковесные блокировки

Доступ к одним и тем же объектам из множества подключений может приводить к большому числу ожиданий из-за «легких блокировок» (Lightweight Locks, LWLock; в других СУБД их часто называют latches — «защелки»). LWLock'и — это внутренний служебный механизм синхронизации PostgreSQL: они защищают внутренние структуры в общей памяти, обеспечивая корректный параллельный доступ (в том числе в реализации MVCC — Multi-Version Concurrency Control) при чтении и запи-

си. При большом числе одновременно активных сессий отдельный LWLock может стать объектом интенсивной конкуренции. При этом LWLock'и не гарантируют «честную» очередь на захват, поэтому при высокой конкуренции процессы могут неоднократно опережать друг друга, что увеличивает время ожидания и снижает общую пропускную способность.

Отдельная разновидность проблем с LWLock'ами на самом деле возникает уже в момент установления нового подключения: запуск backend-процесса и его регистрация в общих структурах сервера (ProcArray/lock manager и др.) сами по себе требуют захвата нескольких LWLock'ов, поэтому даже сравнительно небольшое число одновременно — именно одновременно — устанавливаемых подключений способно сформировать «шторм подключений»¹, который вызовет заметную деградацию сервера еще до выполнения прикладных запросов. Хороший индикатор повышенной конкурентности — частое появление ожиданий с `wait_event_type = 'LWLock'` в представлении мониторинга `pg_stat_activity`.

Чтобы у PostgreSQL оставался запас по производительности и он лучше переживал рост нагрузки или кратковременные пики нагрузки, стоит сознательно пожертвовать задержкой на стороне приложения ради общей производительности сервера базы данных. Вместо того чтобы немедленно открывать столько клиентских подключений, сколько требует приложение, ограничьте это число и заставьте запросы приложения становиться в очередь (надеюсь, ненадолго) за доступом к подключению. То есть приложение должно уметь ожидать, поэтому вам, возможно, придется увеличить на его стороне тайм-ауты.

Если у вас не получится ограничить число подключений для приложения, хорошим решением будет использование внешнего пулера соединений. Пулер соединений, такой как PgBouncer², можно элегантно поставить между приложением и базой данных. Он способен принимать тысячи подключений от приложения, но пропускать к PostgreSQL только заданное их число, заставляя остальных ждать своей очереди. Когда вы попробуете такой подход и будете отслеживать активность базы данных через `pg_stat_activity`, вы, возможно, увидите, что

¹ В PostgreSQL 18 устранено узкое место в подсистеме управления блокировками (LWLock:LockManager): переработана внутренняя организация таблицы блокировок и механизм распределения доступа к ней между процессами, благодаря чему существенно снижена конкуренция за блокировки при большом количестве таблиц и индексов, а также запросов и при высокой параллельности подключений. — *Примеч. науч. ред.*

² PgBouncer — зрелое и широко используемое решение, однако он работает в однопоточном режиме и не поддерживает многопоточную обработку клиентских подключений, что может ограничивать масштабирование по CPU при высокой нагрузке. В качестве альтернатив существуют другие пулеры подключений с многопоточной архитектурой, например Odyssey. — *Примеч. науч. ред.*

реальная конкурентность (то есть сколько задач приложение фактически пытается выполнить одновременно внутри базы данных, а не сколько подключений оно хочет открыть) гораздо ниже, чем вы ожидали. В жизни встречались приложения с 5000 соединений к базе данных, выполняющих одновременно лишь около 15–30 активных задач в базе данных.

Что касается повышенной конкурентности, помните, что PostgreSQL — мультипроцессная система с моделью «один процесс на пользователя». Иными словами, каждый клиент подключается к бэкэнд-процессу, и все это координируется процессом-супервизором `postmaster`. Межпроцессное взаимодействие (Inter-Process Communication, IPC) в PostgreSQL реализовано через семафоры и разделяемую память.

Отсюда следует, что если в нашей системе запущено слишком много процессов, они влекут накладные расходы операционной системы и создают риск чрезмерно частого переключения контекста CPU. Естественно, это снижает производительность сервера.

ПРИМЕЧАНИЕ *Трешинг процессов (process thrashing)* происходит, когда процессов слишком много для их эффективного планирования, и каждый получает крошечный квант времени выполнения, продвигаясь очень медленно, поскольку система тратит больше времени на переключение контекстов, чем на выполнение самих процессов. Кроме того, повышенное использование памяти, вызванное слишком большим числом бэкэнд-процессов, может привести к трешингу памяти, то есть к активному вытеснению ее в своп и обратно, а мы помним, что доступ к диску значительно медленнее, чем к RAM.

Итак, вам может казаться, что распараллеливание сделает работу приложения быстрее, на практике вы можете, наоборот, ухудшить производительность, просто «утопив» сервер БД в слишком большом объеме **одновременной** работы. Сознательно корректируйте его работу или реализуйте задержку на стороне приложения, чтобы сохранить производительность сервера БД. Это звучит контринтуитивно, но, скорее всего, это не замедлит выполнение запросов, потому что неперегруженный сервер выполняет их быстрее!

6.4. Простаивающие соединения

Теперь рассмотрим, почему вам не следует держать «висящие» простаивающие соединения (idle connections) на сервере PostgreSQL.

6.4.1. Для начала: что такое MVCC?

Сейчас полезно кратко обсудить транзакции PostgreSQL и управление конкурентным доступом. Чтобы гарантировать высокий конкурентный доступ и одновременно высокую производительность, PostgreSQL использует механизмы

MVCC (Multi-Version Concurrency Control), а не блокировки. При этом чтение из базы данных никогда не требует ожидания. Запись в БД не блокирует чтение, а чтение не блокирует запись. Как уже понятно из названия, система MVCC работает таким образом: каждая операция записи создает новую версию кортежа (строки). В итоге мы получаем *snapshot isolation* (*изоляцию снимков*): каждая транзакция имеет согласованное на конкретный момент представление (или снимок) данных. Postgres использует временные метки и идентификаторы транзакций (XID) для отслеживания активности, что делает возможным согласование представлений, то есть определение того, какие кортежи видны из каждой транзакции. Каждая транзакция получает от менеджера транзакций Transaction Manager снимок, который содержит самую раннюю все еще активную транзакцию, первую еще не назначенную транзакцию и список всех активных. Получить свежий снимок можно с помощью функции `pg_current_snapshot()`. На рис. 6.2 показано, как выглядит временная шкала транзакций PostgreSQL.

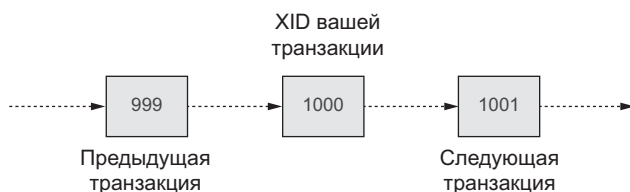


Рис. 6.2. Временная шкала транзакций PostgreSQL, показывающая линейную последовательность XID

Находясь внутри транзакции 1000, вы можете или не можете видеть результаты транзакций 999 и 1001, в зависимости от того, были ли они зафиксированы или отменены.

6.4.2. Проблема с простаивающими соединениями

Можно предположить, что простаивающие (*idle*) сессии безвредны. В конце концов, они не выполняют в базе никаких задач, поэтому логично предположить, что они не влияют на производительность СУБД.

Довольно часто можно увидеть системы в продакшене с тысячами простаивающих соединений, особенно когда используются пулеры соединений из библиотек или фреймворков языков программирования (то есть те, что предоставляет сам язык или *framework*, а не отдельный промежуточный/серверный пулер, например `pgbouncer`). Эти «пулеры» держат в базе данных тысячи сессий **постоянно** открытыми, чтобы иметь пул соединений, доступных для передачи приложению. Обычно самые злостные нарушители — Java-фреймворки, в которых

разработчики настолько отдалены от базы данных через абстракции, что обычно понятия не имеют о том, как работает база данных или что им следует делать (или не следует).

Но мы знаем, что механизм MVCC в PostgreSQL поддерживает снимки для всех транзакций. Это означает, что каждый раз, когда транзакция запрашивает снимок перед началом работы, Postgres должен проверить все открытые соединения, чтобы собрать информацию для снимка. Это следствие необходимости проверять, что делает каждая сессия: мы не знаем, что они простаивают, пока не проверим. Как лаконично выразился опытный разработчик PostgreSQL Том Лейн в ответе на письмо в списке рассылки `pgsql-performance`, вычислительные затраты на создание снимка пропорциональны общему количеству открытых соединений.

В результате даже когда сессия простаивает, она все равно потребляет системные ресурсы и замедляет обработку последующих подключений — такую ситуацию называют *snapshot contention* (*конкуренцией за снимки*).

Откуда берутся простаивающие (*idle*) соединения? Причины могут быть разными:

- просто плохой дизайн приложения: случайно, «на всякий случай», открывают много соединений или осознанно — например, когда очередь задач и «состояние» приложения (прогресс, блокировки, статусы) для множества компонентов и воркеров хранятся в одной общей базе данных;
- попытка сохранить пропускную способность (как мы разбирали выше), увеличивая число одновременных запросов через новые соединения;
- стремление справляться с внезапными всплесками запросов (хотя предпочтительнее поставить пулер между приложением и БД).

При этом, когда мы говорим про простаивающие (*idle*) соединения, это не значит, что они совсем не используются, — достаточно того, что они простаивают большую часть времени (например, приложение редко делает запросы). Тем не менее в реальности такие соединения пагубно влияют на производительность активных соединений, и этот эффект можно измерить.

Оптимизации обработки соединений, в частности процесса получения снимка для новых подключений, появились в PostgreSQL 14, и это решило большую часть проблемы узкого места (хотя и не полностью). Даже после выполненных улучшений получение снимков для множества входящих соединений по-прежнему требует значительных вычислительных ресурсов. Кроме того, даже с учетом выполненных оптимизаций PostgreSQL вам все равно следует помнить, что это многопроцессная система, — каждое подключение требует отдельного бэкэнд-процесса, и таким образом, при тысячах дополнительных процессов

накладные расходы операционной системы для учета операций, планирования и межпроцессного взаимодействия совсем не нулевые.

В заключение отмечу: учитывайте, что приложение может внезапно решить использовать тысячи этих простаивающих соединений, причем одновременно в пиковых сценариях (например, при открытии рынков). В конце концов, вы же сами предоставили ему такое право. И этот сценарий неизбежно вызовет сбой в работе сервера.

Если ваше приложение (или вам так нужно) постоянно открывает множество соединений, которые потом простаивают, применяйте пулер транзакций, такой как PgBouncer в режиме Transaction Mode. PgBouncer просто передаст неиспользуемые соединения другим транзакциям, которым эти соединения нужнее, и таким образом снизит накладные расходы ОС. Это решение лучше, чем использовать `idle_session_timeout` для того, чтобы автоматически закрыть простаивающие сессии, поскольку последнее может нарушить работу клиентских подключений или приложений, в которых не учитывалась эта специфика.

6.5. Управление длительными транзакциями

Специалисты PostgreSQL неизменно советуют избегать длительных транзакций. Но почему? Прежде всего разграничим активные, но длинные сессии и простаивающие сессии внутри транзакций. Начнем с последних.

6.5.1. Простой в транзакции

Если коротко, то сессия, которая находится в состоянии `idle in transaction`, как сообщается в `pg_stat_activity`, — это та, которая инициировала транзакцию, однако не выполняет активный запрос, при этом не откатывает или не фиксирует транзакцию. Распространенный случай: клиент открывает подключение, выполняет `BEGIN` для начала транзакции, а затем оставляет соединение простаивать. Известно, что плохо написанные приложения иногда используют именно такой паттерн доступа к данным. Многие из нас также видели, как администраторы баз данных или разработчики инициируют подключение, а затем оставляют клиентское соединение открытым, уходя на день (или даже в отпуск). Если они успели начать транзакцию перед уходом, их сессия останется в состоянии `idle in transaction`.

Если вы помните из краткого описания MVCC выше, PostgreSQL определяет видимость кортежей с помощью идентификаторов транзакций, и именно так гарантируется свойство **Isolation** (изоляция) в ACID. Напомню, мы рассматривали нарушения свойства изоляции в разделе 5.6.

Блокировки в PostgreSQL

Концепция блокировок в Postgres несложна: с точки зрения пользователя базы данных, блокировки берутся либо на уровне таблицы (например, `ACCESS SHARE`), либо на уровне строк (например, `SELECT ... FOR UPDATE`). Эти блокировки либо разрешают, либо запрещают доступ другим транзакциям, которые пытаются «использовать» тот же объект (то есть тоже получить на него блокировку). Разрешение или запрет определяется тем, конфликтуют ли типы блокировок между собой. Так, если кто-то пытается взять на таблицу блокировку `ACCESS EXCLUSIVE`, она конфликтует со всеми другими типами блокировок, поэтому никто другой не сможет получить на эту же таблицу, например, блокировку `SHARE`.

Теперь рассмотрим сценарий, когда клиент оставляет сессию в состоянии проста в транзакции:

```
Клиент А
-----
BEGIN;
SELECT * FROM mytable;
(...)
```

Оператор `SELECT` завершается и возвращает соответствующий результат. Клиент А оставляет соединение и транзакцию открытыми.

Теперь другой клиент подключается и пытается добавить столбец в `mytable`. Казалось бы, таблица сейчас не используется, верно? Поэтому клиент должен иметь возможность выполнить DDL-команду `ALTER TABLE mytable`.

Но запрос Клиента В ждет вечно (или, по крайней мере, до завершения транзакции Клиента А).

Клиент А		Клиент В
-----	+	-----
BEGIN;		
SELECT * FROM mytable;		
(...)		
		ALTER TABLE mytable
		ADD description text;
		зависает

Транзакция Клиента А заняла самую безобидную из блокировок — блокировку `ACCESS SHARE` — для чтения таблицы. Несмотря на то что этот режим даже не может блокировать других от чтения или записи в таблицу, он конфликтует с блокировкой `ACCESS EXCLUSIVE`, которую требует оператор `ALTER TABLE ADD (COLUMN)`.

Что еще хуже, пока Клиент В ждет своей очереди, чтобы получить нужную ему блокировку на `mytable`, любая другая транзакция или запрос, которым тоже

нужно обратиться к `mytable`, будут заблокированы, потому что им приходится становиться в очередь за ним!

Клиент А	Клиент В	Клиент С
BEGIN; SELECT * FROM mytable; (...)	ALTER TABLE mytable ADD description text; *зависает*	SELECT 1 FROM mytable; *зависает*

Однако все разблокируется, если Клиент А просто завершит первую транзакцию:

Клиент А	Клиент В	Клиент С
BEGIN; SELECT * FROM mytable; (...)	ALTER TABLE mytable ADD description text;	SELECT 1 FROM mytable;
END;	ALTER TABLE	?column? ----- (0 rows)

ПРИМЕЧАНИЕ `END` — нестандартный синоним `COMMIT` в PostgreSQL, то есть альтернативная форма записи команды фиксации транзакции.

Давайте поговорим о `VACUUM`. Механизм MVCC дает отличную производительность при записи и откате транзакций, но за это приходится платить обслуживанием. В частности, речь идет о `VACUUM` — это операция, которая удаляет мертвые кортежи (старые версии строк), которые уже не нужны, поскольку они больше не видны ни одной транзакции. PostgreSQL периодически выполняет эту работу автоматически — это называется `AUTOVACUUM` (автоочистка). Как и любая другая служебная операция, `AUTOVACUUM` использует блокировки и при необходимости ждет их получения.

Тут важно знать то, что клиентские соединения в состоянии `idle in transaction` (простоя в транзакции) могут мешать запуску автоочистки, если они изменили какие-либо данные или используют уровень изоляции, такой как `REPEATABLE READ` или `SERIALIZABLE`. `VACUUM` также не сможет удалить недавно «умершие» версии

строк, которые должны оставаться видимыми для такой открытой транзакции. Если по какой-то причине долго не выполняется `VACUUM/AUTOVACUUM`, то это может привести к разбуханию (bloat) оставшихся мертвых строк внутри таблицы, что серьезно ухудшит производительность и увеличит использование дискового пространства.

ПРЕДУПРЕЖДЕНИЕ Autovacuum выполняет и другие важные задачи, например «заморозку» (freezing) старых версий строк. Если autovacuum длительное время не в состоянии нормально выполняться, это может привести к переполнению идентификаторов транзакций (transaction ID wraparound) — связанному отказу, который мы обсудим в следующем разделе.

Один из способов не допускать, чтобы соединения зависали в состоянии `idle in transaction`, — изначально правильно спроектировать приложение (легко сказать!). Другой вариант — задать значение параметра `idle_in_transaction_session_timeout`. Он принудительно завершает сессии, которые остаются в состоянии `idle in transaction` дольше заданного числа миллисекунд, тем самым не позволяя им слишком долго удерживать блокировки или неоправданно долго мешать операциям очистки. Однако здесь есть нюанс: если приложению действительно нужно какое-то время выполнять обработку после начала транзакции, прежде чем записать изменения и/или завершить транзакцию, тайм-аут должен быть достаточно большим, чтобы это укладывалось в лимит. Более того, ваше приложение должно быть готовым (уметь корректно обрабатывать) к таким типам разрывов соединений по тайм-ауту.

Подробнее см. документацию PostgreSQL:

<https://www.postgresql.org/docs/current/runtime-config-client.html#GUC-IDLE-IN-TRANSACTION-SESSION-TIMEOUT>.

6.5.2. Длительные запросы

Многое из этого справедливо и для длительно выполняющихся запросов. Помните, что даже если вы явно не начали транзакцию, каждый запрос будет выполняться в рамках своей собственной неявной транзакции.

Каждая из этих транзакций получит снимок базы данных, как упоминалось ранее, и будет удерживать его все время, пока выполняется оператор. То же самое будет верно и на все время, пока открыт курсор.

СОВЕТ Говоря о запросах с длительным временем выполнения, мы имеем в виду запросы, которые действительно выполняются (находятся в состоянии `active`), а не те, что «долго не заканчиваются», потому что, например, стоят в ожидании блокировки. В `pg_stat_activity` нужную длительность удобно смотреть как `now() - query_start` — это время, прошедшее с момента запуска текущего запроса.

Autovacuum пропустит очистку «мертвых» кортежей, если какая-либо транзакция удерживает снимок, полученный до того, как эти кортежи были изменены или удалены. Кроме того, если запросу необходимо получить блокировку, которая конфликтует с autovacuum, то autovacuum добровольно уступит ее и прекратит текущую работу. Очевидно, что если запрос выполняется очень долго, autovacuum может так и не получить возможности запуститься и нормально отработать. Все это опять же приводит к проблемам с разбуханием таблицы и к переполнению идентификаторов транзакций.

Наконец, длительные запросы могут вызывать конфликты запросов в сценариях репликации. Например, если на основном сервере идут интенсивные обновления данных, а на реплике одновременно выполняются SELECT по тем же данным, запросы на реплике могут быть отменены. Отмена запроса происходит из-за конфликта, вызванного необходимостью применения WAL (Write-Ahead Log), который повлияет на выбранные данные.

Длительные транзакции также могут вызывать задержку репликации (replication lag) при использовании синхронной или логической репликации, поскольку изменения реплицируются только после фиксации (COMMIT). Эту проблему можно уменьшить, используя асинхронную потоковую репликацию либо передавая изменения больших транзакций по мере их выполнения при логической репликации. Первое достигается заданием параметру `synchronous_standby_names` пустого значения (") в `postgresql.conf`, что отключает ожидание синхронных реплик, а второе — включением параметра `streaming` при настройке подключения логической репликации с помощью `CREATE SUBSCRIPTION`.

Остановить длительно выполняющиеся запросы относительно безопасно (для базы данных, хотя не обязательно для клиентского приложения) можно с помощью `SELECT pg_cancel_backend(<PID>);`. Однако, как мы уже видели, принудительное завершение бэкенд-процессов, например, через оператор `SIGKILL` и `PID` процесса, — это намного более опасное действие, потому что может перевести весь экземпляр сервера в режим восстановления, если PostgreSQL обнаружит, что один из его backend-процессов был остановлен. Безопаснее использовать функцию `pg_terminate_backend(<PID>)`, но этот метод гораздо менее удобен, чем `pg_cancel_backend()`, поскольку он прерывает транзакцию и принудительно отключает пользователя, что может привести к проблемам в работе приложения.

6.6. Высокая частота транзакций

Возможность обрабатывать транзакции с высокой скоростью, как правило, желательна. Но действительно ли длительно поддерживаемая высокая частота транзакций всегда полезна для вашей базы PostgreSQL?

6.6.1. Переполнение идентификаторов транзакций (XID)

Как мы упоминали ранее, PostgreSQL присваивает идентификатор (XID) каждой транзакции, и эти идентификаторы используются в качестве основы для механизма MVCC. Записи попадают в основную область данных таблицы (*heap*), и каждый кортеж имеет *xmin* (XID транзакции, которая вставила эту версию строки) и *xmax* (XID транзакции, которая ее удалила, если кортеж удаляется). Помните, что обновления (UPDATE) в PostgreSQL — это составная операция из вставки и удаления.

СОВЕТ Текущий идентификатор транзакции можно узнать с помощью `SELECT txid_current();`.

XID — это беззнаковые 32-битные числа, поэтому их диапазон значений — около 4,3 миллиарда.

Диапазон значений XID можно представить как круговое пространство с «горизонтом видимости»: и с точки зрения вашей текущей транзакции около 2,15 миллиарда XID находятся в прошлом, и столько же — в будущем (рис. 6.3).

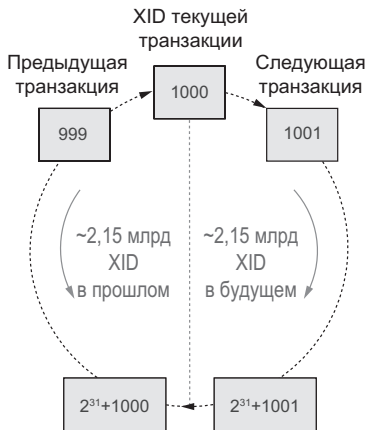


Рис. 6.3. Представление идентификаторов транзакций PostgreSQL как кругового пространства

Представьте, что мы просто «отдалили» картинку с рис. 6.2: то, что выглядело как прямая линия, на самом деле является отрезком окружности. Когда мы перейдем к транзакции 1001, за ней и перед ней все равно будет одинаковое количество миллиардов XID, то есть `txid_current()` напоминает стрелку на барабане в «Поле чудес»!

В предыдущем разделе мы упомянули вид сбоя из-за XID wraparound (*переполнения XID*). Такой сбой происходит, когда вы пытаетесь прочитать очень старый

кортеж, который находится более чем ~2,15 миллиарда XID назад относительно текущей транзакции. Если посмотреть на рис. 6.3, легко понять, почему это опасно: такой XID, с вашей точки зрения, уже попадает в область «будущих» XID и, следовательно, должен считаться невидимым! Таким образом, получается сюрреалистическая картина: мы не знаем, относится кортеж к прошлому или к будущему, что, в свою очередь, грозит несогласованностью или потерей данных. PostgreSQL пытается предотвратить такой wraparound (проблему обрачиваемости и нехватку диапазона значений) и исправить ситуацию любой ценой, запуская непрерывную автоочистку (autovacuum). Когда такое случается, система может существенно замедлиться (вплоть до «ползущего» режима), и избежать этого уже нельзя. Но как не дойти до этой точки?

Мы предотвращаем XID wraparound с помощью приема, называемого *freezing* (заморозка). VACUUM проходит по каждой таблице и помечает очень старые кортежи, которые точно относятся к прошлому, флагом «заморожено» (frozen). Такой флаг указывает, что они очень старые и находятся вне нашего текущего пространства XID — так замороженные кортежи становятся безопасными с точки зрения их xmin. В некотором смысле это возвращение в оборот ранее использованных XID от кортежей.

Единственное, о чем нам теперь нужно позаботиться, — чтобы операция VACUUM FREEZE отработала до наступления XID wraparound, и именно это Postgres спешит сделать. Операция запускается, когда PostgreSQL считает, что вы близки к исчерпанию безопасного диапазона XID (параметр настраивается через autovacuum_freeze_max_age). Как мы и говорили в предыдущем разделе, сессии в состоянии idle in transaction могут удерживать «горизонт» XID и откладывать заморозку старых строк.

6.6.2. Быстрое расходование XID

Пользователи и разработчики часто пользуются преимуществами высокой скорости INSERT/UPDATE в PostgreSQL и просто забрасывают в базу данных огромные объемы данных, не ощущая потери производительности. Однако каждая транзакция записи «использует» XID, и при тяжелых OLTP-нагрузках 2 миллиарда транзакций расходуются очень быстро. Я видел базу данных, которая израсходовала этот объем за неделю! При такой активности следует опасаться того, что скорость расходования XID обгонит скорость автоочистки (и, следовательно, автоматической заморозки) — это снова приближает нас к проблеме XID wraparound.

Таким образом, если у вас есть возможность очень быстро выполнять транзакции, это отнюдь не означает, что вы должны так делать. Как же поступить, когда выяснится, что у вас высокая скорость расходования XID?

Во-первых, посмотрите, вдруг вам подойдет пакетная обработка (`batching`). Действительно ли ваше приложение обязательно должно фиксировать каждую операцию отдельным коммитом и сохранять абсолютную атомарность каждой мелкой записи? Простой факт: использование пакетной фиксации транзакций размером в 1000 транзакций снизит скорость расходования XID приложением в 1000 раз от предыдущей, решая проблему скорости транзакций практически мгновенно. Разработчики также могут попытаться переработать поток входящих данных так, чтобы группировать строки и хранить результаты множества операций в более эффективных структурах.

Другой вариант — повысить эффективность `autovacuum`, настроив соответствующие параметры так, чтобы он работал более агрессивно, что, в свою очередь, сделает заморозку кортежей более своевременной и эффективной.

Изучите настройку `freezing` в документации PostgreSQL по адресу: <https://www.postgresql.org/docs/current/routine-vacuuming.html>.

Небольшой полезный совет по архитектуре: не стоит собирать слишком много данных (которые вы, возможно, никогда не будете использовать) внутри базы данных. Многие организации склонны хранить каждый бит данных, который проходит через их системы, на случай, что он когда-нибудь станет полезным в будущем для анализа или для еще чего-нибудь. Обсудите этот вопрос со стейкхолдерами, объясните им риски и затраты при таком подходе и исключите из поступающего потока те данные, которые точно никто никогда смотреть не будет.

Альтернативное решение для работы с полезными, но очень объемными данными — немедленно переработать или переупаковать их, чтобы хранить только сводки, средние значения или тренды, если в будущем вам будут нужны именно они.

6.7. Отключение `autovacuum/autoanalyze`

Из-за особенностей архитектуры MVCC в PostgreSQL `VACUUM` становится неизбежным злом. По крайней мере, некоторые люди видят в нем зло, потому что воспринимают автоочистку как фоновый процесс, который потребляет ресурсы и будто бы ничего не делает для их прикладной нагрузки.

В результате, когда они узнают, что существует некий неугомонный демон автоочистки, который постоянно запускает `VACUUM` внутри их базы данных, их реакция — попытаться найти способ его отключить. Ведь кажется, что он расходует ресурсы системы, причем нередко в самые неподходящие моменты для повседневной нагрузки.

Таким людям часто кажется, что они правы, когда отключают «демона»; это действительно может приводить к повышению производительности, которая заметна и разработчикам, и конечным пользователям. По крайней мере, на

некоторое время. Чтобы понять, почему это прямой путь к катастрофе, сначала рассмотрим, что делает `VACUUM`.

Мы уже знаем, что `VACUUM` очищает мертвые кортежи из нашей базы данных. Это можно рассматривать как сборку мусора: она освобождает место внутри файлов данных таблицы, поскольку эти кортежи не считаются удаленными окончательно, пока по таблице не пройдет `VACUUM`. Позже Postgres может переиспользовать освобожденное пространство. Далее `VACUUM` делает то же самое для индексов таблицы, выполняя соответствующую очистку. Мы также видели, что он может замораживать старые строки таблицы, чтобы предотвратить `XID wraparound`. Наконец, оператором `VACUUM ANALYZE` мы запускаем операцию `ANALYZE` после очистки, чтобы собрать полезную статистику о содержимом таблицы. Эта статистика помогает планировщику запросов выбирать подходящие планы и учитывать специфику данных в таблице.

Теперь посмотрим, как именно работает `autovacuum`. Название говорит само за себя: он автоматически время от времени «просыпается», проверяет, какие таблицы нуждаются в `vacuum` и/или `analyze`, и затем выполняет `VACUUM` и/или `ANALYZE` для этих таблиц. Благодаря этому процессу мы меньше думаем обо всех нюансах из предыдущего абзаца, поскольку он помогает не раздувать таблицу, оптимизировать планы запросов и снижать риск наступления `XID wraparound`.

Если это еще не очевидно, все перечисленные функции критически важны — настолько, что даже если вы отключите `autovacuum` в конфигурации PostgreSQL, система все равно продолжит следить за ситуациями, близкими к исчерпанию `XID`, и будет готова вмешаться. Суть в том, что, как и с резервными копиями (подробнее об этом в следующей главе), операции обслуживания должны выполняться автоматически и не зависеть от ручного вмешательства.

Пагубный кумулятивный эффект раздувания базы очень быстро перевесит любой краткосрочный прирост в производительности от отключения `autovacuum`. С точки зрения производительности «раздутые» таблицы — это просто дополнительные (фактически мусорные) данные на диске, которые являются бесполезными при сканировании таблиц и индексов, из-за чего запросы выполняются медленнее, чем могли бы.

Без контроля таблицы могут раздуться на диске до таких размеров, что простой `VACUUM` это пространство операционной системе уже не вернет — автоочистка лишь пометит пространство внутри файлов базы данных как незанятое, чтобы PostgreSQL мог использовать его повторно. Для реального освобождения места на диске потребуется запустить другой механизм под названием `VACUUM FULL` — он работает медленно, требует еще больше дискового пространства и крайне разрушителен для возможности совместной работы с таблицей, поскольку берет на указанную таблицу блокировку, фактически исключая любую операцию

с ней. Это означает, что вы никогда не должны вызывать `VACUUM FULL` в текущей таблице, если вы не до конца понимаете последствия.

Кроме того, игнорируя важность сбора статистики для оптимизатора, вы рискуете ухудшить производительность запросов. Оптимизатору необходимо знать характеристики данных, такие как кардинальность, количество уникальных значений и другую статистику, чтобы выбрать наилучший план.

Во многом то же самое относится и к ситуации, когда `autovacuum` оставляет с его очень консервативными настройками по умолчанию: для боевых нагрузок они часто недостаточны. Как мы уже видели в первом разделе этой главы, настройки PostgreSQL по умолчанию слишком низкие для реальных нагрузок. Уменьшение этих параметров еще сильнее снижает эффективность автоочистки и также негативно влияет на раздувание базы и сбор статистики.

Рекомендация здесь, вопреки ожиданиям, заключается в том, чтобы по мере роста интенсивности операций записи в вашей рабочей нагрузке выделять не меньше, а больше ресурсов для `autovacuum` и тем самым *увеличивать* эффективность автоочистки. Вам придется принять тот факт, что часть системных ресурсов будет постоянно выделяться на обслуживание MVCC — на ту самую фоновую работу, которая позволяет базе корректно функционировать под той интенсивной нагрузкой, которую вы на нее возлагаете. Сегодня, пока у PostgreSQL нет дополнительных хранилищ для бэкенда¹, вы не можете избежать `VACUUM`.

Если ваш сервер не справляется одновременно с рабочей нагрузкой и необходимой для нее автоочисткой, вам, вероятно, придется либо рассмотреть вертикальное масштабирование системы (добавить CPU/память/диск и IOPS), либо снизить нагрузку, найдя и оптимизировав самые дорогие запросы, либо пересмотреть архитектуру базы (например, разделить данные, использовать репликацию и т. п.).

СОВЕТ Помните, что параметры `autovacuum` можно переопределять на уровне отдельных таблиц, если общие настройки по умолчанию не подходят для части таблиц.

Параметры конфигурации `autovacuum`, включая те, что позволяют поднять или снизить ее активность через задержки по стоимости (`cost-based delays`) и задержки выполнения, документированы здесь:

<https://www.postgresql.org/docs/17/runtime-config-autovacuum.html>.

¹ Есть разные проекты (расширения для PostgreSQL), которые решают эту проблему, например расширение OrioleDB, реализующее свой собственный «движок» хранения данных для PostgreSQL, в котором нет проблемы с раздуванием таблиц, отсутствует необходимость запуска `vacuum`, а также отсутствует проблема `XID wraparound` из-за использования 64-битного счетчика для `XID`. — *Примеч. науч. ред.*

6.8. Игнорирование возможностей EXPLAIN (ANALYZE)

Вы удивитесь, узнав, как мало людей используют EXPLAIN и EXPLAIN ANALYZE для своих запросов, чтобы понять, как они выполняются базой данных. Возможно, это связано со странным форматом вывода, который многим сперва сложно читать, или с тем, что неясно, что означает, например, `Recheck Cond` в `Bitmap Heap Scan` и где это искать. Однако по сравнению с другими СУБД планы запросов PostgreSQL очень детализированы и хорошо документированы.

Ирония в том, что в большинстве случаев, когда пользователи жалуются на запрос, который выполняется слишком долго, EXPLAIN действительно может помочь выяснить причины. Планировщик/оптимизатор генерирует множество возможных планов выполнения для запроса, оценивает их по своей внутренней модели на основе ранее собранной статистики и настроек конфигурации и в конечном итоге выбирает план с наименьшей стоимостью, потому что с большой долей вероятности он и будет самым быстрым. Именно поэтому в предыдущем разделе я показал, насколько важно позволить автоочистке/автоанализу собирать статистику для планировщика.

EXPLAIN ANALYZE не просто показывает выбранный план запроса, а идет еще дальше. Он сообщает вам фактические временные показатели того, сколько времени заняло выполнение каждого компонента оператора.

ПРЕДУПРЕЖДЕНИЕ EXPLAIN ANALYZE фактически *выполняет* SQL-оператор, в отличие от простого EXPLAIN, который только планирует его. Это означает, что если ваш оператор изменяет данные, EXPLAIN ANALYZE <query>; применит эти изменения! Старайтесь по возможности размещать EXPLAIN ANALYZE между командами BEGIN и ROLLBACK, но помните, что ваш ROLLBACK может создать потребность в запуске VACUUM (например, если вы вставляете много строк, а затем откатываете изменения).

Расширение `auto_explain` позволяет автоматически включать планы выполнения для медленных запросов в журнал PostgreSQL. Однако учитывайте, что запуск EXPLAIN для большого числа запросов может снизить производительность.

В этой книге есть множество примеров использования EXPLAIN (ANALYZE), которые помогают разобраться в скорости и логике выполнения запросов, а также понять, почему тот или иной подход неверен. Еще один пример будет в разделе 6.10.

В дополнение к официальной документации в интернете доступны многочисленные руководства, которые помогают разобраться, как читать и понимать вывод EXPLAIN, а также визуализаторы: они более удобно, по мнению части пользователей, организуют узлы и концепции запросов.

6.9. Явная блокировка

Режимы блокировок, которые предоставляет PostgreSQL, могут использоваться приложениями для управления конкурентным доступом к табличным данным в ситуациях, когда стандартного поведения MVCC недостаточно. Явно блокируя объекты, приложение может обеспечивать согласованность данных, гарантировать, что действия не будут выполнены «в неправильном порядке», или предотвращать рассинхронизацию объектов на стороне приложения.

Однако явные блокировки, известные как *тяжелые блокировки* (heavyweight locks), — это плохая практика, и их следует избегать. Они могут полностью блокировать доступ к чтению и записи для других операторов, что приводит к задержкам.

Как мы видели в примере из раздела 6.5, ситуация, когда мы непреднамеренно заблокировали доступ к таблице, могла бы оказаться катастрофической для работы приложения. Оператор бесконечно ждал, когда снимется необходимая ему блокировка, а другие операторы вставляли за ним в очередь и тоже ждали.

Очень легко увидеть только часть проблемы при решении сложной задачи и далее ошибочно сосредоточиться, например, на необходимости синхронизации объектов приложения, и тогда явные блокировки могут показаться идеальным решением. На практике сессии будут, скорее всего, блокировать друг друга, если только, конечно, ваше приложение не было как-то особенно тщательно продумано (догадка: а оно не было). В результате запросы будут еще медленнее, время отклика — длиннее, а время ожидания еще заметнее для пользователей.

На всякий случай не используйте такие блокировки в коде. Как вариант, сделайте так, чтобы клиентское приложение пыталось получить доступ к объекту и, если объект недоступен, немедленно уведомляло приложение об ошибке, чтобы оно могло повторить попытку позже. Такой подход подразумевает, что вам нужно будет сделать приложение отказоустойчивым, чтобы оно могло допускать сбои, обрабатывать эти ошибки и выполнять повторные попытки.

К счастью, в PostgreSQL есть уровень изоляции, который делает именно это. Уровень изоляции `SERIALIZABLE` реализует *Serializable Snapshot Isolation (SSI)*, который обнаруживает и предотвращает *циклы антизависимостей* (anti-dependency cycles) — ситуации, когда одна транзакция записывает новую версию строки, в то время как другая наблюдает более старую версию этой строки, и такие зависимости замыкаются в цикл.

Цикл антизависимостей (rw-dependency)

Транзакция T1 читает три строки из таблицы с условием `WHERE active = true`, транзакция T2 обновляет одну из этих строк с помощью `SET active = false`, и затем

транзакция T1 пытается вычислить среднее значение с условием `WHERE active = true`, которое усредняет только по двум оставшимся строкам. Это также известно как *перекос записи при чтении по предикату* (write skew on predicate read), или *аномалия сериализации G2*.

SERIALIZABLE — самый строгий уровень изоляции транзакций. При обнаружении нарушения сериализуемости PostgreSQL выдает ошибки вида `ERROR: could not serialize access due to read/write dependencies among transactions`. Из-за накладных расходов на обнаружение аномалий он может несколько снизить уровень параллелизма. Однако его главное преимущество заключается в том, что ничто не блокирует запросы и явные блокировки не требуются. Для некоторых типов приложений уровень **SERIALIZABLE** оказывается лучшим выбором для достижения высокой производительности.

СОВЕТ Пример использования **SERIALIZABLE** можно найти в документации PostgreSQL здесь: <https://www.postgresql.org/docs/current/transaction-iso.html#XACT-SERIALIZABLE>.

Как вариант, если вы хотите работать с синхронизацией на стороне приложения, то можете использовать рекомендательные блокировки (advisory locks) — механизм PostgreSQL, который позволяет разработчикам определять блокировки, имеющие смысл только для приложения, но не принудительно контролируемые самой СУБД.

Подробнее о рекомендательных блокировках см. по адресу <https://www.postgresql.org/docs/current/explicit-locking.html#ADVISORY-LOCKS>.

6.10. Отсутствие индексов

Надеюсь, этот сценарий понятен из названия; он характерен не только для PostgreSQL. Индексы могут существенно повысить производительность запросов, особенно при наличии предикатов (`WHERE`), обеспечивая быстрый доступ к данным за счет индексного сканирования вместо полного последовательного сканирования (Seq Scan). Различные доступные типы индексов (B-Tree, Hash, GIN, BRIN, GiST, SP-GiST и Bloom) охватывают множество различных сценариев, и тем не менее мы постоянно видим медленные запросы, которые можно было бы выполнить моментально при использовании индекса.

Однако, переходя с других СУБД, вы можете предположить, что при определенных условиях индекс будет создаваться автоматически. В одном недавнем реальном случае разработчики исходили из того, что, поскольку они выполняли

запрос по столбцу, на который был определен внешний ключ, PostgreSQL автоматически создаст индекс на таблице, содержащей внешний ключ.

Посмотрим, как это выглядело, на примере двух таблиц из базы данных Frogge Emporium. У Frogge есть таблица счетов-фактур, где столбец `customer` ссылается на таблицу `customers`. Вот запрос, чтобы найти, сколько счетов-фактур есть от конкретного заказчика:

```
SELECT count()
FROM erp.invoices
JOIN erp.customers c
ON customer = c.id
WHERE first_name = 'Aiden'
AND last_name = 'Kowalski';
```

С `SET max_parallel_workers_per_gather=0`; вывод `EXPLAIN ANALYZE` для этого запроса выглядит следующим образом:

```

                                QUERY PLAN
-----
Aggregate(cost=6054.51..6054.52 rows=1 width=8) (actual
➡time=28.057..28.060 rows=1 loops=1)
  -> Hash Join (cost=306.01..6054.46 rows=18 width=0) (actual
    ➡time=2.681..28.053 rows=18 loops=1)
      Hash Cond: (invoices.customer = c.id)
      -> Seq Scan on invoices (cost=0.00..5092.00 rows=250000 width=8)
        ➡(actual time=0.004..12.098 rows=250000 loops=1)
      -> Hash (cost=306.00..306.00 rows=1 width=8) (actual
        ➡time=0.953..0.954 rows=1 loops=1)
        Buckets: 1024 Batches: 1 Memory Usage: 9kB
        -> Seq Scan on customers c (cost=0.00..306.00 rows=1
          ➡width=8) (actual time=0.122..0.950 rows=1 loops=1)
          Filter: ((first_name = 'Aiden'::text) AND (last_name =
            ➡'Kowalski'::text))
          Rows Removed by Filter: 13999
Planning Time: 0.139 ms
Execution Time: 28.083 ms
(11 rows)
```

Мы запрашиваем по внешнему ключу, который ссылается на первичный ключ в таблице `customers` и поэтому индексирован там, но, несмотря на это, операция `Gather` для нашего `count()` выполняет последовательное сканирование (`Seq Scan`) таблицы `invoices`. PostgreSQL не создает индекс на ссылающейся таблице автоматически, поэтому запрос работает без индексной оптимизации. Тем не менее он все равно завершается всего за 28 миллисекунд.

После выполнения команды `CREATE INDEX ON erp.invoices(customer)`; мы получаем следующий план выполнения:

QUERY PLAN

```

-----
Aggregate  (cost=314.84..314.85 rows=1 width=8) (actual time=0.957..0.957
  ↳rows=1 loops=1)
  ->
    Nested Loop
      (cost=0.29..314.79 rows=18 width=0) (actual
        ↳time=0.121..0.954 rows=18 loops=1)
        -> Seq Scan on customers c (cost=0.00..306.00 rows=1 width=8)
          ↳(actual time=0.107..0.938 rows=1 loops=1)
          Filter: ((first_name = 'Aiden'::text) AND (last_name =
            ↳'Kowalski'::text))
          Rows Removed by Filter: 13999
        -> Index Only Scan using invoices_customer_idx on invoices
          ↳(cost=0.29..8.61 rows=18 width=8) (actual time=0.012..0.014 rows=18
            ↳loops=1)
            Index Cond: (customer = c.id)
            Heap Fetches: 0
Planning Time: 1.511 ms
Execution Time: 1.338 ms
(10 rows)

```

Этот план преобразовал доступ к таблице `invoices` в сканирование только по индексу (`Index Only Scan`), которое занимает в общей сложности всего 1,3 миллисекунды! Большое ли это достижение? Ранее время тоже измерялось в миллисекундах.

Однако этот запрос выполнялся в 20 раз быстрее. Представьте, что такой запрос выполняется тысячи раз в час или в минуту. В этом случае миллисекунды, которые обычно игнорируются, могут иметь огромное значение для общей пропускной способности системы.

Несколько разделов назад мы говорили о важности команды `EXPLAIN`. Здесь мы применили ее, чтобы посмотреть, используется ли индекс и помог ли созданный нами индекс, сравнив планы выполнения до и после добавления индекса.

Если вы видите, что выполняется полное сканирование таблицы, изучите специфику нужного столбца и условие использования `WHERE` — весьма вероятно, что вы найдете тип индекса, который ускорит ваш SQL-запрос.

6.11. Лишние индексы

С другой стороны, если бездумно создавать индексы на каждом столбце, это серьезно снизит скорость выполнения `INSERT`, `UPDATE` и `DELETE`. Кроме того, вы можете потратить много места на диске на бесполезные индексы.

Мне запомнился один случай, когда в таблице был индекс на каждом столбце, составной индекс для всех столбцов и индекс для каждой комбинации столбцов! Разумеется, ни один из них не использовался по назначению.

Не буду повторяться насчет EXPLAIN — попробуйте сами и убедитесь, насколько медленнее делается запись в большую таблицу, если к ее столбцам все время добавлять индексы.

Как мы говорили в предыдущем разделе, когда у вас очень интенсивный поток запросов, при котором таблица обновляется постоянно, даже просто снижение на десятки или сотые доли миллисекунд по каждому UPDATE значительно улучшит производительность.

Наконец, индекс нужен только тогда, когда у вас есть условие WHERE, и даже в этом случае только для критичных по времени запросов. Если вы собираетесь индексировать данные, которые запрашиваете только раз в день, и запрос занимает пару секунд, затраты на сопровождение такого индекса могут оказаться неоправданными. Кроме того, когда вы выбираете значительную часть таблицы, сканирование индекса может по факту оказаться медленнее, чем последовательное сканирование таблицы. PostgreSQL знает это и выберет именно Seq Scan вместо сканирования индекса.

Вы можете выявить неиспользуемые индексы, получив представление по pg_stat_all_indexes и посмотрев, у каких индексов idx_scan = 0, а затем удалить их с помощью DROP. Этот параметр показывает, что эти индексы никогда не использовались при сканировании по индексу, поэтому нет причин их сохранять.

СОВЕТ Будьте внимательны и не выполняйте команды создания индекса или ограничения, такие как CREATE INDEX ON invoices(customer);, более одного раза. Без явного указания имени индекса или ограничения будет создан дубликат индекса (с уникальным, сгенерированным системой именем). Дубликаты индексов занимают место в базе данных, замедляют операции DML (INSERT, UPDATE и DELETE) и увеличивают время обслуживания (например, для упомянутой выше автоочистки), не принося при этом никакой пользы для скорости запросов.

6.12. Удаление индексов, которые используются в других местах

Вы последовали совету из предыдущего раздела и удалили все индексы со значением idx_scan = 0. Разумеется, вы довольны собой, потому что поступили грамотно, согласно рекомендациям из книги по PostgreSQL, и в результате состояние базы улучшилось. Через несколько минут неожиданно звонит телефон, вы поднимаете трубку и слышите, что бухгалтерия в ярости. Их запросы по отчетам еле двигаются, и они требуют объяснить, что случилось с базой данных.

Как такое возможно? Оказывается, бухгалтерия для своих запросов использует резервный сервер (в качестве выделенной реплики для чтения), подключенный через физическую репликацию. Если бы вы проверили pg_stat_all_indexes

на этом узле, то увидели бы довольно много сканирований по индексу, и это означает, что индекс активно использовался для некоторых запросов. Такая статистика специфична для конкретного экземпляра PostgreSQL, а не для всего кластера. И разумеется, поскольку между экземплярами PostgreSQL существует физическая репликация, после удаления индекса на основном сервере он также исчез и с резервного.

Когда у вас есть резервные серверы, всегда проверяйте, не используются ли индексы на любом из них, прежде чем решите их удалить.

Итоги

- Параметры конфигурации PostgreSQL по умолчанию очень занижены и в большинстве случаев будут недостаточны для реальной рабочей нагрузки. Это может означать, что вы недогружаете системные ресурсы и упускаете потенциальный прирост производительности.
- Вы можете навредить СУБД, неправильно рассчитав настройки выделения памяти. Учитывайте рабочую нагрузку, в том числе типы запросов, их требовательность к памяти, а также предполагаемый уровень конкурентной клиентской нагрузки.
- Из-за особенностей работы механизма MVCC избыточная параллельность может скорее снизить производительность, чем позволить выполнять больше работы одновременно. Изучите риски, связанные со слишком большим количеством открытых сессий внутри PostgreSQL, а также характер вашей рабочей нагрузки и ограничения конкретного сервера базы данных.
- Соединения, которые в основном или полностью находятся в состоянии простоя, не являются «бесплатными» — они создают вычислительную нагрузку, способную повлиять на производительность как PostgreSQL, так и операционной системы в целом. Следует избегать наличия соединений, не выполняющих реальной работы. Если это невозможно, используйте пулер соединений, который учитывает семантику таких соединений, например PgBouncer в режиме транзакций (Transaction Mode).
- Сессии в состоянии `idle in transaction` и сессии с длительно выполняющимися запросами могут неожиданно блокировать другие запросы, что приводит к задержкам или ошибкам в приложении. Они также могут откладывать или полностью блокировать работу автоочистки, что грозит деградацией производительности из-за раздувания данных или более серьезными ошибками.
- Если позволить частоте транзакций выйти из-под контроля, она превысит возможности автоочистки, механизм которой стремится не допустить сбоя из-за проблемы, известной как `XID wraparound`. В итоге может произойти

сбой и аварийное завершение работы базы данных. Смягчить риск этой проблемы может пакетная обработка, реорганизация и агрегация данных — или даже отказ от загрузки потенциально бесполезных данных.

- Автоочистка необходима для корректной работы базы данных. Принудительное снижение ее активности или полное отключение для экономии системных ресурсов нежелательно, поскольку любой прирост производительности со временем будет нивелирован накоплением эффекта раздувания, неточной статистикой для оптимизатора и принудительной профилактикой от `XID wraparound`. Если, по наблюдениям, запрос выполняется медленно, то проверка его через `EXPLAIN` позволит быстро и точно выявить причины, а затем их устранить.
- Явная блокировка объектов приводит к блокировкам чтения/записи, из-за чего ваше приложение может работать медленно или некорректно. Если требуется синхронизация, попробуйте использовать уровень изоляции `SERIALIZABLE` и реализуйте в приложении возможность повторного выполнения операций.
- Разные индексы могут как значительно улучшить, так и ухудшить производительность ваших запросов; в PostgreSQL доступно множество типов индексов. Анализируйте планы запросов и используйте индексирование для ускорения условий `WHERE`. Сканирования всей таблицы нежелательны, поскольку их скорость линейно снижается по мере роста таблицы.
- Неиспользуемые индексы замедляют запись в таблицу и занимают ценное дисковое пространство. Выявите такие индексы в представлении через `pg_stat_all_indexes` и удалите их.
- При удалении индексов, которые кажутся неиспользуемыми, следует соблюдать осторожность, поскольку статистика по их использованию относится только к локальному серверу, хотя на самом деле индексы могут быть задействованы на другом сервере, куда выполняется физическая репликация.

Нерекомендуемые практики администрирования

В этой главе

- ✓ Почему использование дискового пространства может выйти из-под контроля
- ✓ Чем опасно игнорирование журналов и что при этом можно упустить
- ✓ Чем так важны сбор и анализ статистики
- ✓ Почему нужно обязательно применять обновления

После интереснейшего обсуждения разработки и оптимизации наступает время переводить базу данных в продакшен — это относительно рутинная и относительно тяжелая задача администрирования. Однако без этого PostgreSQL не будет надежным и устойчивым. Это напрямую определяет, насколько успешно будет развернут Postgres, что, в свою очередь, может либо обеспечить работу критически важных систем, либо привести к отказу. Теперь мы рассмотрим ряд ошибок, которые администраторы СУБД могут допускать либо из-за недостаточной осведомленности, либо из-за неопытности при управлении Postgres.

7.1. Отсутствие мониторинга дискового пространства

Трудно поверить, но это до сих пор одна из основных проблем, с которыми сталкиваются специалисты, обслуживающие установленные PostgreSQL. Рано

или поздно кто-то обязательно обращается к ним за помощью, чтобы разобраться с печально известной ошибкой `No space left on device` — «кончилось свободное место на диске». Этот раздел можно было бы назвать «Исчерпание дискового пространства», однако я хочу сразу здесь подчеркнуть: если вы не отслеживаете использование дискового пространства, то рано или поздно место на нем кончится.

Давайте сделаем шаг назад и посмотрим, что происходит, когда ваш развернутый экземпляр PostgreSQL исчерпывает доступное дисковое пространство. Прежде всего, о каком диске идет речь? Риски, связанные с исчерпанием пространства на системном диске или в разделе операционной системы, хорошо известны. Будем надеяться, что каталог с данными PostgreSQL (`PGDATA`) находится на другом диске или разделе, и предположим, что именно на нем закончилось место.

Вероятно, вы столкнетесь с сообщением, похожим на это:

```
ERROR: could not extend file "base/12609/16048": No space left on device
HINT: Check free disk space.
```

Что происходит, когда появляется эта пугающая ошибка?

Все зависит от того, где хранятся файлы данных и WAL, а также находятся ли они на одном томе или нет. Простое заполнение диска с данными само по себе не является фатальной ошибкой, но определенно может помешать приложению выполнять обычные операции с базой данных. Однако если на диске с WAL заканчивается место, то становится невозможно вести журнал и фиксировать изменения. При таких условиях PostgreSQL может перейти в состояние `PANIC` и аварийно завершить работу.

Такая ситуация крайне нежелательна, поэтому администраторы стараются предотвратить ее любой ценой. К сожалению, импульсивные решения могут привести к ошибочным действиям, которые лишь усугубят ситуацию.

Распространенной «мерой по спасению» (и конечно же, заблуждением) в такой момент становится попытка удалить старые данные именно в базе, чтобы освободить место. Однако из-за модели MVCC в PostgreSQL это не даст немедленных результатов или вообще не сработает. PostgreSQL должен записывать WAL для каждого изменения данных, поэтому даже для удаления данных в базе вам все равно требуется свободное дисковое пространство (чтобы записать операцию удаления в WAL). Если же у вас достаточно места для выполнения `DELETE` или `TRUNCATE`, оставшиеся мертвые строки все равно потребуют выполнения `VACUUM` для возврата свободного пространства. Однако и в этом случае пространство не вернется операционной системе, а будет просто помечено как свободное внутри файлов данных таблицы. `VACUUM FULL`¹, который может вернуть свободное про-

¹ Кроме того, по своей сути `VACUUM FULL` выполняет полную перепакровку таблицы: данные записываются в новый физический файл (в том же `tablespace`), и затем происходит переключение на него, то есть операция требует заранее доступного

странство обратно в ОС, получает блокировку ACCESS EXCLUSIVE на таблицу во время ее перезаписи, делая таблицу недоступной на время операции, что также категорически не рекомендуется.

Решение увеличить размер дискового тома также может оказаться ошибкой в условиях продакшена, поскольку многие облачные провайдеры и файловые системы приостанавливают все дисковые операции до завершения операции расширения тома. Это потенциально способно вызвать незапланированный простой СУБД. Кроме того, операция расширения может занять значительное время, поэтому иногда более разумным вариантом будет восстановление базы данных на новом сервере или в облачном экземпляре с большим объемом дискового пространства.

Еще одной быстрой идеей может стать поиск логов для удаления. Именно это нередко приводит неопытных пользователей PostgreSQL к одному из самых распространенных способов серьезно навредить системе.

7.1.1. Удаление журнала предзаписи

В былые времена каталог данных PostgreSQL содержал директории с именами `pg_log`, `pg_clog` и `pg_xlog`. Если известно, что `pg_log` содержит логи, которые можно удалить, легко предположить, что можно также избавиться от с-логов и х-логов. К сожалению, `pg_clog` была директорией, содержащей журнал фиксации (Commit Log), который отслеживает состояние всех транзакций и является неотъемлемой частью механизма MVCC. `pg_xlog` была директорией для WAL, и к этому моменту книги уже должно быть понятно, насколько важен журнал транзакций для базы данных. По сути, удаление этих каталогов уничтожало установленный Postgres, мгновенно делая его непригодным для использования, и единственным оставшимся вариантом становилось восстановление базы данных из резервной копии.

Когда руководство PostgreSQL осознало, что проблема начала приобретать массовый характер, `pg_log` переименовали в `log`, чтобы сделать название однозначным, `pg_clog` — в `pg_xact`, а `pg_xlog` — в `pg_wal`, чтобы люди перестали думать, что `log` — это журналы, которые можно удалять по своему усмотрению. Несмотря на то что эти каталоги намеренно переименовали, до сих пор находятся люди, которые умудряются удалять из них файлы, — это одна из грубейших ошибок при работе с PostgreSQL.

дополнительного дискового пространства (как минимум под новую версию таблицы и перестроение ее индексов, а также с учетом объема WAL). Поэтому запуск VACUUM FULL на системе, где уже фиксировались ошибки «No space left on device», существенно повышает риск срыва операции, длительной блокировки таблицы и, в случае дефицита места под WAL, аварийного завершения инстанса. — *Примеч. науч. ред.*

Честно говоря, кажется, что нет никакой возможности остановить людей, чтобы они не удаляли файлы, назначение и важность которых они не понимают. Похоже, единственное, что остается, — писать об этих ошибках книги и надеяться, что однажды их прочтут.

7.1.2. Потребители дискового пространства

Теперь обсудим, что может постепенно или внезапно поглощать дисковое пространство. Очевидно, что таблицы и индексы будут расти со временем по мере добавления в них данных, плюс этот процесс можно значительно усугубить разбуханием. Если мертвые строки не удаляются своевременно, это может привести к неконтролируемому увеличению размера файлов таблиц, и это дисковое пространство будет потеряно для файловой системы до тех пор, пока не будет специально возвращено. Временные файлы, которые обычно используются только на время операции, не помещающейся в памяти, например крупной сортировки или хеш-соединения (HASH JOIN), также могут сохраняться на диске после некорректного завершения работы Postgres. Затем эти временные файлы потребуются очистить вручную.

Ранее мы упоминали, что WAL критически важен для работы базы данных, однако может генерироваться и накапливаться слишком много этих данных. Если настройки `max_wal_size` и `wal_keep_size` неоптимальны, Postgres, возможно, будет использовать больше дискового пространства, чем необходимо. Кроме того, при отставании репликации или сбое резервного сервера, использующего слоты репликации (replication slots), WAL будет накапливаться локально до тех пор, пока резервный сервер не сможет принять и обработать этот WAL. Если этот процесс останется незамеченным (а такое случается чаще, чем можно предположить), он может исчерпать все дисковое пространство, где находится каталог WAL. Наконец, сбой команды `archive_command` также будет держать на диске WAL, который планировалось отправить в архив, до тех пор, пока журнал не удастся успешно архивировать.

7.1.3. Возможные действия

В качестве меры снижения риска можно разместить базу данных и WAL на отдельных томах¹ (и, конечно, внимательно следить за использованием дискового пространства на этих томах). Более того, *табличные пространства* (tablespaces) в PostgreSQL позволяют определять места хранения для объектов базы данных, поэтому фактически вы можете создавать любую таблицу или индекс в любой директории или томе вашей файловой системы. Помимо прямого повышения производительности и экономии затрат, которые достигаются благодаря гибким

¹ На самом деле размещение базы данных и WAL на отдельных томах является хорошей или даже обязательной практикой в высоконагруженных системах. — *Примеч. науч. ред.*

возможностям перемещения объектов на более быстрые или дешевые дисковые хранилища, можно также получить больший контроль над структурой хранения на диске вашей установки PostgreSQL. Наконец, вы можете добавлять табличные пространства по мере необходимости и временно использовать их до разрешения проблемы с основным диском.

Возможно, самая важная задача мониторинга для администратора базы данных — следить за тем, чтобы диск не заполнился. Теперь вы понимаете, что происходит с развернутым Postgres, когда диск заполнился, а также какие ответные действия могут эту ситуацию усугубить, и вам, полагаю, стала очевидна важность такого постоянного мониторинга. Используйте инструменты мониторинга любого типа, настраивайте предупреждения задолго до порога уровня PANIC и регулярно проверяйте вручную все аномалии поведения, не дожидаясь, когда сработают предупреждения.

СОВЕТ В некоторых файловых системах¹ по мере заполнения диска снижается производительность, поэтому важно начать действовать до полного исчерпания пространства.

7.2. Запись логов в PGDATA

Вы можете считать, что заполнение диска до отказа — довольно банальная ошибка, но если бы такое не происходило на практике, этого бы не было в книге. Если вы действительно так думаете, вот менее очевидная и более коварная ошибка.

По умолчанию PostgreSQL записывает файлы журналов в ту же файловую систему, что и сама база данных (мы называем директорию данных сокращенно PGDATA). Такой стандартный вариант может показаться удобным, но он же может привести к проблемам, если логи начнут бесконтрольно расти. Если они занимают слишком много места, они могут «съесть» свободное пространство и привести к тому, что базе данных некуда будет расти, как мы видели в предыдущем разделе. При правильном мониторинге базы данных вы это обнаружите, но что, если исчерпание произойдет быстрее, чем администраторы баз данных смогут отреагировать? Можно даже утверждать, что такое поведение сформирует вектор атаки типа «отказ в обслуживании» (DoS) через преднамеренную генерацию чрезмерного количества записей в журнале, чтобы специально заполнить диск.

Продемонстрируем, как такое может быть, предположив, что следующий код каким-то образом проходит мимо тестирования приложения и контроля качества (QA). Разработчик из компании Frogge Emporium написал простую процедуру — с проверкой ошибок, допустим, чтобы вычислить общее энергопотребление всех филиалов за предыдущий день и записать результаты в журнал аудита. Для этого нужен запрос, который вычисляет итоги по каждому филиалу с помощью

¹ Это же, к сожалению, справедливо и для современных устройств хранения (для SSD/NVMe дисков), не организованных в управляемые пулы/рейды. — *Примеч. науч. ред.*

оконных функций, затем суммирует все и делает запись в таблице аудита. В коде реализована функция повторных попыток.

Кейс 7.1. Процедура для записи суммы энергопотребления за предыдущий день

```
import psycopg
from datetime import date, timedelta as td

with psycopg.connect("dbname=frogge user=frogge") as conn:
    try:
        while True:
            with conn.cursor() as cur:
                try:
                    # Я усвоил урок насчет использования BETWEEN!
                    cur.execute('''WITH
                        yesterday AS (
                            SELECT *
                            FROM erp.energy_use
                            WHERE reading_time >=
                                date_trunc('d', now())
                                - interval '1d'
                            AND reading_time <
                                date_trunc('d', now()),
                        perbranch AS (
                            SELECT first_value(reading) OVER w,
                                   last_value(reading) OVER w,
                                   row_number() OVER w
                            FROM yesterday
                            WINDOW w AS (
                                PARTITION BY branch_id
                                ORDER BY reading_time
                                RANGE BETWEEN UNBOUNDED PRECEDING
                                AND UNBOUNDED FOLLOWING))
                        SELECT sum(last_value - first_value)
                        FROM perbranch
                        WHERE row_number=1''')
                    total = cur.fetchone()[0]

                    cur.execute('''INSERT INTO audit.audit_log
                        (what, who, tstamp)
                        VALUES (%s, %s, now())''',
                        (f"Energy usage for {date.today() - td(days=1)}: "
                        + f"{total} kWh", "Frogge Emporium"))

                    conn.commit()
                    break
                # Если что-то пошло не так, вероятно, база данных занята
                # и можно просто повторить попытку
            except psycopg.errors.Error:
                conn.rollback()
    finally:
        conn.close()
```

Итак, разработчик из Frogge запускает этот код, и через несколько секунд журналы базы данных начинают заполняться записями:

```
2024-09-20 12:26:19.606 BST [3055342] frogge@frogge ERROR:relation
➡"erp.energy_use" does not exist at character 148
2024-09-20 12:26:19.606 BST [3055342] frogge@frogge STATEMENT:WITH
yesterday AS (
    SELECT *
    FROM erp.energy_use
    WHERE reading_time >=
        date_trunc('d', now())
        - interval '1d'
    AND reading_time <
        date_trunc('d', now())),
perbranch AS (
    SELECT first_value(reading) OVER w,
           last_value(reading) OVER w,
           row_number() OVER w
    FROM yesterday
    WINDOW w AS (
        PARTITION BY branch_id
        ORDER BY reading_time
        RANGE BETWEEN UNBOUNDED PRECEDING
        AND UNBOUNDED FOLLOWING))
    SELECT sum(last_value - first_value)
    FROM perbranch
    WHERE row_number=1
2024-09-20 12:26:19.606 BST [3055342] frogge@frogge ERROR: relation
➡"erp.energy_use" does not exist at character 148
2024-09-20 12:26:19.606 BST [3055342] frogge@frogge STATEMENT: WITH
[...]
```

Ой, похоже, что вместо правильного имени таблицы `energy_usage` они написали `energy_use`. В таком сценарии это приводит к бесконечному выполнению кода, который постоянно отправляет запрос к базе данных, всегда неудачный из-за неправильного имени таблицы. Каждый раз при возникновении этой ситуации ошибка записывается в журнал PostgreSQL.

Если мы посмотрим на временные метки, то увидим множество записей с одной и той же ошибкой в течение одной миллисекунды, что делает наш код весьма продуктивным бесконечным циклом. К сожалению, это особенно плохо для нас, поскольку процесс, который пишет в журнал с такой скоростью, может быстро заполнить диск с журналами. Если не прервать его, он будет продолжаться до исчерпания дискового пространства. Следовательно, если ваша база данных находится на том же диске, что и журналы, это грозит аварийным завершением работы СУБД, как мы видели ранее.

В любом случае, как бы ни был маловероятен предыдущий сценарий, он подчеркивает важность хранения журналов в отдельной файловой системе или

отдельном от основного хранилища базы данных разделе. Так вы можете снизить риск того, что запись в журналы поглотит все дисковое пространство вашей базы данных. В качестве дополнительных мер безопасности также следует убедиться, что в вашей СУБД реализована ротация журналов и установлены ограничения на их размер. Если вы не используете средства управления журналами и их ротацией, встроенные в ОС, вы можете управлять размером журналов напрямую, настраивая такие параметры, как `log_rotation_size`, `log_rotation_age` и `log_truncate_on_rotation`. Впоследствии вам придется найти баланс между накоплением большого количества журналов, которые слишком объемны и сложны для просмотра, и риском удаления журналов с полезной историей ошибок и потенциально нужными деталями.

7.3. Игнорирование журналов

Многие считают, что журналы базы данных нужны только для анализа после возникновения проблем. Но это не так! Журналы PostgreSQL существуют не только для «посмертного» анализа инцидентов; они могут информировать почти в реальном времени о том, что происходит внутри базы данных. Игнорируя их, вы упускаете ключевые сведения, которые способны предотвратить серьезные проблемы в будущем.

Рассмотрим несколько моментов, которые можно упустить, если не проверять журналы регулярно.

7.3.1. Некорректная конфигурация

В первую очередь, вы можете обнаружить ошибки конфигурирования. Например, если в журналах встречается запись подобного вида, это может означать, что репликация на резервный сервер или создание потоковых резервных копий выполняются некорректно:

```
2024-09-21 13:41:10.692 BST [3214372] replicator@[unknown] FATAL: number of
➔requested standby connections exceeds max_wal_senders (currently 10)
```

Скорее всего, этот процесс незаметен на уровне приложения, однако требует к себе немедленного внимания, — в этом случае нужно проверить, какие фактические соединения репликации поступают на сервер БД и какие ограничения `max_wal_senders` есть в конфигурации.

Аналогичным образом в журнале могут встречаться ошибки, связанные с недостаточным количеством доступных соединений:

```
2024-09-21 13:49:39.821 BST [3219655] frogge@frogge FATAL: remaining
➔connection slots are reserved for roles with the SUPERUSER attribute
```

Такие сообщения указывают на одну из двух проблем: либо приложению не предоставляется необходимое количество соединений, либо приложение запрашивает чрезмерное их число. Если вспомнить материал главы 6, возможно, пора уже добавить в конфигурацию пулер соединений, например PgBouncer.

Однако не только ошибки уровня FATAL должны вызывать беспокойство. Существуют ситуации, которые не относятся к ошибкам, но могут указывать на скрытые проблемы, например ошибки конфигурирования памяти. Если у вас включен параметр `log_temp_files`, можно заметить множество записей в журнале, похожих на эту:

```
2024-09-21 13:51:15.133 BST [3226458] frogge@frogge LOG:temporary file:
➔path "base/pgsql_tmp/pgsql_tmp3226458.0", size 8437760
2024-09-21 13:51:15.133 BST [3226458] frogge@frogge STATEMENT:SELECT *
➔FROM erp.payments WHERE tstamp >= date_trunc('y', now()) ORDER BY
➔amount;
```

Это может означать, что установлено слишком низкое значение параметра `work_mem`, и СУБД вынуждена прибегать к записи временных файлов на диск для выполнения запросов, которые не помещаются в оперативную память. Как мы уже выяснили, это серьезно влияет на производительность из-за значительной разницы в скорости и задержках между памятью и диском.

7.3.2. Проблемы производительности

Медленные запросы также требуют внимания, поскольку они могут указывать на факторы, снижающие производительность (плохо написанный SQL-код, отсутствующие индексы), или прогрессирующие проблемы, например постепенную деградацию производительности. Установка значения параметра `log_min_duration_statement` равным примерно 100 миллисекундам позволит регистрировать медленные запросы, подобные этому:

```
2024-09-21 14:17:41.763 BST [3232957] frogge@frogge STATEMENT:SELECT *
➔FROM erp.payments WHERE tstamp >= date_trunc('y', now()) ORDER BY
➔amount;
2024-09-21 14:17:41.764 BST [3232957] frogge@frogge LOG:duration:
➔157.096 ms statement: SELECT * FROM erp.payments WHERE tstamp >=
➔date_trunc('y', now()) ORDER BY amount;
```

Поскольку этот запрос выполнялся 157 миллисекунд, он превысил установленный порог и попал в журнал. В предыдущей главе мы подробно разбирали, насколько длительные¹ запросы пагубны для производительности и надежности системы.

¹ В данном контексте под длительными запросами понимаются те, которые стали таковыми вследствие медлительности их выполнения. — *Примеч. науч. ред.*

Кроме того, характер вашей рабочей нагрузки может приводить к появлению в журналах записей, которые помогут при диагностике производительности базы данных и приложения. Например, если вы обнаружите в журнале, что контрольные точки создаются слишком часто, значит, Postgres слишком часто сбрасывает данные на диск, и это негативно сказывается на производительности:

```
2024-09-21 14:20:27.920 BST [3260055] LOG: checkpoints are occurring too
➔ frequently (2 seconds apart)
2024-09-21 14:20:27.920 BST [3260055] HINT: Consider increasing the
➔ configuration parameter "max_wal_size".
2024-09-21 14:20:27.920 BST [3260055] LOG: checkpoint starting: wal
2024-09-21 14:20:27.963 BST [3260055] LOG:checkpoint complete: wrote 1359
➔ buffers (8.3%); 0 WAL file(s) added, 2 removed, 0 recycled; write=0.155
➔ s, sync=0.019 s, total=0.201 s; sync files=2, longest=0.019 s,
➔ average=0.010 s; distance=33073 kB, estimate=37487 kB; lsn=11/8371F2D0,
➔ redo lsn=11/81A37E70
```

В этом случае подсказка HINT абсолютно уместна: вероятно, следует увеличить параметр `max_wal_size`, при этом также важно исследовать причину всплеска записи на диск в других частях системы. Как уже упоминалось ранее, иные проблемы рабочей нагрузки, например чрезмерно высокая скорость транзакций, также могут проявляться в журнале через сообщения о принудительном VACUUM, чтобы предупредить XID wraparound.

7.3.3. Блокировки

Проблемы с блокировками, зафиксированными в журнале, например ошибки взаимоблокировки (deadlock), также могут указывать на текущие или надвигающиеся проблемы с производительностью. Вместо бесконечного ожидания Postgres завершает один из двух конфликтующих запросов, чтобы разорвать взаимоблокировку. Тогда в файле журнала видно примерно следующее:

```
2024-09-21 14:40:24.541 BST [3279275] frogge@frogge ERROR: deadlock detected
2024-09-21 14:40:24.541 BST [3279275] frogge@frogge DETAIL: Process 3279275
➔ waits for ShareLock on transaction 3810; blocked by process 3279214.
Process 3279214 waits for ShareLock on transaction 3811; blocked by process
➔ 3279275.
Process 3279275: UPDATE audit.audit_log SET tstamp=now() WHERE id=99925;
Process 3279214: UPDATE audit.audit_log SET tstamp=now() WHERE id=500002;
2024-09-21 14:40:24.541 BST [3279275] frogge@frogge HINT: See server log
➔ for query details.
2024-09-21 14:40:24.541 BST [3279275] frogge@frogge CONTEXT: while updating
➔ tuple (1868,57) in relation "audit_log"
2024-09-21 14:40:24.541 BST [3279275] frogge@frogge STATEMENT: UPDATE
➔ audit.audit_log SET tstamp=now() WHERE id=99925;
2024-09-21 14:40:24.747 BST [3279214] frogge@frogge LOG: duration:
➔ 20217.150 ms statement: UPDATE audit.audit_log SET tstamp=now() WHERE
➔ id=500002;
```

Поскольку от взаимоблокировок нет никакого волшебного параметра в конфигурации PostgreSQL, стоит проверить логику приложения, чтобы найти места, где возможны взаимоблокировки, и превентивно их исключить.

7.3.4. Повреждение данных

Одним из наиболее серьезных последствий игнорирования работы журналов может стать угроза нарушения целостности данных. Упоминание ошибок, подобных приведенной ниже, может указывать на проблемы файловой системы или даже аппаратного обеспечения, которые изначально могут не проявляться на уровне приложения:

```
2024-09-21 14:45:11.172 BST [3281235] ERROR: could not read block 8185 in
➔file "base/16384/2613": read only 0 of 8192 bytes
2024-09-21 14:45:11.172 BST [3281235] CONTEXT:while reading block 8185 of
➔relation base/16384/2613
2024-09-21 14:45:11.172 BST [3281235] STATEMENT:SELECT * FROM
➔audit.audit_log WHERE tstamp > '2024-09-21';
2024-09-21 14:45:11.173 BST [3281235] LOG: invalid page in block 8185 of
➔relation base/16384/2613
2024-09-21 14:45:11.173 BST [3281235] HINT: This could be a corruption
➔issue caused by hardware problems or a software bug. Check for hardware
➔issues and consider running pg_checksums or other diagnostic tools.
```

В этом случае журнал смог подсветить ранние признаки повреждения данных и дать нам возможность не доводить до серьезной проблемы — потери данных.

7.3.5. Безопасность

Анализ журналов также помогает выявлять инциденты информационной безопасности. Если повторяются частые неудачные попытки аутентификации, это может указывать на возможные брутфорс-атаки:

```
2024-09-21 14:49:34.300 BST [3306179] frogge@frogge FATAL: password
➔authentication failed for user "frogge"
2024-09-21 14:49:34.300 BST [3306179] frogge@frogge DETAIL: Connection
➔matched file "/etc/postgresql/17/main/pg_hba.conf" line 123: "local
➔all all scram-sha-256"
```

Кроме того, аномальные паттерны входа или подозрительные запросы, такие как попытки SQL-инъекций или нетипичный интерес к таблицам с чувствительными данными, могут свидетельствовать о попытке вторжения или о том, что оно уже происходит.

Поскольку объем журналирования, который может генерировать база данных в продакшен-среде, бывает очень большим, в некоторых случаях просто нецелесообразно просматривать все вручную. Инструмент, который может помочь

в этой ситуации, — анализатор журналов, например pgBadger. Он может суммировать данные за определенные периоды и создавать удобные отчеты в формате HTML. Инструмент фиксирует упомянутые выше события, а также отчитывается об использовании ресурсов; в нем можно настроить определенные пороги или паттерны, при достижении которых pgBadger будет отправлять оповещения.

ПРИМЕЧАНИЕ pgBadger доступен по адресу <https://github.com/darold/pgbadger>.

Но даже не обладая специализированными инструментами, вы все равно можете выполнить необходимую проверку и просканировать файлы журналов с помощью быстрых UNIX-утилит, таких как `grep`. Например, можно написать команду, которая будет искать в журнале строки, указывающие на ошибки `ERROR` или `FATAL`:

```
grep -E 'ERROR|FATAL' /var/log/postgresql/postgresql-17-main.log
```

Таким образом, крайне важно активно отслеживать содержимое журналов PostgreSQL, чтобы иметь полное и актуальное представление о состоянии инстанса базы данных — его конфигурации, производительности, целостности данных, безопасности и обработки всех прочих непредвиденных событий.

7.4. Отказ от мониторинга базы данных

Можно предположить, что настроить систему мониторинга может любой специалист, отвечающий за PostgreSQL, или команда. И тем не менее ниже мы повторим в очередной раз свои рекомендации, поскольку их важность нельзя переоценить.

- Если состояние базы данных должным образом не отслеживать, возникнут проблемы и, что еще хуже, не будет заблаговременного предупреждения или каких-либо признаков того, что что-то пошло не так. Для базы данных в продакшене это категорически неприемлемо из-за сопутствующих операционных рисков, и тем не менее базовые задачи мониторинга иногда до сих пор игнорируются.
- Если не заниматься мониторингом, можно не заметить замедления запросов и, соответственно, не выявить и не устранить узкие места, вызывающие деградацию производительности. Необходимо настроить немедленные уведомления о всплесках нагрузки, поскольку в эти моменты может упасть производительность, что негативно скажется на пользовательском опыте и, как следствие, нанесет ущерб репутации компании.
- Аналогично следует отслеживать использование ресурсов PostgreSQL, поскольку разного рода неэффективность или рост нагрузок могут привести к работе на пределе возможностей; это опять же может снизить производительность или даже вызвать полную неотзывчивость базы данных.

- Мониторинг также позволяет выявлять аномальную активность, такую как чрезмерное потребление ресурсов — случайное или умышленное, а также другие аспекты из предыдущего раздела 7.3, которые могут служить первыми предупреждениями об угрозах безопасности или вредоносных действиях.
- Наконец, мониторинг может указать на необходимость операций обслуживания, таких как `VACUUM` и `ANALYZE`, или на недостаточность текущих настроек `autovacuum`.

Обсудим, какие параметры следует отслеживать, чтобы обеспечить адекватный мониторинг вашей установленной СУБД. Сама PostgreSQL предоставляет обширные внутренние сведения о своей работе, но нужно знать, где их искать. Большинство инструментов мониторинга, даже если в них не включен мониторинг PostgreSQL «из коробки», позволяют определять метрики, проверки и использовать панели мониторинга либо посредством настройки, либо посредством использования плагинов. Полезно отслеживать следующие показатели:

- (разумеется) использование дискового пространства и объем временных файлов;
- метрики по таблицам и индексам из представлений `pg_stat_user_tables` и `pg_stat_user_indexes`;
- количество и общий размер WAL-файлов из `pg_stat_wal` через `pg_ls_waldir()`;
- соединения, выполняемые запросы, их продолжительность и факты блокировок (`pg_stat_activity`);
- состояние репликации — с использованием `pg_stat_replication` и соответствующих функций, связанных с WAL;
- возраст транзакций базы данных — из `pg_stat_database`;
- состояние резервного копирования.

Также можно отслеживать и более продвинутые показатели:

- подробную статистику запросов (собирается через расширение `pg_stat_statements`);
- статистику буферов, контрольных точек и фоновой записи;
- блокировки и ожидания на сервере;
- низкоуровневую I/O-статистику из представлений `pg_statio_*_tables` и `pg_stat_io` (доступно, начиная с Postgres 16).

Ниже приведены некоторые универсальные инструменты мониторинга с открытым исходным кодом, которые используют на практике:

- RRDtool и Cacti;
- Nagios/Icinga;

- Munin Monitoring;
- Zabbix;
- Percona Monitoring and Management (PMM);
- Prometheus для облачных сред;
- PGWatch;
- и даже pgAdmin для просмотра в реальном времени.

Все эти инструменты легко найти с помощью любой поисковой системы. Существуют также проприетарные решения, такие как Postgres Enterprise Manager (PEM) от EDB и SaaS-предложения от DataDog и SolarWinds.

Встроенная (predefined) роль PostgreSQL *pg_monitor* имеет доступ к большинству, если не ко всем, важным метрикам. Некоторые инструменты и платформы могут требовать доступа к более закрытым данным, например ко всему содержимому *pg_stat_activity*. Перед тем как принимать решение о предоставлении доступа, оцените, насколько важен мониторинг по сравнению с безопасностью системы.

Очевидно, что мониторинг помогает эффективнее управлять рисками СУБД, будь то проблемы производительности, истощение ресурсов, сбой резервного копирования или нарушение безопасности. Вы, ваша команда и организация в целом всегда должны помнить закон Мерфи: «Все, что может пойти не так, обязательно пойдет не так», и это должно быть отражено в культуре вашего мониторинга. Не ждите, когда что-то пойдет не так, чтобы начать мониторить соответствующие риски, иначе это может оказаться слишком поздно.

7.5. Отказ от отслеживания статистики

Мониторинг базы данных не сводится только к уведомлению о текущих проблемах для их исправления. Не поймите меня неправильно — мониторинг важен, но он может дать лишь ограниченную информацию об общем состоянии СУБД во времени, если вы не сохраняете статистику и не создаете отчеты за большие периоды времени. Большинство инструментов мониторинга ориентированы на отображение текущего состояния и отправку оповещений о проблемах, но не оптимизированы для долгосрочной аналитики, которая позволяет «отдалиться» и получить представление о работе СУБД в глобальном контексте.

Если вы не отслеживаете статистику PostgreSQL в динамике, вы можете упустить информацию, которая поможет подготовиться к будущему росту базы данных. Наличие исторических данных по трендам производительности, шаблонам использования и другим метрикам позволяет проводить анализы и прогнозы,

что делает планирование мощностей и решений по масштабированию проще и разумнее.

Предположим, что клиентская база Frogge Emporium продолжает стабильно расти. Без доступа к историческим данным о том, как система реагировала на эти постепенные изменения, компании будет сложнее прогнозировать эволюцию архитектуры базы данных или инфраструктуры, необходимую для обработки этого роста.

Вы можете найти инструмент мониторинга, позволяющий сохранять метрики во времени, однако хранение большого объема данных может потребовать значительных ресурсов хранения на стороне этого инструмента. В случае проприетарных инструментов и SaaS-платформ почти наверняка возникают дополнительные затраты, связанные с отслеживанием множества систем и с накоплением большого количества данных от них в течение длительного времени. Вы также можете быть сторонником простоты — использовать для мониторинга кастомные скрипты и стандартные утилиты вместо сложных инструментов с избыточным функционалом.

С учетом этих соображений я разработал расширение под названием `pg_statviz`. Оно может делать снимки важных накапливаемых и динамических показателей статистики PostgreSQL, чтобы можно было отслеживать их эволюцию во времени, проводить анализы и создавать визуализации для лучшего понимания нагрузки на сервер. Ключевое преимущество расширения — его легковесность, и отсутствие необходимости загружать модуль в `shared_preload_libraries` означает, что установка даже не требует перезапуска сервера. Также оно позволяет выполнять этот анализ без накладных расходов внешних инструментов или хранилищ, таких как Prometheus, Logstash или Elasticsearch, или локальных агентов, подобных тем, что используются инструментами типа Dog of Data.

Вы можете настроить любой планировщик заданий для создания этих периодических снимков. При разумном интервале создания снимков в 1 минуту (минимальный интервал в `cron`) объем генерируемых данных очень мал, поэтому почему бы не хранить их внутри самой базы данных? Именно это и делает `pg_statviz`, сохраняя помеченные временными метками снимки в таблицах схемы `pgstatviz`. Сейчас расширение собирает статистику по фоновым операциям записи, контрольным точкам, частоте попадания в кэш, количеству подключений, операциям ввода-вывода, количеству прочитанных/записанных кортежей, блокировкам, событиям ожидания, генерации WAL, транзакциям, конфигурации и другим параметрам. Реализованный механизм ресемплирования позволяет выполнять анализ неограниченного объема данных, агрегируя тысячи и даже миллионы снимков до произвольного числа точек на графике (по умолчанию — 100).

Сопутствующая утилита визуализации, которая поставляется с расширением, умеет читать данные из этих таблиц и может создавать графики (сохраняя их в формате .png), что позволяет анализировать эти временные ряды с одного взгляда, как показано на рис. 7.1.

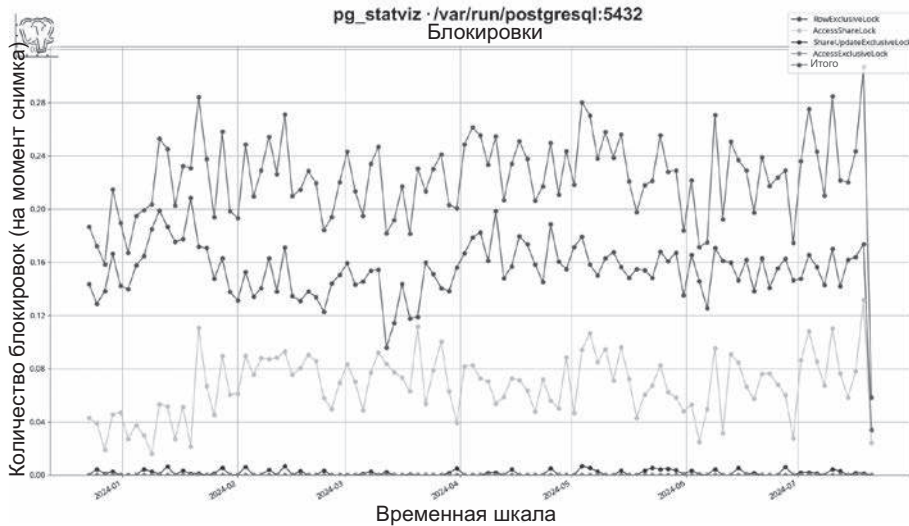


Рис. 7.1. Результат анализа блокировок в pg_statviz. Эта визуализация отображает среднее количество и тип блокировок в базе данных, причем снимки делались каждую минуту в течение нескольких месяцев 2024 года

Вы можете загрузить расширение `pg_statviz` из репозитория сообщества PostgreSQL, установив пакет `pg_statviz_extension` для систем на базе Red Hat или `postgresql-statviz` для систем на базе Debian. Кроме того, его можно установить из PostgreSQL Extension Network (сеть расширений для PostgreSQL). Также доступна ручная установка из исходных файлов — компиляция при этом не требуется.

Вы можете включить расширение в нужной базе данных, выполнив команду (с правами суперпользователя):

```
CREATE EXTENSION pg_statviz;
```

Как только расширение будет активировано, пользователи с соответствующими привилегиями могут немедленно начать делать снимки через:

```
SELECT pgstatviz.snapshot();
NOTICE: created pg_statviz snapshot
        snapshot
```

```
-----
2024-06-27 11:04:58.055453+00
(1 row)
```

Как упоминалось ранее, рекомендуется делать снимки по расписанию, что умеет делать любой инструмент планирования в ОС или СУБД, включая `pg_cron`.

Утилиту визуализации можно также установить из репозитория сообщества PostgreSQL как пакет `pg_statviz` для систем на базе Red Hat или `pg-statviz` для систем на базе Debian. Кроме того, ее можно загрузить из PyPI (сокр. от Python Package Index) следующим образом:

```
pip install pg_statviz
```

Пользоваться утилитой предельно просто:

```
pg_statviz buf --host froggeserver -d frogge -U frogge -D 2024-06-24T23:00
➔2024-06-26
```

Эта команда подключит утилиту к базе данных `frogge` на хосте `froggeserver` под пользователем `frogge` и сгенерирует визуализацию статистики по буферам в период с 23:00 2024-06-24 до 00:00 2024-06-26.

ПРИМЕЧАНИЕ Исходный код и документация доступны по адресу: https://github.com/vyruss/pg_statviz.

7.6. Отказ от обновлений Postgres

На какой версии PostgreSQL вы работаете? Разработка PostgreSQL ведется уже более трех десятилетий, и за это время вышло множество как минорных, так и мажорных релизов. *Минорные* релизы выходят примерно ежеквартально, их можно узнать по последней части номера версии (например, 16.4). В них исправляют ошибки, проблемы безопасности и проблемы, связанные с повреждением данных. В такие релизы никогда не добавляют фичи и не изменяют внутренний формат, а сообщество Postgres считает, что их безопаснее установить, чем проигнорировать. *Мажорные* релизы¹ — это когда меняется (увеличивается) первый элемент в номере версии (например, 16.4). В них появляются новые и тщательно перепроверенные фичи, и в этих релизах может измениться внутренний формат системных таблиц и файлов данных. Соответственно, они могут оказаться несовместимыми с данными в формате прошлого мажорного релиза (но их можно обновить). Короче говоря, именно новые мажорные релизы двигают Postgres

¹ После перехода PostgreSQL Global Development Group в 2017 году на регулярный годовой цикл релизов они выходят раз в год, в последние несколько лет это конец сентября — начало октября. — *Примеч. науч. ред.*

вперед. Каждая мажорная версия обычно поддерживается в течение пяти лет, и за это время предоставляются исправления и обновления безопасности.

Что касается фич, то обратная совместимость практически гарантирована. SQL есть SQL, поэтому код, написанный для Postgres 9.3, скорее всего, будет работать и в Postgres 17. Процесс обновления с версии на версию доказал свою высокую надежность.

Это наблюдение подводит нас к ключевому вопросу: почему же люди опасаются обновлений?

Прежде всего, действует пресловутый принцип «Ну, сейчас же все нормально работает». Можно поддаться ложному чувству безопасности, думая, что если систему хорошо тестируют и мониторят, то она стабильна. В реальности то, что работает сегодня, может перестать работать завтра, если пользователи, к примеру, наткнутся на скрытую ошибку или если приложение вдруг начнет проявлять неожиданное поведение из-за изменения нагрузки или паттернов доступа к данным. Кроме того, не стоит забывать о новых уязвимостях безопасности, которые находят постоянно. Полагаться исключительно на собственное тестирование как гарантию защищенности системы недопустимо.

С этим также тесно связан страх, что обновление может привести к новым проблемам — новым ошибкам, снижению производительности, несовместимости. Причиной может быть и недостаток знаний о PostgreSQL: некоторые администраторы СУБД переживают, что они недостаточно компетентны, чтобы выполнить обновление без риска для системы.

Наконец, существуют организационные причины. Долгий процесс согласования, координация с несколькими отделами и нежелание добавлять в план вынужденные простои тормозят обновления.

Если не обновлять PostgreSQL регулярно, вы подвергаете его значительным рискам. Критически важно, чтобы вы не пропускали обновления безопасности. Как open-source проект PostgreSQL способен выпускать обновления безопасности иногда в течение нескольких часов, что недостижимо для проприетарного ПО. Однако если у вас неподдерживаемая версия или не отлажены процессы обновлений, то вы не сможете воспользоваться плюсами обновлений.

Вторая причина для регулярных обновлений — в них постоянно приходят исправления ошибок. Устраняются как вновь появившиеся регрессионные ошибки, так и давние (которые могут оставаться незамеченными годами). Пропуская обновление, вы легко можете столкнуться с ошибкой, которая уже исправлена в более поздних версиях.

Третья причина — вы упускаете новые фичи и оптимизации, которые есть в каждом следующем мажорном выпуске. Те, кто работает на PostgreSQL 9.6, не

получили нативное (декларативное) партиционирование и логическую репликацию. Те, кто работает на версии 11, не получили `pg_checksums` или генерируемые столбцы, а те, кто застрял на Postgres 13, не получили улучшения пропускной способности при большом количестве соединений и такие продвинутые функции, как потоковая передача крупных транзакций и конвейеризация `libpq`.

Наконец, чем дольше вы остаетесь на старом выпуске, тем сложнее становится его обслуживать и впоследствии — обновлять.

В плане обновлений PostgreSQL замечательна тем, что позволяет мигрировать с одной мажорной версии на другую без промежуточных версий. Однако при таком подходе крайне важно прочитать примечания к релизу каждой отдельной промежуточной мажорной версии.

Всегда читайте примечания к релизу для каждого обновления, даже для минорных версий! Это стоит делать хотя бы потому, что, например, релиз 14.4 исправил ошибку повреждения индекса, и это в некоторых случаях потребовало выполнения `REINDEX`. Далее, в этом же релизе была устранена уязвимость безопасности, но решение могло затронуть пользователей расширения `pg_trgm`.

Говоря о расширениях, важно помнить, что они имеют собственные циклы релизов и собственные исправления ошибок. Всегда следует проверять, что новая версия расширения совместима с вашей текущей версией PostgreSQL.

Обновления не так рискованны и разрушительны, как часто принято считать. В частности, минорные выпуски сопряжены с минимальным риском и позволяют своевременно получать последние исправления и улучшения. Для мажорных обновлений существуют инструменты, такие как `pg_upgrade`, которые со временем были усовершенствованы до уровня, считающегося чрезвычайно надежным (плюсом они также имеют режим предварительной проверки). Альтернативные методы обновления, например использование логической репликации для обновлений с почти нулевым временем простоя, требуют большей подготовки, но позволяют существенно сократить влияние на продуктивную среду.

Наконец, приверженность разработчиков проекта PostgreSQL и его сообщества принципам надежности платформы гарантирует, что обновления протестированы, стабильны и подробно задокументированы через примечания к выпуску.

СОВЕТ Обязательно ознакомьтесь с разделом, который посвящен несовместимым изменениям в документации PostgreSQL, по адресу: <https://www.postgresql.org/docs/current/runtime-config-compatible.html>.

7.7. Отказ от обновления системы

Надеюсь, к этому моменту преимущества регулярного обновления PostgreSQL очевидны. Менее очевидным является то, что можно регулярно обновлять

Postgres, но при этом оставлять операционную систему на одной и той же версии из-за инерции или нежелания со стороны системных администраторов. Нередко продакшен-среды остаются на одной и той же мажорной версии операционной системы в течение четырех или пяти лет. Иногда администраторы избегают обновления до тех пор, пока это возможно, обновляясь только непосредственно перед прекращением всей поддержки ОС, а в некоторых случаях — даже сразу после этого. Крайне плохими примерами такой политики являются предложения Debian, Red Hat и Ubuntu по поддержке устаревших выпусков до 10 лет — срок, превышающий любые разумные пределы.

Расскажу реальный случай, когда эта политика в конечном счете повлияла на работу PostgreSQL в продакшене. У одной базы данных, которая использовала расширение PostGIS для хранения и обработки геопространственных данных, начала падать производительность. Падение никто объяснить не мог, поскольку оно, казалось, было как-то связано с определенными значениями обрабатываемых геопространственных данных!

Например, наблюдалась очень нестабильная производительность для функции PostGIS, такой как `ST_DistanceSphere`, которая возвращает расстояние в метрах между двумя точками на земном шаре. Функция работала значительно медленнее для определенных пар точек данных по сравнению с почти идентичными, например такими:

```
SELECT ST_DistanceSphere('POINT(-150 33)',
                          'POINT(-120.120120 42.488888)');
```

Запрос необъяснимо выполнялся более чем в 50 раз дольше, чем следующий:

```
SELECT ST_DistanceSphere('POINT(-150 33)',
                          'POINT(-120.120120 42.4888881)');
```

Что еще хуже, никто из тех, кто расследовал этот пограничный случай, не смог воспроизвести такое поведение, даже используя абсолютно ту же комбинацию версий PostgreSQL и PostGIS. После долгих проб и ошибок кому-то из команды удалось вызвать это состояние на тестовой системе, но только для этих точных значений данных. Этого было достаточно, чтобы начать расследование, и для анализа на системном уровне были задействованы инструменты профилирования, такие как `perf`.

Было установлено, что процесс бэкенда Postgres тратил большую часть своего времени в математической библиотечной функции `__mul` из `libm`:

```
+      62.78%      1.29%  postgres  libm-2.23.so  [...] __mul
```

Получается, проблема была в умножении?!

Дело в том, что `libm` — часть более широкой библиотеки GNU C (`glibc`), и небольшая проверка подтвердила, что такое anomальное поведение отсутствовало

в системах с другими версиями `glibc`. Дальнейшее расследование выявило, что в `glibc` были обнаружены некоторые медленные алгоритмы для тригонометрических функций синуса и косинуса. Однако проблема с расчетами была устранена в математических функциях *два* года назад. В этой же системе применялась версия `glibc`, которой было *четыре* года.

Поскольку использовалась сильно устаревшая версия без критичных улучшений, она применяла эти медленные алгоритмы, что и стало причиной непонятного резкого замедления в геопространственных математических расчетах.

Это подчеркивает важность своевременного обновления системных библиотек как критически важной меры защиты от подобных трудноуловимых проблем. Даже когда PostgreSQL и его расширения, казалось, работали нормально, системная библиотека, являющаяся базовой зависимостью, в конечном счете снижала производительность на более низком уровне программного стека, что значительно усложнило диагностику и устранение неполадок.

Часто нежелание системных администраторов обновлять компоненты системы или сам Postgres — это ошибочная попытка поддержать стабильность в продакшен-средах. Считается, что, если не выполнять обновления, то снизится риск возникновения новых ошибок или простоев, ведь сейчас же система вроде как работает нормально. Однако, как показывает наш пример, это опасный образ мышления, поскольку пользователи могут столкнуться со скрытыми ошибками, которые появляются в новых сценариях использования или с большими наборами данных.

Отказ от обновления зависимостей (например, версии ОС или ее отдельных библиотек) из-за страха потенциальных поломок неоправдан: риск недополучения критических исправлений ошибок и обновлений безопасности превышает риск появления нового нежелательного поведения, которое, как правило, легко обнаружить и устранить. Более того, чем дольше откладывать обновления, тем сложнее распутывать потом цепочку зависимостей.

Впоследствии становится труднее применить все накопившиеся обновления, а объем тестирования возрастает. Пропуск минорных обновлений, которые обычно не меняют функциональность и не содержат несовместимых изменений, особенно непростителен в современной среде баз данных.

Итоги

- Исчерпание дискового пространства может вызвать серьезные проблемы, поэтому внимательно следите за его использованием. Удаление того, что выглядит как логи или ненужные файлы, изменение размера томов и другие поспешные решения могут ухудшить ситуацию. Определите, на что тратится

дисковое пространство, и устраните проблему, в том числе используя несколько томов.

- Хранение журналов (логов) PostgreSQL в отдельной от базы данных файловой системе помогает снизить риск того, что чрезмерное логирование заполнит диск. Также следует внедрить ротацию логов и установить ограничения на их размер.
- Регулярная проверка логов PostgreSQL крайне важна, чтобы выявлять и устранять ошибки конфигурации, узких мест в производительности, проблем с целостностью данных и инцидентов безопасности практически в реальном времени. Она позволяет решать проблемы заранее, и также для этой задачи можно использовать **pgBadger**.
- Отсутствие мониторинга PostgreSQL может привести к скрытым проблемам с производительностью, угрозам безопасности и исчерпанию важных для работы ресурсов. Используя подходящие инструменты и метрики, чтобы отслеживать снижение производительности, всплески потребления ресурсов и необходимости обслуживания, вы можете оптимизировать работу системы и снизить риски.
- Долгосрочное отслеживание статистики PostgreSQL важно для понимания трендов производительности и принятия обоснованных решений по планированию мощностей. Можно использовать стандартные системы мониторинга или легковесное расширение **pg_statviz**, чтобы собирать и визуализировать статистику во времени без тяжелых инструментов и значительных затрат на хранение.
- Без регулярных минорных обновлений PostgreSQL вы можете столкнуться с рисками безопасности или с последствиями ошибок, которые уже исправлены. Без обновления до последнего мажорного релиза вы можете пропустить актуальные фичи и улучшения. Обновление работает безопасно и стабильно благодаря хорошо протестированным инструментам и исчерпывающей документации. Не бойтесь обновлений; бойтесь остаться без них.
- Отказываясь обновлять операционную систему, а также зависимые библиотеки, вы можете столкнуться со скрытыми проблемами в работе PostgreSQL, ошибками и уязвимостями и свести на нет кажущуюся стабильность, ради которой вы и избегали что-то менять.

Нерекомендуемые практики безопасности

В этой главе

- ✓ Чем грозит неосторожное использование паролей в командной строке
- ✓ Как случайно не предоставить внешним субъектам доступ к базе данных
- ✓ Почему не стоит давать больший доступ, чем необходимо
- ✓ Какие опасности есть при написании и использовании функций

Разработчики СУБД PostgreSQL как авторы уважаемого ПО известны тем, что серьезно относятся к информационной безопасности (ИБ) и придерживаются принципа «безопасность по умолчанию» (security-by-default). В команду ИБ проекта входят опытные контрибьюторы, которые умеют быстро оценивать угрозы и реагировать на них — выпускают минорные релизы с исправлениями уязвимостей.

Однако, к сожалению, большинство инцидентов ИБ происходят не из-за малоизвестных эксплойтов и уязвимостей, а по более прозаическим причинам, например из-за того, что системный администратор забыл изменить учетные данные по умолчанию на продакшен-сервере. Когда не соблюдаются лучшие практики

ИБ, могут возникнуть серьезные последствия: атаки с повышением привилегий и утечкой данных и многое другое. В этой главе мы рассмотрим ошибки оператора (то есть как его небрежность может повлиять на безопасность Postgres) и что можно сделать для их предотвращения.

8.1. Указание `psql -W` или `--password`

Большинство утилит командной строки PostgreSQL, например `psql`, принимают одни и те же опции, такие как `-h` или `--host=` для указания имени хоста подключаемого сервера базы данных или `-U/--username=` для указания пользователя, от имени которого происходит подключение. Нередко пользователи вводят ключ `-W` или `--password` при подключении, чтобы указать пароль.

Администратору СУБД необходимо подключиться к серверу Frogge Emporium для выполнения некоторых действий. Сегодня он уже делал это десятки раз, и это начинает ему надоедать, поскольку для подключения ему приходилось много раз вводить свой пароль, каждый раз выполняя следующую команду:

```
psql -h froggeserver -U frogge -W
```

Время позднее, и он чувствует, что из-за долгой монотонной работы он начинает набирать текст небрежно. Хотя он почти уверен, что ошибся при вводе пароля, `psql` успешно подключается к серверу. Сотрудник удивился, у него даже промелькнула мысль, что, несмотря на усталость, он все равно отлично справляется с вводом текста.

Что же произошло? На самом деле он действительно ввел неверный пароль, но сервер пропустил его и принял соединение. Как такое возможно? Разве он не просил `psql` явно запросить пароль?

Вот почему опцию `-W` в кругах PostgreSQL мы называем бомбой замедленного действия. Ее использование может ввести в заблуждение и нанести ущерб безопасности. Что же с ней не так?

Во-первых, применяя `--password`, вы даете указание любой вызываемой утилите Postgres запросить у вас пароль, независимо от того, нужен ли он серверу. Уточним и сразу же забежим чуть вперед — это именно наш случай: сервер баз данных может быть настроен (кто-то сказал бы: *неправильно* настроен) на прием входящих соединений без требования пароля.

В результате у администратора СУБД из нашего примера возникло ложное чувство безопасности. Поскольку клиент запрашивает пароль, легко предположить, что сервер работает в режиме защиты паролем, хотя на самом деле это не так. Это означает, что любой на его месте мог бы так же легко подключиться без пароля, и администратор даже не догадался бы об этом, потому что думает, что проходит аутентификацию правильно, с паролем.

Более того, указание опции `-W` или `--password` совершенно избыточно¹! Если аутентификация в PostgreSQL сконфигурирована так, чтобы требовать пароль для подключения, сервер автоматически запросит его, даже если вы не передадите эту опцию в командной строке.

К счастью, опция `-W` не позволяет указать пароль в командной строке `psql`, поскольку это было бы серьезным упущением в безопасности. Если бы это было возможно, введенный вами пароль в виде открытого текста был бы виден в системных процессах, отображаемых командами `top` или `ps faux`. Что еще хуже, он был бы записан в истории команд оболочки. Однако, несмотря на то что PostgreSQL защищает вас от этой ошибки, следует быть предельно осторожным и никогда не включать пароли в виде открытого текста в строки подключения в конфигурационных или скриптовых файлах.

Дополнительная проблема, связанная с требованием к клиенту запрашивать пароль, даже когда он не нужен, заключается в том, что неверные пароли также срабатывают, потому что когда серверу не нужен пароль, он просто его игнорирует.

Наконец, привычка применять `-W` может привести к неприятным сюрпризам для пользователя при подключении с другой рабочей станции или через другого клиента. Если для этого соединения применяется аутентификация по паролю, пользователь может быть озадачен, когда перестанет действовать пароль, который вроде бы раньше сервер принимал. Если пароль не проверялся сервером при другом подключении, пользователь мог считать его правильным, даже если он был недействителен.

Лучше всего вообще забыть о существовании этой опции и положиться на автоматический запрос пароля, который PostgreSQL запускает по необходимости. Если сервер не настроен на обязательную парольную аутентификацию, вы должны знать об этом и осознанно принять такое решение, а не пребывать в ложном ощущении защищенности.

8.2. Установка `listen_addresses = '*'`

Поскольку мы упомянули конфигурационные файлы, пришло время рассмотреть одного из лидеров по неправильному использованию параметров в `postgresql.conf`, а именно `listen_addresses`. Он определяет IP-адреса, на

¹ `-W` / `--password` изначально несут иную смысловую и техническую нагрузку, они заставляют, например, `psql` запросить пароль заранее, чтобы сократить число сетевых раундов аутентификации и избежать дополнительного обмена с сервером перед вводом пароля. — *Примеч. науч. ред.*

которых сервер PostgreSQL должен ожидать входящие запросы на подключение. Значение по умолчанию у него довольно строгое, поскольку установлено как `localhost`. Таким образом, когда условный разработчик прекращает экспериментировать локально на своем ноутбуке и хочет протестировать код на реальном сервере баз данных, он обнаружит: то, что работало локально, не позволяет ему подключиться по сети.

Самый простой способ «починить» это — установить `listen_addresses` в значение `'*'`, означающее «любой адрес», перезапустить сервер и получить возможность подключаться. Здорово, не так ли? Вовсе нет.

Причина, по которой такой сценарий плох, заключается в том, что это изменение конфигурации позволяет экземпляру базы данных принимать подключения на любой IP-адрес сервера баз данных, то есть на любой настроенный сетевой интерфейс.

Почему это плохо? Как правило, базы данных должны быть доступны только из определенных частных сетей, таких как защищенный интранет или VPN-соединение. Однако если у сервера настроены/разрешены и другие сетевые подключения, например подключение к интернету, этот параметр сделает PostgreSQL доступным через все интерфейсы, включая общедоступное интернет-соединение. Излишне говорить, что если сервер отвечает на запросы из посторонней сети, возникает большая поверхность атаки и угроза безопасности.

`listen_addresses = 'localhost'` — один из параметров, значение которого выставляется согласно принципу PostgreSQL «безопасность по умолчанию». Параметр должен по умолчанию настраиваться максимально безопасно, и в данном случае такая настройка разрешает подключаться только от локального хоста и игнорирует все, что поступает из сети. Таким образом, сетевое подключение к базе данных должно быть явно разрешено администратором СУБД.

Использование принципа «безопасность по умолчанию»

Указание `listen_addresses = '*'` фактически сводит на нет встроенную в продукт безопасность по умолчанию. Приведу пример, почему выбор в пользу безопасности по умолчанию — это хорошо. В 2020 году исследователи безопасности обнаружили, что — страшно представить — 3,6 миллиона серверов MySQL и MariaDB были доступны из интернета. Вам точно не хочется, чтобы ваш сервер был открыт для потенциальных злоумышленников из интернета (или других небезопасных сетей), которые таким образом сканируют незащищенные сети и выявляют открытые порты на экземплярах СУБД.

Убедитесь, что в списке адресов для прослушивания `listen_addresses` указаны только те интерфейсы и сети, из которых ожидаются подключения клиентов. Например, если PostgreSQL должна быть доступна только из частной сети, укажите только IP-адрес сервера на интерфейсе, подключенном к этой сети, в файле `postgresql.conf`:

```
listen_addresses = '10.10.10.56'
```

Также рекомендуется настроить правила на межсетевом экране (МЭ), чтобы они ограничивались предоставлением доступа к экземпляру сервера только для доверенных подсетей и IP-адресов. Во многих МЭ также реализовано ведение журнала, чтобы было проще обнаруживать вторжения.

8.3. Использование *trust* в `pg_hba.conf`

Пресловутый `pg_hba.conf` — один из самых запутанных и, к сожалению, пугающих файлов конфигурации в «джунглях» PostgreSQL. Он содержит правила *Host-Based Authentication (HBA)* для экземпляра сервера баз данных и определяет, кто может подключаться, откуда, к каким базам и каким способом должен проходить аутентификацию. Его синтаксис четко определен под собственные задачи, но, к сожалению, в файле используется малопонятный сегодня формат. Это может привести к тому, что разработчики и администраторы будут испытывать трудности, поскольку им придется много возиться с записями в этом файле, чтобы PostgreSQL наконец принял их подключения:

```
psql -h froggeserver -U frogge
Password for user frogge:
psql: error: connection to server at "froggeserver" (10.10.10.56), port 5432
➡failed: FATAL: password authentication failed for user "frogge"
```

Иногда, перепробовав множество разных подходов и неизменно получая сообщение выше, администраторы СУБД расстраиваются, отчаиваются и в итоге настраивают сервер так, чтобы он при подключении безусловно им доверял (*trust*). В итоге сервер принимает их подключение без использования пароля, что кажется приемлемым на данный момент, поскольку задача подключения наконец решена.

Бывают и другие, намного более безобидные случаи, когда разработчики уже по привычке прибегают к такому решению. Чтобы избежать работы с паролями, файлами `.pgpass`, сертификатами и т. д., они идут по пути наименьшего сопротивления и для ускорения процесса разрешают подключение с аутентификацией *trust* в `pg_hba.conf`, как указано ниже:

```
Host      all      all      10.10.10.0/24      trust
```

Эта запись означает, что любой (all) пользователь, определенный в Postgres, может подключиться к любой базе данных кластера без пароля из всей подсети 10.10.10.x.

«Хорошо, но в чем проблема?» — спросите вы. В конце концов, внутренняя сеть Frogge Emporium в сети 10.10.10.0/24 является частной, и доступ к ней есть только у авторизованных лиц.

Однако что произойдет, если другое устройство, приложение или скрипт на другом сервере в этой подсети будут скомпрометированы? Тогда, поскольку Postgres доверяет всем подключениям из этой сети, он предоставит полный доступ любому потенциальному злоумышленнику без какой-либо аутентификации.

Файл pg_hba.conf не просто так называется. Аутентификация на основе хоста гарантирует, что только доверенные клиенты с доверенных хостов могут подключаться к серверу, и определяет, как они должны аутентифицироваться для подключения к определенным базам данных.

trust — это не метод аутентификации: он означает «впустить без аутентификации». Его нельзя использовать ни при каких обстоятельствах на машинах, находящихся в продакшене или подключенных к нему, поскольку такой вариант подключения к Postgres представляет собой серьезнейшую уязвимость безопасности. Даже если вы используете trust только для локальных подключений, можете ли вы гарантировать безопасность каждого фрагмента работающего кода на этом сервере?

Ответ прост: аутентификацию через trust в HBA следует полностью исключить. Для защиты доступа к базам данных PostgreSQL следует использовать метод аутентификации не ниже scram-sha-256, поскольку устаревший механизм md5 больше не обеспечивает достаточного уровня криптографической стойкости; SCRAM реализует современный протокол «вызов-ответ», основанный на доказательстве владения паролем без его передачи в явном виде.

Пример настройки метода аутентификации scram-sha-256 приведен ниже:

```
hostssl frogge    frogge    10.10.10.29/32    scram-sha-256
```

Кроме того, не будет лишним принудительно использовать шифрование SSL и разрешать доступ только определенным пользователям на определенных хостах к определенным базам данных, как в предыдущем примере.

Аутентификация trust редко бывает безопасной в любой среде, не говоря уже о продакшене, поскольку она обходит основные уровни безопасности и может поставить под угрозу ваши данные.

8.4. База данных, принадлежащая суперпользователю

Часто встречаются кластеры с базами данных, принадлежащих `postgres` или другому суперпользователю. Это неудивительно, поскольку многие создают такие базы данных по постам из Stack Overflow или бестолковым руководствам, найденным на просторах интернета. В других случаях, как уже говорилось в двух предыдущих разделах, некоторые разработчики или администраторы СУБД устают от того, что не могут выполнить все подготовленные заранее DDL-команды. В результате они производят все действия как суперпользователь, включая создание баз данных и владение ими. Поскольку суперпользователь PostgreSQL обладает неограниченными привилегиями, он может создавать схемы, таблицы и все другие типы объектов базы данных, а также управлять ими.

Рано или поздно кто-то создает новую базу данных и делает ее принадлежащей суперпользователю:

```
CREATE DATABASE frogge_next OWNER postgres;
```

А дальше вы не успеете оглянуться, как ее начинают использовать и другие люди, и такая схема закрепляется надолго. К сожалению, хотя в продакшене такого быть не должно, на практике нередко встречаются рабочие базы данных, принадлежащие суперпользователю (еще хуже, когда пользовательские таблицы и объекты, используемые в продакшене, находятся внутри базы данных `postgres` по умолчанию). Иногда вы даже можете обнаружить, что приложение подключается к базе данных через учетную запись суперпользователя `postgres` по умолчанию. Рассмотрим, почему так делать нельзя.

Архитектура PostgreSQL поддерживает следование принципу наименьших привилегий (Principle of Least Privilege), согласно которому пользователь получает только необходимые для выполнения задачи права. Если вы делаете так, чтобы база данных и содержащиеся в ней объекты принадлежали суперпользователю, то для большинства действий потребуется доступ к ним в качестве суперпользователя. Далее логично предположить, что люди начнут подключаться к этой базе данных с использованием учетных записей с соответствующими привилегиями.

Проблема заключается в том, что в Postgres суперпользователь всемогущ и может обойти практически любую проверку безопасности. При таких возможностях использование учетных записей суперпользователя для обычного владения базой данных и операций становится очень опасным. Давайте посмотрим почему.

В какой-то момент новый разработчик Frogge подключается к базе данных, используя свою непривилегированную учетную запись `jettrodriguez`. Каким-то

образом он запускает хранимую функцию `reset_schema()`, которая очищает базу данных, чтобы на ее место можно было восстановить свежую копию. Однако она была разработана администратором СУБД исключительно для разработчиков и только для сред разработки, где они работают под непривилегированными учетными записями. Функция `reset_schema` приступает к выполнению команды `DROP` для всех таблиц в текущей схеме, что создает катастрофические последствия для базы данных в продакшене. Единственный выход — восстановление данных из резервной копии, и это приводит к значительному простоя и репутационному ущербу для Frogge.

Вы спросите, как так получилось? Просто функция была создана суперпользователем, и он предоставил права на ее выполнение в качестве суперпользователя любому. Таким образом, функция запустилась с особыми правами суперпользователя и начала сеять хаос. Непривилегированная учетная запись разработчика *имела* разрешение на выполнение функций в этой базе данных, эта функция *была* видна учетной записи разработчика, поэтому катастрофа оказалась возможной.

Поскольку функция была создана привилегированной учетной записью, ей разрешили запускаться с соответствующими правами. Функцию предоставили обычным пользователям, и на время ее выполнения все полномочия, показанные на рис. 8.1, стали таким пользователям доступны.

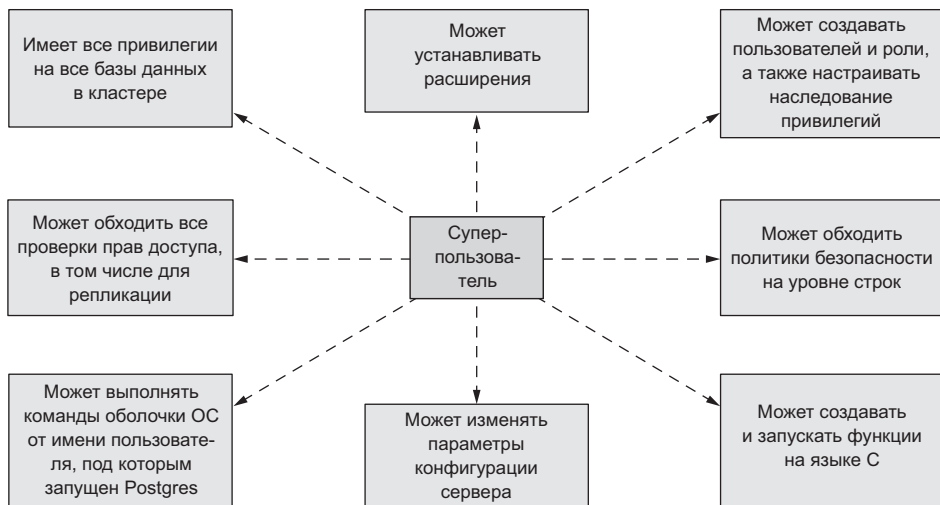


Рис. 8.1. Суперпользователь, аналогичный пользователю `root` в UNIX, может обходить все проверки привилегий и выполнять больше действий, чем любой другой пользователь. Это гораздо опаснее, чем просто иметь все привилегии на все базы данных

В следующем разделе мы увидим, как можно подставиться под злонамеренные действия просто из-за неправильного владения хранимыми функциями или процедурами. Даже если хранимый код сам по себе не задумывался как опасный, выполнение ошибочной функции (спойлер: чужой функции) с привилегиями суперпользователя может значительно усугубить последствия.

Другой пример: в организации реализованы политики безопасности на уровне строк (Row-Level Security, RLS) для всех таблиц. Пользователь подключается с непривилегированной учетной записью, которая, согласно ограничениям RLS, не должна видеть ни одной строки данных в этих таблицах. Пользователь выполняет `SELECT` из представления (VIEW), и, что удивительно, запрос возвращает строки данных. Это представление принадлежит суперпользователю, а они, как мы помним, обходят большинство проверок прав доступа. Следовательно, политика RLS не будет учитываться, и представление вернет строки любому, кто его запрашивает.

COBET Ознакомьтесь еще с одним способом защиты представлений (VIEW) от утечек данных с помощью атрибута `security_barrier` в документации PostgreSQL: <https://www.postgresql.org/docs/current/rules-privileges.html>.

Следовательно, необходимо сделать шаг назад и задаться вопросом: действительно ли суперпользователь должен участвовать в управлении базой данных, учитывая все его привилегии? Не забудьте о двойном риске: и люди подключаются к базе данных как суперпользователи, и объекты с привилегиями суперпользователя по факту позволяют обычным пользователям повышать свои привилегии. Без грамотного управления ролями (ROLE) PostgreSQL можно создать ненужные риски и допустить несанкционированный доступ к данным и случайные деструктивные действия.

Кроме того, если все подключаются с учетной записью суперпользователя, аудит и подотчетность усложняются, поскольку станет трудно отследить, кто какие действия выполнил. Многие нормативные требования предписывают внедрение ролевой модели управления доступом (RBAC) и ведение аудита, поэтому необходимо использовать встроенную систему ролей PostgreSQL.

Большинство повседневных задач не требуют привилегий суперпользователя. Использование суперпользовательских учетных записей следует ограничивать только теми административными операциями, для которых действительно необходимы повышенные права. Базы данных и объекты следует создавать от имени непривилегированных ролей.

Для каждой новой базы данных можно создавать отдельную роль владельца, обладающую только необходимыми правами для управления этой базой. Затем соответствующие привилегии на объекты базы данных следует выдавать (`GRANT`) тем ролям или пользователям, которым они действительно необходимы.

8.5. Неосторожное использование SECURITY DEFINER

В PostgreSQL можно писать код для серверной части базы данных с помощью функций, триггеров и политик RLS (Row-Level Security). Как мы видели, этот код может быть выполнен непреднамеренно и из-за этого может стать своего рода троянским конем.

Разработчика в компании Frogge Emporium с именем пользователя `marionjohnson` отделы продаж и финансов просят создать функцию, которая возвращает общую сумму продаж за текущий месяц, и ему предоставляется разрешение на доступ к этим данным. Он пишет следующую функцию:

```
CREATE OR REPLACE FUNCTION erp.current_month_sales()  
RETURNS numeric AS $$  
SELECT sum(amount)  
FROM erp.payments  
WHERE tstamp BETWEEN date_trunc('month', now()) AND now()  
$$ LANGUAGE SQL SECURITY DEFINER;
```

В PostgreSQL по умолчанию используется режим `SECURITY INVOKER`, это означает, что функция будет выполняться с привилегиями пользователя, который ее вызывает. Поскольку разработчик хочет, чтобы сотрудники из отделов продаж и финансов могли выполнять эту функцию как задумано, он решает указать `SECURITY DEFINER`, чтобы она выполнялась с привилегиями владельца функции, то есть `marionjohnson`.

Функция работает как задумано, и соответствующие отделы довольны, пока однажды утром разработчику не поступает гневный телефонный звонок с вопросом, откуда всем в компании стали известны данные о продажах, хотя руководство Frogge считало эту информацию привилегированной и не ожидало, что неавторизованные сотрудники смогут получить к ней доступ. Использование `SECURITY DEFINER` без ограничения прав на выполнение функции фактически дало доступ любому, кто ее обнаружил.

Функция `SECURITY DEFINER` работает подобно биту `setuid` в системе контроля доступа к файлам UNIX: когда он установлен, файл может быть выполнен с разрешениями владельца файла, а не с разрешениями пользователя, который его выполняет. У такого режима выполнения функции есть легитимные сценарии применения — например, триггерная функция, записывающая данные в таблицу `audit_log`, к которой обычные пользователи не имеют прямого доступа.

По этой причине имеет смысл делать функции `SECURITY DEFINER` максимально простыми, чтобы исключить любое их нецелевое использование и чтобы ими нельзя было злоупотребить через параметры, побочные эффекты или инъекции логики.

ПРИМЕЧАНИЕ Еще одна близкая по смыслу интересная концепция — это объявление о герметичности (LEAKPROOF) в определениях функций. Оно указывает на то, что функции запрещено иметь побочные эффекты. Иными словами, функция не может раскрывать информацию о своих аргументах, кроме как через свое возвращаемое значение. Следовательно, функция, которая выдает сообщение об ошибке с некоторыми (но не всеми) значениями аргументов, считается негерметичной (leaky).

Во-первых, чтобы снизить риски, убедитесь, что любые функции, объявленные как SECURITY DEFINER, могут выполнять только разрешенные роли. Вы можете сделать это, отозвав привилегии выполнения по умолчанию у PUBLIC и выборочно предоставив привилегии для запуска функции, как показано ниже:

```
BEGIN;
  CREATE OR REPLACE FUNCTION erp.current_month_sales()
  ...
  REVOKE ALL ON FUNCTION erp.current_month_sales() FROM PUBLIC;
  GRANT EXECUTE ON FUNCTION erp.current_month_sales() TO sales_role;
  GRANT EXECUTE ON FUNCTION erp.current_month_sales() TO finance_role;
COMMIT;
```

Оборачивание всех действий внутри одной транзакции гарантирует, что никакой неавторизованный пользователь не сможет использовать функцию до того, как будут правильно установлены разрешения.

Во-вторых, убедитесь, что все функции, объявленные как SECURITY DEFINER, хорошо написаны, протестированы и проверены, а также что они правильно валидируют свои входные данные. В противном случае их могут использовать для утечки данных или для повышения привилегий.

Кроме того, всегда устанавливайте search_path для функции SECURITY DEFINER на значение с безопасным порядком (подробнее о том, почему это важно, см. в следующем разделе), например, так:

```
ALTER FUNCTION erp.current_month_sales()
SET search_path = pg_catalog, erp, pg_temp;
```

Наконец, используйте SECURITY DEFINER только там, где это строго необходимо, поскольку в противном случае можно создать сложные сценарии отслеживания разрешений и затруднить управление безопасностью системы.

ПРЕДУПРЕЖДЕНИЕ Как указано в документации, «функции выполняются внутри серверного процесса с полномочиями пользователя операционной системы, запускающего сервер базы данных». Это означает, что если мы напишем функцию на языке программирования, который допускает неконтролируемый доступ к памяти, мы можем написать код, который изменяет внутренние структуры данных сервера PostgreSQL. Вот почему языки в этой категории помечены PostgreSQL как недоверенные (untrusted) и только суперпользователям разрешено создавать функции с их использованием.

8.6. Выбор небезопасного пути поиска

Чтобы сэкономить нам всем немного времени на ввод, PostgreSQL позволяет указывать так называемые неквалифицированные имена объектов (unqualified object name), то есть без уточнения имени схемы, именно поэтому мы можем ввести имя таблицы `customers` вместо `erp.customers`. Возможной эту опцию делает путь поиска схемы `search_path`, поскольку он представляет собой список имен схем, которые PostgreSQL просматривает, чтобы найти объект, на который мы ссылаемся.

Поиск схем (или `schemata`) выполняется в порядке, указанном в `search_path`. Даже если таблица с искомым именем есть в схеме, но ее имени в `search_path` **нет**, она не будет найдена, и PostgreSQL сообщит, что объекты с таким именем не обнаружены. Значение по умолчанию выглядит следующим образом:

```
SHOW search_path;
      search_path
-----
"$user", public
(1 row)
```

Это означает, что сначала СУБД будет искать объект в схеме с тем же именем, что и у текущего пользователя. Если объект там не найден, PostgreSQL переходит к следующей записи, которой является схема `public`. К сожалению, такой поиск по порядку в сочетании с сопоставлением первого найденного элемента означает, что вы можете случайно или намеренно повлиять на поведение запросов пользователей. Поясню: если у вас есть права на создание объектов в схеме, которая находится в `search_path` другого пользователя, то из-за порядка в `search_path` ваш объект может попасть к этому пользователю в его запросах.

Как минимум это может привести к путанице. Например, если используется какое-либо общепотребимое имя функции — `current_month_sales()`, `normalize_phone()` или `get_customer()` — и в схеме, находящейся раньше в `search_path`, существует одноименный объект, то вместо ожидаемого вызова конкретной функции может быть выполнен вызов другого одноименного объекта. В худшем случае злоумышленник может намеренно создать объект в схеме, расположенной раньше в `search_path`, чтобы перехватить разрешение имени. В результате можно непреднамеренно выполнить чтение из подмененной таблицы, записать данные в нее или запустить вредоносный код с вашими привилегиями. Благодаря механизму расширений PostgreSQL подобный риск распространяется не только на таблицы и функции, но и на операторы, типы, последовательности и другие объекты.

Например, предположим, что путь поиска выглядит так:

```
SHOW search_path;
      search_path
-----
"$user", public, erp
(1 row)
```

Злоумышленник может специально создать функцию `public.current_month_sales()`, и она автоматически будет выбрана пользователем (при использовании невалифицированного имени объекта) вместо `erp.current_month_sales()`, что приведет к выполнению его произвольного кода вместо того, который вы ожидали запустить.

По этой причине сейчас пользователи по умолчанию не имеют разрешения создавать объекты в схеме `public`. Однако *будьте осторожны!* До PostgreSQL 14 все пользователи имели привилегии `CREATE` в схеме `public`. Это означает, что любая база данных, обновленная с PostgreSQL 14 или более ранней версии, сохранит эту привилегию. Чтобы удалить ее, можно выполнить команду:

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

Надежнее всего можно защититься от подобных злоупотреблений, ограничив круг лиц, которые могут определять объекты. Пользователи не могут получить доступ к объектам в схеме, которая им не принадлежит, если только владелец не предоставит им привилегии `USAGE` на эту схему. Им также потребуются привилегии `CREATE` на схему для создания объектов и т. д. С помощью ролей контролируйте, к каким схемам разрешен доступ каждому пользователю и в каких схемах им разрешено выполнять `CREATE`. Если это невозможно, удалите из `search_path` все схемы, которые позволяют ненадежным пользователям создавать объекты, и реализуйте в запросах явные ссылки на объекты доверенных владельцев.

Также следует избегать чрезмерного использования схемы `public`. Если у вас сложная система, вероятно, стоит создать отдельные схемы для каждого компонента приложения или команды. Если у вас несколько tenants, их данные также могут находиться в разных схемах.

В целом, хотя схема `public` и `search_path` дают удобство, необходимо тщательно контролировать их использование, чтобы исключить риски от перехвата запросов (`query hijacking`) и повышения привилегий (`privilege escalation`) через переопределение имени объекта.

Итоги

- Использование `psql -w` или `--password` может сбивать с толку и приводить к нарушениям безопасности. Вместо этого полагайтесь на автоматический встроенный механизм запроса пароля в PostgreSQL.

- Установка `listen_addresses = '*'` может сделать сервер СУБД открытым для небезопасных сетей, поэтому следует включать только те доверенные сетевые интерфейсы, которые необходимы для подключения к базе данных.
- Использование метода `trust` в `pg_hba.conf` в продакшен-средах неприемлемо. Всегда следует обеспечивать надлежащую аутентификацию на сервере и максимально ограничивать доступ.
- Владение базами данных и их содержимым суперпользователем может привести к проблемам безопасности и случайному повреждению данных. Вместо этого создавайте роли, которые имеют только соответствующие разрешения для владения и управления этими базами данных, и выборочно предоставляйте разрешения другим ролям.
- Использование `SECURITY DEFINER` при объявлении функции может вызвать утечки данных и сделать возможным повышение привилегий. Чтобы снизить риск, используйте это объявление как можно реже и с безопасным `search_path`, а также используйте по возможности комбинацию `SECURITY INVOKER` с явным предоставлением привилегий через `GRANT`.
- Отсутствие защиты `search_path` может позволить другим перехватывать запросы и повышать свои привилегии. Реализуйте строгий контроль над созданием объектов в схемах и ссылайтесь в запросах только на объекты, принадлежащие доверенным пользователям.

Нерекомендуемые практики, снижающие уровень доступности

В этой главе

- ✓ Как внедрить надлежащую гигиену резервного копирования
- ✓ Почему не стоит игнорировать фичи восстановления PostgreSQL
- ✓ Почему базы данных бывают не готовы к сбою
- ✓ Чем грозит использование неподходящих инструментов

Еще одно обстоятельство, благодаря которому PostgreSQL завоевал репутацию в отрасли, — это его знаменитая отказоустойчивость. Эта способность основана на лучших практиках и специальных инструментах. Да, есть некоторый риск, что из-за самоуспокоенности в духе «Postgres же устойчивый» вопросы высокой доступности (High Availability, HA) могут уходить на второй план. Однако именно высокая доступность (HA) должна быть в центре плана развертывания любого кластера баз данных в продакшене.

Когда в этой главе я говорю о *высокой доступности*, я имею в виду методы и приемы, которые обеспечивают доступность вашей базы данных для пользователей на протяжении всех сценариев сбоев, от незначительных до катастрофических, а также позволяют ей восстанавливаться после этих сбоев. Цель высокой доступности — гарантировать минимальное время простоя или

потери данных с учетом того, что требования и допустимые пределы у каждой организации свои.

К сожалению, на практике мы можем найти системы Postgres, администраторы которых пренебрегают некоторыми (если не всеми!) принципами высокой доступности. У такого поведения несколько причин: заблуждения, чрезмерная уверенность в железе и софте, избыточное снижение расходов или просто полное непонимание важности этой темы. Если ваша база данных откажет или потеряет данные, будет уже слишком поздно вспоминать о высокой доступности. Следовательно, единственный способ гарантировать, что ваши данные останутся в безопасности, — это быть наготове, держать ухо востро и действовать на опережение. Посмотрим, где все может начать идти не так.

9.1. Отсутствие резервного копирования

Дружелюбный системный администратор в какой-то другой компании (не в Frogge Emporium; они уже достаточно пострадали!) знает о рисках, связанных с потерей данных своей организации. Чтобы снизить вероятность такого исхода, он решил использовать стандартные решения: диски, на которых хранится база данных, зеркалируются через RAID 1, так что даже в случае сбоя жесткого диска будет резервный диск, на который можно переключиться.

RAID

RAID, или избыточный массив независимых дисков (Redundant Array of Independent Disks), — технология хранения данных, которая распределяет их по нескольким устройствам; это повышает производительность, обеспечивает избыточность или и то и другое одновременно. Различные схемы организации дисков, которые также определяют роли и обязанности каждого диска в схеме, называются уровнями RAID. Существуют различные уровни, определяющие чередование (striping), зеркалирование (mirroring) и схемы с контролем четности, и эти уровни также можно комбинировать, вкладывая их друг в друга. В нашем случае RAID1 (зеркалирование) хранит побайтовую копию данных на других дисках, поэтому для него требуется минимум два устройства.

Помимо RAID, для пущей безопасности администратор настроил резервный сервер, используя потоковую физическую репликацию. Теперь у них двойная избыточность, как показано на рис. 9.1. Значит, все прекрасно, так?

Но почему же этих мер недостаточно? Во-первых, RAID (любой) не может защитить от ошибок файловой системы. Если логическая структура файла или данные внутри него повреждены, это повреждение идеально дублируется на избыточные диски, и теперь у вас есть два (или более) идентичных диска с поврежденными данными файловой системы.

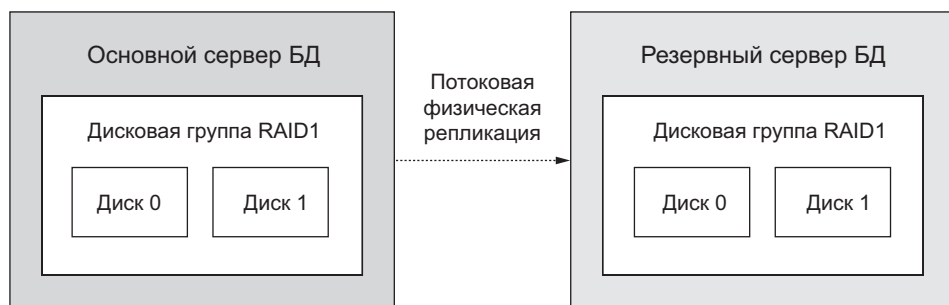


Рис. 9.1. Схема установки PostgreSQL, демонстрирующая двойную избыточность на физическом уровне благодаря резервному серверу, а также зеркалированию дисков на каждом из серверов

А если бы они также включили, используя какую-либо виртуализированную или физическую реализацию, механизм снимков для файловой системы? Для некоторого ПО этого бывает достаточно, чтобы с помощью снимков восстановить все до безопасного состояния. Однако PostgreSQL — это сложная система со своим собственным журналом предзаписи (WAL), буферами, состояниями транзакций, сегментированными файлами данных и т. д. Целостность его данных, как мы знаем, зависит от синхронизации WAL и файлов данных. Следовательно, небезопасно делать копию на уровне файлов, если не гарантируется атомарность снимка (по аналогии, как после вспышки на цифровом фотоаппарате, мы мгновенно получили бы снимок всей файловой системы целиком). Но даже в этом случае результат не идеален, потому что когда мы восстанавливаем этот дисковый снимок и запускаем Postgres, то это выглядит так, как будто произошел сбой. Поэтому при запуске Postgres переходит в режим восстановления после сбоя и приступает к повторному воспроизведению всех зафиксированных транзакций из WAL.

Следует помнить, что восстановление снимка файловой системы поверх директории данных (*data directory*) работающего экземпляра PostgreSQL — это верный путь к катастрофе из-за почти гарантированных несоответствий, которые приведут к повреждению ваших данных. Поэтому не делайте этого. Мы видели, что если Postgres остановлен до восстановления файловой системы, он ведет себя так, как будто произошел сбой при его повторном запуске, но по крайней мере все несоответствия устраняются. На практике единственный безопасный способ сделать снимок — это чистое завершение работы базы данных перед созданием снимка (или перевод ее в режим заморозки путем вызова `pg_backup_start()` до и `pg_backup_stop()` после). Однако для многих пользователей это грозит простым, который они не могут себе позволить.

ВНИМАНИЕ Если данные находятся в нескольких табличных пространствах (`tablespaces`), то есть на разных дисках, то снимки файловых систем этих табличных пространств небезопасно использовать, если только снимок не атомарен для суммы файловых систем, что технически довольно сложно реализовать.

Теперь рассмотрим, почему избыточность в виде физической реплики также недостаточна. Допустим, на основном сервере произошла такая пользовательская оплошность:

```
DROP TABLE patient_data;
```

Какой-то разгильдяй удалил важную таблицу в продакшене. Это изменение немедленно реплицируется на резервный сервер, и теперь таблица исчезла и оттуда. Никакая, даже сверхчеловеческая скорость не поможет вовремя остановить распространение этого на другой сервер. Единственный способ вернуть таблицу — это восстановить ее из резервной копии, а ее нет.

Даже в отсутствие ошибки в SQL-команде репликация не гарантирует безопасности данных. К сожалению, некоторые типы повреждений на основном сервере (например, повреждение журнала WAL из-за ошибки диска или памяти или запуск двух экземпляров Postgres на одной и той же PGDATA) могут быть реплицированы на резервный сервер. Если данные повреждены до того, как они передаются потоком, реплика получит плохие данные.

Отложенные реплики не решают эту проблему. Даже если настроить задержку репликации на 8 или 24 часа, где гарантия, что вы осознаете ошибку не через 8 часов и 1 секунду или, если на то пошло, через 24 часа и 1 миллисекунду? Помните, что ошибка, повреждение или злонамеренное действие могут быть не столь очевидными, как `DROP TABLE patient_data`. Ошибка организации из нашего примера заключалась в том, что, поскольку у них были диски RAID1, снимки серверов и избыточный резервный сервер, они посчитали, что им не нужно делать резервные копии.

Итак, решения на аппаратном уровне, такие как RAID, не могут защитить от повреждений, возникающих на любом уровне выше аппаратного. Решения на уровне блоков или файлов (например, снимки) также нельзя считать адекватной стратегией резервного копирования для PostgreSQL, поскольку ими необходимо тщательно управлять и синхронизировать их с серверным процессом PostgreSQL. В идеале необходимо использовать подходящие инструменты для создания резервных копий, которые осведомлены о семантике процесса PostgreSQL, файлов данных и WAL, чтобы они случайно не нарушили целостность ваших данных. `pg_basebackup` — это безопасный инструмент резервного копирования, который поставляется вместе с PostgreSQL для создания безопасных снимков каталога данных без необходимости остановки СУБД.

ПРИМЕЧАНИЕ Официальную документацию `pg_basebackup` можно найти по адресу <https://www.postgresql.org/docs/current/app-pgbasebackup.html>.

Наконец, ошибочно полагать, что снимки или репликация смогут равноценно или адекватно заменить хорошие резервные копии, и в следующем разделе мы увидим больше примеров этого. Резервные копии дают возможность восстановить данные независимо от вашего оборудования или настройки репликации. Поэтому любой опытный администратор СУБД должен считать отсутствие резервных копий непростительной оплошностью из-за высоких рисков для выполнения операций.

9.2. Отсутствие точки восстановления на момент времени

Наконец администратор СУБД прозрел и решил немедленно начать резервное копирование. Затраты времени на это считаются оправданными, и все рады услышать, что их база данных станет безопаснее. Традиционная концепция резервного копирования базы данных включает дампирование данных с последующим их сохранением в безопасном месте, поэтому они решают использовать `pg_dump`. Согласно документации, это также «утилита для резервного копирования базы данных PostgreSQL»¹, и поскольку она поставляется вместе с Postgres, она, по-видимому, хорошо подходит для этой цели.

Резервные копии создаются. Каждая база данных в кластере успешно копируется по очереди, потому что `pg_dump` может делать дампы только одной за раз. Эти резервные копии гарантированно будут согласованными, поскольку `pg_dump` получает снимок для извлечения данных и нет необходимости прерывать операции с базой данных. Однако стоит отметить: нельзя гарантировать согласованность резервных копий двух взаимозависимых баз данных, пока обе они работают.

Утилита `pg_dumpall` расширяет область резервного копирования на весь кластер, по сути, циклически выполняя `pg_dump` для каждой базы данных. Хотя это удобно и гарантирует включение глобальных объектов, общих для всех баз данных, резервные копии по-прежнему между базами данных не согласованы.

¹ Здесь автор специально допускает упрощение для приводимого примера. Как раз в документации PostgreSQL в описании утилиты `pg_dump` подчеркнуто очевидное: `pg_dump` не подходит для регулярного резервного копирования рабочих баз данных. Это инструмент логического резервного копирования небольших баз данных. — *Примеч. науч. ред.*

Оценим, где мы оказались. У нас есть безопасные дампы данных, которые можно отлично восстановить в пустую базу данных. Отлично! Теперь по порядку рассмотрим проблемы, которые мы упомянули в предыдущем разделе.

Защищены ли мы от потери всех наших данных? В основном да. Мы можем восстановить их по желанию, независимо от того, что произойдет с нашими серверами. И хотя защита не абсолютна, у нас есть данные только до момента последнего резервного копирования и мы можем утратить некоторые из самых последних транзакций, не все потеряно. Защищены ли мы от повреждения данных? Опять же и да и нет. Напомню, у нас есть снимок данных, но только до момента последнего проверенного безопасного резервного копирования. То же самое относится к случайному или злонамеренному повреждению данных. Даже если вы точно определите, когда это произошло, вы сможете откатиться только к моменту, когда была сделана та самая копия, и восстановить ее.

Это означает, что даже если вы ежедневно создаете резервные копии своей базы данных, условная команда `DROP TABLE`, выполненная во второй половине дня, приведет к потере всех данных, созданных с утра, без возможности их восстановления. То же самое справедливо для снимков файловой системы и любого метода полного резервного копирования, а не только для `pg_dump`.

Именно поэтому великий и, увы, ныне покойный Саймон Риггс (Simon Riggs) реализовал функцию *восстановления на момент времени* (*Point-in-Time Recovery, PITR*) в кодовой базе PostgreSQL примерно за 20 лет до написания этих страниц. PITR позволяет эффективно откатить вашу базу данных к точной транзакции, которая вам нужна. Иными словами, можно все восстановить до *конкретной* транзакции *перед* тем, как произошло повреждение, поскольку с PITR вы не ограничены только тем состоянием, которое было зафиксировано в последней полной резервной копии базы данных. Это достигается за счет использования журнала предзаписи (WAL); в конце концов, у нас есть идеальная и воспроизводимая запись каждой транзакции без исключения.

Для выполнения PITR используется комбинация *базовых резервных копий* (*base backups*) (создаваемых утилитой PostgreSQL `pg_basebackup`) и архивированных файлов WAL. Базовые резервные копии представляют собой полные, бинарно-точные снимки всего кластера базы данных, а не только отдельных баз. Архивирование файлов WAL также известно как *непрерывное резервное копирование* (*continuous backup*), поскольку если вы сохраняете все WAL, которые постоянно генерировались PostgreSQL с момента создания базовой резервной копии, это позволяет вам восстановиться до любой точки во времени после создания базовой резервной копии.

Все, что вам нужно сделать, чтобы получить эту возможность, — это настроить архивирование WAL, определив пользовательскую команду `archive_command`

в файле `postgresql.conf`, а затем создать полную резервную копию с помощью `pg_basebackup`. Помните, что базовые резервные копии по сути являются снимками, поэтому для PITR вам также нужна непрерывная цепочка WAL, начиная с периода выполнения резервного копирования и далее — вплоть до целевого момента восстановления. Сам же процесс восстановления PITR сводится к следующему: восстанавливаем сначала полную резервную копию, сделанную `pg_basebackup`, затем выставляем параметр `recovery_target_time` на момент, до которого хотим восстановиться, настраиваем соответствующую пользовательскую команду `restore_command` на источник с архивированными WAL, создаем файл-сигнал `recovery.signal` для запуска процесса восстановления и запускаем PostgreSQL.

В PostgreSQL 17 была представлена возможность, облегчающая резервное копирование очень больших баз данных: создавать инкрементальные резервные копии с использованием `pg_basebackup`. При этом такие инкрементальные резервные копии нельзя использовать напрямую, но их можно объединять (сливать) с базовой полной резервной копией и последующими «инкрементами» с помощью `pg_combinebackup`, получая на выходе новую полную резервную копию.

СОВЕТ Вы можете прочитать о линиях времени (timelines) в разделе «Непрерывное архивирование и восстановление на момент времени» (Continuous Archiving and Point-in-Time Recovery) документации PostgreSQL (<https://www.postgresql.org/docs/current/continuous-archiving.html#BACKUP-TIMELINES>). В нем объясняется, как можно вернуться во времени до фиксации транзакции и создать ответвления на разные линии времени столько раз, сколько потребуется, чтобы точно определить нужный момент для возврата, при этом не перезаписывая и не теряя никаких данных.

Возвращаясь на мгновение к теме `pg_dump`, отмечу, что существуют и другие серьезные проблемы с использованием этой утилиты в качестве инструмента резервного копирования. `pg_dump` выполняет *логическое*, а не *физическое* резервное копирование. Это означает, что он сохраняет описание того, как повторно создать базу данных, а не фактические байты данных. Как следствие, при восстановлении из `pg_dump` PostgreSQL требуется заново создать бинарную структуру каждого восстанавливаемого объекта на диске. Сюда входят индексы, которые не копируются, а повторно создаются на ходу, поэтому восстановление может занять довольно много времени. Кроме того, теряется внутренняя статистика вашего старого кластера (поскольку это уже новый кластер), и статистику таблиц придется накапливать с нуля¹.

¹ В PostgreSQL 18-й версии реализована возможность выгрузки базовой статистики таблиц и индексов утилитами `pg_dump` и `pg_dumpall`. В планах у 19-й версии добавление возможности выгрузки расширенной статистики, включая статистику по выражениям. — *Примеч. науч. ред.*

Если вы не используете PITR, то рискуете получить легко предотвратимую потерю данных, и, возможно, вам придется снова долго и мучительно восстанавливать недостающие данные. PITR и непрерывное архивирование могут потребовать много места на дисках, поскольку нужно хранить полные резервные копии вместе со всеми журналами WAL с момента создания этих копий. Тем не менее PITR считается важной фишкой для систем с высокой нагрузкой и для корпоративного использования, где критически важна высокая надежность.

9.3. Резервное копирование вручную

Итак, с учетом предыдущих разделов прилежный администратор СУБД решил ежедневно выполнять резервное копирование базы данных и, раз уж процесс настолько важен, лично контролировать процесс. Однако жизнь такова, что иногда нужно отдыхать. И вот наступает утро понедельника, нужно сделать резервную копию, а администратора нет на месте. Где же он? А он в свой заслуженный отпуск нежится под теплым солнцем на пляже.

Случилась ошибка?

Нет! Прилежный администратор СУБД добросовестно делегировал эту задачу разработчику. Разработчик действительно выходит на работу в понедельник утром и продолжает ежедневно делать резервную копию базы данных до конца недели, пока администратор СУБД отсутствует.

Когда администратор СУБД возвращается на следующей неделе, чувствуя себя отдохнувшим, он смотрит на сервер и с ужасом обнаруживает, что разработчик сохранил резервные копии из директории `PGDATA /mnt/pgdata` в директорию с названием `/mnt/pgdata/backups`! Это не только неверное место для резервного копирования, но, что еще хуже, оно находится в той же файловой системе и на том же физическом диске, что и база данных. Катастрофы удалось избежать, но лишь чудом, поскольку организации повезло, что за время отсутствия администратора СУБД не произошел сбой сервера.

Из этого следует извлечь два урока. Во-первых, не следует хранить резервную копию на том же железе, что и базу данных; всем известно, что оборудование со временем выходит из строя. Некоторые идут дальше и требуют, чтобы всегда существовала удаленная копия резервных копий на случай, если весь объект сгорит или произойдет что-то еще.

Во-вторых, никогда нельзя полагаться на ручные процессы для защиты данных. Люди совершают ошибки, отвлекаются или забывают, пропускают работу, уходят в отпуск или на другую работу. Они могут делать резервное копирование в разное время дня или даже пропускать дни. Хуже всего, что иногда они склонны хранить все знания о процедуре резервного копирования в своей голове, поэтому

они нигде не записаны. Думаю, все согласится, что всем этим переменчивым и неопределенным факторам не место в ИТ-системе.

Ключ к правильной гигиене резервного копирования — исключить человеческий фактор и полагаться на автоматизацию. Идеальный вариант — это неинтерактивная система, которая автоматически регулярно создает резервные копии базы данных. Разумеется, необходимо следить за этой системой на случай, если она по какой-либо причине выйдет из строя. В то же время нужно использовать непрерывное архивирование WAL (Write-Ahead Log) для целей PITR (Point-in-Time Recovery, восстановление на момент времени).

ВНИМАНИЕ Имейте в виду, что нельзя смешивать и совмещать логические и физические резервные копии. Нельзя выполнить PITR, имея резервную копию, созданную с помощью `pg_dump`, и набор WAL-файлов.

Для автоматизации резервного копирования с использованием `pg_basebackup` можно настроить автоматическое расписание с помощью надежного планировщика задач, такого как `cron`, который доступен в любой UNIX-системе. В мире open-source для задач резервного копирования PostgreSQL существуют и другие популярные специализированные инструменты: Barman (Backup and Recovery Manager) и pgBackRest (Postgres Backup and Restore). Они упрощают PITR и поддерживают расширенные сценарии, такие как сложное расписание, резервное копирование нескольких серверов, несколько расположений для избыточности, политики хранения, выбор методов передачи и параллельное выполнение. Настройка этих инструментов сводится к установке пакетов и добавлению нескольких строк в нужные конфигурационные файлы.

Можно спроектировать архитектуру резервного копирования, которая будет выглядеть примерно как на рис. 9.2. Например, можно автоматически создавать резервные копии кластера баз данных с помощью Barman, используя `pg_basebackup` для получения базовых резервных копий, а репликационное подключение — для получения WAL-файлов для PITR. В качестве альтернативы Barman может настроить команду `archive_command`, которая будет отправлять WAL-файлы через `rsync`. Это позволит параллельно передавать несколько файлов, что поможет справиться с огромным объемом генерируемых WAL-файлов.

ПРИМЕЧАНИЕ Документацию и сами инструменты резервного копирования PostgreSQL можно найти по адресам: <https://pgbarman.org/> (Barman) и <https://pgbackrest.org/> (pgBackRest).

Наконец, помимо того что ручные процессы уязвимы к ошибкам, они плохо масштабируются; бессмысленно ожидать, что кто-то будет регулярно выполнять резервное копирование десятков или сотен серверов баз данных — вполне

реалистичные цифры для нынешних крупных организаций. Признаю, отрицание автоматизации в части резервного копирования не относится прямо к PostgreSQL или даже к базам данных в целом, но эта проблема встречается в отрасли чаще, чем может показаться. И абсолютно непростительно выполнять резервное копирование вручную при таком обилии инструментов резервного копирования, планировщиков и средств для автоматизации в экосистеме PostgreSQL. Можно даже сказать: резервное копирование без автоматизации — это баловство.

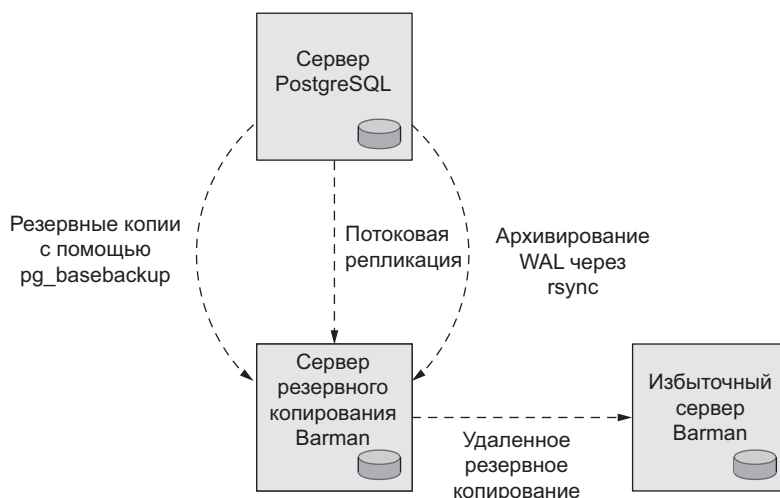


Рис. 9.2. Пример настройки резервного копирования PostgreSQL с выделенным сервером Barman и географической избыточностью, показывающий возможные пути передачи

9.4. Отсутствие тестирования резервных копий

Еще одна проблема, о которой сколько ни говори, все мало. Вспомним печально известные инциденты с участием важных имен в отрасли, когда компании теряли данные, хотя резервные копии у них были, — они просто слишком сильно полагались на то, что эти копии позволят восстановиться без ошибок.

Представим, что все прошлые рекомендации этой главы были учтены: резервные копии с Postgres создаются автоматически и безопасно хранятся в двух местах вместе со всеми необходимыми WAL для PITR. Наступает момент, когда сервер выходит из строя, и администратор СУБД заботливо достает резервную копию для восстановления базы данных. Однако закон Мерфи снова вступает в силу,

и резервная копия не работает: команда восстановления завершается с ошибкой. Теперь у компании нет ни базы данных, ни резервной копии для ее восстановления. Ситуация аховая.

Как все могло пойти настолько плохо? Потенциальных причин для сбоя хватает:

- резервная копия может быть повреждена на стороне источника (например, из-за неправильного вызова команды, что привело к созданию непригодной для использования резервной копии);
- процесс резервного копирования создает пригодную копию, но носитель данных в какой-то момент повреждает ее;
- резервное копирование проходит успешно, но что-то не так с местом хранения, и оно теряет резервную копию после ее размещения.

Как узнать заранее, что нечто подобное произошло? В конце концов, сам по себе процесс резервного копирования не проверяет, можно ли потом будет восстановить данные, и не дает никаких намеков на такие случайности.

Решение одно — надо *тестировать*. Единственный способ на 100 % убедиться, что резервные копии доступны и работают корректно, — это попытаться их восстановить!

Ставки здесь высоки. В конце концов, резервная копия — это то, что должно спасти вас от потери данных, но она может отказать именно тогда, когда нужна больше всего. Если вы не хотите, чтобы это случилось с вами, очевидно, что тестирование резервных копий — это абсолютно необходимый процесс. В конце концов, команды могут быть выполнены неправильно, приложение — дать сбой, диски — выйти из строя, сеть — «поплыть» и т. д.

Тестировать резервные копии PostgreSQL можно, восстанавливая их в тестовом окружении. Наряду с самой резервной копией вы также протестируете процедуру восстановления, что послужит полезной тренировкой для вас и вашей команды на случай реальной чрезвычайной ситуации.

Вот пример того, как протестировать резервную копию, восстановив ее в новом экземпляре Postgres:

1. Скопируйте базовую резервную копию в целевой каталог PGDATA тестового окружения.
2. Проверьте контрольные суммы базовой резервной копии с помощью `pg_verifybackup`.
3. Скопируйте все журналы WAL, получившиеся после создания данной резервной копии, в PGDATA тестового окружения.

4. Вызовите `pg_waldump` для файлов WAL, чтобы убедиться, что они парсятся.
5. Запустите Postgres из каталога PGDATA тестового окружения, чтобы восстановить базу данных и убедиться, что она может прийти к согласованному состоянию.

Чтобы успокоиться полностью, можно просмотреть каждую страницу в восстановленной базе данных и убедиться, что она корректно читается. Для этого достаточно использовать модуль `pageinspect`, чтобы в цикле обработать каждую страницу каждого отношения. Страницу читают с диска функцией `get_raw_page()`, заголовок проверяют через `page_header()`, а контрольную сумму — через `page_checksum()`.

ПРИМЕЧАНИЕ Вы можете ознакомиться с тем, как использовать модуль `pageinspect`, на странице <https://www.postgresql.org/docs/current/pageinspect.html>.

Процедура проверки резервных копий также отлично подходит для автоматизации, и если на каком-то шаге обнаруживаются ошибки, то вам должны приходиться оповещения.

А завершу я этот разговор кейсом, из которого видно, насколько далеко все может зайти, если не соблюдать рекомендации. Компания, лидер отрасли, органично росла на протяжении многих лет. С самого начала она использовала PostgreSQL для своей базы данных, которая играла центральную роль в их операциях. Можно даже сказать, что компания не могла бы существовать без этой базы данных, поскольку в ней содержались критически важные сведения для предоставляемых услуг. В один несчастливый день сервер базы данных упал: отказал жесткий диск.

Они ужаснулись, когда узнали, что их резервная копия непригодна, а последняя рабочая копия была сделана много месяцев назад, что делало большую часть данных неактуальными для текущих операций. Когда компания была еще стартапом, в ней делались резервные копии (что хорошо) с помощью `pg_dump` (что плохо, потому что с тех пор PostgreSQL обзавелся такими фичами, как потоковая репликация, базовые резервные копии и PITR, и следовало бы взять на вооружение их). Это, кстати, еще одна причина, по которой не стоит использовать `pg_dump` в качестве инструмента резервного копирования даже при полностью настроенной автоматизации. Вы сильно рискуете, если полагаетесь на скрипты, написанные несколько десятилетий назад, вместо того чтобы использовать специальную утилиту для резервного копирования, которая получает обновления и исправления. Пользовательские скрипты могут в любой момент тихо отказать — что и произошло в этом случае. Скрипт, который компания всегда использовала для резервного копирования, перестал работать, а они об этом даже не подозревали. Вполне возможно, что ваш «супернадежный» скрипт

перестанет работать, когда вы перенесете его в виртуализированную среду или запустите внутри контейнера Docker, пода Kubernetes и т. д.

Единственная причина, по которой это событие не убило компанию, заключалась в том, что у них была по крайней мере одна старая резервная копия. В любом случае компания была близка к гибели, и это, безусловно, был тревожный звонок. К счастью, им удалось вернуть большую часть содержимого файловой системы с поврежденного диска, но без какой-либо узнаваемой структуры. В конце концов я кропотливо восстановил всю их базу без потери данных, однако это уже другая, очень длинная история, и не для этой книги.

По аналогии с предыдущим разделом можно сказать: резервное копирование без тестирования — это баловство.

9.5. Отсутствие избыточности

Теперь рассмотрим случай, когда стратегия резервного копирования идеальна: Frogge Emporium использует правильные инструменты PostgreSQL, регулярно делает автоматические резервные копии и так же автоматически их тестирует. Теперь они чувствуют себя достаточно защищенными, и их работе, как им кажется, ничто не угрожает.

Что произойдет, если сервер сгорит? Придется приобрести новый и перенести на него резервную копию, а это займет время. Конечно, риск можно уменьшить, подготовив запасной сервер, и именно так поступают облачные провайдеры. Даже если компания пользуется услугами хостинга, ей все равно придется восстанавливать всю систему базы данных на пустом физическом сервере или на новом облачном инстансе.

Хотя такая процедура занимает гораздо меньше времени, простой все еще остается значительным, и здесь мы начинаем говорить о терминах аварийного восстановления: *RPO* (Recovery Point Objective — целевая точка восстановления) и *RTO* (Recovery Time Objective — целевое время восстановления). *RPO* — это объем данных, который мы можем позволить себе потерять в теоретическом сценарии катастрофы. С этим мы справляемся довольно хорошо, поскольку PITR (Point-in-Time Recovery — восстановление на момент времени) позволяет вернуть данные вплоть до последней транзакции, зафиксированной в последнем файле WAL (Write-Ahead Log — журнал предварительной записи), который удалось сохранить. *RTO* — это время простоя, которое мы можем себе позволить до полного восстановления сервиса, и оно полностью зависит от нашей архитектуры высокой доступности.

Доступные средства PostgreSQL для построения решений высокой доступности — немного истории

Со времен доставки журналов (log shipping) репликация была основной частью архитектуры высокой доступности PostgreSQL. Высокая доступность достигалась через непрерывное архивирование, в ходе которого файлы WAL копировались на другой сервер с помощью команды `archive_command`. Из-за этого, начиная с PostgreSQL 8.2, появилась концепция «теплого» резервного сервера (warm standby), который был готов взять на себя функции основного сервера по команде.

С появлением *поточковой репликации* в PostgreSQL 9.0 WAL начал автоматически и асинхронно передаваться на другой сервер. Благодаря этому «теплые» резервные серверы стали получать более актуальные данные, чем это было при копировании целых файлов WAL, что позволило использовать эти серверы также и для обслуживания данных. Возможность подключения к резервному серверу для выполнения запросов только для чтения известна как «горячий» *резервный сервер* (hot standby) (еще одна фишка Риггса).

PostgreSQL 9.1 разрешил *синхронную* репликацию. Благодаря синхронной потоковой передаче WAL резервные серверы всегда гарантированно были актуальны по отношению к основному, поскольку на них фиксировались абсолютно те же транзакции. Это исключает любую потенциальную потерю данных, которые не были переданы на резервный сервер из-за задержки репликации, но за это приходится расплачиваться некоторой потерей производительности (из-за необходимости двустороннего обмена данными для записи транзакции на обеих сторонах).

В версии 9.2 была добавлена возможность *каскадной* репликации, или потоковой передачи данных с одного резервного сервера на другой и от него далее. Используя резервный сервер в качестве ретранслятора, можно разгрузить основной сервер и уменьшить количество соединений репликации, которые ему необходимо обрабатывать.

С появлением Postgres 9.4 появились *слоты репликации* (replication slots), которые решили проблему преждевременного удаления WAL основным сервером до того, как его сможет скачать резервный сервер. Слоты репликации гарантируют, что WAL остается на диске до тех пор, пока он не будет передан последнему резервному серверу, которому он необходим. Однако это создает потенциальную проблему неограниченного накопления WAL на диске, если резервный сервер будет не отвечать или отсутствовать.

В релизе 9.5 добавилось непрерывное архивирование на резервном сервере, благодаря чему реплики смогли иметь собственный отдельный архив WAL. Наконец, в PostgreSQL 10 была представлена нативная логическая репликация, но основное назначение этой возможности заключается отнюдь не в гарантиях высокой доступности.

Имея в своем распоряжении множество разнообразных вариантов репликации, архивирования данных и иных средств, ориентированных на построение решений высокой доступности, PostgreSQL в итоге способен закрыть практически любой сценарий высокой доступности. Если в центре вашего бизнеса стоит СУБД, то ограничиваться одним узлом в продакшен-сценарии нецелесообразно. На момент написания этих строк смиряться со временем простоя (RTO) длительностью в несколько дней (или даже одного дня) практически немыслимо, а в большинстве сценариев счет RTO идет на минуты или секунды. Для очень требовательных приложений существуют даже решения, которые позволяют выполнять *аварийное переключение* PostgreSQL в диапазоне миллисекунд. Напомню, что PostgreSQL — это система с одним мастером (single-master), соответственно, нельзя иметь два primary-сервера для ваших баз данных одновременно. Переключение — это процесс *повышения* резервного (standby) сервера до нового первичного (primary). Мы подробнее рассмотрим разные механизмы переключений в разделе 9.6.

Используя репликацию данных для обеспечения избыточности узлов вашего решения, вы существенно снижаете уязвимость базы данных к таким событиям, как отключение электроэнергии, отказ оборудования или даже повреждение самого дата-центра, где размещено оборудование. Когда нет необходимости ждать восстановления резервных копий и можно просто переключиться на другой сервер и продолжить работу в обычном режиме, это имеет огромное значение для вашей организации; просто представьте, сколько денег может потерять банк или торговая фирма за считанные секунды, если их система недоступна. Для справки: банковская система SWIFT обрабатывает около 11,5 миллиона международных платежей каждый день. Учитывая средний размер платежа в 45 000 долларов США, это означает, что *каждую секунду* совершается транзакций почти на 6 миллионов долларов США. Помимо потери дохода, также следует учитывать репутационный ущерб, который несет организация во время недоступности сервиса.

Если ваше мобильное приложение для соцсети вместо котиков показывало «Извините, что-то пошло не так», то это и была недоступность сервиса с точки зрения пользователя.

Вы можете создать свое решение высокой доступности с учетом своих требований, например, как показано на рис. 9.3. На схеме представленного решения видно, что вместо двух у нас уже три сервера баз данных. Если вы не хотите терять заданный уровень избыточности в период, пока один сервер отключен для обслуживания или полностью вышел из строя, вам потребуется второй резервный (standby) сервер. Чтобы не создавать дополнительную нагрузку на наш первичный (primary) сервер (из-за репликации и резервного копирования), мы

настроили каскадную репликацию с первого резервного сервера как на второй резервный сервер, так и на сервер резервного копирования.

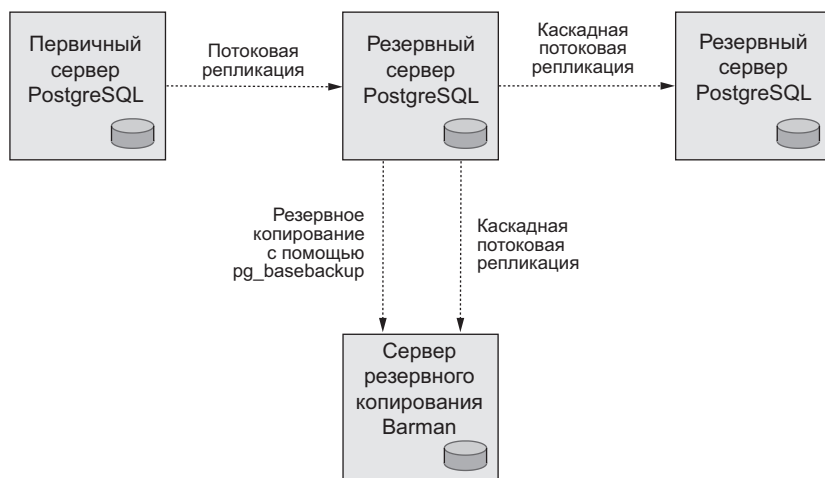


Рис. 9.3. Пример архитектуры решения высокой доступности PostgreSQL с каскадной репликацией для избыточности и резервного копирования

Быстрое восстановление после сбоя — это то, что нужно практически всем, кто профессионально работает с PostgreSQL, и обеспечение избыточности — лучший способ достичь этого. Реализуйте потоковую репликацию и держите один или несколько резервных узлов. В качестве бонуса вы можете использовать их для разгрузки первичного узла, направляя на них трафик с запросами на чтение.

9.6. Игнорирование специальных инструментов высокой доступности

Как и в случае с резервным копированием, ваше решение высокой доступности может значительно выиграть от использования автоматизации. Рассмотрим следующий случай: инженеры Frogge Emporium написали собственные скрипты аварийного переключения на резервный (standby) сервер PostgreSQL при недоступности первичного (primary) сервера по любой причине. Они тщательно протестировали их в лаборатории и уверены в логике скриптов. Однако лабораторная среда отличается от реальной эксплуатации: они не учли возможную задержку в репликации и то, что одна из реплик может отставать. Можете ли вы предположить, что произойдет, если основной сервер выйдет из строя, а переключение выполнится на отстающую реплику?

В данном случае другие реплики уже получают больше транзакций через WAL, чем новый основной сервер. Когда отстающая реплика будет повышена до нового основного сервера, ей будет не хватать этих транзакций. Это приведет к необходимости сложной процедуры извлечения недостающих транзакций из реплик, у которых они есть, и их реинтеграции в линию времени и данные основного сервера. В противном случае выйти на ожидаемый RPO не получится. Более того, этот дополнительный сегмент WAL окажется несовместимым с линией времени нового основного сервера, поскольку линии времени разошлись при повышении реплики до нового основного сервера. В результате ни одна из других реплик не сможет продолжить следовать (*follow*) за новым основным сервером без *перемотки* (*rewinding*) своего WAL до точки, предшествующей разделению линий времени. Под термином «следовать» мы подразумеваем возможность продолжать получать WAL и воспроизводить его с этой точки, а «перемотка» — это просто возврат к более ранней точке.

Именно поэтому не следует изобретать велосипед и разрабатывать свои скрипты и собственную логику высокой доступности, которая должна учитывать все возможные крайние случаи, — разумнее использовать готовые инструменты и решения, созданные экспертами, которые предусмотрели различные сценарии отказов и протестировали свое программное обеспечение.

Рассмотрим другой подобный сценарий. Между двумя центрами обработки данных, в которых расположены основной и резервный узлы соответственно, происходит разрыв сетевого соединения на несколько минут. Скрипт, управляющий переключением на узле-реплике, обнаруживает, что основной сервер Frogge недоступен, и принимает решение сделать реплику новым основным сервером. Однако исходный основной узел продолжает нормально работать и принимать операции записи, поскольку сбой произошел только в соединении между ним и резервным сервером. Когда сетевое соединение восстанавливается, мы получаем два узла Postgres, работающих как основные серверы, с расходящимися данными. Это явление известно как *split-brain*, и возникшая рассинхронизация данных требует ручного разрешения, как и в предыдущем примере.

Специализированные инструменты высокой доступности лучше, поскольку они могут обнаруживать и обрабатывать подобные сбои, используя механизмы, такие как узлы-свидетели (*witness nodes*), кворум (*quorum*) и ограждение (*fencing*), для предотвращения ситуации *split-brain*. Эти инструменты высокой доступности осведомлены обо всех узлах кластера высокой доступности, о состоянии каждого экземпляра PostgreSQL и позиции WAL, чего сам PostgreSQL пока не обеспечивает. Благодаря этому они могут эффективно координировать переключение при отказе и автоматически согласовывать линии времени WAL, используя такие инструменты, как `pg_rewind`. Некоторые инструменты высокой доступности могут даже автоматически клонировать резервные серверы

из основного (или из резервной копии) и включать их в текущий HA-кластер с помощью одной команды. Кроме того, большинство из них имеют встроенные hook-скрипты, механизмы мониторинга, оповещений и условных действий, например автоматическую перенастройку PgBouncer в соответствии с новым состоянием кластера после аварийного или планового переключения.

Вы можете использовать надежные open-source решения, такие как RepMgr (произносится «реп-менеджер», сокращенно от replication manager) и Patroni¹ для развертывания и обслуживания вашего кластера высокой доступности с проверенными механизмами Failover/Switchover переключений. При использовании PostgreSQL в Kubernetes применяется оператор CloudNativePG, который задействует встроенные HA-паттерны Kubernetes для реализации логики переключений при отказе и логики резервного копирования, адаптированной под PostgreSQL.

Ручная обработка аварийных переключений и обеспечение согласованности — задача сложная, подверженная человеческим ошибкам и зачастую требующая больше времени, чем допустимо в вашей организации с учетом требований к HA. Кроме того, пользовательские скрипты часто не могут учесть нюансы правильной логики переключения при отказе PostgreSQL и семантики линий времени, а также обработать пограничные случаи. Вашу архитектуру можно сделать надежнее благодаря многократно проверенным фичам известных инструментов HA — с ними вы будете лучше готовы исключить рассинхронизации данных, простой и потерю информации.

Итоги

- RAID и снимки файловой системы не дадут вам гарантий восстановления после повреждения данных, человеческих ошибок или злонамеренных действий. Лучшая гарантия безопасности данных — создавать резервные копии через подходящие инструменты, такие как `pg_basebackup`.
- Если вы создаете исключительно полные резервные копии, это делает вас уязвимыми к потере данных между моментами создания таких копий. Используйте механизм восстановления на момент времени с непрерывным архивированием WAL, чтобы иметь возможность вернуть базу данных к точке, предшествующей ее повреждению.
- Создавать резервные копии вручную ненадежно и опасно; лучше реализовать автоматическое выполнение резервного копирования, желательно с помощью

¹ На практике Patroni сегодня является де-факто стандартом и одним из наиболее зрелых инструментов для построения решений высокой доступности на базе PostgreSQL с автоматизированным failover/switchover. — *Примеч. науч. ред.*

специальных инструментов, которые учитывают специфику PostgreSQL (например, Barman или pgBackRest), а также обеспечить хранение иной резервной копии во втором, независимом расположении.

- Непроверенные резервные копии могут подвести в самый критический момент, поэтому для подтверждения работоспособности всегда тестируйте их, выполняя полное восстановление. Не полагайтесь исключительно на автоматизацию при тестировании резервных копий, проверяйте каждый шаг. Избегайте самописных скриптов и отдавайте предпочтение проверенным решениям.
- Использование базы данных из одного узла неизбежно приведет к простоям. Обеспечьте избыточность, настроив резервные (standby) серверы и репликацию.
- Ручное аварийное переключение или самописные скрипты грозят длительным простоем, расхождением данных или их потерей. Чтобы надежно и эффективно управлять вашим HA-кластером, отдавайте предпочтение проверенным инструментам, таким как RepMgr, Patroni и CloudNativePG для Kubernetes.

10

Нерекомендуемые практики обновления и миграции

В этой главе

- ✓ Почему важно следить за версией обновления (и за примечаниями к каждому выпуску)
- ✓ Какие последствия могут быть у недостаточно тщательного тестирования перед обновлением или миграцией
- ✓ Как учитывать различия между разными типами и версиями баз данных

Обновление PostgreSQL выполняется довольно просто, тогда как миграция с другой реляционной СУБД на PostgreSQL может быть несколько сложнее. Однако оба процесса критичны и требуют тщательной подготовки, и к ним нельзя относиться легкомысленно. Если что-то пойдет не так, обновление или миграция могут вызвать проблемы с производительностью, простои или даже повреждение данных. Ошибки и недочеты возникают из-за непонимания как процесса обновления баз данных в целом, так и особенностей обновления PostgreSQL в частности.

10.1. Пропуск примечаний к выпуску

В главе 7 я упоминал, что в PostgreSQL отлично реализована обратная совместимость, и, скорее всего, код, написанный для старой версии Postgres, будет прекрасно работать и в самой новой версии. По этой причине, а также благодаря сознательному решению разработчиков Postgres, как правило, не накладывает жестких ограничений на процесс обновления. Это означает, что обычно можно пропускать промежуточные версии, если между ними нет слишком большого разрыва, например, нет необходимости в обновлении с релиза десятилетней давности. Так, переход с версии 13 на версию 16, как правило, проходит надежно и без серьезных рисков.

Наш условный администратор СУБД решает поступить именно так, чтобы воспользоваться новыми возможностями и улучшениями производительности в актуальной версии, поэтому планирует и выполняет обновление с PostgreSQL 13 до 16. Как и ожидалось, все проходит гладко, и, хотя компания выделила на техническое обслуживание 30 минут, утилита `pg_upgrade` быстро справляется с задачей, и обновление завершается всего за несколько минут.

Все идет хорошо, пока в базе данных не начинают происходить странные вещи. В частности, после обновления скидка по программе лояльности для абонементов в спортзал стала рассчитываться неверно, и бухгалтерия спрашивает почему. Компания предлагает постоянным клиентам спортзала одну треть года бесплатно при оформлении абонемента еще на полный год. Например, если кто-то оформляет абонемент 2024-01-01, он должен получить подписку, которая закончится 2025-04-01. Однако теперь бухгалтерия заметила, что та же самая подписка, начинающаяся 2024-01-01, теперь заканчивается 2024-05-01, предоставляя клиенту дополнительный бесплатный месяц. Код приложения, связанный со скидками за лояльность, не менялся, поэтому администратор СУБД счел это расхождение очень странным и решил провести расследование.

Соответствующий фрагмент кода выглядел так:

```
IF existing AND fullyear THEN
    expiry = expiry + '0.333 years';
END IF;
```

Этот фрагмент идиоматически добавляет треть года к дате истечения срока действия подписки только для существующих клиентов, продлевающих абонемент на полный год (не спрашивайте почему — у отдела маркетинга свои соображения).

Чтобы проверить расхождение, администратор СУБД написал небольшой тестовый скрипт:


```
DO $$
DECLARE
    existing boolean; fullyear boolean; expiry timestampz;
BEGIN
    existing := true;
    fullyear := true;
    expiry = '2025-01-01';
    IF existing AND fullyear THEN
        RAISE NOTICE 'Expiry date: %', (expiry + '0.333 years')::text;
    END IF;
END $$;
```

В этом скрипте переменным `existing` и `fullyear` присваивается значение `true`, а также предполагается исходная дата истечения срока действия `expiry` — `2025-01-01`. Администратор СУБД запускает этот код на экземпляре PostgreSQL 13:

```
NOTICE:
Expiry date: 2025-04-01 00:00:00+01
DO
```

Этот результат соответствует ожиданиям. Затем администратор СУБД запускает тот же код на экземпляре PostgreSQL 16:

```
NOTICE:
Expiry date: 2025-05-01 00:00:00+01
DO
```

Следовательно, это подтверждает, что результат зависит от версии PostgreSQL! Но как и почему? После кропотливого изучения документации администратор СУБД находит ответ: в разделе о типе данных `interval` в документации PostgreSQL 13 указано:

«Дробные части единиц, превышающих месяцы, усекаются до целого числа месяцев»,

тогда как в документации PostgreSQL 16 говорится:

«Дробные части единиц, превышающих месяцы, округляются до целого числа месяцев».

Изменение одного-единственного слова — с «усекаются» на «округляются» — имеет решающее значение.

Чтобы проиллюстрировать это развернуто, администратор СУБД выполняет более детализированный запрос. Сначала он запускает его на экземпляре PostgreSQL 13, где получает следующий результат:

```
postgres=# SELECT 1.333*12 as months, ('1.333 years'::interval) as months;
 months |      months
-----+-----
```

```
15.996 | 1 year 3 mons
(1 row)
```

Затем аналогично он запускает его на экземпляре PostgreSQL 16:

```
postgres=# SELECT 1.333*12 as months, ('1.333 years'::interval) as months;
 months |      months
-----+-----
 15.996 | 1 year 4 mons
(1 row)
```

Теперь разница очевидна и соответствует объяснению в документации. Подписка на 15,996 месяца раньше усекалась до 15 месяцев, а теперь, после обновления, округляется до 16 месяцев.

Администратор СУБД прочитал заметки о выпуске PostgreSQL 16, но там не было никакого предупреждения об этом! Однако каким бы неразумным ни казалось добавление интервала «треть года» (*a third of a year*) к типу `timestampz` со стороны беззаботного разработчика, именно это ему поручил менеджер по продукту, которому, в свою очередь, дало указание маркетинговое подразделение.

В данном случае ответственность лежит на администраторе СУБД. Ему следовало прочитать все промежуточные заметки о мажорных релизах, чтобы выявить изменения, внесенные между Postgres 13 и 14, 14 и 15 и, наконец, 15 и 16. Несмотря на то что пропуск обновлений возможен и даже желателен, необходимо быть осмотрительнее и знакомиться со всем набором заметок о релизе. Если бы администратор СУБД это сделал, он бы обнаружил, что это изменение было введено в PostgreSQL 15, и сэкономил бы себе время и силы.

Производительность и поведение PostgreSQL могут незначительно меняться между мажорными выпусками. В нашем примере дробный интервал, указанный в годах, ранее усекался, и срок действия абонемента заканчивался 1 апреля, тогда как, начиная с версии 15, он округляется до ближайшего месяца, продлевая этот срок до 1 мая.

В заметках о выпуске PostgreSQL 15.0 от 13 октября 2022 года указано:

> Если при вводе интервала задано дробное значение для единицы измерения, превышающей месяца, выполнять округление до ближайшего месяца (Брюс Момджан (Bruce Momjian)).

ПРИМЕЧАНИЕ Дробный ввод для интервала объясняется в документации Postgres здесь: <https://www.postgresql.org/docs/current/datatype-datetime.html#DATATYPE-INTERVAL-INPUT>.

Какой вывод можно из этого сделать? Во-первых, неоднозначность в коде, считающем скидки, может стоить денег, но еще важнее то, что обновление без

учета всех изменений в поведении между двумя релизами может привести к всевозможным пограничным случаям, которые неочевидны сразу после обновления. Не нужно игнорировать промежуточные заметки о релизе, даже если вы пропускаете всего одну версию, потому что это рискованно: в одном из промежуточных релизов могло появиться неизвестное вам несовместимое изменение. И это пропущенное изменение может впоследствии потребовать много времени и ресурсов, чтобы его выявить и исправить.

Взгляните на график выпуска PostgreSQL на рис. 10.1 и обратите внимание, сколько минорных версий появляется на протяжении жизненного цикла каждой мажорной версии.

График выпуска версий PostgreSQL

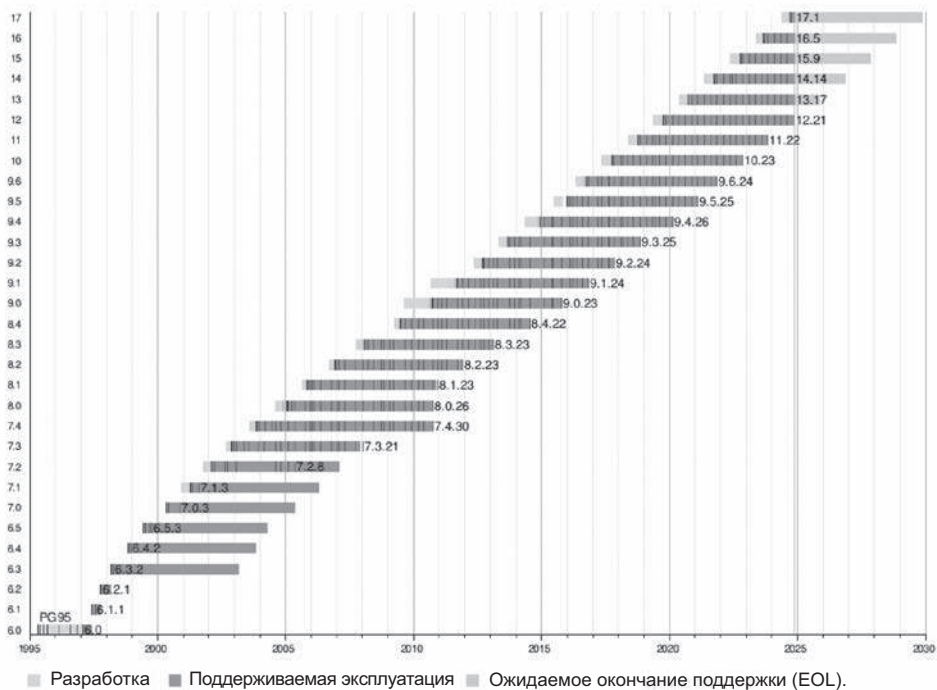


Рис. 10.1. Графическое представление выпуска версий PostgreSQL. Сгенерировано на основе данных, доступных по лицензии Creative Commons Attribution-ShareAlike 4.0, с использованием https://en.wikipedia.org/wiki/Template:Timeline_PostgreSQL

Суть в том, что если вы не прочитали все заметки о релизах между исходной и целевой версиями, то вы грубо нарушили логику процесса обновления. Обязательно обратите внимание на устаревшие или измененные имена функций, усовершенствования в параметрах конфигурации и настройках по умолчанию, а также на обновления синтаксиса SQL.

10.2. Недостаточное тестирование

Ошибку из предыдущего раздела на самом деле можно было бы выявить, если бы перед обновлением проводилось более тщательное тестирование. Теперь рассмотрим другой случай, когда недостаточное тестирование перед обновлением или миграцией также может привести к неприятным последствиям.

Компания Frogge Emporium при обновлении PostgreSQL с 11 до 12 начала замечать некоторые проблемы с производительностью. В частности, один из их запросов (который мы рассматривали в разделе 2.3) формирует список адресов электронной почты клиентов, которым было отправлено уведомление о неоплаченном счете за услуги (а не за приобретенные товары). В инструменте мониторинга этот запрос стал неожиданно работать хуже. Но разве мы не говорили в других частях этой книги, что код, написанный для PostgreSQL 11, будет нормально работать в PostgreSQL 12?

Инженеров Frogge заинтересовало такое поведение, и они решили провести исследование. Они развернули два экземпляра PostgreSQL: один — 11-й версии, второй — 12-й версии, чтобы выполнить сравнительное тестирование. Далее, чтобы выявить различия, запустили один и тот же запрос с оператором EXPLAIN ANALYZE на разных экземплярах.

Запрос на экземпляре с PostgreSQL 11:

```
EXPLAIN (ANALYZE, COSTS OFF, TIMING OFF)
  WITH unp AS (
    SELECT id, customer c, order_group AS og
    FROM erp.invoices
    WHERE paid = false
  ),
  ni AS (
    SELECT og.id
    FROM erp.order_groups og
    JOIN erp.orders o ON o.order_group = og.id
    WHERE o.item IS NULL
  )
  SELECT DISTINCT email
  FROM erp.customer_contact_details ccd
  JOIN unp ON unp.c = ccd.id
  JOIN ni ON ni.id = unp.og
  JOIN erp.sent_emails se ON se.invoice = unp.id
  AND se.email_type = 'Invoice reminder';
```

Полученный план выполнения, несколько сокращенный, выглядит так:

```

-----
QUERY PLAN
-----
HashAggregate (actual rows=9 loops=1)
  Group Key: ccd.email
  CTE unp
    -> Gather (actual rows=1350 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        -> Parallel Seq Scan on invoices (actual rows=450 loops=3)
            [...]
  CTE ni
    -> Gather (actual rows=1350 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        -> Nested Loop (actual rows=450 loops=3)
            -> Parallel Seq Scan on orders o (actual rows=450 loops=3)
                [...]
            -> Index Only Scan using order_groups_pkey on order_groups
                └og (actual rows=1 loops=1350)
                    [...]
    -> Merge Join (actual rows=9 loops=1)
        Merge Cond: (unp.og = ni.id)
        [...]
    -> Hash Join (actual rows=1350 loops=1)
        Hash Cond: (unp.id = se.invoice)
        -> Hash Join (actual rows=1350 loops=1)
            Hash Cond: (unp.c = ccd.id)
            [...]
Planning Time: 0.216 ms
Execution Time: 50.033 ms

```

Мы видим, что два выражения СТЕ запустили параллельное сканирование таблиц. Отфильтрованные из них результаты стали доступны для последующих хеш-соединений (HashJoin) и, далее, для хеш-агрегации (HashAggregate), которая нужна для группировки. Обе эти операции работают относительно быстро для больших наборов данных. Время выполнения запроса составило 50 миллисекунд.

Затем сотрудники компании запускают запрос с тем же оператором EXPLAIN на экземпляре с PostgreSQL 12 и с удивлением обнаруживают иной план выполнения для того же запроса!

```

-----
QUERY PLAN
-----
Unique (actual rows=9 loops=1)
  -> Sort (actual rows=9 loops=1)
      Sort Key: ccd.email
      Sort Method: quicksort      Memory: 25kB
  -> Nested Loop (actual rows=9 loops=1)
      -> Nested Loop (actual rows=9 loops=1)
          [...]

```

```

-> Parallel Hash Join (actual rows=3 loops=3)
    Hash Cond: (o.order_group = invoices.order_group)
    -> Parallel Seq Scan on orders o (actual
        rows=450 loops=3)
        [...]
    -> Parallel Hash (actual rows=450 loops=3)
        [...]
        -> Parallel Index Scan using
            ↪invoices_pk on invoices (actual rows=450 loops=3)
            [...]
            -> Sort (actual rows=1347 loops=3)
                [...]
                -> Seq Scan on
                    ↪sent_emails se (actual rows=1350 loops=3)
                    Filter: (email_type
                        ↪ = 'Invoice reminder'::email_type)
                        [...]
Planning Time: 0.538 ms
Execution Time: 85.025 ms

```

Как видим, план выполнения PostgreSQL 12 отличается: в нем нет упоминания СТЕ и есть два уровня вложенных циклов (*Nested Loop*). Внутренний вложенный цикл выполняет сканирование индекса (*index scan*), которое из-за внешнего цикла выполняется многократно. Фильтрация также происходит после соединений (*join*), а это значит, что ненужные данные удаляются позже, после обработки большего объема информации. Наконец, результаты сортируются, а затем проходят через операцию *Unique*. В результате запрос теперь выполняется за 85 миллисекунд.

Похоже, что Postgres сначала построил СТЕ в план, а потом выбрал другой путь выполнения. Это не обязательно означает, что такой план запроса хуже во всех случаях, но так уж получилось, что в этом случае он медленнее.

Как мы упоминали ранее, поведение PostgreSQL может незначительно меняться между основными релизами, и это один из таких случаев. Начиная с версии PostgreSQL 12, СТЕ стали автоматически встраиваться в план, когда на них ссылаются только один раз в запросе, при условии, что они не рекурсивны и не имеют побочных эффектов.

Еще одно изменение между релизами 11 и 12, которое многих застало врасплох, — включение¹ по умолчанию JIT-компиляции (*JIT, just-in-time*). Это привело к тому, что некоторые быстрые и оптимальные запросы стали медленнее, поскольку Postgres тратил дополнительное процессорное время, напрасно пытаясь их оптимизировать с помощью JIT. Если не проводить регрессионное

¹ После долгих споров в PostgreSQL 19 JIT снова будет выключен по умолчанию. На момент подготовки данного комментария версия PostgreSQL 19 еще не вышла, но патч, выключающий JIT для PostgreSQL 19, уже принят. — *Примеч. науч. ред.*

тестирование на реальных запросах, используя 11-ю версию в качестве базовой, то такого рода замедление будет трудно обнаружить. Причина необходимости тестирования запросов в том, что запросы могут превысить порог `jit_above_cost`, который запускает автоматический оптимизатор JIT.

Оба этих примера показывают: успешное обновление не гарантирует, что все будет работать как раньше. Нужно тщательно протестировать приложение и код со свежей версией Postgres. СУБД способна оптимизировать запросы по-новому, что может привести к совершенно иным планам выполнения и повлиять на производительность.

Тестируйте запросы на реальных данных (или максимально приближенных к ним), чтобы выявить возможные проблемы. Идеально проводить такие проверки в предпрод-среде перед обновлением продакшена, включая в проверку все ключевые запросы и пограничные случаи. Обязательно проводите нагрузочное тестирование — оно покажет, как обновление скажется на общей производительности системы.

10.3. Погружение в хаос кодировок

Многие компании переходят на PostgreSQL с других СУБД. Часто мигрируют с MySQL или MariaDB — именно такую задачу получил администратор СУБД в следующем примере. Информация копилась годами, база данных организации объемна, в нее входят персональные данные людей со всего мира, в именах которых попадаются спецсимволы.

На первый взгляд миграция не должна вызвать трудностей: в ПО использовался стандартный SQL, и оно прошло тестирование на полную совместимость с Postgres. Администратор сделал дамп базы MySQL в SQL-файл, затем импортировал его в Postgres:

```
psql service=staging -f prod_dump.sql
```

Терминал начал заполняться ошибками:

```
ERROR:
invalid byte sequence for encoding "UTF8": 0xe9
```

Как такое возможно? Unicode (в данном случае UTF-8) должен справляться со всеми именами в базе, поскольку поддерживает международные символы. Найдя строку данных, вызвавшую ошибку, администратор определил, что `0xe9` — это шестнадцатеричное значение символа `é` в кодировке `latin1`.

При дальнейшем расследовании выяснилось, что инсталляция MySQL, которая использовалась на момент запуска приложения, применяла `latin1` как кодировку по умолчанию. Хотя с тех пор ее обновили до версии, где по умолчанию

используется `utf8mb4`, база продолжала работать с `latin1`, так как таблицы никогда не конвертировали явно.

Что такое `utf8mb4`? Почему в MySQL по умолчанию не `utf8`? Все из-за того, что `utf8` в MySQL — это неполноценная реализация UTF-8, поддерживающая только символы длиной до 3 байт. Это означает, что она не может кодировать многие международные символы Unicode или эмодзи. Поскольку это не настоящий UTF-8, в MySQL добавили кодировку `utf8mb4`, поддерживающую 4-байтные символы. Вместо того чтобы исправить плохую реализацию, MySQL выбрала запутанное нестандартное имя для хорошо известной кодировки UTF-8, сохранив старый и сырой `utf8` для обратной совместимости.

После конвертации дампа из `latin1` в UTF-8 администратор повторил импорт, но снова получил ошибку от PostgreSQL:

```
ERROR: invalid byte sequence for encoding "UTF8": 0x00
```

Оказалось, что часть данных содержала *нулевые символы* (ASCII NUL), представленные шестнадцатеричным `\x00` или восьмеричным `\0`. В документации PostgreSQL объясняется: «Строковая константа не может содержать нулевые символы».

Это еще одна ловушка: даже если другой источник данных или СУБД (в этом случае MySQL) допускает у себя строки с нулевыми символами, это не гарантирует, что их примет PostgreSQL, даже во «всеядной» кодировке UTF8.

Ошибка заключалась в том, что администратор не учел различия в кодировках между MySQL и PostgreSQL. Не определив правильную исходную и целевую кодировки и не выполнив необходимую конвертацию при миграции, администратор столкнулся с ошибками *invalid byte sequence*, то есть «неверная байтовая последовательность». Кроме того, некорректная обработка недопустимых символов в исходных данных привела к сбоям импорта.

Основные различия, которые следует учитывать:

- PostgreSQL по умолчанию использует UTF8 и строго проверяет корректность UTF-8 кодировки, отвергая символы с неверными байтовыми последовательностями и нулевыми символами.
- Старые версии MySQL и MariaDB по умолчанию работают с `latin1` (также известной как ISO-8859-1, или кодовая страница Windows-1252), тогда как новые версии могут использовать `utf8` или `utf8mb4`. Как уже упоминалось, `utf8` в этих СУБД нужно до 3 байт на символ, что исключает поддержку многих символов Unicode.
- В Oracle, начиная с версии 8.1.7, UTF8 означает только Unicode revision 3.0, поэтому полная поддержка современного UTF-8 требует кодировок с еще более странными названиями, например AL32UTF8.

- Поддержка текста в Microsoft SQL Server, как и можно было ожидать, ориентирована на кодовые страницы Windows. Начиная с SQL Server 2012, Unicode поддерживается через тип данных NVARCHAR, использующий кодировку UTF-16. Поддержка UTF-8 появилась только в SQL Server 2019.

Упуская из виду подобные нюансы, вы ставите под угрозу всю миграцию: можно не только вызвать задержки, но и нарушить целостность данных, незаметно испортив их. Например, если кодировка в целевой СУБД принимает символы, которые затем невозможно корректно декодировать обратно, данные могут быть безвозвратно утеряны.

Безопасный сценарий миграции данных выглядит следующим образом:

1. Определите исходную кодировку.
2. Экспортируйте данные в той же кодировке.
3. Преобразуйте данные в UTF-8. Пример с использованием UNIX-утилиты `iconv`:

```
iconv -f latin1 -t utf8 prod_dump_latin1.sql -o prod_dump_utf8.sql
```

4. Очистите данные, удалив символы, которые PostgreSQL не принимает. Например, удалите нулевые байты с помощью UNIX-утилит `tr` или `sed`

```
tr -d '\000' < prod_dump_utf8.sql > prod_dump_clean.sql
sed 's/\x0//g' prod_dump_utf8.sql > prod_dump_clean.sql
```

5. Убедитесь, что целевая база данных сконфигурирована так, чтобы иметь возможность использовать кодировку UTF8 и подходящие параметры локали:

```
ENCODING = 'UTF8'
LC_COLLATE = 'en_US.UTF-8'
LC_CTYPE = 'en_US.UTF-8'
```

6. Импортируйте очищенные данные.

И последнее — хотя я об этом уже не раз говорил: перед началом конвертации и миграции тщательно все протестируйте на реальных или приближенных к реальным данным.

СОВЕТ Инструменты вроде `pgloader` и `ora2pg` помогают в задачах миграции: автоматически управляют наборами символов и конвертируют данные. `pgloader` доступен по адресу <https://github.com/dimitri/pgloader>, а `ora2pg` — <https://github.com/darold/ora2pg>.

10.4. Разный BOOLEAN

Еще одна распространенная ошибка, связанная с предыдущей, — невнимание к тому, как в базе данных используются логические значения. Приведем простой

пример, похожий на ситуацию с миграцией из предыдущего раздела: экспорт таблицы с биевыми значениями из, скажем, MariaDB.

Определение таблицы такое:

```
CREATE TABLE Election2024 (
  VoterID INT AUTO_INCREMENT KEY,
  Voted BOOLEAN NOT NULL DEFAULT false
);
```

Администратор СУБД экспортирует таблицу и пытается загрузить ее в PostgreSQL. Возникает ошибка:

```
psql:prod_dump.sql:24: ERROR: column "Voted" is of type boolean but
expression is of type integer
LINE 1:... "Voting"."Election2024" ("VoterID", "Voted") VALUES (500, 0);
                                         ^
```

HINT: You will need to rewrite or cast the expression.

Минуточку: столбец `Voted` в таблице определен как `BOOLEAN`. Посмотрим, что показывает MariaDB при запросе структуры таблицы:

```
DESCRIBE Election2024;
```

Field	Type	Null	Key	Default	Extra
VoterID	int(11)	NO	PRI	NULL	auto_increment
Voted	tinyint(1)	NO		NULL	

Оказывается, MariaDB и MySQL хранят `BOOLEAN` внутри как `TINYINT(1)`. Документация этих СУБД по типам `BOOL/BOOLEAN` подтверждает это:

Эти типы являются синонимами для `TINYINT(1)`. Нулевое значение считается ложью, ненулевые — истиной.

Такое представление сложно назвать стандартным для логических типов! Это означает, что импортировать числовые данные нельзя без предварительного преобразования в корректные SQL-значения `BOOLEAN`, поскольку PostgreSQL воспринимает экспортированные целочисленные литералы именно как целые числа.

Проблема с логическими типами существует и в других СУБД:

- В Microsoft SQL Server обычно используют тип `bit`, принимающий значения `0`, `1` и `NULL` для представления логических значений.
- IBM DB2 начал поддерживать `BOOLEAN` как тип столбца только после версии 11.1.1.1, выпущенной в 2016 году. Однако система допускает строковые значения `'0'` и `'1'`, а также числа `0` и `1` в таких столбцах.

- Oracle исторически имел ограниченную поддержку типа `BOOLEAN`. Изначально стандартной практикой было использование `CHAR(1)` для хранения `Y`, то есть `yes`, да, и `N` — `no`, нет (как это англоцентрично, правда?). Позже большинство перешло на `NUMBER(1)`. Удивительно, но полноценная поддержка `BOOLEAN` появилась только в версии 23с, выпущенной в 2023 году.

PostgreSQL полностью поддерживает стандартный SQL-тип `BOOLEAN`, делая `true` и `false` полноценными значениями. Строгое следование стандарту означает, что другие значения в логических столбцах хранить невозможно.

Если исходный столбец определен как `BOOLEAN`, но содержит нестандартные значения, потребуется преобразовать эти символьные или числовые значения в `true` и `false` в дамп-файле перед загрузкой, чтобы далее нативный `BOOLEAN` тип PostgreSQL корректно их принял при загрузке.

Даже если в исходной СУБД столбцы определены как псевдологические числовые или символьные типы, незачем сохранять эту практику в PostgreSQL. Следует учитывать, что запросы, неявно ожидающие логический тип, могут завершиться ошибкой из-за несоответствия типов. Для согласованности и соответствия стандарту преобразуйте такие столбцы в `boolean` после загрузки данных.

Предположим, таблица после миграции выглядит следующим образом:

```
Table "Voting.Election2024"
Column | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
VoterID | integer   |           | not null | generated always as identity
Voted   | integer   |           | not null | 0
Indexes:
    "Election2024_pkey" PRIMARY KEY, btree ("VoterID")
```

А данные содержат следующие значения:

```
VoterID | Voted
-----+-----
    500 |    0
    501 |    1
```

Выполним преобразование:

```
ALTER TABLE "Voting"."Election2024" ALTER "Voted" DROP DEFAULT;
ALTER TABLE "Voting"."Election2024" ALTER "Voted" SET DATA TYPE BOOLEAN
    USING CASE WHEN "Voted" = 1 THEN true ELSE false END;
ALTER TABLE "Voting"."Election2024" ALTER "Voted" SET DEFAULT false;
```

После преобразования структура таблицы принимает такой вид:

```
Table "Voting.Election2024"
Column | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
```

```

VoterID | integer |          | not null | generated always as identity
Voted   | boolean |          | not null | false
Indexes:
"Election2024_pkey" PRIMARY KEY, btree ("VoterID")

```

Данные теперь корректно хранятся:

```

VoterID | Voted
-----+-----
    500 | f
    501 | t

```

Таким образом, чтобы не возникали проблемы при миграции между СУБД, убедитесь, что логические столбцы используют стандартный тип `BOOLEAN`, а нестандартные псевдологические значения (например, целые числа или символы) преобразованы в `true` и `false`.

10.5. Некорректная обработка различий в типах данных

В предыдущих разделах мы уже рассмотрели некоторые проблемы, возникающие при переходе между СУБД, поэтому сосредоточимся на ключевых моментах.

Такие базы данных, как MySQL/MariaDB, SAP Sybase/Microsoft SQL Server, Informix и другие, используют нестандартный тип `DATETIME` для хранения даты и времени, подобно `TIMESTAMP`. Однако реализация этого непоследовательна: одни СУБД позволяют хранить доли секунд, другие обеспечивают точность до нескольких миллисекунд, третьи способны учесть миллисекунды или их доли. Еще одна проблема — некоторые системы допускают некорректные значения `DATETIME`: например, MySQL и MariaDB в нестрогом режиме (долгое время он использовался по умолчанию) принимают любые невалидные даты.

Наконец, самая серьезная проблема: при попытке преобразовать большинство этих `DATETIME` в соответствующий стандарту SQL тип PostgreSQL `TIMESTAMP WITH TIME ZONE` выясняется, что предыдущие реализации не поддерживают часовые пояса и должны рассматриваться как наивные (локальные) метки времени. Подробнее о том, почему это плохо и почему не следует использовать `TIMESTAMP (WITHOUT TIME ZONE)`, см. в разделе 3.1.

Непонятно, почему некоторые СУБД, например Oracle, используют тип данных `DATE` для хранения информации о дате и времени, но с точностью только до одной секунды! (Тип `TIMESTAMP` в Oracle также хранит дату и время, но включает доли секунд.)

Эта проблема выходит за рамки типов даты/времени. `NUMBER` в Oracle может хранить как целые числа, так и значения с плавающей точкой, но PostgreSQL

требует явного различения типов `numeric`, `integer` и `float/double precision`. Неправильное сопоставление типов данных может привести к потере точности или некорректным вычислениям.

Из этого следует, что по названию типа данных нельзя определить, что он хранит, и что между СУБД существуют серьезные несоответствия, — возможно, из-за (неудачных) проектных решений на ранних этапах их разработки. Не надо исходить из того, что типы данных с одинаковыми названиями полностью совместимы; всегда проверяйте их определение и поведение в документации. Используйте функции преобразования типов данных PostgreSQL, чтобы обеспечить совместимость, и применяйте инструменты миграции, такие как `pgloader` и `ora2pg`.

Итоги

- Всегда изучайте примечания к релизу (release notes) для всех версий между текущей и целевой версиями PostgreSQL перед обновлением. Пренебрежение этим может привести к неожиданному поведению из-за новых оптимизаций или критических изменений, а их диагностика и исправление потребуют значительного времени.
- Обновления PostgreSQL могут вносить, казалось бы, незначительные изменения в поведение или производительность, но это в итоге может стать проблемой. Тщательно тестируйте приложение и запросы в тестовой среде перед обновлением, используя реалистичные данные и нагрузки, чтобы выявить потенциальные проблемы заранее.
- Если вы игнорируете различия в кодировках символов и недопустимых символов при миграции базы данных, то рискуете обнаружить ошибки импорта или даже потерять данные. При миграции между различными СУБД обязательно корректно указывайте исходную и целевую кодировки, выполняйте необходимую конвертацию и очищайте данные от таких символов, как ASCII NUL. В этом помогут специализированные инструменты миграции.
- При переносе булевых данных в PostgreSQL убедитесь, что используется правильный тип `BOOLEAN`. Преобразуйте нестандартные представления (например, целые числа или символы) в `true` и `false`, чтобы обеспечить согласованность и предотвратить ошибки несоответствия типов.
- Несогласованность в именовании типов данных, их функциональности и точности между СУБД приводит к ошибкам миграции. Всегда проверяйте и учитывайте определения и поведение типов данных, а также используйте инструменты преобразования и утилиты миграции PostgreSQL для обеспечения совместимости.

11

О PostgreSQL, лучших практиках и о вас: заключительные выводы

В этой главе

- ✓ Какие существуют типы пользователей и профили использования PostgreSQL
- ✓ Как проактивное планирование помогает не накапливать технический долг и не усугублять проблемы
- ✓ С чего начать улучшать плохо спроектированную унаследованную базу данных PostgreSQL
- ✓ Как осмысленно использовать PostgreSQL, чтобы повысить производительность и надежность

11.1. К какому типу пользователей вы относитесь

Подробно разобрав ошибки и возможные проблемные сценарии, давайте сделаем шаг назад и взглянем на опыт работы с PostgreSQL более широко. Одна из причин, почему PostgreSQL так распространен и нравится совершенно непохожим друг на друга людям, заключается в том, что эта система способна удовлетворять их разнообразным требованиям. Да, пользователи PostgreSQL действительно очень сильно отличаются. Из профессионального опыта и общения с коллегами на конференциях я сделал вывод, что существует огромный спектр людей,

которые выполняют совершенно разные задачи с помощью этой СУБД, и каждый день появляются новые варианты ее использования!

Когда кто-то делится историей успеха, которого удалось добиться благодаря Postgres, это по-настоящему здорово — можно узнать о новом или необычном варианте его применения. Рассказ о неудачной попытке реализовать что-то средствами Postgres не менее интересен, поскольку понимание причин провала помогает избежать аналогичных ситуаций и выбрать альтернативный путь. Замечу, что именно на этом принципе построена вся инженерия надежности и безопасности, а также культура вокруг этой дисциплины.

Культура надежности и безопасности учит нас, что ошибки возникают из-за человеческого фактора, сбоев процессов, недостаточной подготовки или простого недопонимания. В этой книге мы видели ошибки, которые могут случиться практически по любой из этих причин. Признаем, что не все пользователи Postgres одинаковы — их роль в организации формирует определенный угол зрения или предвзятость в подходе к использованию базы данных. Осознание этого помогает понять, как узконаправленный фокус может привести к ошибкам, которых можно было бы избежать при комплексном и объективном рассмотрении проблемы.

Вот несколько типов пользователей и их характерные особенности. Это не полный список, а скорее некоторые наблюдения. PostgreSQL может использоваться как в профессиональной среде, так и в академических или домашних условиях.

11.1.1. Любитель

Вы — фулстек-разработчик или инженер данных. Вы следите за современными технологиями и в какой-то момент узнали о Postgres по сарафанному радио или из статей, рекламирующих его достоинства (или из книг вроде этой!). Возможно, вы энтузиаст, работающий над личным проектом с базой данных. Решив разобраться, в чем дело, вы начинаете погружаться в Postgres, чтобы научиться извлекать данные или проводить исследования, и он кажется подходящим решением.

Однако масштабирование работает не так хорошо, как обещали. Интеграция с привычными инструментами тоже вызывает сложности. Перед вами два пути: уйти в Postgres с головой, чтобы глубоко изучить технологию и экосистему и наконец достичь цели, либо отказаться и поискать «более простое» решение, не требующее тонкой настройки.

Первый путь заставит вас глубоко изучить базу данных и ее экосистему и превратит в настоящего профессионала или глубоко погруженного в тему любителя — как минимум эксперта в этой области. Второй путь может привести к тому, что вы будете пользоваться «волшебным» инструментом, который «просто

работает», но часто такая технология, обещающая все, покрывает лишь две трети ваших потребностей, остальное приходится доводить до ума кастомным кодом или ручной обработкой данных.

Итог каждого пути зависит от того, для чего именно вы хотите использовать Postgres. Подходит ли ваша задача для реляционных или документных (JSON) баз данных? Если да, то, скорее всего, вы сможете выстроить корректное и устойчивое решение. Если же вы опираетесь на иную парадигму данных — графовую модель, MapReduce или специфические форматы хранения вроде Parquet и Iceberg, — тогда успех может зависеть от сторонних компонентов: расширений PostgreSQL, механизмов Change Data Capture (CDC) или иных инструментов трансформации и репликации данных.

Но поиск идеального инструмента может превратиться в безрезультативную погоню за совершенством. Если вы знакомы со специалистами по данным, то наверняка заметили: больше всего их заботит точность и корректность данных. Удобство использования они ценят лишь постольку-поскольку. Несколько лишних кликов их не смущают — главное, чтобы в итоге получился стабильный источник достоверных данных и надежный метод их извлечения и анализа.

Такой теоретический любитель часто использует неправильно возможности PostgreSQL или типы данных, игнорирует соглашения SQL и лучшие практики, неверно понимает (или вовсе не читает) документацию и ищет быстрые решения, дающие мгновенный результат. Как мы видели в главах 2 и 3, подобные упрощения могут привести к серьезным проблемам в будущем.

11.1.2. Осторожный хранитель

Вы — добротный администратор баз данных и защитник информации в своей области. Вы досконально знаете лучшие практики благодаря лучшим результатам в учебной группе или многолетнему успешному опыту работы. Возможно, вы — опытный DevOps-инженер или SRE-инженер.

Однако некоторые вещи могут вас поставить в тупик. Один разработчик просит использовать JSON(B) для нереляционного хранения данных, другие обращаются с вопросами о распределенных технологиях, шинах событий и потоковой обработке. Вы неизменно возвращаете их с небес на землю, поскольку PostgreSQL — реляционная СУБД, работающая в продакшене. Все эти новомодные подходы могут нарушить целостность данных из-за отсутствия реализации ACID или снизить производительность системы из-за чрезмерных запросов или требований репликации — список можно продолжать. Вы постоянно концентрировались на основных принципах администрирования баз данных и могли упустить новейшие возможности PostgreSQL или тенденции последнего времени в технологиях данных.

Такой осторожный хранитель склонен придерживаться знакомых подходов, будь то опыт работы с другой СУБД или устаревшие практики вроде резервного копирования через логический дамп и восстановление. Нежелание пробовать новые или специфичные для PostgreSQL решения (см. главу 4) может приводить к негибкости, в результате чего будут упущены возможности синергии с современными технологиями. Как было сказано в главе 7, отставание от современных тенденций, включая отказ от обновлений, может завести в тупик.

11.1.3. Беспечный разработчик

Вы можете даже не знать, что под капотом PostgreSQL, и не интересоваться, как он работает на уровне данных. Для вас база данных — это некая константа, она «должна просто работать» с кодом, будь то язык программирования или фреймворк. Все, что вам нужно, — это просто одно подключение к БД (или несколько тысяч подключений, если на то пошло). С вашей точки зрения, база данных второстепенна для задачи, а «люди, отвечающие за БД», должны сами как-нибудь разобраться с запросами, которые ваше приложение или ORM шлет в сторону СУБД.

Возможно, вас интересует тема искусственного интеллекта и вы хотите погрузиться в тему векторизации и эмбедингов. Или у вас есть другие специализированные задачи, например работа с ГИС через PostGIS или использование других расширений PostgreSQL. Однако узкая специализация может помешать увидеть всю картину, а она заключается в том, что мимо СУБД вы не пройдете, все данные все равно хранятся и обрабатываются в ней. Еще вариант: вы слишком полагаетесь на возможности ORM и фреймворка и недооцениваете ключевые преимущества реляционных СУБД.

Такого человека часто называют *наивным конечным пользователем* или «беспечным разработчиком», и он склонен допускать базовые ошибки при извлечении данных или работе с SQL (при наличии доступа к SQL-интерфейсу), например порождать проблему N+1 запросов или выполнять фильтрацию на уровне приложения вместо базы. Эти проблемы мы обсуждали в главе 6. Это тот тип пользователя, который, если предоставить ему полную свободу действий, способен допустить аномалии сериализации в своем приложении или хранить даты в виде текстовых строк. Обычно помогает исправить ситуацию восстановление контекста — чтобы увидеть, как все взаимосвязано и как все компоненты могут оптимально работать вместе.

11.1.4. Выживальщик

Вы оказались в незавидной ситуации: вас «забросили» в эпицентр эксплуатации сложной системы или вы хорошо знакомы с реляционными СУБД в целом, но

не с PostgreSQL в частности. С другой стороны, ну чем таким этот PostgreSQL может отличаться от того, с чем вы сталкивались раньше?

Конечно, *много чем*. В мире существует огромное разнообразие сценариев использования Postgres, и благодаря кастомизации Postgres может занимать такие ниши, о которых обычный человек даже не задумывается. Все эти ниши и соответствующие рабочие процессы часто требуют специфических знаний, например понимания, что таблицы с высокой частотой обновлений нуждаются в агрессивном использовании `autovacuum`. Соответственно, вас может несколько ошеломить сложность существующей настройки, функций и техник PostgreSQL. Отказ от помощи или консультации из-за гордыни или желания показать, что вы держите ситуацию под контролем, может создать ненужные риски.

Выживальщик часто склонен без критического анализа следовать инструкциям из блогов, которые лишь приблизительно соответствуют поставленной задаче, или делегировать ее агентам языковых моделей с сомнительным результатом. Как и любой мощный инструмент, языковые модели могут творить чудеса в умелых руках, но становятся опасными при неграмотном использовании. Ощущая себя в ситуации выживания, такой специалист может заикнуться на решении конкретной задачи, упуская необходимость постоянно применять лучшие практики, особенно в вопросах безопасности и надежности (вспомните главу 9 о вредных практиках для высокой доступности).

Я описываю этих гипотетических пользователей не для того, чтобы создавать негативные образы, а чтобы стимулировать самоанализ. Для самопознания крайне важно определить конкретные потребности вас как пользователя PostgreSQL и сформулировать ожидания, ограничения, соглашения и предубеждения, связанные с вашей ролью в организации. Поняв свои особенности как пользователя СУБД, вы яснее осознаете свои естественные склонности и зоны, требующие повышенного внимания. Речь не о том, чтобы повесить на себя ярлык определенного типа, а о том, чтобы увидеть сильные и слабые стороны и эффективно и результативно использовать PostgreSQL.

Потому важно осознавать свои особенности, сопоставлять практический опыт с должностными обязанностями, а также осознанно углублять знания или обращаться за помощью к коллегам — все это помогает предотвращать ошибки.

11.2. Будьте проактивны: действуйте на опережение

В управлении базами данных каждое решение может вызвать каскад последствий для производительности, надежности и обслуживания. СУБД PostgreSQL становится отлаженной не отчаянным тушением пожаров, а тщательным анализом проблем и продуманным планированием.

Если выполнение запроса замедлилось или ухудшилось, значит, пора принять меры. Не ждите, пока:

- упадет производительность, чтобы начать оптимизировать запросы или добавлять индексы;
- произойдет потеря данных, чтобы приступить к внедрению надежной стратегии резервного копирования;
- упадет сервер, чтобы настроить и протестировать механизмы резервирования и отказоустойчивости;
- вам срочно понадобится функциональность новой версии PostgreSQL, чтобы в спешке планировать обновление;
- у вас начнут заканчиваться идентификаторы первичного ключа, чтобы искать решение для ключевого столбца;
- разрастется таблица до гигабайтов из-за мусора, чтобы приступить к оценке и корректировке настройки автоочистки;
- накопится миллиард строк в таблице, чтобы решить, что необходимо партиционирование;
- взлетят затраты на железо или облако, чтобы начать задумываться об оптимизации базы.

Когда видишь такое перечисление сценариев, они кажутся очевидными для любого профессионального администратора СУБД. Тем не менее примеры таких ситуаций появляются снова и снова. Зачастую, когда сильно занят тем, чтобы побыстрее получить результат, отвлекаешься от вопросов, связанных с долгосрочной стабильностью и производительностью.

Бдительность — первый шаг к тому, чтобы действовать на опережение, поэтому:

- проверяйте код разработчиков, чтобы выявлять ошибки и упущения;
- обращайте внимание на паттерны доступа к данным, которые могут стать будущими узкими местами в системе;
- анализируйте паттерны использования базы данных, чтобы выявлять аномалии и точки роста.

Составьте план действий. Протестируйте предлагаемое решение, развернув его в предпрод-окружении, максимально приближенном к продакшену. Затем внедряйте изменения в удобном темпе, с запасом по времени. Никто не хочет, чтобы проблема переросла в катастрофу, и работа на опережение от этого защищает.

Когда нужно что-то быстро разработать, многие аспекты можно проигнорировать и на время отложить решение. Однако применять лучшие практики, равно как и планировать масштабируемость, производительность, удобство обслуживания,

надо с первого дня. Действуя таким образом, вы сможете минимизировать или полностью исключить накопление технического долга. Профилактика проблем предотвращает их повторное появление в будущем.

В этом процессе вы не одиноки — используйте всю экосистему PostgreSQL себе на пользу. В ней есть инструменты для высокой доступности, резервного копирования, мониторинга и оповещений, анализа и визуализации статистики — и множество других способов и инструментов, чтобы заранее узнавать о потенциальных проблемах и решать их до того, как они приведут к сбоям.

Не забывайте о постоянном совершенствовании. Следите за развитием PostgreSQL и смежных технологий, выделяйте время на обучение (даже в формате неформальных сессий обмена знаниями). Участвуйте в сообществе PostgreSQL через публичные форумы и мероприятия. Благодаря постоянному движению вы расширяете кругозор и учитесь на чужом опыте. Разве не ради этого вы читаете эту книгу?

Создавайте и поддерживайте культуру, где ценится прогнозирование и подготовка. Поощряйте коллег и членов команды, чтобы они мыслили на перспективу, делились наблюдениями и обменивались знаниями.

11.3. Итак, вам досталась проблемная база данных

Такое случается часто. Вы приходите в проект и видите, что состояние базы данных оставляет желать лучшего. Однако это еще не конец света. Разберемся почему.

11.3.1. «Так исторически сложилось»

Иногда приходится работать с базами данных, спроектированными предшественником. Бывает, что в команде раньше не было администратора СУБД. Эти исторические наслоения иногда приводят к неприятным открытиям: например, можно обнаружить базы данных с кодировкой `SQL_ASCII`.

Доставшаяся вам база данных может оказаться настоящим кошмаром. Она могла стать результатом сомнительных решений, сжатых сроков или самонадеянности архитекторов. Важно помнить: те, кто ее создавал, вероятно, использовали все доступные на тот момент инструменты и знания, а текущий хаос мог возникнуть из-за органичного роста кодовой базы и набора данных.

11.3.2. Что дальше?

Что делать, когда столько всего нужно привести в порядок? Не паникуйте и вспомните старую поговорку: «Нет худа без добра». Воспринимайте ситуацию

как возможность начать исправлять проблемы, не доводя дело до катастрофы, и извлеките уроки из этого опыта.

В системе может быть множество проблем и недостатков: неудачный выбор кодировки или именования объектов, слишком широкие таблицы с сотнями столбцов и множественным назначением, чрезмерное количество узких таблиц, неэффективная архитектура мультитенантности с массой повторений или запутанный код. Администратор СУБД мог пренебречь основами: нормализацией, обеспечением целостности данных и созданием индексов.

Сделайте глубокий вдох; вспомните, что вы не одиноки и что все решаемо. Сначала определите, что требует немедленного вмешательства. Спросите тех, кто работает с системой, о главных проблемах — они с радостью поделятся этой информацией. Возможно, они жалуются на медленную работу, постоянную нехватку места на диске или частые перезагрузки из-за исчерпания памяти сервера.

Если есть возможность устранить наиболее серьезные и известные точки отказа, начните с этих быстрых побед, чтобы выиграть время для дальнейшего поэтапного и более системного улучшения архитектуры. Не попадайтесь в ловушку *«неправильно поставленной задачи» (XY problem)*: определите реальную причину неисправности, изучите лучшие практики и возвращайтесь к базовым принципам проектирования БД при исправлении ситуации. Неправильно поставленная задача — это распространенная ситуация, когда пользователь просит помочь ему решить вопрос, убрав лишь симптомы, вместо того чтобы устранять истинную, более глубокую, причину.

11.3.3. Начните с главного

Первым делом оцените ситуацию и изучите то, с чем столкнулись. Используйте `pg_dump` и другие инструменты вроде `pgAdmin` или `DBeaver` для извлечения, визуализации и анализа структуры схемы. Затем можно приступить к проверке данных и их качества с помощью исследовательских запросов. Это поможет глубже понять природу содержимого базы данных. Изучите конфигурацию PostgreSQL и проверьте, соответствует ли она тому, что вы обнаружили ранее. Проверьте журналы и работу базы данных через `pg_stat_activity`, `pg_stat_statements` и другие подходящие инструменты.

Эти помогут вам выявить подсказки, где искать источник проблемы: в коде приложения, в схеме или в конфигурации базы данных, может, даже в конвейере загрузки данных. Не пытайтесь исправить все сразу, лучше разбейте намеченные улучшения на проекты посильного размера с измеримыми целями.

Наследование проблемной базы данных заставляет не только исправлять существующие проблемы, но и готовить эту базу к будущему (рис. 11.1). Чтобы усилия окупались:

- документируйте все что можно. Поддерживайте актуальность документации по схеме, связям, конфигурациям и ключевым контекстным заметкам;
- автоматизируйте как можно больше процессов, которые обеспечивают высокую доступность, аварийное восстановление и обслуживание. Мы уже обсуждали важность этого, особенно в главах о нерекондуемых практиках администрирования (глава 7) и нерекондуемых практиках для высокой доступности (глава 9);
- поделитесь с командой списком рекомендаций и лучших практик по безопасности, проектированию схемы и запросов, а также индексации и оптимизации.

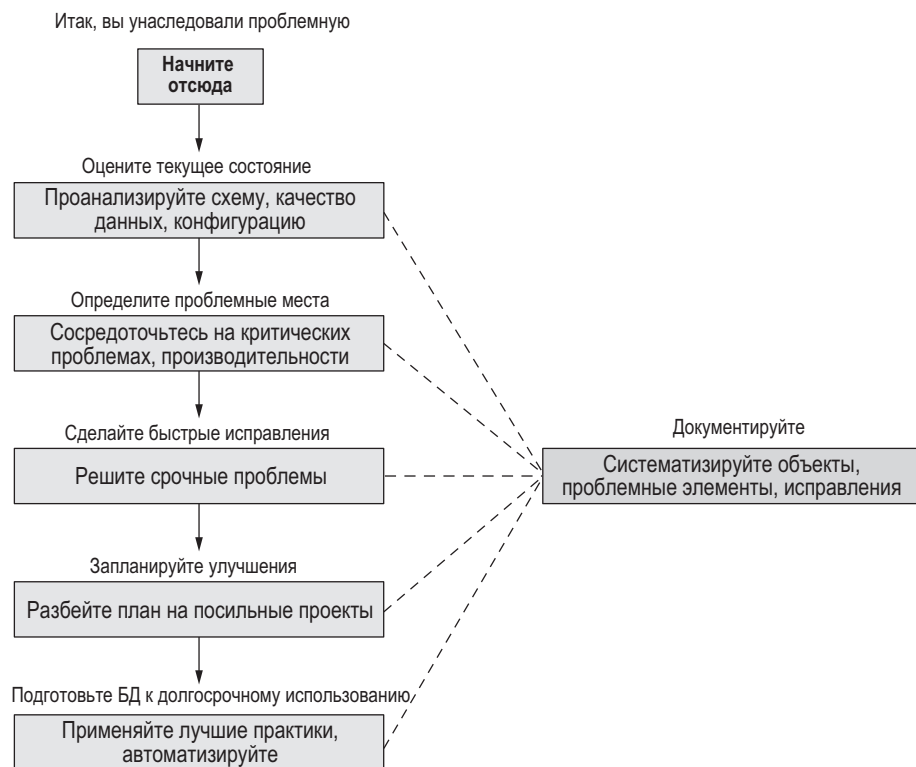


Рис. 11.1. Дорожная карта для систематической оценки, исправления, документирования и улучшения унаследованной базы данных

Задача принять на сопровождение существующую базу данных может казаться пугающей, но это также возможность действительно повлиять на ситуацию. Даже если вам не удастся превратить ее в образцовую базу для работы с данными, вы вполне способны преобразовать плохо функционирующую систему в пригодную для эксплуатации, устойчивую и надежную. Расставьте приоритеты в устранении проблем и двигайтесь постепенно, шаг за шагом. Методичный подход, следование лучшим практикам и стратегическое мышление позволят сделать базу полезной как для вас, так и для других, а также дадут ценный практический опыт.

11.4. Относитесь к Postgres с уважением, и она ответит вам тем же

PostgreSQL — это не просто движок СУБД, а универсальная мощная система, заслуженно получившая репутацию надежной и гибкой платформы. Однако, как и с любым инструментом, успех зависит от правильности использования и бережного отношения к нему. Как вычислительная система Postgres во многом отражает то, как с ним взаимодействуют.

Именно эта универсальность делает PostgreSQL особенным, позволяя решать самые разные задачи: от сложных реляционных операций с терабайтами данных до гибридных подходов с JSONB или узкоспециализированных работ вроде геопространственной обработки. Но именно из-за этой мощности он требует уважительного и аккуратного обращения.

На примере Frogge Emporium, которая прошла через все перипетии экосистемы PostgreSQL на страницах этой книги, становится очевидной фундаментальная истина: учиться на своих (и чужих) ошибках — полезно. Когда видно, где лежат грабли, ты лучше понимаешь, откуда брать лучшие практики.

Привожу возможные плюсы грамотного использования PostgreSQL:

- Если правильно оптимизировать запросы, делать эффективные индексы и точно конфигурировать систему, то можно создать отзывчивую и производительную среду.
- Если грамотно измерять показатели и продуманно планировать работу, то можно масштабировать систему соразмерно росту данных и пользовательской базы.
- Если вести проактивный мониторинг производительности и выполнять профилактику через VACUUM и другие методы, то можно добиться стабильности и надежности (об автоочистках см. главу 6).
- Если реализовать и обеспечить соблюдение мер безопасности, избыточности, а также стратегий резервного копирования и отказоустойчивости,

то можно гарантировать защиту данных и устойчивость эксплуатации (см. главы 8 и 9).

- Если использовать расширенный синтаксис для SQL-выражений в PostgreSQL вместе с его NoSQL возможностями, то можно получить необходимую гибкость во время выбора архитектурного подхода при построении приложения.

Что же нужно, чтобы такой вдумчивый подход вошел в привычку?

- Знайте свои потребности. Отдавайте себе отчет в том, что именно вы делаете, и соизмеряйте цели с возможностями PostgreSQL и его экосистемы.
- Следуйте лучшим практикам. Создавайте чистые схемы и проектируйте безопасные архитектуры.
- Всегда учитесь. Следите за новыми функциями и расширениями, поддерживайте связь с сообществом.

В определенном смысле PostgreSQL можно воспринимать как партнера на вашем профессиональном пути. Он способен значительно помочь, но требует встречного движения и уважения к своим особенностям. Например, соблюдение проверенных шаблонов проектирования и регулярное выполнение `VACUUM` напрямую повышают производительность и надежность. Если вы выполняете свою часть работы, PostgreSQL будет работать не менее усердно.

Конец книги не финал, поскольку работа с PostgreSQL выходит далеко за пределы этих страниц. Пусть ошибки, решения, концепции и идеи, о которых мы говорили, вдохновят вас на дальнейшие исследования, обучение и инновации.

Итоги

- Несоответствие между тем, как вы используете PostgreSQL, и вашими реальными потребностями может быть следствием недостаточной профессиональной осознанности и приводить к предсказуемым ошибкам. Лучшее понимание своего пользовательского профиля, специфики должностных обязанностей, собственных склонностей и ограничений помогает сосредоточиться на проблемных областях, чтобы избежать ошибок и оптимизировать взаимодействие с базой данных. Понимание себя как пользователя, следование лучшим практикам и готовность обратиться за советом позволяют более объективно взглянуть на потенциальные проблемы.
- Вы сильно рискуете, если не вмешиваетесь в незначительную проблему с производительностью, надежностью или масштабируемостью и дожидаетесь, когда эта проблема станет более значимой. Составляйте планы с учетом будущего роста и оптимизируйте все с самого начала, чтобы избежать технического долга. Упреждающие меры, такие как код-ревью и анализ паттернов

использования, в сочетании с регулярным мониторингом и своевременным устранением слабых мест крайне важны и могут избавить от множества проблем.

- Прежде чем постепенно улучшать плохо спроектированную базу данных, устраните первоочередные проблемы. Проведите системную оценку базы с помощью инструментов для проверки ее схемы, анализа данных и мониторинга производительности. В процессе документируйте все, что обнаружили и изменили, автоматизируйте процессы, делитесь лучшими практиками и избегайте проблемы неправильно поставленной задачи.
- Не поддавайтесь иллюзии, что PostgreSQL универсальна: игнорирование ее специфики снижает эффективность. Используйте эту СУБД грамотно: оптимизируйте запросы, применяйте эффективные индексы и проводите профилактическое обслуживание, чтобы создать производительную, стабильную и надежную систему. Будьте в курсе новостей и воспринимайте Postgres, его экосистему и сообщество как надежных партнеров в работе с данными. Понимайте типовые ошибки и следуйте лучшим практикам, чтобы раскрыть весь потенциал PostgreSQL и построить устойчивую, надежную систему.

База данных *Frogge Emporium*

В этом приложении вы найдете информацию о том, как создать схему базы данных Frogge Emporium и заполнить ее тестовыми данными.

A.1. Схема базы данных *Frogge Emporium*

Для воссоздания базы данных Frogge Emporium и пользователя выполните следующие команды в `psql` под пользователем `postgres`:

```
CREATE USER frogge PASSWORD '<password>';
CREATE DATABASE frogge OWNER frogge;
```

Затем создайте схему, выполнив файл `schema.sql` из командной строки:

```
psql -U frogge -f schema.sql
```

Файл `schema.sql` доступен по адресу: <https://github.com/vyruss/postgresql-mistakes/>

Содержимое `schema.sql`:

```
CREATE SCHEMA erp;
CREATE SCHEMA audit;
CREATE SCHEMA support;
CREATE SCHEMA test;

-- Customers go in this table.
CREATE TABLE erp.customers (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    first_name text NOT NULL,
    middle_name text,
    last_name text,
    marketing_consent boolean DEFAULT false NOT NULL
);

-- This is where we hold contact details for customers.
CREATE TABLE erp.customer_contact_details (
    id bigint PRIMARY KEY REFERENCES erp.customers(id),
```

```

    email text DEFAULT '' NOT NULL,
    street_address text,
    city text,
    state text,
    country text,
    phone_no text
);
CREATE INDEX ON erp.customer_contact_details (email);

-- We represent order status by an enumeration.
CREATE TYPE erp.order_status AS ENUM (
    'Placed',
    'Fulfilled',
    'Cancelled'
);

-- Order groups aggregate large orders for multiple items.
CREATE TABLE erp.order_groups (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    status erp.order_status,
    placed_at timestamptz,
    updated_at timestamptz,
    customer bigint REFERENCES erp.customers(id)
);

-- Table to hold orders for individual items or services.
CREATE TABLE erp.orders (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    order_group bigint REFERENCES erp.order_groups(id),
    status erp.order_status,
    placed_at timestamptz,
    updated_at timestamptz,
    item integer,
    service integer
);

-- Each invoice for an order group goes in here.
CREATE TABLE erp.invoices (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    amount numeric NOT NULL,
    customer bigint REFERENCES erp.customers(id),
    paid boolean DEFAULT false NOT NULL,
    order_group bigint REFERENCES erp.order_groups(id),
    updated_at timestamptz DEFAULT CURRENT_TIMESTAMP,
    created_at timestamptz DEFAULT CURRENT_TIMESTAMP
);

-- We hold payments for specific invoices in here.
CREATE TABLE erp.payments (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    tstamp timestamptz NOT NULL,
    amount numeric NOT NULL,
    invoice bigint REFERENCES erp.invoices(id)

```

```

);
CREATE INDEX ON erp.payments (tstamp);

-- Our list of suppliers and their details.
CREATE TABLE erp.suppliers (
    id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    company_name text,
    state text,
    country text,
    phone_no text,
    email text
);

-- We represent the type of each email sent by an enumeration.
CREATE TYPE erp.email_type AS ENUM (
    'Invoice reminder',
    'Welcome',
    'Account closed',
    'Happy birthday'
);

-- This table is the history of all emails sent out to customers.
CREATE TABLE erp.sent_emails (
    tstamp timestamptz PRIMARY KEY DEFAULT CURRENT_TIMESTAMP,
    customer bigint REFERENCES erp.customers(id),
    email_type erp.email_type,
    invoice bigint REFERENCES erp.invoices(id)
);

-- This table records energy usage readings for each of the branches.
CREATE TABLE erp.energy_usage (
    branch_id integer NOT NULL,
    reading_time timestamptz NOT NULL DEFAULT CURRENT_TIMESTAMP,
    reading numeric NOT NULL,
    unit varchar DEFAULT 'kwh' NOT NULL
);

-- This table holds customer service ticket details.
CREATE TABLE support.tickets (
    id integer PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    content text,
    status smallint,
    opened_at timestamptz DEFAULT CURRENT_TIMESTAMP NOT NULL,
    closed_at timestamptz
);

-- Logging of user activity for audit purposes.
CREATE TABLE audit.audit_log (
    id bigint PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    what text,
    who text,
    tstamp timestamptz
);

```

A.2. Данные базы Frogge Emporium

Заполните базу данными о клиентах и их контактах, загрузив файл `customer_dump.sql` из командной строки:

```
psql -U frogge -f customer_dump.sql
```

Файл `customer_dump.sql` доступен по адресу: <https://github.com/vyruss/postgresql-mistakes/>.

Затем создайте остальные данные, выполнив `create_data.sql` из командной строки:

```
psql -U frogge -f create_data.sql
```

Файл `create_data.sql` доступен по адресу: <https://github.com/vyruss/postgresql-mistakes/>.

Содержимое `create_data.sql`:

```
-- Data for suppliers table
INSERT INTO erp.suppliers (company_name, state, country, email) VALUES
('Omni Consumer Products', 'MI', 'United States of America', 'ocp@example.com'),
('Yoyodyne', null, 'Japan', 'yoyodyne@example.com');
-- Data for orders, order_groups, invoices, payments, sent_emails tables
DO $$
DECLARE _id bigint;
DECLARE _t1 timestamptz;
DECLARE _t2 timestamptz;
BEGIN
SELECT CURRENT_DATE - INTERVAL '1y' INTO _t1;
SELECT CURRENT_DATE INTO _t2;
FOR i IN 1 .. 50000 LOOP
    INSERT INTO erp.order_groups (status, placed_at, updated_at, customer)
    VALUES ('Fulfilled',
        (_t1 + (i * INTERVAL '1 s')),
        (_t1 + (i * INTERVAL '1 s')),
        TRUNC(RANDOM() * 14000 + 1)) RETURNING id INTO _id;
    INSERT INTO erp.orders (order_group, status, placed_at, updated_at,
        item) VALUES
        (_id, 'Fulfilled',
        (_t1 + (i * INTERVAL '1 s')),
        (_t1 + (i * INTERVAL '1 s')),
        TRUNC(RANDOM() * 1000 + 1));
    INSERT INTO erp.invoices (amount, customer, paid, order_group,
        created_at, updated_at) VALUES
        (59.95, (SELECT customer FROM erp.order_groups WHERE id=_id), 't',
        _id,
        (_t1 + (i * INTERVAL '1 s')),
        (_t1 + (i * INTERVAL '1 s')
        + INTERVAL '30 s')) RETURNING id INTO _id;
```

```

INSERT INTO erp.payments (tstamp, amount, invoice)
VALUES ((_t1 + (i * INTERVAL '1 s')
        + INTERVAL '30 s'), 59.95, _id);
END LOOP;
FOR i IN 1 .. 200000 LOOP
INSERT INTO erp.order_groups (status, placed_at, updated_at, customer)
VALUES ('Placed', _t2 - INTERVAL '2 d' + (i * INTERVAL '1 s'),
        _t2 - INTERVAL '2 d' + (i * INTERVAL '1 s'),
        TRUNC(RANDOM() * 14000 + 1)) RETURNING id INTO _id;
INSERT INTO erp.orders (order_group, status, placed_at, updated_at,
item) VALUES
        (_id, 'Placed', _t2 - INTERVAL '2 d' + (i * INTERVAL '1 s'),
        _t2 - INTERVAL '2 d' + (i * INTERVAL '1 s'),
        TRUNC(RANDOM() * 1000 + 1));
INSERT INTO erp.invoices (amount, customer, paid, order_group,
created_at, updated_at) VALUES
        (59.95, (SELECT customer FROM erp.order_groups WHERE id=_id), 't',
        _id,
        _t2 - INTERVAL '2 d' + (i * INTERVAL '1 s'),
        _t2 - INTERVAL '2 d' + (i * INTERVAL '1 s') + INTERVAL '30 s')
        RETURNING id INTO _id;
INSERT INTO erp.payments (tstamp, amount, invoice)
VALUES (_t2 - INTERVAL '2 d' + (i * INTERVAL '1 s') + INTERVAL '30 s',
        59.95, _id);
END LOOP;
WITH o AS (SELECT id FROM erp.orders ORDER BY RANDOM() LIMIT 1350
FOR UPDATE)
UPDATE erp.orders SET item = NULL, service = 21
FROM o WHERE orders.id=o.id;
WITH i AS (SELECT id FROM erp.invoices ORDER BY RANDOM() LIMIT 1350
FOR UPDATE)
UPDATE erp.invoices SET paid='f' FROM i WHERE invoices.id=i.id;
WITH i AS (SELECT id, created_at, customer FROM erp.invoices
WHERE paid='f')
INSERT INTO erp.sent_emails (tstamp, customer, email_type, invoice)
SELECT i.created_at + INTERVAL '1 d', i.customer,
        'Invoice reminder', i.id FROM i;
END $$ LANGUAGE plpgsql;
-- Data for tickets table
INSERT INTO support.tickets (status, content, opened_at, closed_at)
SELECT 20, 'issue text',
CURRENT_DATE - INTERVAL '2y' + n * (INTERVAL '1 m'),
CURRENT_DATE - INTERVAL '2y' + n * (INTERVAL '1 m') + INTERVAL '1 d'
FROM generate_series(1,100000) n;
INSERT INTO support.tickets (status, content, opened_at)
SELECT 10, 'issue text',
CURRENT_DATE - INTERVAL '1y' + n * (INTERVAL '1 m')
FROM generate_series(1,500) n;

```

В

Памятка

Таблица В.1. Некорректное использование SQL

Использование NOT IN для исключения	Не используйте NOT IN для исключения списка значений, который может содержать хотя бы одно NULL — это вернет пустой результат. Рассмотрите применение NOT EXISTS
Выбор диапазонов с помощью BETWEEN	Фильтрация с BETWEEN — например, между двумя временными метками — может возвращать пересекающиеся результаты в последующих запросах, поскольку диапазоны включают границы
Отказ от использования CTE	СТЕ не только повышают читаемость запросов, но и могут улучшить производительность, позволяя оптимизатору объединять и переупорядочивать части запроса
Использование идентификаторов в верхнем регистре	Заключение идентификаторов в кавычки делает их регистрозависимыми, тогда как соглашение Postgres предполагает регистронезависимость всех идентификаторов. Это может снизить удобство использования и приводить к ошибкам
Деление целых чисел INTEGER	Деление целых чисел дает усеченный целочисленный результат, что часто оказывается неожиданным
Подсчет значений NULL в COUNT	count () игнорирует значения NULL, поэтому при подсчете столбца, допускающего NULL, вы получите не общее количество строк, а число строк без NULL в этом поле
Запросы к индексированным столбцам через выражения	При запросе к индексированным данным, к которым применено приведение к типу, Postgres может вообще не использовать индекс
UPSERT с NULL в составном уникальном ключе	По возможности избегайте использования столбцов с NULL в составных уникальных ключах. PostgreSQL 15 вводит NULLS NOT DISTINCT для решения проблем с UPSERT, но лучше применять более корректную схему данных

Выборка и получение всех данных	Не замедляйте запросы выборкой большего объема данных, чем строго необходимо, и не получайте на стороне приложения больше данных, чем нужно. Фильтрацию и операции с данными выполняйте на стороне СУБД
Игнорирование преимуществ чекеров/линтеров или больших языковых моделей	Применяйте инструменты для проверки и линтинга SQL на корректность и потенциальное влияние на продакшен. Используйте генеративный ИИ для упрощения работы, но учитывайте ограничения — всегда перепроверяйте ответы и принимайте окончательное решение самостоятельно

Таблица В.2. Некорректное использование типов данных

TIMESTAMP (WITHOUT TIME ZONE)	Использование TIMESTAMP (WITHOUT TIME ZONE) не дает преимуществ и может приводить к ошибкам в расчетах времени из-за отсутствия информации о часовом поясе и переходе на летнее время. TIMESTAMP WITH TIME ZONE — правильный тип данных для записи временных меток как конкретных моментов времени
TIME WITH TIME ZONE	У TIMETZ и CURRENT_TIME сомнительная полезность, поскольку часовые пояса теряют смысл без контекста дат. Опять же предпочтительнее использовать TIMESTAMPTZ, даже если не требуется отображать часть даты во временной метке
MONEY	MONEY не хранит информацию о валюте и обладает ограниченной и несовершенной реализацией. Следует избегать его использования в пользу NUMERIC или других числовых форматов, которые точно хранят значения, возможно, в сочетании с хранением валюты в отдельном столбце
SERIAL	Два серийных типа SERIAL и BIGSERIAL фактически вытеснены столбцами-идентификаторами, которые обладают более предсказуемым поведением в вопросах ролевой принадлежности и использования последовательностей, а также обеспечивают ясность относительно принадлежности последовательности к конкретной таблице
CHAR(n), VARCHAR(n)	Ограниченные символьные типы CHAR(n) и VARCHAR(n) не экономят место для хранения, а пробелы, сохраняемые в CHAR(n), могут ухудшить производительность. Кроме того, существует риск столкнуться с особенностями SQL и загнать себя в угол, выставляя максимальные ограничения по длине. TEXT — более удачный вариант
XML	XML — неудачный вариант для хранения документов, если только вы не копируете неизменяемые XML-данные внутри базы. Для запросов и манипуляций с данными следует использовать JSON(B)

Таблица В.3. Ошибки при работе с таблицами и индексами

Наследование таблиц	Если вы открыли для себя механизм наследования таблиц и считаете, что он вам нужен, скорее всего, вы ошибаетесь. Реализуйте отношения родитель–потомок через внешние ключи и триггеры при необходимости
Пренебрежение партиционированием таблиц	Партиционирование таблиц упрощает управление крупными таблицами и их обслуживание, а также ускоряет запросы к ним
Партиционирование по нескольким ключам	Будьте осторожны с субпартиционированием — партиционирование по нескольким ключам не то же самое. К сожалению, из документации неочевидно, что делает синтаксис многоключевого партиционирования, когда его лучше применять и каковы его последствия
Использование неправильного типа индекса	Каждый тип индекса в PostgreSQL имеет свои сильные и слабые стороны. Адаптируя стратегию индексирования под тип данных и характер запросов, можно оптимизировать производительность и использование места. При правильном подходе запросы могут выполняться значительно быстрее

Таблица В.4. Неправильное использование возможностей

Выбор кодировки SQL_ASCII	SQL_ASCII — это скорее отсутствие кодировки, чем сама кодировка. Чтобы избежать риска необратимого смешения кодировок и использовать удобство автоматического преобразования наборов символов, применяйте для базы данных UTF-8
CREATE RULE	Правила (RULE) в PostgreSQL не имеют отношения к правилам в других СУБД. Они сложны для понимания и в основном служат внутренним механизмом Postgres. В большинстве случаев они работают не так, как ожидает пользователь, поэтому вместо них лучше использовать триггеры (TRIGGER)
Реляционный JSON	Использование значений JSON(B) с реляционными шаблонами доступа приводит к менее эффективным SQL-запросам, которые сложнее читать и которые могут работать медленнее. Не следует смешивать парадигмы SQL и NoSQL — лучше применять каждый инструмент для решения соответствующих задач
Повсеместное использование UUID	UUID занимают больше места для хранения, чем даже bigint, а их индексирование менее эффективно. Они гарантируют диапазон значений, который может быть избыточным для конкретной задачи, тогда как целочисленный индекс часто достаточно для обеспечения уникальности

Самописная multi-master репликация	Двунаправленная или multi-master репликация сложнее, чем кажется на первый взгляд. Для практического использования в продакшен-системе существует длинный список требований, соображений и ограничений. Если задача действительно оправдывает настройку multi-master системы, лучше не изобретать велосипед, а использовать готовое решение, разработчики которого уже учли эти аспекты
Самописные распределенные системы	Все предыдущие пункты относятся и к самописным распределенным системам. Создавать их не рекомендуется, если только область применения не очень узкоспециализированная. Необходимо учитывать возможность аномалий распределенной сериализации и реальные компромиссы, описанные в теоремах CAP и PACELC

Таблица В.5. Нерекомендуемые практики, снижающие производительность

Конфигурация по умолчанию в продакшене	Используемые значения по умолчанию в PostgreSQL очень занижены по ресурсам и в большинстве случаев будут неоптимальны для реальных рабочих нагрузок. Это означает, что системные ресурсы будут использоваться не полностью, а потенциальный прирост производительности останется нереализованным
Некорректное распределение памяти	Можно серьезно ухудшить работу системы, неправильно рассчитав настройки распределения памяти. Необходимо учитывать характер рабочей нагрузки: типы выполняемых запросов, их требования к памяти и ожидаемый уровень многопользовательского доступа
Слишком много подключений	Избыточная многопользовательская нагрузка может не повысить, а снизить производительность из-за особенностей работы механизма MVCC (Multi-Version Concurrency Control). Важно понимать риски, связанные со слишком большим числом открытых сессий в PostgreSQL, и учитывать как специфику рабочей нагрузки, так и ограничения конкретного сервера базы данных
Простаивающие соединения	Простаивающие соединения создают вычислительную нагрузку, которая может влиять на производительность PostgreSQL и операционной системы. Следует избавляться от соединений, в которых не выполняется полезная работа. Если это невозможно, используйте пулер соединений, который учитывает эту семантику, например PgBouncer в режиме Transaction Mode
Управление длительными транзакциями	Сессии в состоянии idle in transaction и сессии с длительными запросами могут неожиданно блокировать другие операции, вызывая задержки или ошибки в приложениях. Они также способны откладывать или полностью блокировать работу autovacuum, что приводит к падению производительности из-за раздувания данных или более серьезных ошибок

Высокая частота транзакций	Изменения в базе данных от большого и неуправляемого количества транзакций могут превышать скорость работы механизма самоочистки <code>autovacuum</code> , работающего над предотвращением <code>XID wraparound</code> (переполнения <code>XID</code> транзакций), что, в свою очередь, может привести к падению базы данных. Снизить риск этой проблемы можно, используя пакетную обработку, реорганизацию и агрегацию данных — или даже отказываясь от потенциально бесполезных данных
Отключение <code>autovacuum/</code> <code>autoanalyze</code>	<code>Autovacuum</code> критически важен для корректной работы базы данных. Снижение его эффективности или полное отключение ради экономии системных ресурсов — ошибочный подход, поскольку любой выигрыш в производительности со временем будет перечеркнут накопленным эффектом раздувания данных, неточной статистикой оптимизатора и принудительным использованием защиты от <code>XID wraparound</code>
Игнорирование возможностей <code>EXPLAIN</code> (<code>ANALYZE</code>)	Если обнаруживаются признаки медленного запроса, необходимо проанализировать план его выполнения через <code>EXPLAIN</code> — это быстрый и точный способ выявить причины проблем и помочь в их устранении
Явная блокировка	Явная блокировка объектов может приводить к блокировкам чтения/записи, из-за чего приложение начинает работать медленно или с ошибками. Если требуется синхронизация, используйте уровень изоляции <code>SERIALIZABLE</code> и реализуйте в приложении механизм повторных попыток выполнения операций
Отсутствие индексов	Индексы критически влияют на скорость выполнения запросов, а PostgreSQL предлагает множество их типов. Анализируйте планы запросов и применяйте индексы для того, чтобы ускорить выполнение условий <code>WHERE</code> . Последовательные сканирования нежелательны, поскольку их скорость снижается вместе с ростом таблицы
Лишние индексы	Ненужные индексы замедляют операции записи в таблицу и занимают ценное дисковое пространство. Выявляйте неиспользуемые индексы через представление <code>pg_stat_all_indexes</code> и удаляйте их
Удаление индексов, используемых в других местах	Соблюдайте осторожность при удалении индексов, которые кажутся неиспользуемыми. Статистика по использованию индекса актуальна только для локального узла, но этот же индекс может использоваться на другом узле, куда выполняется физическая репликация

Таблица В.6. Нерекомендуемые практики администрирования

Отсутствие мониторинга дискового пространства	Исчерпание дискового пространства может вызвать серьезные проблемы, поэтому тщательно отслеживайте его использование. Поспешные решения вроде удаления файлов, похожих на логи или на ненужные данные, а также изменения размера томов могут усугубить ситуацию. Определите, что именно потребляет пространство, и решите проблему, используя несколько томов
Запись логов в PGDATA	Хранение логов PostgreSQL в отдельной файловой системе от базы данных снижает риск исчерпания дискового пространства, связанный с чрезмерным логированием. Также помогает настройка ротации логов и ограничение их размера
Игнорирование логов	Регулярная проверка логов PostgreSQL критична для своевременного выявления и устранения ошибок конфигурации, узких мест производительности, проблем целостности данных и инцидентов безопасности. Проверка позволяет начать решать проблемы до их эскалации, а для упрощения задачи можно использовать pgBadger
Отказ от мониторинга базы данных	Если не внедрять мониторинг PostgreSQL, будет трудно выявлять неочевидные проблемы в работе системы, угрозы безопасности и исчерпание ресурсов, которое также влияет на работу системы. Чтобы обеспечить оптимальную производительность и снизить риски, используйте необходимые инструменты и метрики для отслеживания низкой производительности, скачков нагрузки и всплеска запросов
Отказ от отслеживания статистики во времени	Продолжительное отслеживание статистики PostgreSQL важно для того, чтобы понимать тренды производительности и принимать информированные решения по планированию мощностей. Для сбора и визуализации статистики можно использовать стандартные инструменты мониторинга или легковесное расширение pg_statviz, которое не требует ресурсоемких инструментов и значительных затрат на хранение
Отказ от обновлений PostgreSQL	Если не обновлять регулярно PostgreSQL минорными версиями, можно получить риски ИБ или ошибки, которые были в этих версиях исправлены. Отказ от перехода на следующую основную (мажорную) версию лишает доступа к новым функциям и улучшениям. Безопасным и надежным обновление делают хорошо протестированные инструменты и подробная документация. Не бойтесь обновлений — бойтесь их пропустить
Отказ от обновления системы	Если вы не обновляете операционную систему, а также зависимые библиотеки, у вас могут возникнуть неочевидные проблемы с работой PostgreSQL, равно как ошибки и уязвимости, что сведет на нет видимость стабильности, ради которой вы избегали обновлений

Таблица В.7. Нерекомендуемые практики безопасности

Указание <code>psql -W</code> или <code>--password</code>	Применение <code>psql -W</code> или <code>--password</code> может сбивать с толку и создавать уязвимости. Вместо этого используйте встроенный механизм автоматического запроса пароля в PostgreSQL
Установка <code>listen_addresses = '*'</code>	Установка параметра в <code>listen_addresses = '*'</code> может открыть сервер базы данных для небезопасных сетей, поэтому включайте в список только доверенные сетевые интерфейсы, необходимые для подключения к базе данных
Использование <code>trust</code> в <code>pg_hba.conf</code>	Применение метода <code>trust</code> в <code>pg_hba.conf</code> в продакшен-средах недопустимо. Всегда требуйте надлежащей аутентификации на сервере и по максимуму ограничивайте доступ
База данных, принадлежащая суперпользователю	Если владельцем баз данных и их содержимого указан суперпользователь, это может привести к проблемам безопасности и случайному повреждению данных. Я рекомендую создавать роли с минимально необходимыми правами для владения базами данных и предоставлять разрешения другим ролям выборочно
Неосторожное использование <code>SECURITY DEFINER</code>	Объявление функций с атрибутом <code>SECURITY DEFINER</code> может вызвать утечки данных и эскалацию привилегий. Чтобы снизить риски, применяйте этот вариант крайне осторожно, с указанием безопасного <code>search_path</code> , и отдавайте предпочтение комбинации <code>SECURITY INVOKER</code> с явными командами <code>GRANT</code>
Выбор небезопасного пути поиска	Отсутствие защиты у <code>search_path</code> позволяет перехватывать запросы и повышать привилегии. Обеспечьте строгий контроль создания объектов в схемах и ссылайтесь в запросах только на объекты, принадлежащие доверенным пользователям

Таблица В.8. Нерекомендуемые практики, снижающие уровень доступности

Отсутствие резервного копирования	RAID и снимки файловой системы не гарантируют защиты от повреждения данных, человеческих ошибок или вредоносных действий. Гарантировать сохранность данных можно только с помощью регулярного резервного копирования специализированными инструментами, такими как <code>pg_basebackup</code>
Отсутствие точки восстановления на момент времени	Полные резервные копии без использования возможностей PITR (Point-in-Time Recovery) создают риск потери данных между бэкапами. Используйте механизм PITR с непрерывным архивированием, чтобы иметь возможность откатить базу данных на нужный момент до повреждения
Резервное копирование вручную	Резервное копирование вручную ненадежно и нестабильно. Настройте автоматизированное расписание бэкапов, предпочтительно с помощью специализированного ПО для PostgreSQL (например, <code>Barman</code> или <code>pgBackRest</code>), и обязательно сохраняйте резервные копии на другом физическом носителе

Отсутствие тестирования резервных копий	Непроверенные бэкапы могут выдать ошибку в самый ответственный момент. Для гарантии работоспособности регулярно выполняйте пробное полное восстановление. Не полагайтесь исключительно на автоматизацию — проверяйте каждый этап. Отдавайте предпочтение проверенным решениям вместо самодельных скриптов
Отсутствие избыточности	Одноузловой сервер баз данных без механизма отказоустойчивости неизбежно приведет к простоям. Обеспечьте избыточность, настроив резервные узлы и репликацию на них
Игнорирование специальных инструментов высокой доступности	Ручное переключение или кастомные скрипты чреваты рисками продолжительных простоев, расхождения данных или их потерь. Используйте проверенные инструменты для высокой доступности, такие как RepMgr, Patroni и CloudNativePG для Kubernetes, чтобы надежно управлять кластером

Таблица В.9. Нерекомендуемые практики обновления и миграции

Пропуск примечаний к выпуску	Перед обновлением обязательно изучайте примечания к релизам (release notes) ко всем версиям между текущей и целевой PostgreSQL. Пренебрежение этим правилом приводит к неожиданному поведению СУБД из-за появления в ней новых оптимизаций или критических изменений, диагностика и исправление которых требуют значительного времени
Недостаточное тестирование	Обновления PostgreSQL могут вносить незначительные изменения в поведение или производительность, недооценка которых в итоге может создать проблемы. Тщательно тестируйте приложение и запросы в предпрод-среде перед обновлением, используя реалистичные данные и нагрузки для того, чтобы заблаговременно выявлять потенциальные проблемы
Погружение в хаос кодировок	Игнорирование различий в кодировках символов, а также использование недопустимых символов при миграции базы данных приводит к ошибкам импорта или даже потере данных. При переносе данных между разными СУБД обязательно указывайте кодировки символов и соответствующее преобразование между ними, а также очищайте данные от нулевых символов ASCII. Инструменты миграции помогают решить эту задачу
Разный BOOLEAN	При переносе булевых данных в PostgreSQL убедитесь, что они используют правильный тип BOOLEAN. Преобразуйте нестандартные представления (например, целые числа или символы) в true и false для сохранения согласованности и предотвращения ошибок несоответствия типов
Некорректная обработка различий в типах данных	Несогласованность в именовании типов данных, их функциональности и точности между разными СУБД приводит к ошибкам миграции. Всегда проверяйте типы данных и их поведение и используйте инструменты преобразования PostgreSQL и утилиты миграции, чтобы обеспечить совместимость

Таблица В.10. О PostgreSQL, лучших практиках и о вас: заключительные выводы

К какому типу пользователей вы относитесь	Несоответствие использования PostgreSQL вашим потребностям может быть связано с недостатком самоанализа, что приводит к предсказуемым ошибкам. Определите свой профиль пользователя, специфические склонности, предубеждения и ограничения, и вы сможете сосредоточиться на областях, которые требуют улучшения, предотвратить ошибки и оптимизировать взаимодействие с базой данных. Понимание своего типа пользователя, соблюдение лучших практик и обращение за советом позволяют более объективно видеть потенциальные проблемы
Будьте проактивны: действуйте на опережение	Вы идете на неоправданный риск, если предпочитаете не вмешиваться в незначительную проблему с производительностью, надежностью или масштабируемостью и дожидаться, когда эта проблема станет более значимой. Создавайте план будущего роста и оптимизируйте все заранее, чтобы предотвратить накопление технического долга. Упреждающие меры, такие как ревью кода и анализ паттернов использования, в сочетании с регулярным мониторингом и своевременным устранением слабых мест имеют решающее значение и могут избавить от множества проблем
Итак, вам досталась проблемная база данных	В случае наследования плохо спроектированной базы данных в первую очередь следует устранить основные болевые точки, прежде чем углубляться в детали. Проведите системную оценку базы с помощью инструментов для проверки ее схемы, анализа данных и мониторинга производительности. В процессе документируйте все, что обнаружили и изменили, автоматизируйте процессы, делитесь лучшими практиками и избегайте «проблемы неправильно поставленной задачи»
Относитесь к Postgres с уважением, и он ответит вам тем же	Не поддавайтесь иллюзии, что PostgreSQL универсален: игнорирование его специфики снижает его эффективность. Используйте СУБД разумно: оптимизируйте запросы, грамотно применяйте индексы и проводите профилактическое обслуживание для создания производительной, стабильной и надежной системы. Будьте в курсе событий и воспринимайте Postgres, его экосистему и сообщество как надежных партнеров на вашем пути работы с данными. Выявляйте распространенные ошибки, следуйте лучшим практикам, и вы сможете раскрыть потенциал PostgreSQL и построить устойчивую, надежную систему

Read IT Club

Комьюнити рецензентов и переводчиков ИТ-литературы

Миссия участников клуба – обеспечить высокое качество профессиональной переводной литературы на русском языке. «Книжные дебагеры» проверяют корректность терминологии и подписей на схемах и иллюстрациях, чтобы сделать книги более понятными русскоязычному читателю. Стать участником Read IT Club может любой ИТ-специалист, готовый поделиться опытом с сообществом.



присоединиться к нам